



Устройство движка нейросети Ollama Квест Инженера ИИ LLM

Модуль 1 - Миссия 1. Пробуждение ядра Ollama

Урок 1 - Вход в лабораторию как работают трансформеры

Шаг 01:

Миссия 1

Разработчик Игорь запускает свою первую модель в Ollama и замечает: ответы модели странно повторяют одну и ту же фразу. Он подозревает, что проблема — в архитектуре трансформеров. Что является ключевым элементом, который позволяет модели понимать контекст предыдущих токенов?

Подсказка: Подумай, какой механизм отвечает за связь слов между собой и называется вниманием.

Ответ:

- **Механизм внимания (attention)**
- Градиентный спуск
- Функция активации ReLU
- Слой нормализации BatchNorm

Шаг 02:

Миссия 2

 Инженер Аня исследует работу слоёв трансформера. Она видит блоки Encoder и Decoder. Какую роль выполняет Encoder в трансформерной архитектуре?

Подсказка: Вспомни, что Encoder преобразует входные данные в представление, понятное модели.

Ответ:

- **Преобразует входной текст в контекстное представление**
- Генерирует выходные токены по одному
- Удаляет шум из входных данных
- Хранит историю предыдущих ответов

🔮 Шаг 03:

Миссия 3

👤 Разработчик Дима замечает, что Ollama выдаёт ответы, словно запоминая структуру предложений. Он интересуется, почему трансформеры обрабатывают все токены параллельно, в отличие от RNN. Что позволяет это делать?

Подсказка: Подумай о том, что заменяет последовательную обработку в RNN на матричные операции.

Ответ:

- **Механизм самовнимания (self-attention)**
- Конволюционные фильтры
- Градиентный шаг обучения
- Случайная инициализация весов

🔮 Шаг 04:

Миссия 4

👤 Программист Катя хочет объяснить стажёру, как трансформеры понимают порядок слов, если они обрабатывают все токены одновременно. Что добавляется к токенам, чтобы модель различала их позиции?

Подсказка: Вспомни, что позиции токенов кодируются дополнительными векторами.

Ответ:

- **Позиционные эмбединги (positional embeddings)**
- Градиенты обучения
- Случайные смещения весов
- Dropout-слои

🔮 Шаг 05:

Миссия 5

🔍 Младший инженер Пётр тестирует Ollama и хочет понять, почему модель иногда выдаёт бессмысленные ответы. Он слышал о 'маскировании' в attention. Что делает маскирование?

Подсказка: Маскирование ограничивает внимание модели, чтобы она не видела будущее при генерации текста.

Ответ:

- **Скрывает будущие токены при обучении или генерации**
- Отключает все веса слоя
- Удаляет токены из входной последовательности
- Случайно обнуляет часть слоёв сети

🔮 Шаг 06:

Миссия 6

Разработчица Ника анализирует слой softmax в attention. Она замечает, что выходные вероятности всегда суммируются в 1. Зачем это нужно?

Подсказка: Это помогает модели распределить внимание между токенами.

Ответ:

- **Чтобы нормализовать внимание между токенами**
- Чтобы ускорить обучение
- Чтобы увеличить количество слоёв
- Чтобы сделать модель устойчивее к шуму

🔮 Шаг 07:

Миссия 7

Задача от наставника Олега: сопоставь ключевые элементы трансформера с их функциями.

Подсказка: Каждый элемент выполняет свою уникальную роль в потоке данных.

Ответ:

- **Self-Attention** — Рассчитывает важность каждого токена по отношению к другим
- **Feed-Forward Network** — Обрабатывает результаты внимания через нелинейные преобразования
- **Positional Encoding** — Добавляет информацию о позиции токенов
- **Layer Normalization** — Стабилизирует обучение между слоями

🔮 Шаг 08:

Миссия 8

👤 Разработчик Саша изучает, как обучаются трансформеры. Соотнеси термин с его правильным определением.

Подсказка: Вспомни, что loss-функция помогает понять, насколько хорошо модель предсказывает токены.

Ответ:

- **Loss Function** — Мера ошибки между предсказанием и правильным ответом
- **Optimizer** — Изменяет веса модели для уменьшения ошибки
- **Gradient Descent** — Метод поиска минимума функции ошибки
- **Learning Rate** — Определяет, насколько сильно корректируются веса

🔮 Шаг 09:

Миссия 9

👥 В команде Ollama обсуждают структуру модели. Сопоставь часть трансформера с её типичным аналогом в коде.

Подсказка: Encoder обрабатывает входные данные, а Decoder генерирует выход.

Ответ:

- **Encoder** — Функция анализа входного текста
- **Decoder** — Функция генерации выходного текста
- **Attention Block** — Связывает части входа по смыслу
- **Embedding Layer** — Преобразует токены в числовые вектора

🔮 Шаг 10:

Миссия 10

Тест инженера Марии: соотнеси известные модели с их особенностями.

Подсказка: GPT — генератор, BERT — понимает контекст, T5 — переводчик смыслов.

Ответ:

- **GPT** — Автоагрессивная модель для генерации текста
- **BERT** — Модель двунаправленного понимания контекста
- **T5** — Модель перевода задачи в формат 'текст-в-текст'
- **LLAMA** — Оптимизированная модель Meta для локальных запусков

🔮 Шаг 11:

Миссия 11

👤 Инженер Алексей проводит ревью кода Ollama. Сопоставь термин с его назначением.

Подсказка: Tokenizer — вход, attention — связь, softmax — распределение.

Ответ:

- **Tokenizer** — Разбивает текст на токены
- **Attention** — Определяет важность токенов
- **Softmax** — Нормализует вероятности
- **Output Layer** — Преобразует вероятности в слова

🔮 Шаг 12:

Миссия 12

Расположи шаги обработки текста в трансформере в правильном порядке.

Подсказка: Сначала текст превращается в числа, затем проходит внимание, после — преобразования и выход.

Ответ:

- Текст разбивается на токены
- Токены преобразуются в эмбединги
- Обработка через слои внимания и нейронные блоки
- Формирование выходных вероятностей и слов

🔮 Шаг 13:

Миссия 13

👤 Инженер Илья документирует процесс генерации текста. Расположи этапы в правильном порядке.

Подсказка: Генерация всегда начинается с промпта и завершается выбором лучшего токена.

Ответ:

- Получение входного промпта
- Кодирование токенов
- Расчёт attention и вероятностей
- Выбор следующего токена и генерация ответа

🔮 Шаг 14:

Миссия 14

 Наставник Вадим даёт практику: выстрой путь обучения модели.

Подсказка: Начни с данных, закончи улучшением весов.

Ответ:

- Сбор и подготовка датасета
- Инициализация модели
- Обучение с помощью оптимизатора
- Оценка и сохранение модели

 **Шаг 15:**

Миссия 15

Расположи логические этапы при обработке промпта Ollama внутри трансформера.

Подсказка: Вспомни: сначала токенизация, потом внимание, потом выход.

Ответ:

- Токенизация входного текста
- Формирование эмбеддингов
- Пропуск через блоки self-attention
- Генерация выходного текста

 **Шаг 16:**

Миссия 16

 Стажёр Лена хочет понять внутреннюю логику трансформера. Упорядочи компоненты в порядке прохождения данных.

Подсказка: От текста к токенам, через внимание, к выходу.

Ответ:

- Входной текст
- Токенизация
- Эмбеддинги
- Attention и Feed-Forward
- Генерация ответа

Урок 2 - Из чего сделана LLaMA и Mistral

 **Шаг 01:**

Миссия 1

Ты только прибыл в лабораторию Ollama. На мониторе вспыхивает надпись: «Разбери, из чего состоит LLaMA». Что лежит в основе любой LLM вроде LLaMA?

Подсказка: Подумай, с чего начинается обучение моделей — с огромных массивов данных и архитектуры нейронных сетей.

Ответ:

- **Архитектура трансформера и большие текстовые датасеты**
- Просто случайные наборы чисел
- Классические нейросети без внимания
- Машинное зрение и картинки

🔮 Шаг 02:

Миссия 2

👤 Разработчик Артём спорит с коллегой: «Почему Mistral быстрее, чем LLaMA?». Что делает Mistral более эффективной?

Подсказка: Mistral — это не магия, а оптимизация внимания и слияние экспертов.

Ответ:

- **Использует Sparse Mixture of Experts и оптимизированное внимание**
- Обучается на меньших данных без потери качества
- Использует RNN вместо трансформеров
- Работает только на CPU

🔮 Шаг 03:

Миссия 3

👤 Инженер Юлия запускает тест Ollama и замечает: память GPU расходуется быстро. Что является ключевым элементом, определяющим размер модели?

Подсказка: Вспомни, что хранится в каждом слое трансформера — матрицы весов.

Ответ:

- **Количество параметров (весов) модели**
- Размер входных данных
- Количество токенов в обучающем корпусе
- Версия CUDA

🔮 Шаг 04:

Миссия 4

📖 Программист Дима изучает структуру LLaMA и находит слой под названием «Attention». Какую задачу он решает?

Подсказка: Этот механизм помогает модели «смотреть» на важные части текста.

Ответ:

- **Выделяет важные токены и контекст в предложении**
- Хранит обученные веса всех токенов
- Отвечает за синтаксический анализ
- Проверяет орфографию

🔮 Шаг 05:

Миссия 5

На утреннем стендапе инженер говорит: «Параметры LLaMA — 7B». Что это значит?

Подсказка: Это не версия модели, а её масштаб по числу обученных весов.

Ответ:

- **У модели 7 миллиардов обучаемых параметров**
- У неё 7 различных архитектур
- Модель обучена 7 часов
- 7 бит на токен

🔮 Шаг 06:

Миссия 6

🔍 Наставник предлагает тебе сопоставить архитектурные элементы LLaMA и их роли. Это поможет понять, как рождаются ответы модели.

Подсказка: Вспомни, где происходит обработка токенов, нормализация и сбор контекста.

Ответ:

- **Embedding слой** — Преобразует токены в числовые векторы
- **Attention слой** — Находит важные связи между токенами
- **Feed Forward слой** — Обработывает признаки после внимания
- **LayerNorm** — Стабилизирует распределения значений между слоями

🔮 Шаг 07:

Миссия 7

В журнале инженера записаны ключевые концепции. Сопоставь термины и их объяснения.

Подсказка: Контекст, токен и эмбединг — три кита LLM.

Ответ:

- **Токен** — Минимальная единица текста, которую понимает модель
- **Контекстное окно** — Количество токенов, которое модель учитывает при ответе
- **Эмбединг** — Векторное представление смысла слова
- **Attention Head** — Механизм фокусировки на разных частях текста

🔮 Шаг 08:

Миссия 8

На доске три формулы. Вычисли, что к чему относится в архитектуре трансформера.

Подсказка: Q, K, V — это не просто буквы, а ключ к вниманию.

Ответ:

- **Q (Query)** — Что ищем в контексте
- **K (Key)** — Ключи, по которым ищем
- **V (Value)** — Значения, которые используем
- **Softmax** — Нормализует веса внимания

🔮 Шаг 09:

Миссия 9

⚙ На стенде Mistral изображён блок «Rotary Embedding». Определи, какие функции выполняют различные виды позиционных кодировок.

Подсказка: Они помогают модели понимать порядок слов.

Ответ:

- **Absolute Positional Encoding** — Фиксированные синусоиды для позиции
- **Rotary Embedding** — Вращает векторы для сохранения относительного порядка
- **Relative Position Bias** — Добавляет смещение в зависимости от расстояния между токенами
- **No Positional Encoding** — Используется в экспериментах без учёта порядка

🔮 Шаг 10:

Миссия 10

🕒 Вечером в офисе инженер Мила сравнивает две модели по параметрам. Сопоставь модель и её характеристики.

Подсказка: Mistral меньше по параметрам, но умнее за счёт архитектуры.

Ответ:

- **LLaMA 7B** — Базовая модель Meta с 7 млрд параметров
- **LLaMA 70B** — Модель для сложных задач, требует мощного GPU
- **Mistral 7B** — Быстрая и эффективная с MoE
- **Mixtral 8x7B** — Состоит из 8 экспертов, активируются только нужные

🔮 Шаг 11:

Миссия 11

🔮 Ты исследуешь Ollama и находишь ключевые файлы модели. Сопоставь файл и его назначение.

Подсказка: Model card описывает, а weights — обучают.

Ответ:

- **model.bin** — Основные веса модели
- **tokenizer.json** — Определяет, как разбивать текст на токены
- **config.json** — Содержит гиперпараметры и архитектуру
- **model_card.md** — Описание модели, источники данных и лицензия

🔮 Шаг 12:

Миссия 12

🔧 Тебе нужно запустить новую модель LLaMA в Ollama. Расставь шаги в правильном порядке.

Подсказка: Начни с подготовки модели, а закончи проверкой запуска.

Ответ:

- Скачать модель и конфигурацию
- Поместить файлы в каталог models/
- Добавить конфигурацию в Ollama.yaml
- Запустить ollama serve и проверить ответ

🔮 Шаг 13:

Миссия 13

🔧 Тебе поручено оптимизировать модель Mistral. Расставь этапы оптимизации в правильной последовательности.

Подсказка: Сначала собираем данные, потом квантуем и тестируем.

Ответ:

- Провести профилирование модели
- Выбрать формат квантования (Q4/Q5)
- Выполнить конвертацию
- Проверить точность и скорость

💡 Шаг 14:

Миссия 14

🧠 Ты анализируешь, как проходит один проход обучения. Расставь шаги в правильном порядке.

Подсказка: Модель читает токены, делает прогноз, считает ошибку и корректирует веса.

Ответ:

- Токенизация входного текста
- Прямое распространение (forward pass)
- Подсчёт функции потерь
- Обратное распространение (backpropagation)

💡 Шаг 15:

Миссия 15

📦 Команда Ollama готовит релиз модели. Расставь этапы деплоя в правильной последовательности.

Подсказка: Сначала упаковка, потом тест, потом релиз.

Ответ:

- Подготовить веса и конфигурацию
- Упаковать модель в формат Ollama
- Протестировать локально
- Опубликовать в репозитории Ollama Hub

📖 Урок 3 - 💡 Ollama как интерфейс к глубинам нейросети

💡 Шаг 01:

Миссия 1

Ты впервые запускаешь Ollama и видишь команду ``ollama run llama``. Что ожидаешь увидеть как результат?

Подсказка: Ollama оборачивает модель, чтобы ты мог взаимодействовать с ней через CLI или API.

Ответ:

- **Ответ модели на твой запрос (текст)**
- График производительности GPU
- Список всех моделей на диске
- Логи сервера Ollama

💡 Шаг 02:

Миссия 2

⚙️ Ты наблюдаешь за log-файлом Ollama и видишь «model loaded». Что это означает?

Подсказка: Это не только загрузка файла — модель становится готовой к запросам.

Ответ:

- **Ollama инициализировал веса модели и готов принимать запросы**
- Модель завершила обучение
- Модель выгружена из памяти
- Сервер Ollama завершил работу

🔗 Шаг 03:

Миссия 3

🔗 Ты тестируешь интерфейс: отправляешь prompt «Привет, мир». Что происходит внутри Ollama?

Подсказка: Ollama передаёт prompt в модель и возвращает выдачу наружу.

Ответ:

- **CLI/API пересылает prompt в модель, получает текст и возвращает пользователю**
- Ollama шифрует prompt и отправляет его в облако
- Ollama сохраняет prompt в базу данных
- Ollama транслирует prompt на GPU как файл данных

🔗 Шаг 04:

Миссия 4

В интерфейсе Ollama ты задаёшь «max_tokens=50». Что это ограничение на?

Подсказка: Модель ограничивает длину ответа в токенах.

Ответ:

- **Максимальное число токенов в выдаче**
- Максимальное число символов
- Максимальное число слов
- Максимальное число байт в ответе

🔗 Шаг 05:

Миссия 5

🔍 Ты смотришь документацию Ollama. Сопоставь команды и их описания.

Подсказка: `ollama list` показывает, что уже загружено; `ollama remove` удаляет модель.

Ответ:

- **ollama run** — Запустить модель и сделать запрос
- **ollama list** — Показать загруженные модели
- **ollama remove** — Удалить модель из Ollama
- **ollama serve** — Запустить модель как сервер API

🔗 Шаг 06:

Миссия 6

Разработчик Вика конфигурирует параметры Ollama. Сопоставь параметр и его роль.

Подсказка: `temperature` влияет на случайность, `top\_p` — на порог вероятности.

Ответ:

- **temperature** — Контролирует степень случайности ответа
- **top_p** — Выбирает токены с накопленной вероятностью p
- **max_tokens** — Ограничивает длину ответа модели
- **stop_sequences** — Останавливает генерацию при встрече с шаблоном

🔗 Шаг 07:

Миссия 7

👂 Ты загружаешь модель в Ollama и наблюдаешь, как он создаёт слои. Сопоставь слои интерфейса и их задачи.

Подсказка: Ollama добавляет «adapter» и управляет вводом-выводом.

Ответ:

- **Input preprocessing** — Токенизирует и нормализует текст
- **Inference engine** — Запускает прямой проход модели
- **Adapter layer** — Интеграция API или форматирования вывода
- **Output postprocessing** — Декодирует токены в текст

🔮 Шаг 08:

Миссия 8

🔗 На твоём экране — статус API-сервера Ollama. Сопоставь термин статус и его значение.

Подсказка: `running`, `stopped`, `error` — типичные состояния сервисов.

Ответ:

- **running** — Сервер запущен и готов к запросам
- **stopped** — Сервер не запущен
- **error** — Произошла ошибка при запуске
- **restarting** — Сервер перезапускается

🔮 Шаг 09:

Миссия 9

📁 Ты изучаешь рабочую папку Ollama. Сопоставь файл / папку и их назначение.

Подсказка: `models/` содержит модели, `logs/` — вывод запуска.

Ответ:

- **models/** — Хранит папки с загруженными моделями
- **logs/** — Содержит журналы запуска и ошибок
- **config.yaml** — Настройки Ollama (порт, пути)
- **cache/** — Кеширует результаты генерации

🔮 Шаг 10:

Миссия 10

🔗 Ты разрабатываешь pipeline: от запроса пользователя до ответа модели через Ollama.

Расставь шаги в правильном порядке:

Подсказка: Сначала запрос, потом интерфейс, потом модель, затем возвращаем ответ.

Ответ:

- Пользователь отправляет текстовый prompt
- Ollama принимает prompt через API/CLI
- Модель выполняет inference
- Ollama возвращает сгенерированный текст пользователю

🔮 Шаг 11:

Миссия 11

🔗 Ты готовишь новое обновление Ollama. Нужно пересобрать компоненты. Расставь этапы:

Подсказка: Сначала собираешь код, потом тестируешь, затем деплоишь.

Ответ:

- Компиляция / сборка проекта
- Запуск модульных и интеграционных тестов
- Сборка дистрибутива (binary / контейнер)
- Развёртывание в production

 Шаг 12:

Миссия 12

 Ты анализируешь логи Ollama при падении inference. Расставь шаги от причины к отладке:
Подсказка: Начни с симптомов, затем диагностики, затем исправления.

Ответ:

- Обнаружение симптомов (ошибка, timeout)
- Сбор трейсбеков и метрик
- Идентификация узкого места
- Внесение патча и пересборка

 Шаг 13:

Миссия 13

 Ты интегрируешь Ollama в продакшн. Расставь этапы развёртывания:
Подсказка: Сначала подготовка среды, потом миграции, затем запуск.

Ответ:

- Настроить окружение (сервер, GPU, зависимости)
- Залить модели и конфигурации
- Применить миграции (если есть)
- Запустить сервис Ollama и мониторинг

 Шаг 14:

Миссия 14

 Ты модернизируешь интерфейс Ollama: добавляешь новую команду. Расставь этапы от идеи до релиза:

Подсказка: Сначала дизайн, потом код, потом тест и деплой.

Ответ:

- Спроектировать поведение и синтаксис команды
- Написать реализацию внутри кода Ollama
- Написать тесты CLI / API
- Включить в сборку и выпустить обновление

 Шаг 15:

Миссия 15

 Ты следишь за SLA Ollama в продакшене. Расставь шаги реагирования на перегрузку API:

Подсказка: Монитор → тревога → масштабирование → проверка.

Ответ:

- Сбор метрик и мониторинг нагрузки
- Срабатывание тревоги (alert)

- Вертикальное или горизонтальное масштабирование
- Проверка устойчивости и стабильности

🔮 Шаг 16:

Миссия 16

🔮 Финальный тест: ты провел нагрузочное тестирование Ollama в продакшн-сценарии. Расставь действия по шагам проверки стабильности:

Подсказка: От простого к комплексному: базовые запросы → стресс → устойчивость.

Ответ:

- Отправить лёгкие запросы и измерить латентность
- Увеличить нагрузку до пиковых значений
- Наблюдать отклики, ошибки, таймауты
- Проверить, что система не упала и ответы корректны

📖 Урок 4 - 🔍 Разведка архитектура attention и токенизация

🔮 Шаг 01:

Миссия 1

Вы — инженер Ollama: ядро проснулось, нужно выбрать начальный шаг. Что вы делаете первым?

Подсказка: подумайте, что нужно знать, прежде чем что-то менять.

Ответ:

- **Проверяю, как ядро обрабатывает токены на входе**
- Немедленно запускаю обучение attention
- Удаляю все временные файлы модели
- Начинаю оптимизировать скорости GPU

🔮 Шаг 02:

Миссия 2

Вы заметили, что токенизация разделяет слово «unbelievable» неправильно. Что вы проверяете первым?

Подсказка: нужно увидеть, какие правила токенизации сейчас загружены.

Ответ:

- **Конфигурацию словаря и правила BPE**
- Версию CUDA ядра
- Как компилировался модуль attention
- Сетевые настройки сервера

🔮 Шаг 03:

Миссия 3

Вы видите, что attention веса сильно концентрированы на крайних токенах. Какая гипотеза наиболее вероятна?

Подсказка: часто причина — ошибки в нормализации или маскировке.

Ответ:

- **Неправильно применена маска padding-a**

- learning rate слишком низкий
- все слои attention отключены
- ошибка в загрузке эмбедингов

🔧 Шаг 04:

Миссия 4

Ваш напарник предлагает сменить библиотеку токенизации. Что сначала проверите?

Подсказка: важно, чтобы новые токены соответствовали старым идентификаторам.

Ответ:

- **Совпадение id-токенов между словарями**
- Размер файла библиотеки
- Скорость сериализации модели
- Количество параметров attention

🔧 Шаг 05:

Миссия 5

Вы экспериментируете с multi-head attention, но замечаете, что один из голов весит ноль. Что вероятнее всего случилось?

Подсказка: проверьте градиенты и нормировки для каждой головы.

Ответ:

- **Градиенты этой головы исчезают (vanishing)**
- Проблема в dropout на остальных головах
- Ошибка сериализации весов
- Модель зафиксировала голову как постоянную

🔧 Шаг 06:

Миссия 6

Сопоставьте этап токенизации с его описанием:

Подсказка: вспомните, какие шаги входят в BPE / WordPiece.

Шаги:

Ответ:

- Чистка текста
- Разбиение на базовые символы / байты
- Слияние частых пар токенов
- Построение словаря

Описания:

1. Удаление незначащих символов, нормализация регистра
2. Инициализация алфавита как отдельные символы / байты
3. Поиск и объединение наиболее частых соседних пар токенов
4. Сохранение финального набора токенов в словарь модели

- **Чистка текста** — Удаление незначащих символов, нормализация регистра
- **Разбиение на базовые символы / байты** — Инициализация алфавита как отдельные символы / байты

- **Слияние частых пар токенов** — Поиск и объединение наиболее частых соседних пар токенов
- **Построение словаря** — Сохранение финального набора токенов в словарь модели

💡 Шаг 07:

Миссия 7

Сопоставьте термин LLM/attention с его ролью внутри ядра Ollama:

Подсказка: подумайте, где attention, где контекст, где позиционная эмбединг.

Термины:

Ответ:

- Position embedding
- Query-Key-Value
- Attention mask
- Context window

Роли:

1. Добавляет информацию о позиции токенов
 2. Контекст, через который происходит вычисление внимания
 3. Ограничивает, к каким токенам можно «смотреть»
 4. Основные вычислительные ветви внимания
- **Position embedding** — Добавляет информацию о позиции токенов
 - **Query-Key-Value** — Основные вычислительные ветви внимания
 - **Attention mask** — Ограничивает, к каким токенам можно «смотреть»
 - **Context window** — Контекст, через который происходит вычисление внимания

💡 Шаг 08:

Миссия 8

Сопоставьте тип ошибки с её наименованием, которое вы могли бы встретить в дебаге ядра Ollama:

Подсказка: подумайте об overflow, underflow, nan, invalid mask.

Ошибки:

Ответ:

- NaN в весах attention
- Overflow активации
- Подсчет attention к паддингу без маски
- Underflow края softmax

Наименования:

1. invalid_mask_error
 2. softmax_underflow
 3. activation_overflow
 4. nan_weight_error
- **NaN в весах attention** — nan_weight_error
 - **Overflow активации** — activation_overflow
 - **Подсчет attention к паддингу без маски** — invalid_mask_error
 - **Underflow края softmax** — softmax_underflow

🔮 Шаг 09:

Миссия 9

Сопоставьте модули ядра Ollama с задачами, которые они решают:

Подсказка: вспомните, какие модули участвуют в attention pipeline.

Модули:

Ответ:

- Token embedding
- Self-attention layer
- Layer normalization
- Feed-forward network

Задачи:

1. Суммировать вклад из всех токенов
 2. Преобразовать эмбединги токенов в контекстное пространство
 3. Нормализация пред активацией для стабильности
 4. Применение нелинейного трансформации и расширение возможностей модели
- **Token embedding** — Преобразовать эмбединги токенов в контекстное пространство
 - **Self-attention layer** — Суммировать вклад из всех токенов
 - **Layer normalization** — Нормализация пред активацией для стабильности
 - **Feed-forward network** — Применение нелинейного трансформации и расширение возможностей модели

🔮 Шаг 10:

Миссия 10

У инженера Ollama возникла гипотеза проблемы: attention не распределяет вес по всем токенам. Отсортируйте шаги диагностики от начала до конца:

Подсказка: сначала сбор данных, затем проверка масок, затем визуализация, затем коррекция.

1. Визуализировать матрицы attention
2. Запустить инструмент логов для фильтрации токенов с нулевым весом
3. Проверить mask padding и causal mask
4. Сравнить распределения весов между head'ами

Ответ:

- Запустить инструмент логов для фильтрации токенов с нулевым весом
- Проверить mask padding и causal mask
- Визуализировать матрицы attention
- Сравнить распределения весов между head'ами

🔮 Шаг 11:

Миссия 11

Вам нужно восстановить порядок этапов построения ВРЕ-словаря при обучении ядра Ollama. Расположите в правильной последовательности:

Подсказка: сначала инициализация, потом статистика, потом слияния, потом финализация.

1. Подсчет частот всех соседних пар символов
2. Инициализация токенов как отдельных символов / байтов

3. Итерации объединения наиболее частых пар до порога
4. Сохранение итогового словаря

Ответ:

- Инициализация токенов как отдельных символов / байтов
- Подсчет частот всех соседних пар символов
- Итерации объединения наиболее частых пар до порога
- Сохранение итогового словаря

 Шаг 12:

Миссия 12

Когда ядро уже загружается, вы хотите отладить последовательность операций attention. Упорядочите этапы внутренне:

Подсказка: сначала формируются Q/K/V, затем внимание, затем суммирование, затем переход к next layer.

1. Применение linear к входам для Q/K/V
2. Softmax нормализация весов внимания
Умножение весов на значение V и суммирование
3. Переход к feed-forward слою

Ответ:

- Применение linear к входам для Q/K/V
- Softmax нормализация весов внимания
- Умножение весов на значение V и суммирование
- Переход к feed-forward слою

 Шаг 13:

Миссия 13

Вам нужно воспроизвести внутренние шаги при mask'ировании в attention ядре Ollama. Упорядочите:

Подсказка: сначала строится базовая маска, потом применяется causal маска и padding, потом слей комбинированная маска.

1. Составление маски padding (где токен = padding)
2. Составление causal маски (токен не может смотреть вперед)
3. Объединение масок (логическое И / сложение с минус бесконечностью)
4. Наложение маски на logits before softmax

Ответ:

- Составление маски padding (где токен = padding)
- Составление causal маски (токен не может смотреть вперед)
- Объединение масок (логическое И / сложение с минус бесконечностью)
- Наложение маски на logits before softmax

 Шаг 14:

Миссия 14

При интеграции ядра в систему вы хотите логировать стадии attention. Расположите этапы логирования в хронологическом порядке:

Подсказка: сначала вход, затем QKV, затем веса, затем итоговый контекст.

1. Логировать итоговый контекст (output)
2. Логировать веса attention после softmax
3. Логировать Q, K, V матрицы
4. Логировать входные эмбединги токенов

Ответ:

- Логировать входные эмбединги токенов
- Логировать Q, K, V матрицы
- Логировать веса attention после softmax
- Логировать итоговый контекст (output)

Шаг 15:

Миссия 15

Вы отлаживаете multi-layer attention: нужно перестроить порядок слоёв. От простого к более абстрактному:

Подсказка: lower layer действует на токены, верхние — на контексты.

1. Последний слой attention
2. Первый слой attention
3. Средний слой attention
4. Вывод результирующего эмбединга

Ответ:

- Первый слой attention
- Средний слой attention
- Последний слой attention
- Вывод результирующего эмбединга

Шаг 16:

Миссия 16

Вы готовитесь к стресс-тесту ядра с длинным вводом. Упорядочите проверочные шаги:

Подсказка: сначала на коротких входах, потом средних, потом масштабирование, потом анализ.

1. Запуск ядра на очень длинных последовательностях
2. Сравнение поведения с короткими вводами
3. Постепенное увеличение длины вводов
4. Анализ производительности и ошибок

Ответ:

- Сравнение поведения с короткими вводами
- Постепенное увеличение длины вводов
- Запуск ядра на очень длинных последовательностях
- Анализ производительности и ошибок

Шаг 17:

Миссия 17

Финальное задание: порядок этапов раскрытия attention ядра Ollama от входа до выхода:

Подсказка: подумайте весь pipeline: токенизация, embedding, attention, нормализация, выход.

1. Обработка токенов ядром attention (QKV → вес → контекст)
2. Нормализация и residual connection
3. Токенизация и embedding
4. Финальный вывод / преобразование на выход

Ответ:

- Токенизация и embedding
- Обработка токенов ядром attention (QKV → вес → контекст)
- Нормализация и residual connection
- Финальный вывод / преобразование на выход

Урок 5 - 🕒 Квест найди узкое место в цепочке генерации

Шаг 01:

Миссия 1

Вы — инженер ядра Ollama. На этапе токенизации вы замечаете, что один из этапов сильно отстаёт по времени. Что вы проверите в первую очередь?

Подсказка: первым делом стоит удостовериться, что узкий этап — не I/O или ожидание внешнего ресурса.

Ответ:

- **Профилирование времени выполнения токенайзера**
- Увеличение размера батча сразу
- Изменение архитектуры модели без анализа

Шаг 02:

Миссия 2

Вы внедрили кэш на уровне сегментации, но генерация всё ещё медленна на большом тексте. Что разумнее попробовать дальше?

Подсказка: подумайте о слоях выше — внимание, декодирование.

Ответ:

- **Ограничить окно внимания (attention) для дальних токенов**
- Увеличить слой эмбединга
- Удалить кэш полностью
- Переобучить модель на новых данных

Шаг 03:

Миссия 3

Вы анализируете профиль и видите, что вычисления трансформера занимают 90 % времени, а ввод-вывод — лишь 5 %. Что это говорит?

Подсказка: узкое место — не в диске или сети.

Ответ:

- **Узкое место** — в вычислениях трансформера
- **Узкое место** — в загрузке данных
- **Узкое место** — в кэше

🔮 Шаг 04:

Миссия 4

При эксперименте вы уменьшили размер hidden dimension, и система стала быстрее, но качество текста страдает. Что выбрать дальше?

Подсказка: компромисс производительность / качество — не всегда линейный.

Ответ:

- **Применить смешанное прецизионное вычисление (mixed precision)**
- Снизить learning rate
- Отключить dropout
- Увеличить batch size без анализа

🔮 Шаг 05:

Миссия 5

Вы заметили всплески памяти на этапе размещения attention matrices. Как вы будете действовать первым шагом?

Подсказка: попробуйте изменить стратегию распределения памяти.

Ответ:

- **Активировать шаг подсвертки (gradient checkpointing)**
- Уменьшить batch size сразу
- Увеличить hidden dimension

🔮 Шаг 06:

Миссия 6

Сопоставьте тип узкого места и признак, который вы могли бы наблюдать при профилировании:

Тип узкого места:

Ответ:

- Вычислительное (compute-bound)
- Память (memory-bound)
- Ввод-вывод (I/O-bound)
- Синхронизация (synchronization-bound)

Признаки:

1. Сильный рост дерева вызовов на операции GPU
2. Частые page fault, swap, нехватка памяти
3. Ожидание операций с диском или сетью
4. Блокировки между потоками, ожидание мьютексов

🔮 Подсказка:

Тип узкого места часто виден по характеру задержек и по тому, где ждёт система.

- **Вычислительное (compute-bound)** — Сильный рост дерева вызовов на операции GPU
- **Память (memory-bound)** — Частые page fault, swap, нехватка памяти
- **Ввод-вывод (I/O-bound)** — Ожидание операций с диском или сетью
- **Синхронизация (synchronization-bound)** — Блокировки между потоками, ожидание мьютексов

🔗 Шаг 07:

Миссия 7

Разработчики команды предложили гипотезы узких мест. Сопоставьте гипотезу и способ проверить её:

Гипотезы:

Ответ:

- Внимание распространяется слишком далеко по всему контексту
- Кэш сегментов работает плохо
- Граф вычислений (computation graph) имеет дублирующиеся узлы
- Синхронизация блокирует потоки при батче

Способы проверки:

1. Отключить кэш и замерить время
2. Срезать attention окно и замерить производительность
3. Профилировать граф и искать дублирующие операции
4. Измерить задержку ожидания mutex/lock в профиле

🔗 Подсказка:

Подумайте, как именно можно «отключить» или «ограничить» функцию, чтобы проверить гипотезу.

- **Внимание распространяется слишком далеко по всему контексту** — Срезать attention окно и замерить производительность
- **Кэш сегментов работает плохо** — Отключить кэш и замерить время
- **Граф вычислений (computation graph) имеет дублирующиеся узлы** — Профилировать граф и искать дублирующие операции
- **Синхронизация блокирует потоки при батче** — Измерить задержку ожидания mutex/lock в профиле

🔗 Шаг 08:

Миссия 8

Сопоставьте метрику профиля и то, что она показывает:

Метрики:

Ответ:

- GPU utilization
- Memory bandwidth
- Disk I/O latency
- Lock wait time

Значения/симптомы:

1. GPU загружен < 10 %
2. Потребность в чтении/записи к диску занимает значительное время
3. Пиковое потребление памяти превышает пропускную способность памяти
4. Потоки простаивают, ожидая захвата блокировок

🔗 Подсказка: Métrica часто прямо говорит, где система простаивает.

- **GPU utilization** — GPU загружен < 10 %

- **Memory bandwidth** — Пиковое потребление памяти превышает пропускную способность памяти
- **Disk I/O latency** — Потребность в чтении/записи к диску занимает значительное время
- **Lock wait time** — Потоки простаивают, ожидая захвата блокировок

🧙 Шаг 09:

Миссия 9

Расположите шаги поиска узкого места в цепочке генерации:

1. Провести профилирование всех этапов
2. Построить гипотезы о возможных узких местах
3. Проверить гипотезы экспериментально
4. Усилить узкий компонент (оптимизация, замена) и повторить цикл

🧙 Подсказка:

Сначала измеряем, потом думаем, затем экспериментируем и улучшаем.

Ответ:

- Провести профилирование всех этапов
- Построить гипотезы о возможных узких местах
- Проверить гипотезы экспериментально
- Усилить узкий компонент (оптимизация, замена) и повторить цикл

🧙 Шаг 10:

Миссия 10

У вас есть четыре версии цепочки генерации: A, B, C, D. Вы получили их замеры от профиля (время в мс): A=1200, B=950, C=1100, D=1000. Расположите их от самой быстрой к самой медленной.

🧙 Подсказка:

Меньшее время = лучше.

Ответ:

- B
- D
- C
- A

🧙 Шаг 11:

Миссия 11

У вас есть четыре компонента цепочки: токенизация, эмбединг, внимание, декодер. Расположите их по времени, от того, что обычно занимает наибольшее время до того, что наименьшее.

🧙 Подсказка:

Общее правило: внимание часто главенствует.

Ответ:

- Внимание
- Декодер
- Эмбединг
- Токенизация

🔗 Шаг 12:

Миссия 12

Протокол улучшения узкой части:

1. Изменить код оптимизированно
2. Запустить тесты на корректность
3. Повторно профилировать после изменений
4. Вернуться к шагу гипотезирования, если осталось узкое место

🔗 Подсказка:

Это итеративный цикл.

Ответ:

- Изменить код оптимизированно
- Запустить тесты на корректность
- Повторно профилировать после изменений
- Вернуться к шагу гипотезирования, если осталось узкое место

🔗 Шаг 13:

Миссия 13

После оптимизаций вы заметили, что производительность упала для коротких запросов, хотя для длинных выросла. Что могло пойти не так?

Подсказка: возможно, надстройки или кэш влияют по-разному на длину.

Ответ:

- **Оверхед кэша или переключения режима стал заметен на коротких запросах**
- Модель потеряла способность к генерации длинных текстов
- Декодер теперь обрёл на работу только с длинными контекстами

🔗 Шаг 14:

Миссия 14

Вы подозреваете, что у вас два узких участка: внимание и декодер. Что вы сделаете?

Подсказка: возможно, стоит разделить оптимизацию по областям.

Ответ:

- **Оптимизировать внимание и декодер поочерёдно, тестируя каждый**
- Оптимизировать только более медленный, игнорируя второй
- Сразу переписать всю цепочку под новую архитектуру
- Отключить внимание на время эксперимента

🔗 Шаг 15:

Миссия 15

Вы заметили, что при большом количестве параллельных запросов возрастает задержка. Сопоставьте причину и симптом:

Причины:

Ответ:

- Контенция за памятью GPU
- Синхронизация при batching

- Загрузка диска при swap
- Очереди запросов на входе

Симптомы:

1. Увеличение latency при удержании GPU usage стабильно высоким
2. Пики ожидания блокировок, увеличенные lock wait time
3. Высокая задержка дисковых операций
4. Заметные очереди на входном порту, рост time_in_queue

🔮 **Подсказка: Посмотрите, на каком ресурсе система начинает «задышаться» первой.**

- **Контенция за памятью GPU** — Увеличение latency при удержании GPU usage стабильно высоким
- **Синхронизация при batching** — Пики ожидания блокировок, увеличенные lock wait time
- **Загрузка диска при swap** — Высокая задержка дисковых операций
- **Очереди запросов на входе** — Заметные очереди на входном порту, рост time_in_queue

🔮 **Шаг 16:**

Миссия 16

Команда предлагает три варианта оптимизации: квантование, distillation, sparsity. Сопоставьте метод и потенциальный риск снижения качества:

Методы:

Ответ:

- Квантование
- Distillation (дистилляция)
- Разреженность (sparsity)

Риски:

1. Шумы и артефакты в тексте
2. Упущение редких конструкций из-за усреднения знаний
3. Неоптимальное размещение структуры с нулевыми весами

🔮 **Подсказка: Каждый метод имеет свои слабые стороны — подумайте, что именно он теряет.**

- **Квантование** — Шумы и артефакты в тексте
- **Distillation (дистилляция)** — Упущение редких конструкций из-за усреднения знаний
- **Разреженность (sparsity)** — Неоптимальное размещение структуры с нулевыми весами

🔮 **Шаг 17:**

Миссия 17

Финальный челлендж: у вас есть четыре гипотезы узких мест, и вы хотите протестировать все. Какие шаги вы выполните в правильном порядке?

1. Сформулировать гипотезы
2. Написать бенчмарки/тесты для каждой гипотезы
3. Запустить измерения и профилирование
4. Сравнить результаты и выбрать узкие звенья
5. Оптимизировать выбранные узкие звенья
6. Провести повторную проверку цепочки

💡 **Подсказка:** Это полный цикл от гипотезы до проверки и оптимизации.

Ответ:

- Сформулировать гипотезы
- Написать бенчмарки/тесты для каждой гипотезы
- Запустить измерения и профилирование
- Сравнить результаты и выбрать узкие звенья
- Оптимизировать выбранные узкие звенья
- Провести повторную проверку цепочки

📖 **Модуль 2 - ⚙️ Миссия 2. Архитектор Ollama Engine**

📖 **Урок 1 - 🏠 Компоненты Ollama daemon, backend, CLI**

💡 **Шаг 01:**

Миссия 1

Вы запускаете daemon Ollama впервые. Что первым делом проверит CLI?

Подсказка: CLI часто проверяет связь с daemon через сокет или порт.

Ответ:

- **Подключение к daemon через Unix-сокет или TCP**
- Запуск backend модели напрямую
- Обновление CLI до последней версии

💡 **Шаг 02:**

Миссия 2

CLI отправляет запрос daemon: "model.load". Но backend жалуется «модель не найдена». Какая часть архитектуры отвечает за хранение модели?

Подсказка: backend обычно управляет ресурсами и модельными артефактами.

Ответ:

- **backend**
- CLI
- daemon

💡 **Шаг 03:**

Миссия 3

При попытке остановить daemon через CLI команда "daemon.stop" висит долго без ответа. Где может быть блокировка?

Подсказка: daemon может ждать завершения фоновых задач или закрытия каналов.

Ответ:

- **Ожидание завершения фоновых задач в daemon**
- Зависание CLI-интерфейса
- Ошибка в backend ресурсах
- Сбой в модели

💡 **Шаг 04:**

Миссия 4

CLI запрашивает статус у daemon, но ответ занимает много времени. Что можно предположить?

Подсказка: возможно, daemon занят тяжёлой задачей или перегружен.

Ответ:

- **daemon загружен обработкой запроса модели**
- CLI тормозит сам по себе
- backend выключен
- сеть не работает

Шаг 05:

Миссия 5

Вы обнаружили, что daemon и backend запущены, но CLI выдаёт ошибку «permission denied». Где можно проверить привилегии?

Подсказка: сокет файловой системы или права доступа к IPC.

Ответ:

- **Права на Unix-сокет daemon**
- Права на чтение модели
- Права на запуск backend

Шаг 06:

Миссия 6

Сопоставьте компонент Ollama и его роль:

Компоненты:

Ответ:

- daemon
- backend
- CLI
- IPC / коммуникационный слой

Роли:

1. Запускает сервер, управляет жизненным циклом
2. Обработывает запросы модели, выделяет ресурсы
3. Интерфейс команд пользователя к системе
4. Обеспечивает обмен сообщениями между CLI и daemon/backend

 **Подсказка: Подумайте: кто слушает запросы и кто их исполняет.**

- **daemon** — Запускает сервер, управляет жизненным циклом
- **backend** — Обработывает запросы модели, выделяет ресурсы
- **CLI** — Интерфейс команд пользователя к системе
- **IPC / коммуникационный слой** — Обеспечивает обмен сообщениями между CLI и daemon/backend

Шаг 07:

Миссия 7

При разработке новой функции вы должны решить, куда логировать: в daemon или в backend. Сопоставьте сценарий и предпочтительный компонент для логирования:

Сценарии:

Ответ:

- Ошибка старта daemon
- Ошибки модели при генерации
- Переход CLI в режим interactive
- События IPC-сообщений

Компоненты логирования:

1. daemon
2. backend
3. CLI
4. daemon / IPC

🔗 **Подсказка: Логи лучше писать там, где возникает контекст события.**

- **Ошибка старта daemon** — daemon
- **Ошибки модели при генерации** — backend
- **Переход CLI в режим interactive** — CLI
- **События IPC-сообщений** — daemon / IPC

🔗 **Шаг 08:**

Миссия 8

При нагрузочном тесте вы фиксируете задержки. Сопоставьте метрику и как её можно измерить на уровне компонента:

Метрики:

Ответ:

- latency (задержка)
- throughput (пропускная способность)
- ошибки RPC
- время отклика IPC

Подсчёт / метод:

1. **разница времени reply** — запрос
2. количество успешных операций в секунду
3. процент неудачных вызовов
4. усреднённое время обмена между CLI и daemon

🔗 **Подсказка: Метрики обычно измеряют время, количество и ошибки.**

- **latency (задержка)** — разница времени reply — запрос
- **throughput (пропускная способность)** — количество успешных операций в секунду
- **ошибки RPC** — процент неудачных вызовов
- **время отклика IPC** — усреднённое время обмена между CLI и daemon

🔗 **Шаг 09:**

Миссия 9

У вас есть стадии запуска Ollama Engine: конфигурирование, инициализация backend, запуск daemon, ожидание запроса CLI. В каком порядке они выполняются?

1. конфигурирование (чтение конфигов)

2. запуск daemon
3. инициализация backend (модели, ресурсы)
4. ожидание запроса CLI

💡 **Подсказка: Сначала настраиваем, потом запускаем и поднимаем службы.**

Ответ:

- конфигурирование (чтение конфигов)
- запуск daemon
- инициализация backend (модели, ресурсы)
- ожидание запроса CLI

💡 **Шаг 10:**

Миссия 10

У вас есть четыре события: CLI → request, daemon → dispatch, backend → compute, backend → respond. Расположите их по хронологии:

1. CLI → request
2. daemon → dispatch
3. backend → compute
4. backend → respond

💡 **Подсказка: Подумайте, как запрос путешествует через слои.**

Ответ:

- CLI → request
- daemon → dispatch
- backend → compute
- backend → respond

💡 **Шаг 11:**

Миссия 11

Во время shutdown подозреваем порядок закрытия: backend, daemon, IPC, потом CLI. Правильный порядок?

1. завершение обработки запросов backend
2. закрытие IPC / коммуникационных каналов
3. остановка daemon сервера
4. CLI завершает сессию

💡 **Подсказка: Закрываем сначала рабочую часть, затем коммуникации и процессы.**

Ответ:

- завершение обработки запросов backend
- закрытие IPC / коммуникационных каналов
- остановка daemon сервера
- CLI завершает сессию

💡 **Шаг 12:**

Миссия 12

CLI выдаёт «RPC timeout» при обращении к daemon. Где начать диагностику?

Подсказка: проверьте, жив ли daemon и не перегружен ли он.

Ответ:

- **Проверить, запущен ли daemon и отвечает ли он**
- Переустановить CLI
- Поменять модель на другую

🔮 Шаг 13:

Миссия 13

backend падает с ошибкой OOM (Out of Memory) при загрузке модели, хотя daemon работает. Что вы измените?

Подсказка: backend управляет ресурсами модели.

Ответ:

- **Ограничить память, выделенную backend (memory cap)**
- Остановить daemon
- Обновить CLI
- Удалить конфигурацию

🔮 Шаг 14:

Миссия 14

После добавления нового backend типа (например, for GPU), CLI-daemon-backend цепочка не работает. Что вероятнее всего забыли?

Подсказка: вероятно, не обновили маршрутизацию запросов daemon.

Ответ:

- **Не добавлена регистрация этого backend в daemon**
- CLI не установлен
- Конфигурация модели отсутствует

🔮 Шаг 15:

Миссия 15

Вы проектируете расширяемость Ollama: сопоставьте требование и архитектурное решение:

Требования:

Ответ:

- Поддержка множества backend (CPU, GPU, TPU)
- Изоляция ресурсов между пользователями
- Гибкость расширения CLI командами

Решения:

1. Плагиновая система backend
2. Namespace или контейнеризация ресурсов per user
3. Модульная архитектура команд CLI с регистрацией команд

🔮 **Подсказка: Как правило, каждому требованию соответствует своя архитектурная абстракция.**

- **Поддержка множества backend (CPU, GPU, TPU)** — Плагиновая система backend
- **Изоляция ресурсов между пользователями** — Namespace или контейнеризация ресурсов per user

- **Гибкость расширения CLI командами** — Модульная архитектура команд CLI с регистрацией команд

🔗 Шаг 16:

Миссия 16

Вы рассматриваете варианты коммуникации между CLI и daemon: HTTP, gRPC, Unix-сокеты. Сопоставьте способ и его преимущество:

Способы:

Ответ:

- HTTP / REST
- gRPC
- Unix-сокеты (local IPC)

Преимущества:

1. легковесность, минимальная обвязка
2. низкая латентность, двоичный протокол, стриминг
3. максимально быстрая локальная коммуникация без сети

🔗 **Подсказка: Подумайте про накладные расходы формата и транспорт.**

- **HTTP / REST** — легковесность, минимальная обвязка
- **gRPC** — низкая латентность, двоичный протокол, стриминг
- **Unix-сокеты (local IPC)** — максимально быстрая локальная коммуникация без сети

🔗 Шаг 17:

Миссия 17

Итеративный цикл проектирования архитектуры Ollama Engine:

1. Сбор требований пользователей и сценариев CLI
2. Проектирование API и коммуникаций между компонентами
3. Реализация daemon, backend, IPC согласно проекту
4. Тестирование взаимодействия (CLI ↔ daemon ↔ backend)
5. Нагрузочное тестирование и обнаружение узких мест
6. Повторное проектирование и улучшение архитектуры

🔗 **Подсказка: Архитектура — это не одноразовое решение, а цикл улучшений.**

Ответ:

- Сбор требований пользователей и сценариев CLI
- Проектирование API и коммуникаций между компонентами
- Реализация daemon, backend, IPC согласно проекту
- Тестирование взаимодействия (CLI ↔ daemon ↔ backend)
- Нагрузочное тестирование и обнаружение узких мест
- Повторное проектирование и улучшение архитектуры

📖 Урок 2 - 📄 Вскрываем Modelfile сердце модели

🔗 Шаг 01:

Миссия 1

Вы открываете ****Modelfile**** и видите секцию ``metadata``. Что вы смотрите сначала?

Подсказка: metadata обычно содержит версию и идентификатор модели.

Ответ:

- **Поле версии (version) модели**
- Список слоёв сети
- Граф вычислений

Шаг 02:

Миссия 2

В Modelfile вы находите секцию ``layers`` с массивом описаний. Какой атрибут слоя поможет вам понять форму параметров?

Подсказка: слои обычно имеют ``shape`` или ``dimensions``.

Ответ:

- **shape / dimensions**
- color
- visibility
- alias

Шаг 03:

Миссия 3

При попытке загрузить модель из Modelfile, backend жалуется на неизвестный атрибут ``foo``. Что делать первым шагом?

Подсказка: возможно, Modelfile содержит устаревшее поле или ошибку в имени.

Ответ:

- **Проверить документацию версии Modelfile для поддерживаемых атрибутов**
- Игнорировать атрибут foo
- Удалить весь слой с foo

Шаг 04:

Миссия 4

Вы видите в Modelfile секцию ``optimizer_config``. В CLI вы хотите вывести её содержимое. Какой формат предпочтительнее для чтения?

Подсказка: JSON или YAML читаются легче, чем бинарные форматы.

Ответ:

- **Читать как JSON / YAML**
- Интерпретировать как двоичный blob
- Выводить хэш секции
- Игнорировать optimizer_config

Шаг 05:

Миссия 5

В Modelfile вы видите ``dependencies`` — список других файлов. Что важно перед загрузкой модели?

Подсказка: убедитесь, что все зависимости доступны.

Ответ:

- Проверить наличие всех файлов из **dependencies**
- Пропустить **dependencies**
- Удалить раздел **dependencies**
- Загрузить модель без зависимостей

🔮 Шаг 06:

Миссия 6

Сопоставьте атрибут **Modelfile** и его смысл:

Атрибуты:

Ответ:

- **version**
- **layers**
- **dependencies**
- **metadata**

Смыслы:

1. Информация о модели (имя, автор и др.)
2. Список файлов, от которых зависит модель
3. Описание архитектурных слоёв и их параметров
4. Версия формата **Modelfile**

🔮 **Подсказка: Посмотрите, как Modelfile организован — часть отвечает за слои, часть — за справочные данные.**

- **version** — Версия формата **Modelfile**
- **layers** — Описание архитектурных слоёв и их параметров
- **dependencies** — Список файлов, от которых зависит модель
- **metadata** — Информация о модели (имя, автор и др.)

🔮 Шаг 07:

Миссия 7

Вы встречаете атрибут ``weight_init`` в слое. Сопоставьте значение и его эффект:

Значения:

Ответ:

- **zeros**
- **random_normal**
- **xavier**
- **he_uniform**

Эффекты:

1. **Все веса нулевые** — нет обучения
2. Инициализация из нормального распределения вокруг 0
3. Инициализация по формуле Xavier
4. Инициализация по He uniform диапазону

🔮 **Подсказка: Весовая инициализация часто влияет на старт обучения или стабильность.**

- **zeros** — Все веса нулевые — нет обучения

- **random_normal** — Инициализация из нормального распределения вокруг 0
- **xavier** — Инициализация по формуле Xavier
- **he_uniform** — Инициализация по He uniform диапазону

🔮 Шаг 08:

Миссия 8

В Modelfile есть поля ``dropout_rate``, ``activation``, ``padding``. Сопоставьте их с предназначением:

Поля:

Ответ:

- `dropout_rate`
- `activation`
- `padding`

Назначения:

1. Указывает вероятность отключения нейронов во время обучения
2. Функция активации слоя (ReLU, tanh и др.)
3. Как обрабатываются границы при свёртке (valid, same и др.)

🔮 **Подсказка: Каждое поле прямо говорит о своей роли в модели.**

- **dropout_rate** — Указывает вероятность отключения нейронов во время обучения
- **activation** — Функция активации слоя (ReLU, tanh и др.)
- **padding** — Как обрабатываются границы при свёртке (valid, same и др.)

🔮 Шаг 09:

Миссия 9

Вы видите в Modelfile записи ``uint8``, ``float16``, ``float32``. Сопоставьте тип и его преимущество/ограничение:

Типы:

Ответ:

- `uint8`
- `float16`
- `float32`

Преимущества / ограничения:

1. Минимальный размер, но низкая точность
2. Баланс между размером и точностью
3. Высокая точность, но больше памяти

🔮 **Подсказка: Подумайте, как разные форматы влияют на размер и качество.**

- **uint8** — Минимальный размер, но низкая точность
- **float16** — Баланс между размером и точностью
- **float32** — Высокая точность, но больше памяти

🔮 Шаг 10:

Миссия 10

В Modelfile слои перечислены: Embedding, Transformer, Output, Norm. Каков правильный порядок прохождения данных через эти слои?

1. Embedding
2. Transformer
3. Norm
4. Output

🔗 **Подсказка: Сначала представление токенов, затем трансформации, нормализация и вывод.**

Ответ:

- Embedding
- Transformer
- Norm
- Output

🔗 Шаг 11:

Миссия 11

Порядок этапов чтения и обработки Modelfile:

1. Открыть файл и прочитать байты
2. Десериализовать структуру Modelfile (JSON / бинарно)
3. Валидация схемы и версий
4. Загрузка зависимостей и ресурсов
5. Инициализация слоёв и параметров

🔗 **Подсказка: Нельзя инициализировать слои до проверки валидности и загрузки ресурсов.**

Ответ:

- Открыть файл и прочитать байты
- Десериализовать структуру Modelfile (JSON / бинарно)
- Валидация схемы и версий
- Загрузка зависимостей и ресурсов
- Инициализация слоёв и параметров

🔗 Шаг 12:

Миссия 12

Вы анализируете список слоёв: Linear, Dropout, Activation, ResidualAdd. В каком порядке они обычно располагаются?

1. Linear
2. Activation
3. Dropout
4. ResidualAdd

🔗 **Подсказка: Сначала линейная трансформация, потом активация, затем возможно дропаут, и добавление остатка.**

Ответ:

- Linear
- Activation

- Dropout
- ResidualAdd

🔗 Шаг 13:

Миссия 13

В Modelfile вы обнаружили поле ``version = 2``, но документация описывает только версию 1. Что вы сделаете?

Подсказка: возможно, нужно обработать старую и новую версии.

Ответ:

- **Проверить миграции или схемы для версии 2**
- Игнорировать версию и загрузить как v1
- Отказаться от загрузки модели

🔗 Шаг 14:

Миссия 14

Вы хотите добавить поле ``notes`` в Modelfile для человеческой информации, как комментарий. Как лучше сделать это без нарушения совместимости?

Подсказка: добавьте optional поле, которое не влияет на другие поля.

Ответ:

- **Ввести поле ``notes`` как опциональное необязательное поле**
- Переписать весь Modelfile формат
- Удалить другое поле, чтобы вместить ``notes``

🔗 Шаг 15:

Миссия 15

Вы хотите оптимизировать Modelfile размер. Сопоставьте приём и потенциальный эффект:

Приёмы:

Ответ:

- Удалить камеры (unused layers)
- Использовать float16 вместо float32
- Сжать Modelfile gzip'ом

Эффекты:

1. Снижение объёма файла за счёт удаления неиспользуемых параметров
2. Снижение памяти на хранение весов вдвое (примерно)
3. Дополнительная компрессия, но требует разжатия при загрузке

🔗 **Подсказка: Каждый приём имеет свои плюсы и минусы для загрузки и использования.**

- **Удалить камеры (unused layers)** — Снижение объёма файла за счёт удаления неиспользуемых параметров
- **Использовать float16 вместо float32** — Снижение памяти на хранение весов вдвое (примерно)
- **Сжать Modelfile gzip'ом** — Дополнительная компрессия, но требует разжатия при загрузке

🔮 Шаг 16:

Миссия 16

В Modelfile встречаются поля ``initializer``, ``regularizer``, ``constraint``. Сопоставьте и назначение:
Поля:

Ответ:

- `initializer`
- `regularizer`
- `constraint`

Назначения:

1. Определяет начальную инициализацию весов
2. Применяет штрафы к весам (L1, L2 и др.)
3. Ограничивает значения весов (например, максимальное значение)

🔮 Подсказка: Эти три поля часто сопровождают слои в библиотеке нейросетей.

- **`initializer`** — Определяет начальную инициализацию весов
- **`regularizer`** — Применяет штрафы к весам (L1, L2 и др.)
- **`constraint`** — Ограничивает значения весов (например, максимальное значение)

🔮 Шаг 17:

Миссия 17

Полный порядок этапов работы с Modelfile в runtime:

1. Считать raw данные файла
2. Десериализовать в структуру данных (JSON / бинарно)
3. Проверить schema / версию и валидность
4. Разрешить зависимости (загрузить файлы)
5. Инициализировать слои и веса
6. Вернуть объект модели в backend

🔮 Подсказка: Нельзя инициализировать слои до того, как зависимости будут доступны и схема проверена.

Ответ:

- Считать raw данные файла
- Десериализовать в структуру данных (JSON / бинарно)
- Проверить schema / версию и валидность
- Разрешить зависимости (загрузить файлы)
- Инициализировать слои и веса
- Вернуть объект модели в backend

📖 Урок 3 - 🔁 Поточковая генерация токенов под капотом

🔮 Шаг 01:

Миссия 1

Вы запускаете Ollama Engine и видите лог: «начало потоковой генерации токенов». Что происходит первым?

Подсказка: Двигатель сначала инициализирует контекст, а не сразу отправляет токены.

Ответ:

- **Инициализация внутреннего состояния генератора**
- Непосредственная передача первого токена клиенту
- Закрытие соединения с клиентом

 **Шаг 02:**

Миссия 2

Во время работы потоковой генерации токенов вы хотите контролировать скорость выдачи. Что вы примените?

Подсказка: Часто используют back-pressure или приостановку эмиссии.

Ответ:

- **Механизм back-pressure / пауз между токенами**
- Отправка всех токенов сразу
- Отключение потока

 **Шаг 03:**

Миссия 3

Вы запускаете эксперимент: потоковая генерация выходит медленнее ожидаемого. Что вы проверите в первую очередь?

Подсказка: Узкое место может быть либо в декодере, либо в I/O.

Ответ:

- **Задержку декодера / модельного шага**
- Скорость загрузки исходного текста
- Количество токенов в словаре

 **Шаг 04:**

Миссия 4

Во время отладки вы видите, что потоковая генерация останавливается на половине. Какова вероятная причина?

Подсказка: Может быть превышен таймаут или клиент больше не читает.

Ответ:

- **Клиент перестал читать / таймаут на стороне протокола**
- Модель закончила выдачу
- Не хватило памяти

 **Шаг 05:**

Миссия 5

Сопоставьте термин и его роль в поточной генерации Ollama Engine:

Ответ:

- Буфер токенов
- Back-pressure
- Контекст внимания
- Пул потоков

1. Механизм замедления генерации при перегрузке

2. Очередь готовых токенов к отправке клиенту
3. Хранение ранее сгенерированных токенов для исходного внимания
4. Параллельные потоки декодирования запросов

Подсказка: Представьте, что система должна управлять скоростью, памятью, конкуренцией и состоянием внимания одновременно.

- **Буфер токенов** — Очередь готовых токенов к отправке клиенту
- **Back-pressure** — Механизм замедления генерации при перегрузке
- **Контекст внимания** — Хранение ранее сгенерированных токенов для исходного внимания
- **Пул потоков** — Параллельные потоки декодирования запросов

Шаг 06:

Миссия 6

Сопоставьте стадию в потоке и что она делает:

Ответ:

- Token sampling
 - Scoring / логит-шкалирование
 - Нормализация распределения
 - Отправка клиенту
1. Преобразование логитов в вероятности
 2. Выбор вероятного токена
 3. Математическая стабилизация распределения
 4. Токен передаётся через протокол клиента

Подсказка: Логиты → вероятности → отбор → передача.

- **Token sampling** — Выбор вероятного токена
- **Scoring / логит-шкалирование** — Математическая стабилизация распределения
- **Нормализация распределения** — Преобразование логитов в вероятности
- **Отправка клиенту** — Токен передаётся через протокол клиента

Шаг 07:

Миссия 7

Сопоставьте параметры генерации с эффектом в потоке:

Ответ:

- temperature
 - top_k
 - top_p
 - repetition_penalty
1. Ограничивает выбор ближайшими кандидатами
 2. Вероятностно отсекает редкие варианты по кумулятивной вероятности
 3. Умножает логиты повторяющихся токенов
 4. Размывает / усиливает распределение

Подсказка: Знайте standard формулы — temperature влияет на «жидкость», top_k и top_p — это срезы, repetition_penalty — штрафы.

- **temperature** — Размывает / усиливает распределение

- **top_k** — Ограничивает выбор ближайшими кандидатами
- **top_p** — Вероятностно отсекает редкие варианты по кумулятивной вероятности
- **repetition_penalty** — Умножает логиты повторяющихся токенов

💡 Шаг 08:

Миссия 8

Сопоставьте механизм ограничения задержки с задачей:

Ответ:

- Rate limiter
- Windowed buffer
- Client ack / feedback
- Выделенный токен-чекпоинт

1. Ограничивает число токенов в единицу времени
2. Держит фиксированное окно токенов перед отправкой
3. Клиент подтверждает приём и влияет на скорость
4. Сохраняет положение, с которого можно возобновить

Подсказка: Представьте, как сервер избегает «одаривание» клиента слишком быстро.

- **Rate limiter** — Ограничивает число токенов в единицу времени
- **Windowed buffer** — Держит фиксированное окно токенов перед отправкой
- **Client ack / feedback** — Клиент подтверждает приём и влияет на скорость
- **Выделенный токен-чекпоинт** — Сохраняет положение, с которого можно возобновить

💡 Шаг 09:

Миссия 9

Упорядочьте этапы обработки одного токена в Ollama Engine (от внутреннего логита до выдачи клиенту):

1. Scoring / логит → поправки
2. Нормализация распределения
3. Выбор токена (sampling)
4. Отправка клиенту

Подсказка: Логиты → вероятности → отбор → передача.

Ответ:

- Scoring / логит → поправки
- Нормализация распределения
- Выбор токена (sampling)
- Отправка клиенту

💡 Шаг 10:

Миссия 10

Упорядочьте шаги реакции на перегрузку потока (flow overload):

1. Замедление эмиссии токенов
2. Увеличение буферизации
3. Обратная связь клиенту
4. Откат или сброс лишних токенов

Подсказка: Как правило, система сначала пытается мягко справиться, затем – жёсткими мерами.

Ответ:

- Замедление эмиссии токенов
- Увеличение буферизации
- Обратная связь клиенту
- Откат или сброс лишних токенов

Шаг 11:

Миссия 11

Упорядочите приёмы оптимизации throughput токенов:

1. Батчинг / группировка токенов
2. Асинхронная отправка
3. Пул потоков декодирования
4. Минимизация копирования данных

Подсказка: Подумайте, как ускорить движение данных и снизить накладные расходы.

Ответ:

- Батчинг / группировка токенов
- Асинхронная отправка
- Пул потоков декодирования
- Минимизация копирования данных

Шаг 12:

Миссия 12

Упорядочите уровни буферов в системе потоковой генерации токенов (от ближнего к клиенту до глубинного):

1. Выходной токен-буфер перед клиентом
2. Буфер готовых логитов / кандидатов
3. Буфер промежуточных представлений модели
4. Буфер ввода / контекстного состояния

Подсказка: Ближе к клиенту — готовые токены, дальше — исходные слои и контексты.

Ответ:

- Выходной токен-буфер перед клиентом
- Буфер готовых логитов / кандидатов
- Буфер промежуточных представлений модели
- Буфер ввода / контекстного состояния

Шаг 13:

Миссия 13

Упорядочите уровни воздействия параметров генерации на задержку:

1. temperature / крипторазмазка
2. top_k / top_p фильтрация
3. repetition_penalty
4. back-pressure / паузы

Подсказка: Некоторые параметры влияют прямо на распределение, другие — на скорость потока.

Ответ:

- temperature / крипторазмазка
- top_k / top_p фильтрация
- repetition_penalty
- back-pressure / паузы

Шаг 14:

Миссия 14

Упорядочите шаги восстановления после сбоя во время потоковой генерации:

1. Проверка состояния чекпоинта
2. Откат к безопасному состоянию
3. Повтор генерации с этого места
4. Синхронизация с клиентом / уведомление об ошибке

Подсказка: Сначала проверяем, потом откатываем, затем повторяем и сообщаем.

Ответ:

- Проверка состояния чекпоинта
- Откат к безопасному состоянию
- Повтор генерации с этого места
- Синхронизация с клиентом / уведомление об ошибке

Шаг 15:

Миссия 15

Упорядочите слои, через которые проходит токен при передаче по сети:

1. Протокол приложения / HTTP / WebSocket
2. Буфер транспортного уровня (TCP / TLS)
3. Сетевой стек / IP
4. Канал физической передачи / сокет

Подсказка: От высокоуровневого приложения к физическому каналу.

Ответ:

- Протокол приложения / HTTP / WebSocket
- Буфер транспортного уровня (TCP / TLS)
- Сетевой стек / IP
- Канал физической передачи / сокет

Шаг 16:

Миссия 16

Упорядочите стадии масштабирования Ollama Engine при росте нагрузки:

1. Горизонтальное масштабирование узлов генерации
2. Увеличение глубины буферов / очередей
Оптимизация кода декодера / логики потока
3. Балансировка запросов между нодами

Подсказка: Сначала улучшаем на месте, затем добавляем машины и распределяем нагрузку.

Ответ:

- Горизонтальное масштабирование узлов генерации
- Увеличение глубины буферов / очередей
- Оптимизация кода декодера / логики потока
- Балансировка запросов между нодами

🔮 **Шаг 17:**

Миссия 17

Упорядочите мероприятия для диагностики артефактов (мерцание, пропущенные токены) в стриминговом выводе:

1. Проверка логов буферов и задержек
2. Использование чекпоинтов и возобновления
3. Снижение скорости эмиссии и плавное сглаживание
4. Сбор статистики по latency / jitter токенов

Подсказка: Сначала собираем данные, затем локализуем, потом корректируем, затем проверяем устойчивость.

Ответ:

- Проверка логов буферов и задержек
- Использование чекпоинтов и возобновления
- Снижение скорости эмиссии и плавное сглаживание
- Сбор статистики по latency / jitter токенов

📖 **Урок 4 - 🏠 Исследование кэша и буферов памяти**

🔮 **Шаг 01:**

Миссия 1

Вы запускаете Ollama Engine и замечаете, что часть данных загружается из кэша, а не из диска. Что первично проверите?

Подсказка: Часто сначала смотрят настройки кэширования, а не сам диск.

Ответ:

- **Параметры разрешения кэширования (cache enabled)**
- Пропускной канал сети
- Версию операционной системы

🔮 **Шаг 02:**

Миссия 2

Инженер замечает, что буфер памяти переполняется быстро. Что скорее всего происходит?

Подсказка: Возможно, очищение или сброс буфера не работает.

Ответ:

- **Не срабатывает механизм освобождения буфера**
- Дисковая подсистема работает медленно
- Сеть перестала отвечать

🔮 Шаг 03:

Миссия 3

Один из инженеров предложил увеличить размер кэша, чтобы уменьшить доступы к диску. Какой риск есть у этого подхода?

Подсказка: Большой кэш может съедать память, мешая другим подсистемам.

Ответ:

- **Высокое потребление ОЗУ и конкуренция за память**
- Уменьшение пропускной способности сети
- Непредсказуемые ошибки на клиенте

🔮 Шаг 04:

Миссия 4

При нагрузочном тестировании кэш часто промахивается (cache miss). Что скорее всего вы сделаете?

Подсказка: Можно проанализировать шаблоны доступа к данным или политику замещения.

Ответ:

- **Проверю схему доступа данных и алгоритм замещения**
- Увеличу число потоков генерации
- Отключу кэш полностью

🔮 Шаг 05:

Миссия 5

Сопоставьте термин с его функцией в подсистемах кэша / буфера:

Ответ:

- Cache line
- Eviction policy
- Dirty buffer
- Write-through

1. Выбор, какие данные удалить из кэша при переполнении
2. Стратегия записи данных сразу и в кэш, и в основной носитель
3. Блок памяти единичного размера, загружаемый из памяти в кэш
4. Буфер, где хранятся изменённые данные до записи на диск

Подсказка: Подумайте об «грязных» данных и алгоритмах замещения.

- **Cache line** — Блок памяти единичного размера, загружаемый из памяти в кэш
- **Eviction policy** — Выбор, какие данные удалить из кэша при переполнении
- **Dirty buffer** — Буфер, где хранятся изменённые данные до записи на диск
- **Write-through** — Стратегия записи данных сразу и в кэш, и в основной носитель

🔮 Шаг 06:

Миссия 6

Сопоставьте тип буфера с сценарием, где он применим в Ollama Engine:

Ответ:

- Read buffer

- Write buffer
- Prefetch buffer
- LRU cache

1. Буфер предварительной загрузки данных перед доступом
2. Буфер чтения входных данных из внешнего хранилища
3. Буфер записи выходных результатов или изменений
4. Кэш с политикой наименее часто используемых элементов

Подсказка: Read/Write — прямое чтение и запись, prefetch — предзагрузка.

- **Read buffer** — Буфер чтения входных данных из внешнего хранилища
- **Write buffer** — Буфер записи выходных результатов или изменений
- **Prefetch buffer** — Буфер предварительной загрузки данных перед доступом
- **LRU cache** — Кэш с политикой наименее часто используемых элементов

Шаг 07:

Миссия 7

Сопоставьте алгоритм замещения с его характеристикой:

Ответ:

- LRU
 - FIFO
 - LFU
 - Random
1. Удаляется элемент, к которому давно не обращались
 2. Удаление в порядке поступления
 3. Удаляется наименее часто используемый элемент
 4. Удаляется случайно выбранный элемент

Подсказка: Подумайте о «давности», «частоте» и «случайности».

- **LRU** — Удаляется элемент, к которому давно не обращались
- **FIFO** — Удаление в порядке поступления
- **LFU** — Удаляется наименее часто используемый элемент
- **Random** — Удаляется случайно выбранный элемент

Шаг 08:

Миссия 8

Сопоставьте стратегию записи с её поведением:

Ответ:

- Write-back
 - Write-through
 - Write-around
 - Write coalescing
1. **Изменения записываются сначала в кэш, а потом** — на диск при необходимости
 2. Каждая запись идёт сразу в кэш и на диск
 3. Запись сначала минует кэш и сразу идёт на диск
 4. Объединение нескольких мелких записей в одну запись на диск

Подсказка: Разницу между агрессивной записью и «скрытыми» буферами.

- **Write-back** — Изменения записываются сначала в кэш, а потом — на диск при необходимости
- **Write-through** — Каждая запись идёт сразу в кэш и на диск
- **Write-around** — Запись сначала минует кэш и сразу идёт на диск
- **Write coalescing** — Объединение нескольких мелких записей в одну запись на диск

🔮 Шаг 09:

Миссия 9

Упорядочите этапы обращения данных при кэш-промахе (cache miss):

1. Запрос на основной носитель (диск / память)
2. Возврат данных в кэш
3. Удаление старого элемента в кэше (если переполнение)
4. Возврат данных клиенту

Подсказка: Промах → запрос → загрузка → осаждение → ответ.

Ответ:

- Запрос на основной носитель (диск / память)
- Возврат данных в кэш
- Удаление старого элемента в кэше (если переполнение)
- Возврат данных клиенту

🔮 Шаг 10:

Миссия 10

Упорядочите уровни памяти / буферов, по которым проходят данные в Ollama Engine:

1. Кэш L1 / регистра
2. Кэш L2 / промежуточный буфер
3. Буфер кэша моделей (application buffer)
4. Основная память / диск

Подсказка: От самых быстрых к более медленным уровням.

Ответ:

- Кэш L1 / регистра
- Кэш L2 / промежуточный буфер
- Буфер кэша моделей (application buffer)
- Основная память / диск

🔮 Шаг 11:

Миссия 11

Упорядочите действия очистки буфера (flush) перед завершением сеанса:

1. Запись «грязных» данных на диск
2. Сброс кэша на нулевые значения
3. Ожидание завершения всех асинхронных операций
4. Закрытие файловых дескрипторов / освобождение ресурсов

Подсказка: Сначала нужно привести к консистентному состоянию, затем закрыть ресурсы.

Ответ:

- Запись «грязных» данных на диск
- Сброс кэша на нулевые значения
- Ожидание завершения всех асинхронных операций
- Закрытие файловых дескрипторов / освобождение ресурсов

 Шаг 12:

Миссия 12

Упорядочите фазы профилирования буферов / кэшей:

1. Сбор статистики попаданий / промахов
2. Анализ распределения обращений (горячие участки)
3. Моделирование изменений в политике кэша / размерах буферов
4. Проведение повторного теста и контроль метрик

Подсказка: Профилирование => гипотезы => симуляция => проверка.

Ответ:

- Сбор статистики попаданий / промахов
- Анализ распределения обращений (горячие участки)
- Моделирование изменений в политике кэша / размерах буферов
- Проведение повторного теста и контроль метрик

 Шаг 13:

Миссия 13

Упорядочите стадии, при которых данные могут попасть из кэша в основной буфер памяти:

1. Кэш признан устаревшим / сброшен
2. Буфер dirty записан на диск / память
3. Следующее обращение к данным запрашивает свежую копию из памяти / диск
4. Кэш обновлён и заполнен новой копией

Подсказка: Сначала invalidate, затем flush, затем запрос, затем refill.

Ответ:

- Кэш признан устаревшим / сброшен
- Буфер dirty записан на диск / память
- Следующее обращение к данным запрашивает свежую копию из памяти / диск
- Кэш обновлён и заполнен новой копией

 Шаг 14:

Миссия 14

Упорядочите уровни отказа кэша, если один уровень не отвечает:

1. Попадание в L1 / быстрый кэш
2. Переход к кэшу следующего уровня (L2 / приложение)
3. Запрос к основной памяти / диску
4. Запись результата обратно в все уровни кэша

Подсказка: Пробуем самый быстрый уровень, затем идём вниз, и после — заполняем обратно.

Ответ:

- Попадание в L1 / быстрый кэш
- Переход к кэшу следующего уровня (L2 / приложение)
- Запрос к основной памяти / диску
- Запись результата обратно в все уровни кэша

 Шаг 15:

Миссия 15

Упорядочите уровни буферов при крупной генерации текста:

1. Буфер токенов готовых к отправке
2. Внутренний модельный кэш слоёв / attention
3. Буфер контекстных представлений
4. Буфер записи изменений (dirty buffer)

Подсказка: Данные движутся от модели к сети и обратно, буферы организованы по роли.

Ответ:

- Буфер токенов готовых к отправке
- Внутренний модельный кэш слоёв / attention
- Буфер контекстных представлений
- Буфер записи изменений (dirty buffer)

 Шаг 16:

Миссия 16

Упорядочите шаги при снятии «шумов» или неконсистентностей в кэше:

1. Инвалидация (отметка как устаревшего)
2. Сброс dirty данных (flush)
3. Обновление по читаемому запросу (fetch и refill)
4. Синхронизация между нодами / репликами

Подсказка: Сначала пометить старое, затем очистить, затем наполнить вновь, и синхронизировать.

Ответ:

- Инвалидация (отметка как устаревшего)
- Сброс dirty данных (flush)
- Обновление по читаемому запросу (fetch и refill)
- Синхронизация между нодами / репликами

 Шаг 17:

Миссия 17

Упорядочите этапы адаптивной корректировки размеров кэша / буферов при нагрузке:

1. Мониторинг метрик (промахи, задержки)
2. Анализ закономерностей и узких мест
3. Изменение параметров размера / политик кэша / буферов
4. Проверка эффекта на нагрузочном тесте

Подсказка: Сначала смотрим, затем решаем, применяем и проверяем.

Ответ:

- Мониторинг метрик (промахи, задержки)
- Анализ закономерностей и узких мест
- Изменение параметров размера / политик кэша / буферов
- Проверка эффекта на нагрузочном тесте

 Урок 5 - Квест отладь сбой генерации и восстанови поток вывода

 Шаг 01:

Миссия 1

На вашем LLM-движке внезапно перестал выводиться ответ: запрос принимается, но генерация останавливается на середине. Что первое вы проверите?

Подсказка: Иногда поток останавливается из-за таймаута или переполнения буфера.

Ответ:

- **Настройки таймаута генерации**
- Шифрование трафика между модулями
- Версию ОС на сервере
- Цветовую тему в интерфейсе

 Шаг 02:

Миссия 2

Вы обнаружили, что генератор прерывается при встрече спецсимвола «\n\n». Какое ограничение может быть причиной?

Подсказка: Иногда используется максимально допустимая длина токенов без переноса строки.

Ответ:

- **Ограничение на длину буфера токенов**
- Ограничение по размеру файла конфигурации
- Лимит на количество open file descriptors
- Ограничение оперативной памяти процесса

 Шаг 03:

Миссия 3

Вы добавили логирование, но видите только статус «начало генерации», а не частичные фрагменты текста. Что могло быть отключено?

Подсказка: Проверьте, не отключен ли flush буфера вывода.

Ответ:

- **Авто-flush буфера вывода**
- Уровень логирования DEBUG
- SSL-шифрование логов
- Ротация логов по размеру

 Шаг 04:

Миссия 4

После исправления flush генерация снова прекращается, но теперь ошибка «broken pipe». Что вероятнее всего происходит?

Подсказка: Разрыв канала может быть вызван закрытием клиента раньше времени.

Ответ:

- **Клиент закрыл соединение до завершения генерации**
- SSD диск сервера вышел из строя
- Неправильная кодировка заголовка HTTP
- Ошибка DNS-разрешения

Шаг 05:

Миссия 5

Вы активировали keep-alive соединение, но генерация всё ещё не доходит до конца. Что добавить?

Подсказка: Иногда нужен heartbeat или «ping» между сообщениями.

Ответ:

- **Периодические сигналы heartbeat между фрагментами**
- Автоматическое переинициализирование сессии
- Увеличение max_connections
- Шифрование heartbeat

Шаг 06:

Миссия 6

Сопоставьте причину остановки генерации с возможным следствием:

Причины:

Ответ:

- Таймаут соединения
- Переполнение буфера на стриме
- Закрытие клиента преждевременно
- Ошибка сериализации объекта

Следствия:

1. Получен только фрагмент ответа
2. Сбой формата JSON в середине
3. Соединение разорвано с ошибкой broken pipe
4. Запрос завершился без ответа

Подсказка: Подумайте, чем грозит каждый из этих сценариев для вывода.

- **Таймаут соединения** — Запрос завершился без ответа
- **Переполнение буфера на стриме** — Получен только фрагмент ответа
- **Закрытие клиента преждевременно** — Соединение разорвано с ошибкой broken pipe
- **Ошибка сериализации объекта** — Сбой формата JSON в середине

Шаг 07:

Миссия 7

Упорядочите шаги восстановления потока вывода после сбоя:

1. Добавление heartbeat / keep-alive
2. Восстановление логики flush и буферизации

3. Анализ логов и трассировки ошибки
4. Тестирование сценариев с прерываниями
5. Корректировка таймаутов соединения

Подсказка: Сначала нужно понять, что и где сломалось, потом восстановить систему шаг за шагом.

Ответ:

- Анализ логов и трассировки ошибки
- Восстановление логики flush и буферизации
- Корректировка таймаутов соединения
- Добавление heartbeat / keep-alive
- Тестирование сценариев с прерываниями

Шаг 08:

Миссия 8

После внедрения heartbeat поток работает, но нагрузка возросла и задержка выросла. Что оптимизировать первым делом?

Подсказка: Посмотрите на интервал heartbeat и частоту.»

Ответ:

- **Уменьшить частоту heartbeat**
- Увеличить блокирующие операции
- Добавить больше логирования
- Переход на HTTP/1.0

Шаг 09:

Миссия 9

Вы заметили, что ошибка появляется только при генерации больших текстов (>1000 токенов). Что может помочь?

Подсказка: Иногда стоит разбивать генерацию на чанки.

Ответ:

- **Делить запрос на части и объединять результаты**
- Запрашивать весь текст за один вызов независимо от длины
- Снижать размер модели
- Отключать безопасные фильтры

Шаг 10:

Миссия 10

Сопоставьте тип сбоя генерации с корректным патчем:

Тип сбоя:

Ответ:

- Остановка на спецсимволе
- Таймаут
- Закрытие соединения клиентом
- Сбой JSON-парсера

Патчи:

1. Разбиение по чанкам и escape спецсимволов
2. Увеличение таймаута / повтор на таймаут
3. Удержание соединения через heartbeat и retry
4. Экранирование или отладка сериализации

Подсказка: Сопоставьте проблему с наиболее прямым средством её устранения.

- **Остановка на спецсимволе** — Разбиение по чанкам и escape спецсимволов
- **Таймаут** — Увеличение таймаута / повтор на таймаут
- **Закрытие соединения клиентом** — Удержание соединения через heartbeat и retry
- **Сбой JSON-парсера** — Экранирование или отладка сериализации

Шаг 11:

Миссия 11

Сопоставьте уровень логов с тем, какие сообщения вы хотите видеть:

Уровень:

Ответ:

- DEBUG
- INFO
- WARNING
- ERROR

Сообщения:

1. Детальные шаги flush, heartbeat, буферизации
2. Старт и завершение сессий
3. Необычные, но не критичные события
4. Критичные сбои, приводящие к падению

Подсказка: DEBUG = максимально подробный, ERROR = серьёзные сбои.

- **DEBUG** — Детальные шаги flush, heartbeat, буферизации
- **INFO** — Старт и завершение сессий
- **WARNING** — Необычные, но не критичные события
- **ERROR** — Критичные сбои, приводящие к падению

Шаг 12:

Миссия 12

Упорядочите приоритетные участки кода для отладки сбоя потока:

1. Слой сетевого стека и соединения
2. Модуль сериализации / форматирования вывода
3. Буферизация и flush логики
4. Механизм heartbeat / keep-alive
5. Обработка исключений и retry

Подсказка: Начните с внешних слоёв, потом внутренние модули, затем fallback.

Ответ:

- Слой сетевого стека и соединения
- Буферизация и flush логики

- Модуль сериализации / форматирования вывода
- Механизм heartbeat / keep-alive
- Обработка исключений и retry

🔗 Шаг 13:

Миссия 13

После всех исправлений вы замечаете периодические пропуски частей текста (фрагменты исчезают). Что вероятно виновато?

Подсказка: Частые удаления могут быть из-за переполнения буфера или дубликатов.

Ответ:

- **Обрезание буфера слишком агрессивно**
- Неправильный порядок ключей JSON
- Низкий уровень логирования
- Несовместимость версии Python

🔗 Шаг 14:

Миссия 14

Сопоставьте среду (локал / стейдж / продакшн) с возможным поведением сбоя:

Среда:

Ответ:

- Локал
- Стейдж
- Продакшн

Поведение:

1. Вывод без прерываний
2. Прерывание на спецсимволах
3. Остановка на таймауте / broken pipe

Подсказка: В боевом окружении чаще всего — самые строгие ограничения.

- **Локал** — Вывод без прерываний
- **Стейдж** — Прерывание на спецсимволах
- **Продакшн** — Остановка на таймауте / broken pipe

🔗 Шаг 15:

Миссия 15

Упорядочите финальные шаги верификации генератора после исправления:

1. Нагрузочное тестирование на длинных запросах
2. Валидация JSON во всех частичных фрагментах
3. Мониторинг памяти / утечек при долгом стриме
4. Сценарии прерываний и retry
5. Ручная проверка на спецсимволах и граничных случаях

Подсказка: Начните с автоматических тестов, затем нагрузка, затем ручные кейсы.

Ответ:

- Валидация JSON во всех частичных фрагментах

- Сценарии прерываний и retry
- Нагрузочное тестирование на длинных запросах
- Мониторинг памяти / утечек при долгом стриме
- Ручная проверка на спецсимволах и граничных случаях

Модуль 3 - Миссия 3. Форматы и код ДНК моделей

Урок 1 - ДНК модели формат gguf и структура весов

Шаг 01:

Миссия 1

Разработчик Алиса открывает файл модели в формате GGUF и видит непонятный бинарный заголовок. Что она проверит первым?

Подсказка: В GGUF в начале должны быть сигнатура и версия формата.

Ответ:

- **Сигнатуру (магическое число) и версию GGUF**
- Кодировку исходного текста
- Размер файла в байтах
- CRC контрольную сумму файла

Шаг 02:

Миссия 2

Боб пытается загрузить веса модели из GGUF, но операция падает с ошибкой «unexpected tag». Что вероятно неправильно?

Подсказка: GGUF использует именованные теги-чанки с типом данных.

Ответ:

- **Чанк тег имеет неизвестный идентификатор**
- Файл не читается в бинарном режиме
- Неправильный порядок загрузки библиотек
- Версия Python несовместима

Шаг 03:

Миссия 3

При просмотре структуры GGUF Alice находит секцию «metadata», но без ключей. Что могло быть пропущено?

Подсказка: Метаданные в GGUF обычно хранятся как словарь ключ→значение.

Ответ:

- **Сохранение map-энтри в метаданных**
- Очищение кеша модели
- Фильтрация нестандартных тегов
- Сжатие метаданных

Шаг 04:

Миссия 4

При загрузке весов Bob получает, что блок attention имеет нехватку байтов. Что может быть причиной?

Подсказка: Возможно, смещение в файле указано неправильно.

Ответ:

- **Неправильный офсет чанк-данных весов**
- Недостаток оперативной памяти
- Конфликт версии GGUF
- Отсутствие лицензии на модель

Шаг 05:

Миссия 5

Сопоставьте элемент GGUF с его ролью:

Элемент:

Ответ:

- Magic / сигнатура
- Version
- Metadata
- Weights chunk

Роль:

1. **Указывает, что файл** — GGUF
2. Хранит числовые веса слоёв
3. Версия формата GGUF
4. Дополнительные атрибуты модели (имя, токенизация и пр.)

Подсказка: Сигнатура — в начале, затем версия, метаданные и далее веса.

- **Magic / сигнатура** — Указывает, что файл — GGUF
- **Version** — Версия формата GGUF
- **Metadata** — Дополнительные атрибуты модели (имя, токенизация и пр.)
- **Weights chunk** — Хранит числовые веса слоёв

Шаг 06:

Миссия 6

При чтении весов встречаются теги float32, float16, int8. Сопоставьте тип тегов с их преимуществами:

Тип тега:

Ответ:

- float32
- float16
- int8

Преимущество:

1. Максимальная точность (меньше потерь)
2. Уменьшенный объём памяти с сохранением плавности
3. Максимальное сжатие и ускорение вывода

Подсказка: float32 — точный, но большой; int8 — компактный, но с потерями.

- **float32** — Максимальная точность (меньше потерь)
- **float16** — Уменьшенный объём памяти с сохранением плавности

- **int8** — Максимальное сжатие и ускорение вывода

Шаг 07:

Миссия 7

Упорядочите этапы открытия GGUF файла в память:

1. Чтение сигнатуры и проверка формата
2. Чтение версии и проверка совместимости
3. Интерпретация метаданных модели
4. Чтение чанков весов
5. Валидация контрольных сумм / целостности

Подсказка: Начинайте с базовых заголовков, затем метаданные и далее — веса + проверка.

Ответ:

- Чтение сигнатуры и проверка формата
- Чтение версии и проверка совместимости
- Интерпретация метаданных модели
- Чтение чанков весов
- Валидация контрольных сумм / целостности

Шаг 08:

Миссия 8

При сохранении GGUF Alice забыла записать контрольные суммы. Что может произойти при загрузке?

Подсказка: Без проверки целостности сбои менее очевидны, но могут повредить веса.

Ответ:

- **Не удастся обнаружить повреждённые части весов**
- Файл станет текстовым
- Метаданные будут перепутаны
- Магическая сигнатура изменится

Шаг 09:

Миссия 9

Bob хочет сохранить модель с квантованием (int8) в GGUF. Какой шаг он должен обязательно сделать перед сохранением?

Подсказка: Нужно привести веса к нужному типу данных.

Ответ:

- **Конвертировать веса в int8 формат перед упаковкой**
- Отключить метаданные
- Удалить сигнатуру GGUF
- Зашифровать файл модели

Шаг 10:

Миссия 10

Сопоставьте фразу с действием при работе с GGUF:

Фраза:

Ответ:

- «GGUF лёгкий»
- «GGUF строгий»
- «fallback weights»
- «metadata override»

Действие:

1. Допускает отсутствующие чанки с запасом
2. Требуется наличие всех обязательных тегов
3. Использует запасные веса, если основной чанк отсутствует
4. Позволяет переопределить метаданные вручную

Подсказка: «Лёгкий» — более гибкий, «строгий» — строгий по требованиям.

- **«GGUF лёгкий»** — Допускает отсутствующие чанки с запасом
- **«GGUF строгий»** — Требуется наличие всех обязательных тегов
- **«fallback weights»** — Использует запасные веса, если основной чанк отсутствует
- **«metadata override»** — Позволяет переопределить метаданные вручную

 Шаг 11:

Миссия 12

Алиса анализирует три файла весов: model.gguf, model.gguf.gz, model.gguf.snappy. Сопоставьте суффикс с описанием:

Суффикс:

Ответ:

- .gguf
- .gguf.gz
- .gguf.snappy

Описание:

1. Несжатый формат
2. Сжатие gzip
3. Сжатие snappy для быстрого распаковки

Подсказка: gzip — классическое сжатие, snappy быстрее на декодировку.

- **.gguf** — Несжатый формат
- **.gguf.gz** — Сжатие gzip
- **.gguf.snappy** — Сжатие snappy для быстрого распаковки

 Шаг 12:

Миссия 12

Упорядочите способы сжатия GGUF по скорости распаковки (от самого быстрого к самому медленному):

1. Snappy
2. Нет сжатия
3. Gzip
4. LZ4 (если поддерживается)

Подсказка: Обычно лёгкие алгоритмы распаковываются быстрее, но менее эффективно сжимаются.

Ответ:

- Snappy
- Нет сжатия
- Gzip
- LZ4 (если поддерживается)

🔗 Шаг 13:

Миссия 13

В процессе отладки Bob видит, что веса одного слоя умножаются на нулевой коэффициент при загрузке. Что вероятно произошло?

Подсказка: Возможно, применяется неправильная масштабная нормировка или множитель пустой.

Ответ:

- **Неверно применён scale factor = 0**
- Тип данных float32 переписан как int8
- Смещение весов неверно считано
- Неправильный порядок загрузки слоёв

🔗 Шаг 14:

Миссия 14

После успешной загрузки GGUF Alice хочет инспектировать веса слоя. Какой инструмент ей поможет?

Подсказка: Нужно что-то, что может прочитать бинарные массивы и показать значения.

Ответ:

- **Просмотр через Python + numpy / view как массив**
- Текстовый редактор Notepad
- Image Viewer
- Блокнот Windows

🔗 Шаг 15:

Миссия 15

Сопоставьте термин с его значением в контексте GGUF:

Термин:

Ответ:

- Chunk alignment
- Offset table
- Padding bytes
- Checksum

Значение:

1. Смещение начала чанков в файле
2. Дополнительные нули между чанками для выравнивания
3. Таблица смещений к каждому чанку
4. Контроль целостности данных

Подсказка: offset table — карта, alignment / padding — выравнивание, checksum — проверка.

- **Chunk alignment** — Дополнительные нули между чанками для выравнивания
- **Offset table** — Таблица смещений к каждому чанку
- **Padding bytes** — Смещение начала чанков в файле
- **Checksum** — Контроль целостности данных

🔧 Шаг 16:

Миссия 16

Упорядочите проверки, которые Alice выполняет перед экспортом модели в GGUF:

1. Проверка, что все слои имеют допустимые типы данных
2. Вычисление контрольных сумм чанков
3. Установка выравнивания для каждого чанка
4. Запись метаданных и мета-тегов
5. Запись сигнатуры и версии формата

Подсказка: Сначала — сигнатура, затем метаданные, потом слои, выравнивание и контроль целостности.

Ответ:

- Запись сигнатуры и версии формата
- Запись метаданных и мета-тегов
- Проверка, что все слои имеют допустимые типы данных
- Установка выравнивания для каждого чанка
- Вычисление контрольных сумм чанков

📖 Урок 2 - 🧠 Кванты Q4-Q8 как AI теряет и сохраняет память

🔧 Шаг 01:

Миссия 1

Вы — инженер LLM, и модель Q4 — это ваша первая лиговая “квантовая ячейка”. Что из этого лучше описывает, что Q4 ****не может**** делать по сравнению с Q8?

Подсказка: Q4 — это агрессивное сжатие, многие детали теряются.

Ответ:

- **Сохранять тонкие детали долгосрочной памяти**
- Работать быстрее, чем Q8
- Работать в формате целых чисел

🔧 Шаг 02:

Миссия 2

🧠 Команда столкнулась с «размытием памяти» после подачи длинного текста: в Q4 модель забывает, что упоминалось в начале. Какой из подходов поможет смягчить потерю контекста?

Подсказка: Используйте вспомогательные сигналы, а не полагаясь только на чистый квант.

Ответ:

- **Инжектировать краткие маркеры контекста (anchoring)**
- Увеличить batch size
- Перекодировать всё в float32
- Обрезать начало текста всегда

Шаг 03:

Миссия 3

В одной из сессий инженер заметил: «Q6 иногда ведёт себя почти как Q8, но с артефактами». Что из списка лучше всего объясняет это поведение?

Подсказка: Часто Q6 сохраняет средние степени свободы, но компромиссы видны в крайних битах.

Ответ:

- **Он пытается сохранить наиболее значимые биты, жертвуя менее значимые**
- Он автоматически переключается в Q8 режим
- Он добавляет шум на выходе
- Он дублирует входные токены

Шаг 04:

Миссия 4

👤 Девушка-инженер Юлия проверяет ответы модели после перехода на Q4. Она замечает странный сбой: модель «забывает» цифры года. Почему это может быть?

Подсказка: Q4 уменьшает точность представления числовых различий.

Ответ:

- **Потеря точности при округлении квантования**
- Модель вообще не поддерживает числа
- Ошибка декодирования UTF-8
- Проблема с токенизацией слов

Шаг 05:

Миссия 5

Вы хотите снизить искажения в Q4 при обработке финансовых данных. Какой стратегический шаг будет наиболее полезен?

Подсказка: Выбирайте стратегию, которая минимизирует потери в чувствительных значениях.

Ответ:

- **Использовать гибридный режим — Q4 для текста, Q8 для чисел**
- Отказаться от квантования
- Квантовать всё усреднённо
- Игнорировать числовые токены

Шаг 06:

Миссия 6

👤 Сопоставьте характеристику квантования с её описанием.

Характеристика:

Ответ:

- Динамический диапазон
- Квантование с шагом
- Сторонняя ошибка (bias)
- Квантизация случайного шума (dithering)

Описание:

1. Добавление небольшого шума перед квантованием, чтобы сгладить ошибки.
2. Разница между минимальным и максимальным значением, которое можно выразить.
3. Систематическая смещённость представлений после квантования.
4. Шаг, через который дискретизируются значения (например, $\Delta = 0.125$).

Подсказка: Подумайте, какая характеристика отвечает за «скольжение» диапазона, а какая — за смещение.

- **Динамический диапазон** — Разница между минимальным и максимальным значением, которое можно выразить.
- **Квантование с шагом** — Шаг, через который дискретизируются значения (например, $\Delta = 0.125$).
- **Сторонняя ошибка (bias)** — Систематическая смещённость представлений после квантования.
- **Квантизация случайного шума (dithering)** — Добавление небольшого шума перед квантованием, чтобы сгладить ошибки.

Шаг 07:

Миссия 7

Сопоставьте уязвимость с её проявлением в LLM при агрессивном квантовании.

Уязвимость:

Ответ:

- Аномалии в крайних токенах
- Забывание первых строк
- Нестабильность генерации дробей
- Симметрия битовых шаблонов

Проявление:

1. Код “2023” превращается в “2020” или “2025”.
2. Модель “теряет” введение, забывает упомянутые факты.
3. Некоторые символы становятся зеркальными в битах.
4. Ошибки проявляются чаще на границах последовательностей.

Подсказка: Подумайте, как квантование влияет на токены, стоящие на краях или которые содержат тонкие структуры.

- **Аномалии в крайних токенах** — Ошибки проявляются чаще на границах последовательностей.
- **Забывание первых строк** — Модель “теряет” введение, забывает упомянутые факты.
- **Нестабильность генерации дробей** — Код “2023” превращается в “2020” или “2025”.
- **Симметрия битовых шаблонов** — Некоторые символы становятся зеркальными в битах.

Шаг 08:

Миссия 8

🔗 Сопоставьте стратегию компенсации потерь информации с её кратким объяснением.

Стратегия:

Ответ:

- Residual correction

- Clipping
- Mixed precision fallback
- Adaptive quant step

Объяснение:

1. Обрезание экстремальных значений, чтобы избежать выхода за диапазон.
2. Использование более точной квантовки для чувствительных элементов.
3. Хранение остаточной ошибки (разницы), чтобы потом её компенсировать.
4. Подстройка шага квантования в зависимости от сегмента данных.

Подсказка: Одна стратегия сохраняет “остаток”, другая — изменяет шаг вплоть до локальной адаптации.

- **Residual correction** — Хранение остаточной ошибки (разницы), чтобы потом её компенсировать.
- **Clipping** — Обрезание экстремальных значений, чтобы избежать выхода за диапазон.
- **Mixed precision fallback** — Использование более точной квантовки для чувствительных элементов.
- **Adaptive quant step** — Подстройка шага квантования в зависимости от сегмента данных.

🔮 Шаг 09:

Миссия 9

В реальной системе QA-интерфейса модель даёт некорректные ответы из-за искажений. Сопоставьте источник искажения с примером поведения.

Источник искажения:

Ответ:

- Ошибка округления
- Смещение битов
- Нелинейность активации после квантования
- Кумулятивная ошибка

Пример поведения:

1. Несколько мелких искажений на каждом уровне суммируются и проявляются в конце.
2. Один ключевой бит смещён вверх, и результат сдвинут систематически.
3. При квантовании активированной функции некоторые значения “телепортируются” к крайним.
4. Плавное значение “0.47” округляется к “0.5”, нарушая пропорцию.

Подсказка: Подумайте, как ошибка на каждом слое может “накопиться”, и как битовые смещения влияют.

- **Ошибка округления** — Плавное значение “0.47” округляется к “0.5”, нарушая пропорцию.
- **Смещение битов** — Один ключевой бит смещён вверх, и результат сдвинут систематически.
- **Нелинейность активации после квантования** — При квантовании активированной функции некоторые значения “телепортируются” к крайним.
- **Кумулятивная ошибка** — Несколько мелких искажений на каждом уровне суммируются и проявляются в конце.

Шаг 10:

Миссия 10

 Расположите этапы, по которым инженер переходил от Q8 к Q4, минимизируя потери памяти:

1. Обнаружение чувствительных сегментов (имён, цифр, дат)
2. Настройка fallback-режима для этих сегментов
3. Квантование всей остальной части в Q4
4. Внедрение residual correction / остаточной ошибки
5. Тестирование на сложных примерах
6. Мониторинг распределения битовых ошибок

Подсказка: Подумайте, что делается перед квантованием, что после, и как тестирование встроено в цепочку.

Ответ:

- Обнаружение чувствительных сегментов (имён, цифр, дат)
- Настройка fallback-режима для этих сегментов
- Квантование всей остальной части в Q4
- Внедрение residual correction / остаточной ошибки
- Тестирование на сложных примерах
- Мониторинг распределения битовых ошибок

Шаг 11:

Миссия 11

Упорядочите уровни действий, когда после квантования модель начала допускать ошибки в длинных последовательностях:

1. Выявление сегмента, где память ломается
2. Оценка амплитуды ошибок на этом участке
3. Внедрение адаптивного шага квантования для этого участка
4. Коррекция остаточной ошибки (если возможно)
5. Повторное тестирование всей последовательности

Подсказка: Распределение ошибок меняется после каждого этапа — логично идти от анализа к исправлению и тесту.

Ответ:

- Выявление сегмента, где память ломается
- Оценка амплитуды ошибок на этом участке
- Внедрение адаптивного шага квантования для этого участка
- Коррекция остаточной ошибки (если возможно)
- Повторное тестирование всей последовательности

Шаг 12:

Миссия 12

 Расставьте по порядку действия инженера при встроенном тестировании квантованной модели:

1. Сформировать набор контрольных вопросов с датами, именами и цифрами
2. Запустить модель в режиме Q4 и собрать ответы
3. Сравнить ответы с эталоном, измерить расхождения

4. Определить сегменты с ошибками и их природу
5. Настроить стратегию компенсации (fallback, residual, адаптивный шаг)
6. Повторить цикл до приемлемого качества

Подсказка: Это цикл «проверка — анализ — корректировка — повторение».

Ответ:

- Сформировать набор контрольных вопросов с датами, именами и цифрами
- Запустить модель в режиме Q4 и собрать ответы
- Сравнить ответы с эталоном, измерить расхождения
- Определить сегменты с ошибками и их природу
- Настроить стратегию компенсации (fallback, residual, адаптивный шаг)
- Повторить цикл до приемлемого качества

Шаг 13:

Миссия 13

Упорядочите уровни сложности при отладке искажений в Q4:

1. Простые числовые токены (1234, 2025)
2. Ключевые имена / названия (Alice, Zeta)
3. Сочетания слов + цифр (Model2023)
4. Большие текстовые абзацы с фактами
5. Контексты с взаимосвязями (спойлеры, упоминания ранее)

Подсказка: Чем дальше контекст, тем сложнее сохранить корректность.

Ответ:

- Простые числовые токены (1234, 2025)
- Ключевые имена / названия (Alice, Zeta)
- Сочетания слов + цифр (Model2023)
- Большие текстовые абзацы с фактами
- Контексты с взаимосвязями (спойлеры, упоминания ранее)

Шаг 14:

Миссия 14

 Упорядочите итерации оптимизации квантования, которые инженер обычно делает:

1. Базовое квантование в Q4
2. Добавление residual correction
3. Включение fallback на Q8 для чувствительных частей
4. Подстройка шага квантования адаптивно
5. Тестирование и мониторинг в нагрузке

Подсказка: Следуйте логике: сначала простое — потом надстройки — потом проверка.

Ответ:

- Базовое квантование в Q4
- Добавление residual correction
- Включение fallback на Q8 для чувствительных частей
- Подстройка шага квантования адаптивно
- Тестирование и мониторинг в нагрузке

Шаг 15:

Миссия 15

Упорядочите фазы, которые команда проходит при переходе на агрессивное квантование в продакшене:

1. Лабораторные эксперименты на данных разработчиков
2. Масштабирование тестов на контент пользователей
3. Выявление “горячих” токенов
4. Настройка компенсаций (residual, fallback)
5. Мониторинг ошибок в живой нагрузке

Подсказка: Постепенный rollout снижает риск.

Ответ:

- Лабораторные эксперименты на данных разработчиков
- Масштабирование тестов на контент пользователей
- Выявление “горячих” токенов
- Настройка компенсаций (residual, fallback)
- Мониторинг ошибок в живой нагрузке

Шаг 16:

Миссия 16

 Упорядочите действия инженера, когда после выпуска наблюдается всплеск ошибок в цифрах в реальных ответах:

1. Сбор примеров с ошибками из логов
2. Анализ паттернов ошибок (в каких местах чаще всего ошибаются)
3. Внедрение локального fallback на Q8 для таких мест
4. Добавление residual correction именно для этих токенов
5. Проверка улучшения на тех же логах

Подсказка: Это цикл реактивной отладки: лог → анализ → исправление → проверка.

Ответ:

- Сбор примеров с ошибками из логов
- Анализ паттернов ошибок (в каких местах чаще всего ошибаются)
- Внедрение локального fallback на Q8 для таких мест
- Добавление residual correction именно для этих токенов
- Проверка улучшения на тех же логах

Шаг 17:

Миссия 17

Финальный квест: вы — инженер LLM, и ваша цель — минимизировать искажения памяти при Q4 до уровня приемлемого качества. Упорядочите окончательные шаги в стратегии:

1. Проверить распределение ошибок на разных токенах и позициях
2. Выбрать порог чувствительности (например, цифры, имена)
3. Настроить fallback на Q8 для пороговых токенов
4. Внедрить residual correction или перенос ошибки
5. Подстроить шаг квантования локально на участках с шумом
6. Запустить “боевое” тестирование на живых данных

7. Мониторинг и обновление политики квантования при drift'e

Подсказка: Это финальный боевой конвейер: от анализа к политике к мониторингу.

Ответ:

- Проверить распределение ошибок на разных токенах и позициях
- Выбрать порог чувствительности (например, цифры, имена)
- Настроить fallback на Q8 для пороговых токенов
- Внедрить residual correction или перенос ошибки
- Подстроить шаг квантования локально на участках с шумом
- Запустить "боевое" тестирование на живых данных
- Мониторинг и обновление политики квантования при drift'e

Урок 3 - Токенизация и кодирование смысла

Шаг 01:

Миссия 1

Анна, инженер LLM, обрабатывает текст «Москва 2025 год». Какой формат токенизации сработает наиболее просто?

Подсказка: Разделение по пробелам часто сохраняет базовую читаемость.

Ответ:

- **Whitespace-токенизация (разделение по пробелам)**
- Байт-уровневая (byte-level) токенизация
- Символьная токенизация (по символам)
- Случайная агрегация подстрок

Шаг 02:

Миссия 2

 Вася пробует использовать BPE (Byte Pair Encoding) для фраз «программирование» и «программисты». Что он получит?

Подсказка: BPE разбивает на часто встречающиеся подстроки, даже внутри слова.

Ответ:

- **Общие подстроки «программ», «ирован», «исты» с частичным разделением**
- Идентичные токены для двух слов
- Полное деление на символы
- Одна большая подстрока «программи» и остатки

Шаг 03:

Миссия 3

Катя экспериментирует с субсловами: как лучше кодировать новое длинное слово, которого не было в обучении?

Подсказка: Субсловный подход разбивает слово на уже знакомые части.

Ответ:

- **Разбить на субслова, уже известные модели**
- Назначить новый уникальный токен целиком
- Игнорировать новое слово (удалить)
- Квантовать как шумовый токен

Шаг 04:

Миссия 4

🧑 Инженер Лена проверяет частотность токенов: что будет с очень редким словом «энтеропсихология»?

Подсказка: Редкие слова часто дробятся на субслова или символы.

Ответ:

- **Разбьётся на субслова или символы**
- Останется целым как один токен
- Будет удалено
- Будет повторяться как шумовой токен

Шаг 05:

Миссия 5

Иван экспериментирует с токенизацией чисел: «1234567890». Какой подход сохранит максимальную информацию?

Подсказка: Можно разбивать числа на части или сохранить как единый токен, если модель знает это.

Ответ:

- **Сохранить как единый токен (если модель его знает) или разбить на знакомые части**
- Разбить на единичные цифры всегда
- Удалить половину цифр
- Превратить в текстовый токен “число”

Шаг 06:

Миссия 6

🧑 Сопоставьте термин токенизации с его описанием.

Термин:

Ответ:

- Субсловная токенизация
- Byte-level токенизация
- Символьная токенизация
- Whitespace-токенизация

Описание:

1. Токенизация, где каждое отдельное слово разделяется пробелами.
2. Использование символов Unicode как единичных токенов.
3. Разбиение на часто встречающиеся части слова (субслова).
4. Токенизация на уровне байтов, включая нестандартные символы.

Подсказка: Подумайте: какой метод гибкий для новых слов, а какой работает даже с любыми байтами?

- **Субсловная токенизация** — Разбиение на часто встречающиеся части слова (субслова).
- **Byte-level токенизация** — Токенизация на уровне байтов, включая нестандартные символы.
- **Символьная токенизация** — Использование символов Unicode как единичных токенов.

- **Whitespace-токенизация** — Токенизация, где каждое отдельное слово разделяется пробелами.

Шаг 07:

Миссия 7

Сопоставьте проблемы токенизации с примерами, которые могут их иллюстрировать.

Проблема:

Ответ:

- Оверсегментация
- Недосегментация
- Неизвестные токены (OOV)
- Неправильное объединение

Пример:

1. Слово «playground» разбито как «pla y grounds».
2. Фраза «машинное обучение» закодирована как один токен «машинноеобучение».
3. Слово «гиперпрограммирование» не входит в словарь и распадается на символы.
4. Два слова «New York» объединены как «NewYork» без разделения.

Подсказка: Оверсегментация — слишком мелко, недосегментация — слишком грубо.

- **Оверсегментация** — Слово «playground» разбито как «pla y grounds».
- **Недосегментация** — Фраза «машинное обучение» закодирована как один токен «машинноеобучение».
- **Неизвестные токены (OOV)** — Слово «гиперпрограммирование» не входит в словарь и распадается на символы.
- **Неправильное объединение** — Два слова «New York» объединены как «NewYork» без разделения.

Шаг 08:

Миссия 8

 Сопоставьте метод кодирования с его характеристикой.

Метод:

Ответ:

- One-hot
- Токен ид (token id)
- Эмбединг (векторное представление)
- Subword backoff

Характеристика:

1. Каждый токен соответствует единичному вектору с одним «1».
2. Каждому токenu присваивается уникальный целочисленный идентификатор.
3. Токены кодируются как плотные векторы, несущие смысл.
4. Если токен неизвестен, используется ближайшая субсловная часть.

Подсказка: Один метод — дискретный, другой — числовой, третий — семантический.

- **One-hot** — Каждый токен соответствует единичному вектору с одним «1».
- **Токен ид (token id)** — Каждому токenu присваивается уникальный целочисленный идентификатор.

- **Эмбе́ддинг (векторное представление)** — Токены кодируются как плотные векторы, несущие смысл.
- **Subword backoff** — Если токен неизвестен, используется ближайшая субсловная часть.

🔮 Шаг 09:

Миссия 9

🧠 Упорядочите шаги процесса токенизации и кодирования смысла:

1. Исходный текст (фраза, предложение)
2. Токенизация (разбиение на токены, субслова или байты)
3. Преобразование токенов в идентификаторы (token ids)
4. Кодирование идентификаторов в векторы (эмбе́ддинги)
5. Использование эмбе́ддингов в модели для предсказания

Подсказка: Подумайте: сначала ломаем текст, потом даём числовые формы, затем смысловые представления.

Ответ:

- Исходный текст (фраза, предложение)
- Токенизация (разбиение на токены, субслова или байты)
- Преобразование токенов в идентификаторы (token ids)
- Кодирование идентификаторов в векторы (эмбе́ддинги)
- Использование эмбе́ддингов в модели для предсказания

🔮 Шаг 10:

Миссия 10

Упорядочите этапы, когда новая фраза неизвестна модели, и нужно обработать её:

1. Токенизация на субслова или байты
2. Проверка словаря субслов
3. Замена неизвестных субслов или fallback к байт-токенам
4. Получение token ids для известных частей
5. Кодирование в эмбе́ддинги и подача в модель
6. Интерпретация результата модели

Подсказка: Подумайте, как обрабатывать части, которых модель не знает, и как дальше интегрировать.

Ответ:

- Токенизация на субслова или байты
- Проверка словаря субслов
- Замена неизвестных субслов или fallback к байт-токенам
- Получение token ids для известных частей
- Кодирование в эмбе́ддинги и подача в модель
- Интерпретация результата модели

🔮 Шаг 11:

Миссия 11

🧠 Упорядочите этапы, когда инженер исправляет искажения смысла после токенизации:

1. Выявление токенов с сильными искажениями значений

2. Анализ, на каком этапе (токенизация, id, эмбединг) произошло искажение
3. Изменение токенизации или fallback-стратегии для этих токенов
4. Перекодирование и повторное тестирование значений
5. Мониторинг и корректировка на новых данных
6. Ретренинг или адаптация словаря в крайних случаях

Подсказка: Исправление смысла — это итеративный цикл анализа и правки.

Ответ:

- Выявление токенов с сильными искажениями значений
- Анализ, на каком этапе (токенизация, id, эмбединг) произошло искажение
- Изменение токенизации или fallback-стратегии для этих токенов
- Перекодирование и повторное тестирование значений
- Мониторинг и корректировка на новых данных
- Ретренинг или адаптация словаря в крайних случаях

Шаг 12:

Миссия 12

Упорядочите уровни сложности при понимании кодирования смысла токенов:

1. Простые слова из словаря (кот, дом)
2. Слова с суффиксами (программирование, программирования)
3. Составные слова (машиностроение, нейронсеть)
4. Неологизмы (гиперпрограмминг)
5. Слова + emoji / символы (hello😊, C++17)

Подсказка: Чем более экзотичен токен, тем сложнее правильно его обработать.

Ответ:

- Простые слова из словаря (кот, дом)
- Слова с суффиксами (программирование, программирования)
- Составные слова (машиностроение, нейронсеть)
- Неологизмы (гиперпрограмминг)
- Слова + emoji / символы (hello😊, C++17)

Шаг 13:

Миссия 13

🧠 Упорядочите шаги, если в модели появляются семантические ошибки после кодирования:

1. Проанализировать, какие токены регулярно неправильно интерпретируются
2. Вернуться к токенизации и проверить, как они разбиваются
3. Попробовать альтернативную токенизацию (exit to byte-level или подслова)
4. Перекодировать и проверить эмбединги этих токенов
5. Встроить fallback или дообучение эмбедингов
6. Мониторинг семантической стабильности на тестах

Подсказка: Корректировка смысла требует корректировки сразу нескольких этапов.

Ответ:

- Проанализировать, какие токены регулярно неправильно интерпретируются
- Вернуться к токенизации и проверить, как они разбиваются
- Попробовать альтернативную токенизацию (exit to byte-level или подслова)

- Перекодировать и проверить эмбединги этих токенов
- Встроить fallback или дообучение эмбедингов
- Мониторинг семантической стабильности на тестах

🔧 Шаг 14:

Миссия 14

Упорядочите фазы, когда команда добавляет новую поддержку токенов (новые слова, символы) в модель:

1. Сбор списка новых токенов из пользовательских данных
2. Анализ, как они разбиваются текущей токенизацией
3. **Решение** — добавить в словарь субслов или fallback к байтам
4. Обновление карты token ids / словаря модели
5. Перетренирование или дообучение эмбедингов для новых токенов
6. Тестирование и мониторинг на пользовательской выборке

Подсказка: Добавление токенов — это не просто словарь, это весь путь от токенов к смыслу.

Ответ:

- Сбор списка новых токенов из пользовательских данных
- Анализ, как они разбиваются текущей токенизацией
- **Решение** — добавить в словарь субслов или fallback к байтам
- Обновление карты token ids / словаря модели
- Перетренирование или дообучение эмбедингов для новых токенов
- Тестирование и мониторинг на пользовательской выборке

🔧 Шаг 15:

Миссия 15

🧠 Упорядочите действия инженера, если эмбединги двух семантически близких слов далеки в пространстве:

1. Выявить пары слов, которые должны быть близки по смыслу
2. **Проверить их токенизацию** — часто они разбиты по-разному
3. Обеспечить одинаковую субсловную основу там, где возможно
4. Переобучить или скорректировать эмбединги этих токенов вместе
5. Проверить семантическую близость на отложенной выборке
6. Внедрить регуляризацию близости в эмбединг-пространстве

Подсказка: Семантика — это не только кодировка, но и тренировочный сигнал.

Ответ:

- Выявить пары слов, которые должны быть близки по смыслу
- **Проверить их токенизацию** — часто они разбиты по-разному
- Обеспечить одинаковую субсловную основу там, где возможно
- Переобучить или скорректировать эмбединги этих токенов вместе
- Проверить семантическую близость на отложенной выборке
- Внедрить регуляризацию близости в эмбединг-пространстве

🔧 Шаг 16:

Миссия 16

Финальный квест: вы — инженер LLM, и ваша задача — построить полный конвейер токенизации и кодирования смысла, минимизируя ошибки. Упорядочите окончательные шаги:

1. Анализ корпуса текстов, выявление редких, новых токенов
2. Проектирование токенизации (subword + byte fallback)
3. Создание и расширение словаря токенов / субслов
4. Назначение token ids и обновление словаря модели
5. Инициализация или дообучение эмбеддингов для новых токенов
6. Тестирование на контрольных текстах с цифрами, именами, символами
7. Мониторинг семантических ошибок и периодическое обновление

Подсказка: Это ваш «боевой» pipeline: от корпуса до мониторинга на живых данных.

Ответ:

- Анализ корпуса текстов, выявление редких, новых токенов
- Проектирование токенизации (subword + byte fallback)
- Создание и расширение словаря токенов / субслов
- Назначение token ids и обновление словаря модели
- Инициализация или дообучение эмбеддингов для новых токенов
- Тестирование на контрольных текстах с цифрами, именами, символами
- Мониторинг семантических ошибок и периодическое обновление

Урок 4 - Визуализация слоёв и attention

Шаг 01:

Миссия 1

Анна запустила модель и увидела, что визуализация слоя показывает почти одинаковые картины для разных входов. Что могло пойти не так?

Подсказка: подумайте о весах слоёв и их случайной инициализации.

Ответ:

- **Веса слоя ещё не обучены, близки к нулю**
- Граф визуализации слоёв повреждён
- Входные данные оказались идентичны
- Код визуализации неправильно построен

Шаг 02:

Миссия 2

Боб хочет визуализировать attention-карты. Какой из вариантов лучше всего подойдёт?

Подсказка: attention — это матрица весов, показывающая связь между токенами.

Ответ:

- **Тепловая карта (heatmap) между токенами**
- Гистограмма частот attention
- 3D-граф сети слоёв
- Текстовый лог коэффициентов

Шаг 03:

Миссия 3

Визуализатор показывает attention лишь для первого слоя, а пользователь хочет увидеть его для последнего слоя. Что изменить?

Подсказка: проверьте параметр, через который задаётся номер слоя для визуализации.

Ответ:

- **Указать нужный индекс слоя в функцию визуализации**
- Изменить размер входной последовательности
- Перезапустить модель с другим seed
- Отключить dropout

🔮 Шаг 04:

Миссия 4

Чарли увидел, что attention-карты слишком «размыты», почти равномерны. Что может быть причиной?

Подсказка: подумайте о распределении весов и регуляризации.

Ответ:

- **Слишком сильная маска внимания или регуляризация**
- Неправильное кодирование токенов
- Ошибки при нормализации softmax
- Неправильно указан batch size

🔮 Шаг 05:

Миссия 5

Девелопер Дана хочет визуализировать активации (outputs) скрытого слоя как распределение значений. Какой график выбрать?

Подсказка: активации — набор чисел, удобно показать их распределение.

Ответ:

- **Гистограмма распределения значений активации**
- Сетевой граф слоёв
- Тепловая карта между нейронами
- Круговая диаграмма

🔮 Шаг 06:

Миссия 6

Задача: Сопоставьте тип визуализации с тем, что он показывает.

Типы визуализации:

Ответ:

- Heatmap attention
- Гистограмма активаций
- Scatter plot токенов
- Bar chart слоёв

Пояснения:

1. Отображает связи между токенами через веса внимания.
2. Показывает распределение значений нейронов.
3. Сравнивает два признака токенов на плоскости.

4. Сравнивает среднюю активность разных слоёв.

Подсказка: подумайте, какой визуализацией удобно показать связи, распределение, сравнение.

- **Heatmap attention** — Отображает связи между токенами через веса внимания.
- **Гистограмма активаций** — Показывает распределение значений нейронов.
- **Scatter plot токенов** — Сравнивает два признака токенов на плоскости.
- **Bar chart слоёв** — Сравнивает среднюю активность разных слоёв.

🔮 Шаг 07:

Миссия 7

Инженер Элли хочет сопоставить термины визуализации и их определения в контексте LLM:

Термины:

Ответ:

- Layer output
- Attention score
- Query-Key product
- Masking

Определения:

1. Результирующие активации слоя для каждого токена.
2. Произведение запросов и ключей до softmax.
3. Маска, запрещающая внимание к определённым токенам.
4. **После softmax** — нормированные веса внимания.

Подсказка: вспомните полный путь вычисления attention: query, key, score, softmax, mask.

- **Layer output** — Результирующие активации слоя для каждого токена.
- **Attention score** — После softmax — нормированные веса внимания.
- **Query-Key product** — Произведение запросов и ключей до softmax.
- **Masking** — Маска, запрещающая внимание к определённым токенам.

🔮 Шаг 08:

Миссия 8

Алиса строит сопоставление между способами визуализации и их ограничениями:

Варианты визуализации:

Ответ:

- Heatmap
- Гистограмма
- Scatter plot
- Bar chart

Ограничения / слабости:

1. Не показывает связи между элементами.
2. Трудно отобразить большое количество слоёв.
3. Может накладывать точки при плотных данных.
4. Цветовая интерпретация может сбиваться.

Подсказка: подумайте: heatmap — про связи, бар — про сравнение.

- **Heatmap** — Цветовая интерпретация может сбиваться.
- **Гистограмма** — Не показывает связи между элементами.
- **Scatter plot** — Может накладывать точки при плотных данных.
- **Bar chart** — Трудно отобразить большое количество слоёв.

🔮 Шаг 09:

Миссия 9

Улан хочет правильно упорядочить шаги визуализации attention между токенами:

1. Normalize (softmax) attention-scores
2. Compute query·key for каждый токен
3. Применить маску запрещённых связей
4. Построить heatmap по нормированным весам

Подсказка: вспомните порядок операции attention: score → mask → softmax → визуализация.

Ответ:

- Compute query·key for каждый токен
- Применить маску запрещённых связей
- Normalize (softmax) attention-scores
- Построить heatmap по нормированным весам

🔮 Шаг 10:

Миссия 10

При визуализации слоёв Бен хочет упорядочить этапы:

1. Сбор активаций слоя
2. Преобразование активаций в пригодный формат (reshape, reduce)
3. Выбор типа графика
4. Отображение графика пользователю

Подсказка: сначала получить данные, потом подготовить, выбрать формат и визуализировать.

Ответ:

- Сбор активаций слоя
- Преобразование активаций в пригодный формат (reshape, reduce)
- Выбор типа графика
- Отображение графика пользователю

🔮 Шаг 11:

Миссия 11

Кира должна упорядочить уровни агрегации визуализации:

1. По слою целиком (одно число на слой)
2. По нейрону в слое (высокое разрешение)
3. По группе нейронов (subset)
4. По токенам (attention к отдельному токenu)

Подсказка: от самого грубого к самому детальному уровень визуализации.

Ответ:

- По слою целиком (одно число на слой)
- По группе нейронов (subset)
- По нейрону в слое (высокое разрешение)
- По токенам (attention к отдельному токenu)

🔮 **Шаг 12:**

Миссия 12

Элвин хочет правильно упорядочить этапы построения full attention view:

1. Получить attention-weights из модели
2. Выбрать слои и головы, которые интересны
3. Нормировать и подготовить веса
4. Отобразить heatmap с токенами на осях

Подсказка: сначала извлекаем данные, потом фильтруем, нормируем, визуализируем.

Ответ:

- Получить attention-weights из модели
- Выбрать слои и головы, которые интересны
- Нормировать и подготовить веса
- Отобразить heatmap с токенами на осях

🔮 **Шаг 13:**

Миссия 13

В финальной части курса Алма хочет упорядочить уровни детализации визуализации:

1. Среднее значение внимания по всем головам и слоям
2. Внимание по каждому слою среднего по головам
3. Внимание по каждой голове отдельного слоя
4. Внимание по каждому токenu в каждой голове

Подсказка: от самого обобщённого к самому детальному уровню.

Ответ:

- Среднее значение внимания по всем головам и слоям
- Внимание по каждому слою среднего по головам
- Внимание по каждой голове отдельного слоя
- Внимание по каждому токenu в каждой голове

🔮 **Шаг 14:**

Миссия 14

После визуализации attention в модель включён dropout, и attention-карты сильно шумят. Что можно сделать?

Подсказка: подумайте об отключении стохастичности при визуализации.

Ответ:

- **Отключить dropout / режим обучения**
- Увеличить learning rate
- Уменьшить batch size
- Сгладить карты при помощи сгущения

🔗 Шаг 15:

Миссия 15

Сопоставьте тип проблем визуализации и способ их устранения:

Проблемы:

Ответ:

- Карты attention слишком бледные
- Слишком много слоёв на визуализации
- Нечитаемые подписи токенов
- Высокий шум из-за dropout

Способы устранения:

1. Увеличить контраст цветовой шкалы
2. Фильтровать или агрегировать слои
3. Показывать сокращённые токены или первые буквы
4. Отключить dropout на фазе визуализации

Подсказка: сопоставьте каждую проблему с тем, что её прямо исправляет.

- **Карты attention слишком бледные** — Увеличить контраст цветовой шкалы
- **Слишком много слоёв на визуализации** — Фильтровать или агрегировать слои
- **Нечитаемые подписи токенов** — Показывать сокращённые токены или первые буквы
- **Высокий шум из-за dropout** — Отключить dropout на фазе визуализации

🔗 Шаг 16:

Миссия 16

При визуализации attention слой содержит головы, но разработчик хочет усреднить по головам. Как это сделать?

Подсказка: подумайте о размерности attention-тензора.

Ответ:

- **Средний по размерности heads**
- Сложить и отобразить как одну голову
- Игнорировать все, кроме первой головы
- Выбрать максимум по головам

🔗 Шаг 17:

Миссия 17

Финальный вызов: упорядочите этапы продвинутой визуализации attention + активаций:

1. Извлечь и сохранить активации и attention-weights
2. Выбрать интересующие слои, головы и токены
3. Нормализация / агрегация / усреднение
4. Построить визуализацию (heatmap, гистограммы, scatter и т.д.)

Подсказка: это комбинированный процесс — сначала сбор, фильтрация, преобразование, затем визуализация.

Ответ:

- Извлечь и сохранить активации и attention-weights
- Выбрать интересующие слои, головы и токены

- Нормализация / агрегация / усреднение
- Построить визуализацию (heatmap, гистограммы, scatter и т.д.)

Урок 5 - 🕒 Квест расшифруй токенизацию Mistral и найди лишний токен

Шаг 01:

Миссия 10

Наташа хочет упорядочить шаги поиска лишнего токена:

1. Токенизировать строку
2. Сравнить с эталоном исходной строки
3. Найти токен, который не участвует в сборке строки
4. Проверить корректность оставшихся токенов

Подсказка: сначала разбить, затем анализировать и выявлять.

Ответ:

- Токенизировать строку
- Сравнить с эталоном исходной строки
- Найти токен, который не участвует в сборке строки
- Проверить корректность оставшихся токенов

Шаг 02:

Миссия 11

Глеб хочет упорядочить проверку токенов по степени подозрительности:

1. Совпадает ли с подстрокой исходной?
2. Количество символов необычное?
3. Необычные символы или пробел внутри?
4. Редкость токена в словаре?

Подсказка: сначала простые проверки, затем более «интеллектуальные».

Ответ:

- Совпадает ли с подстрокой исходной?
- Количество символов необычное?
- Необычные символы или пробел внутри?
- Редкость токена в словаре?

Шаг 03:

Миссия 12

Игорь хочет упорядочить этапы при сравнении двух токенизаций (А и В) для нахождения лишнего:

1. Сравнить общий набор токенов
2. Исключить совпадающие токены
3. **Остаток** — кандидат на лишний
4. Проверить, что остаток корректно не образует часть исходной строки

Подсказка: метод «разностного» анализа.

Ответ:

- Сравнить общий набор токенов
- Исключить совпадающие токены
- **Остаток** — кандидат на лишний
- Проверить, что остаток корректно не образует часть исходной строки

 Шаг 04:

Миссия 13

Юлия хочет упорядочить уровни детализации разбора токенов:

1. Токенизация всего текста
2. Анализ символов внутри токена
3. Сравнение с базовой токенизацией
4. Выделение лишнего токена

Подсказка: от общего к частному.

Ответ:

- Токенизация всего текста
- Анализ символов внутри токена
- Сравнение с базовой токенизацией
- Выделение лишнего токена

 Шаг 05:

Миссия 14

Сергей выбрал три токенизации и хочет упорядочить этапы, чтобы выделить лишний:

1. Сгенерировать три варианта токенов
2. Построить частотный индекс токенов
3. Найти токен с низкой частотой в сравнении
4. Проверить его корректность относительно строки

Подсказка: сначала собираем, затем статистика, затем исключение.

Ответ:

- Сгенерировать три варианта токенов
- Построить частотный индекс токенов
- Найти токен с низкой частотой в сравнении
- Проверить его корректность относительно строки

 Шаг 06:

Миссия 15

При сравнении токенов Марк обнаружил «<» как отдельный токен в одной вариации. Он уверен, что это лишний. Вы согласны?

Подсказка: символы пунктуации могут быть нормальными токенами.

Ответ:

- **Нет** — символ «<» может быть валидным токеном
- **Да** — скорее всего лишний
- Нужно проверить контекст
- Нельзя судить без модели

🔮 Шаг 07:

Миссия 16

Костя токенизировал строку дважды: вариант А и В. Он видит, что в варианте А есть токен «хуз» — лишний. Что лучше сделать?

Подсказка: проверьте, можно ли «хуз» составить из соседних токенов.

Ответ:

- **Попробовать склеить «хуз» из соседних токенов**
- Всегда удалять «хуз»
- Игнорировать отличие
- Добавить ещё токен похожий

🔮 Шаг 08:

Миссия 17

В финале Алиса хочет упорядочить весь алгоритм расшифровки лишнего токена в Mistral:

1. Получить несколько вариантов токенизации
2. Сравнить наборы токенов между вариантами
3. Выделить кандидата (лишний токен)
4. Проверить, что кандидат не участвует в реконструкции строки

Подсказка: это комбинированный метод: генерация, сравнение, исключение, верификация.

Ответ:

- Получить несколько вариантов токенизации
- Сравнить наборы токенов между вариантами
- Выделить кандидата (лишний токен)
- Проверить, что кандидат не участвует в реконструкции строки

🔮 Шаг 09:

Миссия 9

Вася пытается понять, что такое токенизация в модели Mistral. Что из ниже — корректное упрощённое определение?

Подсказка: Токен — не всегда отдельное слово, часто часть слова.

Ответ:

- **Преобразование текста в последовательность элементов (токенов) модели**
- Шифрование текста в двоичный код
- Сжатие текста без потерь

🔮 Шаг 10:

Миссия 10

Марина экспериментирует: при токенизации слова «running» Mistral разбивает его на «run» + «ning». Как это называется?

Подсказка: Часто токены — это субсловные фрагменты.

Ответ:

- **Subword токенизация (субсловная)**
- Словная токенизация
- Байт-уровневая токенизация

🔮 Шаг 11:

Миссия 11

Илья заметил, что некоторые символы (например, эмодзи 😊) превращаются в несколько токенов. Почему?

Подсказка: Эмодзи могут быть сложными в представлении, они часто занимают больше «места» в кодировке.

Ответ:

- **Потому что модель разбивает их на составные кодовые блоки (субтокены)**
- Потому что они удаляются токенизатором
- Потому что они считаются одним токеном всегда

🔮 Шаг 12:

Миссия 12

Катя удалила из словаря Mistral один токен «@@ing». Почему это может быть ошибкой?

Подсказка: Удаление часто используемых субтокенов мешает корректной генерации.

Ответ:

- **Потому что многие глаголы используют этот субтокен, модель начнёт генерировать ошибки**
- Потому что словарь не важен
- Потому что модель сама восстановит этот токен

🔮 Шаг 13:

Миссия 13

Сопоставьте термины (слева) и их значения (справа):

Ответ:

- Токен
 - Словарь (vocab)
 - Субтокен
 - Unknown token (UNK)
1. Элемент, не находящийся в словаре модели
 2. Минимальная единица, на которую разбивается текст
 3. Набор токенов, которые знает модель
 4. Часть слова, используемая в субсловной токенизации

Подсказка: Словарь — это «знание» модели, а субтокены — внутри слов.

- **Токен** — Минимальная единица, на которую разбивается текст
- **Словарь (vocab)** — Набор токенов, которые знает модель
- **Субтокен** — Часть слова, используемая в субсловной токенизации
- **Unknown token (UNK)** — Элемент, не находящийся в словаре модели

🔮 Шаг 14:

Миссия 14

При токенизации фразы «I love NLP» вы видите, что «love» разбилось на «lo» + «ve». Что это говорит о словаре?

Подсказка: Если таких субтокенов «lo», «ve» в словаре — значит словарь субтокенов минимален.

Ответ:

- **Словарь содержит субтокены, а не целые слова**
- Словарь содержит только слова целиком
- Это ошибка токенизатора

Шаг 15:

Миссия 15

Саша подозревает, что в словаре Mistral есть лишний токен, который почти никогда не используется. Как он может это проверить?

Подсказка: Частотность токенов в корпусе текста — важный показатель.

Ответ:

- **Посчитать частоту использования токена в большом корпусе и посмотреть, близка ли она к нулю**
- Удалить токен сразу и посмотреть, что произойдёт
- Добавить токен в словарь снова

Шаг 16:

Миссия 16

Сопоставьте причины (слева) и возможные последствия (справа), если в словаре есть лишний токен:

Ответ:

- Увеличение словаря без надобности
 - Снижение плотности распределения вероятностей
 - Увеличение расхода памяти
 - Замедление поиска токена
1. Меньше уверенности модели, что распределение «склеено» на полезные токены
 2. Потребление большего объёма оперативки и диска
 3. Увеличение времени на сопоставление токенов в словаре
 4. Словарь растёт, хотя многие токены неиспользуемы

Подсказка: Лишний токен — это не просто «высокий индекс», это нагрузка для модели.

- **Увеличение словаря без надобности** — Словарь растёт, хотя многие токены неиспользуемы
- **Снижение плотности распределения вероятностей** — Меньше уверенности модели, что распределение «склеено» на полезные токены
- **Увеличение расхода памяти** — Потребление большего объёма оперативки и диска
- **Замедление поиска токена** — Увеличение времени на сопоставление токенов в словаре

Шаг 17:

Миссия 17

Вы проверили частоты и нашли токен «Ġqz» с частотой почти ноль. Что логично сделать дальше?

Подсказка: Удаление должно быть обоснованным и безопасным.

Ответ:

- **Провести контрольный тест генерации без этого токена**
- Удалить любой случайный токен

- Удалить сразу много токенов
- Удалить «Ġqz» и считать, что всё работает

🔗 Шаг 18:

Миссия 18

После удаления предполагаемого лишнего токена генерация в некоторых фразах ухудшилась. Почему?

Подсказка: Даже редко используемые токены могут выполнять «заполнительную» функцию.

Ответ:

- **Поскольку модель ожидала его как «пробел» или маркер в некоторых случаях**
- Удаление токена не может ухудшить результат
- Это баг в токенизаторе, не из-за удаления

🔗 Шаг 19:

Миссия 19

Сопоставьте шаги анализа словаря (слева) и краткое описание (справа):

Ответ:

- Сбор корпуса текстов
 - Токенизация корпуса
 - Подсчёт частот токенов
 - Оценка влияния удаления
1. Посмотреть, как удаление повлияет на генерацию
 2. Разбить тексты на токены по словарю модели
 3. Собрать реальные тексты задачи
 4. Подсчитать, как часто каждый токен встречается

Подсказка: Анализ начинается с данных, проходит через токенизацию, статистику и тесты.

- **Сбор корпуса текстов** — Собрать реальные тексты задачи
- **Токенизация корпуса** — Разбить тексты на токены по словарю модели
- **Подсчёт частот токенов** — Подсчитать, как часто каждый токен встречается
- **Оценка влияния удаления** — Посмотреть, как удаление повлияет на генерацию

📖 Модуль 4 - 🧠 Миссия 4. Генетическая модификация модели (Fine-tuning)

📖 Урок 1 - Что такое Fine-tuning и LoRA

🔗 Шаг 01:

Миссия 1

Мария хочет дообучить модель под задачу классификации отзывов. Какой вариант метода fine-tuning будет менее ресурсоёмким?

Подсказка: Помните: LoRA часто экономит память и вычисления.

Ответ:

- **Использовать LoRA (Low-Rank Adaptation) на части слоёв**
- Дообучать все веса модели полностью

- Заморозить модель и не делать fine-tuning вовсе

🧙 Шаг 02:

Миссия 2

После выбора метода дообучения Иван задаётся вопросом: что важнее при fine-tuning — размер данных или качество данных?

Подсказка: Иногда меньше, но точнее — выигрывает.

Ответ:

- **Качество данных важнее, чем просто большой объём**
- Всегда нужно как можно больше данных, даже если они шумные
- **Данные не важны** — модель всё сама разберёт

🧙 Шаг 03:

Миссия 3

Оля хочет применять LoRA. Какое из следующих утверждений правильно относительно того, что делает LoRA?

Подсказка: LoRA вносит адаптационные матрицы в существующие слои.

Ответ:

- **LoRA добавляет низкоранговые матрицы, не изменяя основные веса модели**
- LoRA удаляет некоторые слои для экономии
- LoRA полностью заменяет слои модели новыми

🧙 Шаг 04:

Миссия 4

Сопоставьте понятия (слева) с их описаниями (справа):

Понятия:

Ответ:

- Adapter
- Prefix tuning
- LoRA
- Full fine-tuning

Определения:

1. Дообучение всех параметров модели
2. Добавление небольших слоёв или адаптеров между слоями модели
3. Настройка только префиксных токенов для трансформера
4. Добавление низкоранговых адаптаций к существующим весам

Подсказка: Каждый метод — компромисс между гибкостью и ресурсами.

- **Adapter** — Добавление небольших слоёв или адаптеров между слоями модели
- **Prefix tuning** — Настройка только префиксных токенов для трансформера
- **LoRA** — Добавление низкоранговых адаптаций к существующим весам
- **Full fine-tuning** — Дообучение всех параметров модели

🧙 Шаг 05:

Миссия 5

Вы запускаете fine-tuning с LoRA на задаче генерации кода. Какой риск может быть особенно серьёзным?

Подсказка: Обратите внимание на «конфликт» между прежними знаниями и дообучением.

Ответ:

- **Переобучение (catastrophic forgetting)** — модель забудет прежние знания
- Модель начнёт выдавать только один и тот же ответ на всё
- **LoRA не будет работать** — ничего не дообучится

🔮 Шаг 06:

Миссия 6

Максим собирает данные для дообучения. Какую стратегию стоит применить, чтобы не «загрязнить» модель некорректными примерами?

Подсказка: Лучше меньше, но качественно проверенных данных.

Ответ:

- **Фильтровать и вручную проверять примеры перед дообучением**
- Добавлять всё, что удалось собрать, без проверки
- Использовать синтетические ответы без верификации

🔮 Шаг 07:

Миссия 7

Ваша задача: сопоставить этапы цикла fine-tuning с их описаниями.

Этапы:

Ответ:

- Инициализация адаптеров / LoRA
- Заморозка базовых слоёв
- Обучение на новых данных
- Интеграция в сервис и валидация

Описания:

1. Подключение адаптивных компонентов
2. Оставить веса модели неизменными в части слоёв
3. Обновление параметров LoRA/адаптеров через градиенты
4. Проверка качества и деплой в продакшен

Подсказка: Подумайте, что происходит сначала, а что — в конце.

- **Инициализация адаптеров / LoRA** — Подключение адаптивных компонентов
- **Заморозка базовых слоёв** — Оставить веса модели неизменными в части слоёв
- **Обучение на новых данных** — Обновление параметров LoRA/адаптеров через градиенты
- **Интеграция в сервис и валидация** — Проверка качества и деплой в продакшен

🔮 Шаг 08:

Миссия 8

В критический момент вы замечаете: loss на валидации начинает расти, хотя на обучении продолжает падать. Что это может означать?

Подсказка: Это классическая ситуация при переобучении.

Ответ:

- **Модель начала переобучаться на тренировочном наборе**
- Данные валидации некорректны
- LoRA перестал работать и адаптации ломаются

Шаг 09:

Миссия 9

Сопоставьте термины (слева) и преимущества LoRA/адаптеров (справа):

Термины:

Ответ:

- Память
- Скорость обучения
- Совместимость
- Гибкость

Преимущества:

1. **Легкий вес** — мало дополнительных параметров
2. Можно обучать быстрее, чем всю модель целиком
3. Можно интегрировать с существующими моделями без изменений
4. Можно гибко настраивать адаптационные слои / ранги

Подсказка: Подумайте, какие аспекты оптимизированы при использовании LoRA.

- **Память** — Легкий вес — мало дополнительных параметров
- **Скорость обучения** — Можно обучать быстрее, чем всю модель целиком
- **Совместимость** — Можно интегрировать с существующими моделями без изменений
- **Гибкость** — Можно гибко настраивать адаптационные слои / ранги

Шаг 10:

Миссия 10

Вы решили протестировать разные ранги в LoRA ($\text{rank} = 4, 8, 16$). Что вы ожидаете при увеличении ранга?

Подсказка: Чем выше ранг — больше свободы адаптации, но и нагрузка.

Ответ:

- **При увеличении ранга модель может лучше подстроиться, но ресурсозатраты растут**
- При увеличении ранга нагрузка уменьшается
- Ранг не влияет ни на что

Шаг 11:

Миссия 11

Вы запустили fine-tuning с LoRA и адаптерами одновременно (гибрид). В чем может быть смысл такого подхода?

Подсказка: Комбинация может дать лучшие свойства от каждого метода.

Ответ:

- **Гибрид позволяет использовать силу адаптеров там, где нужен, и LoRA там, где эффективнее**

- Это просто усложняет архитектуру без смысла
- **LoRA и адаптеры несовместимы** — это ошибка

🔮 Шаг 12:

Миссия 12

Расположите шаги от запуска доёркировки LoRA в правильном порядке:

1. Инференс / тестирование качества модели
2. Обучение адаптационных параметров LoRA
3. Инициализация LoRA и интеграция в архитектуру модели
4. Подготовка и очистка данных для дообучения

Подсказка: Сначала нужно подготовить данные, затем встроить LoRA, потом обучать, и в конце — оценка.

Ответ:

- Подготовка и очистка данных для дообучения
- Инициализация LoRA и интеграция в архитектуру модели
- Обучение адаптационных параметров LoRA
- Инференс / тестирование качества модели

🔮 Шаг 13:

Миссия 13

Во время валидации вы замечаете, что с некоторыми входами модель выдает странные ответы. Что стоит проверить первым?

Подсказка: Иногда проблема не в модели, а в данных.

Ответ:

- **Качество и корректность валидационных данных и формат входов**
- Увеличить количество эпох до обучения
- Сразу менять ранг LoRA вверх

🔮 Шаг 14:

Миссия 14

Вы готовитесь к деплою модели с LoRA. Какой подход полезен, чтобы минимизировать риск на продакшене?

Подсказка: Используйте B-копии и “canary” развертывания.

Ответ:

- **Запустить canary-версию и постепенно переключать трафик**
- Сразу заменить на новую модель на весь трафик
- Не развёртывать никогда, держать экспериментальную модель в тени

🔮 Шаг 15:

Миссия 15

Расположите по порядку возможные причины, почему fine-tuning с LoRA мог “не сработать”:

1. Неправильное значение learning rate
2. Слишком мало данных или плохое качество данных
3. Ранг LoRA слишком низкий, адаптация слишком слабая

4. Недостаточное число эпох обучения

Подсказка: Все эти факторы влияют; подумайте, в каком порядке стоит проверять.

Ответ:

- Неправильное значение learning rate
- Слишком мало данных или плохое качество данных
- Ранг LoRA слишком низкий, адаптация слишком слабая
- Недостаточное число эпох обучения

Урок 2 - Подготовка данных и фильтрация текстов

Шаг 01:

Миссия 1

 Ваша команда хочет начать с фильтрации данных — какой первый критерий стоит проверить при подготовке корпуса для дообучения модели?

Подсказка: Начинайте с самых грубых, легко проверяемых признаков.

Ответ:

- **Удаление очень коротких или пустых строк**
- Анализ семантической близости
- Переведение текста в эмбединги и кластеризация

Шаг 02:

Миссия 2

Вы обнаружили, что в данных есть HTML-теги. Какое действие следует предпринять на этапе очистки?

Подсказка: Теги не несут смысловой нагрузки для языковой модели.

Ответ:

- **Удалить все HTML-теги**
- Оставить теги как есть
- Заменить теги на теги Markdown
- Закодировать теги как специальные токены

Шаг 03:

Миссия 3

 Вам прислали данные от внешнего источника с подозрением на спам и токсичность. Какой этап стоит сделать первым?

Подсказка: Проверьте неочевидное перед семантикой.

Ответ:

- **Фильтрация по стоп-словам и черному списку слов**
- Кластеризация на основе эмбедингов
- Запуск предсказателя токсичности
- Удаление пунктуации

Шаг 04:

Миссия 4

 Вы хотите нормализовать текст. Что из списка лучше применить на этом этапе?

Подсказка: Приведение к единому виду помогает сократить шум.

Ответ:

- **Приведение к нижнему регистру**
- Замена слов на эмбединги
- Удаление заглавных букв только в начале
- Разбивка на предложения без изменения регистра

🔮 Шаг 05:

Миссия 5

🔍 Вы хотите убрать «слишком длинные» примеры текста. Какой порог разумен на этапе подготовки (в смысле числа токенов)?

Подсказка: Люди редко пишут сверхдлинные абзацы в диалогах.

Ответ:

- **Около 512-2048 токенов**
- 10000-20000 токенов
- 50 токенов
- **Без порога** — пускай будут все

🔮 Шаг 06:

Миссия 6

🔍 Сопоставьте тип ошибки с причиной её появления при фильтрации:

Тип ошибки:

Ответ:

- Ложно положительный (over-filtering)
- Ложно отрицательный (under-filtering)
- Шум (noise)
- Бесполезный пример (useless sample)

Причины:

1. Пример удалён, хотя был качественным
2. Пример оставлен, хотя содержал спам
3. Пример слишком короткий или бессмысленный
4. Пример содержит случайный набор символов

Подсказка: Ложно положительный = «удалили что нельзя было».

- **Ложно положительный (over-filtering)** — Пример удалён, хотя был качественным
- **Ложно отрицательный (under-filtering)** — Пример оставлен, хотя содержал спам
- **Шум (noise)** — Пример содержит случайный набор символов
- **Бесполезный пример (useless sample)** — Пример слишком короткий или бессмысленный

🔮 Шаг 07:

Миссия 7

Сопоставьте метод очистки с его назначением:

Методы:

Ответ:

- Стоп-словный фильтр
- Регулярные выражения (regex)
- Качество языка (лингвистические правила)
- Метрика длины (токены/символы)

Назначения:

1. Удалять часто встречающиеся малоинформативные слова
2. Исключать нежелательные шаблоны как ссылки или скрипты
3. Отсеивать слишком короткие или шумные строки
4. Проверять соответствие языковой модели правилу грамматики или словарю

Подсказка: Подумайте, что делает regex — ищет шаблоны.

- **Стоп-словный фильтр** — Удалять часто встречающиеся малоинформативные слова
- **Регулярные выражения (regex)** — Исключать нежелательные шаблоны как ссылки или скрипты
- **Качество языка (лингвистические правила)** — Проверять соответствие языковой модели правилу грамматики или словарю
- **Метрика длины (токены/символы)** — Отсеивать слишком короткие или шумные строки

🔮 Шаг 08:

Миссия 8

📖 Сопоставьте формат данных с его преимуществом в подготовке корпуса:

Форматы:

Ответ:

- JSONL с полями «prompt»/«completion»
- Plain text, одна строка = один пример
- CSV с колонками source и target
- XML / HTML с тегами

Преимущества:

1. Легко парсить и сохранять дополнительные метаданные
2. **Минимальный формат** — просто текст
3. Поддерживает двунаправленные пары input-output
4. Может быть тяжеловесным из-за тегов и вложенности

Подсказка: JSONL часто используется в API OpenAI.

- **JSONL с полями «prompt»/«completion»** — Легко парсить и сохранять дополнительные метаданные
- **Plain text, одна строка = один пример** — Минимальный формат — просто текст
- **CSV с колонками source и target** — Поддерживает двунаправленные пары input-output
- **XML / HTML с тегами** — Может быть тяжеловесным из-за тегов и вложенности

🔮 Шаг 09:

Миссия 9

🔗 Сопоставьте шаг фильтрации с его характерным действием:

Шаги:

Ответ:

- Удаление дубликатов
- Нормализация юникода (NFKC/NFC)
- Очистка от странных символов
- Фильтрация по языковому детектору

Действия:

1. Оставить только примеры на нужном языке
2. Привести разные представления символов к единому виду
3. Удалить полностью идентичные строки
4. Исключить символы вне базового набора (не-печатные и мусорные)

Подсказка: Unicode-нормализация полезна при разных формах записи одинаковых символов.

- **Удаление дубликатов** — Удалить полностью идентичные строки
- **Нормализация юникода (NFKC/NFC)** — Привести разные представления символов к единому виду
- **Очистка от странных символов** — Исключить символы вне базового набора (не-печатные и мусорные)
- **Фильтрация по языковому детектору** — Оставить только примеры на нужном языке

 Шаг 10:

Миссия 10

🔧 Упорядочите этапы подготовки данных перед fine-tuning (от начала к концу):

1. Очистка HTML / тегов / скриптов
2. Нормализация регистра, Unicode
3. Фильтрация по длине / спаму
4. Удаление дубликатов
5. Проверка на язык / языковая фильтрация

Подсказка: Начинаем с базовой «чистки», затем нормализация, затем фильтры, затем финальные проверки.

Ответ:

- Очистка HTML / тегов / скриптов
- Нормализация регистра, Unicode
- Фильтрация по длине / спаму
- Удаление дубликатов
- Проверка на язык / языковая фильтрация

 Шаг 11:

Миссия 11

Упорядочите сценарии по степени риска: какой сценарий может нанести наибольший вред модели при плохой фильтрации?

1. Добавление токсичных фраз из пользовательского ввода
2. **Слишком агрессивная фильтрация** — удаление редких стилей речи
3. Оставление url-спама и скриптов
4. Оставление логов или ошибок кода как текстов
5. Нормальное пользовательское диалоговое содержание

Подсказка: Вредные данные > спам > удаление редких стилей > обычные тексты.

Ответ:

- Добавление токсичных фраз из пользовательского ввода
- Оставление url-спама и скриптов
- Оставление логов или ошибок кода как текстов
- **Слишком агрессивная фильтрация** — удаление редких стилей речи
- Нормальное пользовательское диалоговое содержание

Шаг 12:

Миссия 12

 Упорядочите форматы, исходя из удобства для моделей и людей (от наиболее удобного к наименее):

1. JSONL prompt/completion
2. CSV source-target
3. Plain text пер строкам
4. XML / HTML

Подсказка: Чем меньше лишнего — тем удобнее для обработки.

Ответ:

- JSONL prompt/completion
- CSV source-target
- Plain text пер строкам
- XML / HTML

Шаг 13:

Миссия 13

Упорядочите методы очистки от шума по «жёсткости» (от мягких к агрессивным):

1. Нормализация регистра / Unicode
2. Удаление стоп-слов
3. Удаление по длине
4. Фильтрация по языковому детектору
5. Удаление по черному списку токсичных фраз

Подсказка: Сначала мягкие преобразования, затем грубые фильтры.

Ответ:

- Нормализация регистра / Unicode
- Удаление стоп-слов
- Удаление по длине
- Фильтрация по языковому детектору
- Удаление по черному списку токсичных фраз

Шаг 14:

Миссия 14

🕒 При подготовке данных к гибриднему обучению (с нуля + дообучение), какую фильтрацию лучше выполнить позже, а не сразу?

Подсказка: Иногда лучше сначала сохранить как можно больше примеров, затем фильтровать.

Ответ:

- **Кластерный отбор на основе семантики и удаление редких стилей**
- Удаление пустых строк
- Удаление HTML-тегов сразу

Шаг 15:

Миссия 15

🔑 При подготовке данных для диалоговой модели вы хотите держать контекст. Какую фильтрацию стоит делать осторожно?

Подсказка: Контекст важен, не ломайте последовательности.

Ответ:

- **Не удалять отдельные реплики, которые разрывают цепочку диалога**
- Фильтрация по длине внутри каждого сообщения
- Нормализация регистра

Шаг 16:

Миссия 16

🔑 Сопоставьте признаки примера с решением: какой признак, какое действие?

Признаки:

Ответ:

- Слишком частый шаблон «Buy now!»
- Много ссылок (URL)
- Низкая языковая модельная вероятность (anomaly)
- Текст на другом языке

Действия:

1. Удалить как спам / рекламный шаблон
2. Удалить пример как пустой или мусорный
3. Удалить или перевести на нужный язык
4. Проверка вручную или удаление по порогу вероятности

Подсказка: «Buy now» — классическая рекламная фраза.

- **Слишком частый шаблон «Buy now!»** — Удалить как спам / рекламный шаблон
- **Много ссылок (URL)** — Удалить пример как пустой или мусорный
- **Низкая языковая модельная вероятность (anomaly)** — Проверка вручную или удаление по порогу вероятности
- **Текст на другом языке** — Удалить или перевести на нужный язык

Шаг 17:

Миссия 17

🔑 Финальный квест: упорядочите компоненты фильтрационной пайплайна (от начала до конца):

1. Нормализация текста (регистр, Unicode)
2. Удаление тегов, кодовых фрагментов (HTML, скрипты)
3. Фильтрация по длине / пустоте / мусору
4. Фильтрация по языку или языковой модели
5. Удаление дубликатов

6. Фильтрация по стоп-словам, шаблонам, токсичности
7. Ручная проверка спорных примеров

Подсказка: Подумайте, что логично делать автоматически, а что — вручную в конце.

Ответ:

- Нормализация текста (регистр, Unicode)
- Удаление тегов, кодовых фрагментов (HTML, скрипты)
- Фильтрация по длине / пустоте / мусору
- Фильтрация по языку или языковой модели
- Удаление дубликатов
- Фильтрация по стоп-словам, шаблонам, токсичности
- Ручная проверка спорных примеров

Урок 3 - Обучение с помощью PEFT unsloth

Шаг 01:

Миссия 1

Вася решил применить PEFT (Parameter-Efficient Fine-Tuning). Что из перечисленного — ключевая идея PEFT?

Подсказка: PEFT старается менять минимум параметров.

Ответ:

- **Модифицировать лишь небольшую часть весов или добавлять адаптеры**
- Дообучать всю модель целиком
- Заморозить модель и не изменять ничего

Шаг 02:

Миссия 2

Лена хочет выбрать между LoRA и адаптерами. Что преимущественно у LoRA?

Подсказка: LoRA — Low Rank адаптация.

Ответ:

- **Малое число дополнительных параметров, матрицы низкого ранга**
- Полная копия слоёв модели
- Обучение с нуля без использования базовой модели

Шаг 03:

Миссия 3

 При использовании unsloth инженер хочет адаптировать под новые задачи. Что важнее задать корректно?

Подсказка: Уточните, какие части должны быть адаптированы.

Ответ:

- **Какие слои будут «размораживаться» и какие — фиксированы**
- Полный режим обучения все параметры
- Не учитывать слоёв модели вообще

Шаг 04:

Миссия 4

📖 Сопоставьте технику PEFT с её описанием:

Техники:

Ответ:

- LoRA
- Adapter (адаптеры)
- Prefix Tuning
- Prompt Tuning

Описание:

1. Вставка небольших слоёв внутрь слоёв Transformer
2. Обучение вектора префикса, который добавляется к входу
3. Обучение только токен-промпта, не трогая веса модели
4. Низкоранговая матричная адаптация весов

Подсказка: Помните: LoRA = low-rank, prompt tuning — минимальное вмешательство.

- **LoRA** — Низкоранговая матричная адаптация весов
- **Adapter (адаптеры)** — Вставка небольших слоёв внутрь слоёв Transformer
- **Prefix Tuning** — Обучение вектора префикса, который добавляется к входу
- **Prompt Tuning** — Обучение только токен-промпта, не трогая веса модели

💡 Шаг 05:

Миссия 5

Сопоставьте термин с его ролью в PEFT/Unsloth:

Термины:

Ответ:

- Freeze (заморозка)
- Unfreeze (разморозка)
- Fine-tune head
- Rank-decomposition

Роли:

1. Обучать только верхний слой классификатора или генератора
2. Оставлять веса неизменными в слоях модели
3. Разрешить изменение ранее фиксированных слоёв
4. Разложение весовых матриц на низкоранговые компоненты

Подсказка: Freeze / unfreeze — противоположные операции.

- **Freeze (заморозка)** — Оставлять веса неизменными в слоях модели
- **Unfreeze (разморозка)** — Разрешить изменение ранее фиксированных слоёв
- **Fine-tune head** — Обучать только верхний слой классификатора или генератора
- **Rank-decomposition** — Разложение весовых матриц на низкоранговые компоненты

💡 Шаг 06:

Миссия 6

⚙ Сопоставьте параметр PEFT-конфигурации с его значением:

Параметры:

Ответ:

- r (rank)
- alpha
- dropout
- target_modules

Значения:

1. Какие слои модели подлежат адаптации (например, «q_proj», «v_proj»)
2. Скалирование низкоранговых матриц
3. Вероятность дропаут при адаптерах
4. Ранг матриц LoRA (малая величина)

Подсказка: r — это rank, dropout — вероятность, alpha — масштабирование.

- **r (rank)** — Ранг матриц LoRA (малая величина)
- **alpha** — Скалирование низкоранговых матриц
- **dropout** — Вероятность дропаут при адаптерах
- **target_modules** — Какие слои модели подлежат адаптации (например, «q_proj», «v_proj»)

 Шаг 07:

Миссия 7

Сопоставьте метод регуляризации с эффектом при fine-tuning:

Методы:

Ответ:

- Dropout внутри адаптера
- Weight decay на адаптеры
- Gradient clipping
- Gradient checkpointing

Эффекты:

1. Ограничение величины градиентов
2. Снижение переобучения адаптеров
3. Сохранять память при обратном проходе
4. Снижение избыточной адаптации и стабилизация

Подсказка: Weight decay — стандартный L2-регуляризатор.

- **Dropout внутри адаптера** — Снижение избыточной адаптации и стабилизация
- **Weight decay на адаптеры** — Снижение переобучения адаптеров
- **Gradient clipping** — Ограничение величины градиентов
- **Gradient checkpointing** — Сохранять память при обратном проходе

 Шаг 08:

Миссия 8

🗒 Упорядочите этапы настройки PEFT-обучения (от начала к концу):

1. Выбор target_modules и конфигурации r/alpha/dropout
2. Заморозка базовой модели (freeze)
3. Инициализация адаптеров или LoRA-модулей
4. Проход forward + backward на тренировочных данных

5. Обновление только адаптеров / LoRA-параметров
6. Оценка на валидации и сохранение чекпоинтов

Подсказка: Сначала конфигурация, затем freeze, затем инициализация, потом обучение и оценка.

Ответ:

- Выбор target_modules и конфигурации r/alpha/dropout
- Заморозка базовой модели (freeze)
- Инициализация адаптеров или LoRA-модулей
- Проход forward + backward на тренировочных данных
- Обновление только адаптеров / LoRA-параметров
- Оценка на валидации и сохранение чекпоинтов

Шаг 09:

Миссия 9

✂ Упорядочите виды токенов, к которым чаще всего применяют адаптерные методы (от наиболее вероятного к менее):

1. Внутренние представления self-attention (q, k, v)
2. Выходной слой logits / head
3. Embedding слой input tokens
4. Позиционные эмбединги

Подсказка: Чаще адаптируют внимание, реже — позиционные.

Ответ:

- Внутренние представления self-attention (q, k, v)
- Выходной слой logits / head
- Embedding слой input tokens
- Позиционные эмбединги

Шаг 10:

Миссия 10

🐞 Петя запустил обучение с PEFT: но ошибка растёт. Что он мог забыть сделать?

Подсказка: Убедитесь, что только адаптеры обучаются.

Ответ:

- **Не заморозил базовую модель, и она тоже обновляется**
- Не указал целевой модуль (target_modules)
- Не инициализировал адаптеры

Шаг 11:

Миссия 11

📊 Наташа хочет контролировать размер модели после PEFT. Что полезно отслеживать?

Подсказка: Сколько дополнительных параметров добавляется?

Ответ:

- **Количество обучаемых параметров адаптера / LoRA**
- Общее количество токенов в данных
- Время загрузки модели из файлов

🔮 Шаг 12:

Миссия 12

🔗 Сопоставьте проблему обучения PEFT с решением:

Проблемы:

Ответ:

- Градиенты расходятся (напряжение)
- Переобучение адаптеров
- Стабильность при больших шагах обучения
- Пересечение слоёв с параметрами модели

Решения:

1. Gradient clipping или понижение learning rate
2. Добавить weight decay или dropout в адаптеры
3. Использовать регуляризацию, раннюю остановку
4. Убедиться, что target_modules выбраны корректно, не дублируются

Подсказка: Переобучение — когда адаптер подгоняет под шум.

- **Градиенты расходятся (напряжение)** — Gradient clipping или понижение learning rate
- **Переобучение адаптеров** — Добавить weight decay или dropout в адаптеры
- **Стабильность при больших шагах обучения** — Использовать регуляризацию, раннюю остановку
- **Пересечение слоёв с параметрами модели** — Убедиться, что target_modules выбраны корректно, не дублируются

🔮 Шаг 13:

Миссия 13

🔗 Сопоставьте критерий валидации с тем, что он проверяет при PEFT:

Критерии:

Ответ:

- Loss на валидации
- Метрика качества (например, accuracy, BLEU)
- Количество обновлённых параметров
- Скорость обучения (learning dynamics)

Что проверяет:

1. Как хорошо модель обобщается на новых данных
2. Какова эффективность обучения адаптера (стабильность)
3. Насколько большой прирост в параметрах вы сделали
4. Наблюдение за колебаниями и скачками loss / метрики

Подсказка: Loss + метрика — стандартные индикаторы качества.

- **Loss на валидации** — Как хорошо модель обобщается на новых данных
- **Метрика качества (например, accuracy, BLEU)** — Наблюдение за колебаниями и скачками loss / метрики
- **Количество обновлённых параметров** — Насколько большой прирост в параметрах вы сделали

- **Скорость обучения (learning dynamics)** — Какова эффективность обучения адаптера (стабильность)

🔮 Шаг 14:

Миссия 14

🔧 Упорядочите шаги переноса модели PEFT на прод (от подготовки до развёртывания):

1. Сохранение адаптера / LoRA чекпоинта
2. Интеграция адаптера в базовую модель (слияние весов или runtime wrap)
3. Тестирование итоговой модели на контрольных сценариях
4. Оптимизация inference (сжатие, ускорение)
5. Развёртывание на продовую инфраструктуру

Подсказка: Сначала сохранить, затем встроить, проверить, оптимизировать и запустить.

Ответ:

- Сохранение адаптера / LoRA чекпоинта
- Интеграция адаптера в базовую модель (слияние весов или runtime wrap)
- Тестирование итоговой модели на контрольных сценариях
- Оптимизация inference (сжатие, ускорение)
- Развёртывание на продовую инфраструктуру

🔮 Шаг 15:

Миссия 15

🔧 Упорядочите уровни вмешательства в модель (от наименее к наиболее):

1. Prompt Tuning
2. Prefix Tuning
3. Adaptation через адаптеры
4. LoRA (низкоранговые изменения весов)
5. Полный fine-tuning всей модели

Подсказка: Чем больше слоёв и параметров затрагиваются, тем выше вмешательство.

Ответ:

- Prompt Tuning
- Prefix Tuning
- Adaptation через адаптеры
- LoRA (низкоранговые изменения весов)
- Полный fine-tuning всей модели

🔮 Шаг 16:

Миссия 16

🧠 В продовой конфигурации инженер хочет уменьшить накладные расходы от адаптеров. Что можно сделать?

Подсказка: Подумайте про merge/свёртку или оптимизацию рантайма.

Ответ:

- **Слить адаптеры в базовые веса (merge) для inference**
- Оставить два набора весов и выбирать динамически
- Запускать весь адаптер на каждой итерации inference

Шаг 17:

Миссия 17

🏁 Финальный квест: упорядочите основные аспекты успеха PEFT-дообучения (от базового к комплексному):

1. Выбор target_modules и конфигурации адаптера
2. Заморозка не нужных слоёв модели
3. Обучение только адаптеров / LoRA-параметров
4. Валидация и контроль метрик качества
5. Слияние адаптера при inference (merge) или легкое включение
6. Мониторинг производительности и drift после развёртывания
7. Управление регуляризацией и гиперпараметрами по ходу

Подсказка: Подумайте про жизненный цикл от настройки до эксплуатации и мониторинга.

Ответ:

- Выбор target_modules и конфигурации адаптера
- Заморозка не нужных слоёв модели
- Обучение только адаптеров / LoRA-параметров
- Валидация и контроль метрик качества
- Слияние адаптера при inference (merge) или легкое включение
- Мониторинг производительности и drift после развёртывания
- Управление регуляризацией и гиперпараметрами по ходу

Урок 4 - Инъекция LoRA в Ollama

Шаг 01:

Миссия 1

👤 Инженер Маша работает над инъекцией LoRA в модель Ollama. Что первое хочет проверить?

Подсказка: Начинайте с самого базового шага — проверка совместимости форматов.

Ответ:

- **Совместимость LoRA-формата с версией Ollama**
- Оптимизация скорости inference
- Компиляция ядра LoRA
- Удаление старых слоёв модели

Шаг 02:

Миссия 2

Иван заметил, что LoRA-файл весит слишком много. Что логично сделать?

Подсказка: Иногда разумная компрессия важнее предельной точности во всех случаях.

Ответ:

- **Применить квантование LoRA**
- Разделить файл на части
- Сжать .zip архивом и подгружать на лету
- Удалить половину весов вручную

🔮 Шаг 03:

Миссия 3

🧐\ Пётр хочет протестировать, как встроенный LoRA влияет на латентное пространство. Что стоит сделать первым?

Подсказка: Начните с визуализации эмбеддингов до и после.

Ответ:

- **Сделать t-SNE проекцию эмбеддингов**
- Запустить полный fine-tuning
- Удалить LoRA для теста
- Поменять архитектуру модели

🔮 Шаг 04:

Миссия 4

\ После инъекции LoRA на продакшене появилась деградация качества на длинных запросах. Что наиболее вероятно поможет?

Подсказка: LoRA может быть менее устойчив к длинным сессиям.

Ответ:

- **Увеличить контекстные окна и применять постепенную адаптацию**
- Отключить LoRA на длинных запросах полностью
- Применить агрессивную дропаут-регуляцию
- Сократить длину шага обучения

🔮 Шаг 05:

Миссия 5

🗣 Сопоставьте термин в технологии LoRA с его описанием:

Термины:

Ответ:

- Rank
- Adapter
- Delta-weights
- Merge LoRA

Определения:

1. Разница весов, добавленная к базовой модели
2. Малые слои, вставляемые между слоями базовой сети
3. Объединение LoRA-веса с базовой моделью для финального веса
4. Число базовых направлений в Low-rank аппроксимации

Подсказка: Rank — это размерность низкого ранга, adapter вводит добавочные слои.

- **Rank** — Число базовых направлений в Low-rank аппроксимации
- **Adapter** — Малые слои, вставляемые между слоями базовой сети
- **Delta-weights** — Разница весов, добавленная к базовой модели
- **Merge LoRA** — Объединение LoRA-веса с базовой моделью для финального веса

Шаг 06:

Миссия 6

 Сопоставьте шаги инъекции LoRA в Ollama с их назначением:

Шаги:

Ответ:

- Загрузка .safetensors файла
- Интерпретация метаданных
- Привязка LoRA слоёв
- Сохранение объединённой модели

Назначения:

1. Интеграция LoRA в архитектуру модели
2. Подгрузка весов
3. Сохранение итогового состояния модели
4. Проверка совместимости версии и формата

Подсказка: Сначала читаем файл → метаданные → встраиваем → сохраняем.

- **Загрузка .safetensors файла** — Подгрузка весов
- **Интерпретация метаданных** — Проверка совместимости версии и формата
- **Привязка LoRA слоёв** — Интеграция LoRA в архитектуру модели
- **Сохранение объединённой модели** — Сохранение итогового состояния модели

Шаг 07:

Миссия 7

 Сопоставьте эффект настройки LoRA с потенциальным изменением производительности:

Эффекты:

Ответ:

- Снижение расхода памяти
- Увеличение задержки inference
- Улучшение качества генерации на тонких доменах
- Повышенная нестабильность при длинных сессиях

Потенциальные причины:

1. Дополнительные вычисления на адаптерах
2. LoRA адаптирован к специфику задачи
3. Модули LoRA удалены из main-памяти
4. Накопленные ошибки при длинном контексте

Подсказка: LoRA уменьшает память, но добавляет вычисления; при длительных цепочках накапливаются ошибки.

- **Снижение расхода памяти** — Модули LoRA удалены из main-памяти
- **Увеличение задержки inference** — Дополнительные вычисления на адаптерах
- **Улучшение качества генерации на тонких доменах** — LoRA адаптирован к специфику задачи
- **Повышенная нестабильность при длинных сессиях** — Накопленные ошибки при длинном контексте

🔧 Шаг 08:

Миссия 8

👉 Упорядочите шаги от простого к сложному, которые нужно проверить при диагностике ошибок после инъекции LoRA:

1. Проверка версии Ollama-движка
2. Визуализация градиентов в LoRA-модулях
3. Проверка формата .safetensors и метаданных
4. Проверка поведения базовой модели без LoRA
5. Сравнение выхода модели до и после

Подсказка: Начинайте с базового («что работает без LoRA»), затем по форматам и версиям, потом сравнение, затем градиенты.

Ответ:

- Проверка поведения базовой модели без LoRA
- Проверка версии Ollama-движка
- Проверка формата .safetensors и метаданных
- Сравнение выхода модели до и после
- Визуализация градиентов в LoRA-модулях

🔧 Шаг 09:

Миссия 9

📐 Упорядочите конфигурации LoRA по возрастанию риска переобучения:

1. rank = 1, alpha = 4
2. rank = 4, alpha = 16
rank = 8, alpha = 32
3. rank = 16, alpha = 64
4. rank = 32, alpha = 128

Подсказка: Чем выше rank и alpha, тем сильнее влияние LoRA — выше риск переобучения.

Ответ:

- rank = 1, alpha = 4
- rank = 4, alpha = 16
- rank = 8, alpha = 32
- rank = 16, alpha = 64
- rank = 32, alpha = 128

🔧 Шаг 10:

Миссия 10

🔧 Упорядочите этапы отладки LoRA, если модель отвечает странно:

1. Отключение LoRA (возврат к базовой модели)
2. Анализ входных данных (валидность токенов)
3. Генерация простых запросов-контролей
4. Сравнение ответов с эталонной моделью
5. Измерение влияния LoRA слоёв по слоям

Подсказка: Сначала проверить данные, затем базовую модель, затем контролы и сравнения, затем детальный layer-level анализ.

Ответ:

- Анализ входных данных (валидность токенов)
- Отключение LoRA (возврат к базовой модели)
- Генерация простых запросов-контролей
- Сравнение ответов с эталонной моделью
- Измерение влияния LoRA слоёв по слоям

 Шаг 11:

Миссия 11

Марина наблюдает, что при одном и том же prompt с LoRA модель генерирует разный ответ каждый раз. Что может быть причиной?

Подсказка: LoRA вводит адаптивные модули — они могут дополнительно шуметь или быть нестабильными.

Ответ:

- **Случайность / стохастичность адаптеров LoRA**
- Флюктуации базовой модели
- Ошибка токенизации
- Плохой seed во всем pipeline

 Шаг 12:

Миссия 12

🕒 После внедрения LoRA инженеры решили fine-tune только часть слоёв. Почему?

Подсказка: LoRA — это метод адаптации, не полное переобучение.

Ответ:

- **Чтобы уменьшить риск переобучения и ускорить обучение**
- Чтобы обойти защита модели
- Чтобы полностью удалить базовые веса
- Чтобы увеличить размер модели

 Шаг 13:

Миссия 13

Петя предлагает использовать negative LoRA (отрицательные веса) как корректировку. Что важно проверить прежде всего?

Подсказка: Убедитесь, что результирующее weight не выходит за допустимые границы.

Ответ:

- **Что итоговая модель не содержит нулевых или отрицательных весов**
- Что negative LoRA равен базе
- Что negative LoRA применится ко всем слоям
- Что negative LoRA увеличивает память

 Шаг 14:

Миссия 14

Сопоставьте ситуации с подходящими стратегиями при использовании LoRA:

Ситуации:

Ответ:

- Быстрый prototyping на малых данных
- Продакшен-модель, стабильность важнее качества
- Специфичный домен с узкой тематикой
- Очень длинные контексты (1000+ токенов)

Стратегии:

1. Низкий rank LoRA, агрессивный дропаут
2. Merge LoRA с базовой моделью и отключение адаптеров
3. Повышенный rank и обучение с большим alpha
4. Избегать LoRA, оставить только базовый fine-tuning

Подсказка: При длинных контекстах адаптеры могут мешать; в продакшене часто проще объединить заранее.

- **Быстрый prototyping на малых данных** — Низкий rank LoRA, агрессивный дропаут
- **Продакшен-модель, стабильность важнее качества** — Merge LoRA с базовой моделью и отключение адаптеров
- **Специфичный домен с узкой тематикой** — Повышенный rank и обучение с большим alpha
- **Очень длинные контексты (1000+ токенов)** — Избегать LoRA, оставить только базовый fine-tuning

🔮 Шаг 15:

Миссия 15

📖 Сопоставьте термин с его смыслом в контексте Ollama + LoRA:

Термины:

Ответ:

- Context window
- Prompt engineering
- Adapter fusion
- Weight decay

Определения:

1. Метод объединения адаптивных модулей в базовую модель
2. Регуляризация градиентов в fine-tuning
3. Размер контекста, который модель видит за раз
4. Формирование запроса так, чтобы LoRA активировался оптимально

Подсказка: Prompt engineering влияет как активируются адаптеры, а fusion — это слияние.

- **Context window** — Размер контекста, который модель видит за раз
- **Prompt engineering** — Формирование запроса так, чтобы LoRA активировался оптимально
- **Adapter fusion** — Метод объединения адаптивных модулей в базовую модель
- **Weight decay** — Регуляризация градиентов в fine-tuning

🔮 Шаг 16:

Миссия 16

⚙ Упорядочите методы борьбы с дестабилизацией после внедрения LoRA от самым слабым к самым мощным:

1. Уменьшение learning rate
2. Увеличение регуляризации (weight decay)
3. Partial freeze (заморозка части слоёв)
4. Grad clipping на адаптерах
5. Early stopping

Подсказка: Сначала снизить агрессивность обучения, затем контролировать градиенты, далее заморозка и останов.

Ответ:

- Уменьшение learning rate
- Увеличение регуляризации (weight decay)
- Grad clipping на адаптерах
- Partial freeze (заморозка части слоёв)
- Early stopping

🔮 **Шаг 17:**

Миссия 17

Упорядочите этапы проверки качества после полной интеграции LoRA:

1. Сравнение метрик (BLEU / ROUGE / perplexity)
2. Тестирование на «edge-case» запросах
3. Проверка латентных представлений
4. Нагрузочное тестирование с несколькими параллельными запросами
5. Ручное прочтение сгенерированных ответов экспертами

Подсказка: Начинайте с количественных метрик, потом углубляйтесь в edge-case, потом нагрузка и ручка.

Ответ:

- Сравнение метрик (BLEU / ROUGE / perplexity)
- Проверка латентных представлений
- Тестирование на «edge-case» запросах
- Нагрузочное тестирование с несколькими параллельными запросами
- Ручное прочтение сгенерированных ответов экспертами

📖 **Урок 5 - 🧙 Квест обучи модель понимать HTML и интегрируй в Ollama**

🔮 **Шаг 01:**

Миссия 1

Алекс начинает курс по обучению модели HTML-парсингу. С чего он должен начать?

Подсказка: Сначала стоит удостовериться, что модель вообще «видит» теги как отдельные токены.

Ответ:

- Проверить токенизацию HTML-тегов моделью
- Запустить полный fine-tuning сразу
- Инжектировать CSS как текст
- Игнорировать теги, работать только с текстом

🔮 Шаг 02:

Миссия 2

👤 Марина хочет, чтобы модель различала `
` и `

Ответ:

- Провести mini-классификацию тегов по типу
- Непосредственно объединить с Ollama
- Обучить CSS синтаксис параллельно
- Удалить все атрибуты тегов

🔮 Шаг 03:

Миссия 3

👤 Иван заметил, что модель путает вложенные теги. Как можно помочь?

Подсказка: Учитывание структуры важно — не просто линейный текст.

Ответ:

- Добавить примеры с вложенной структурой тегов в обучающую выборку
- Просто увеличить batch size
- Убрать все вложенные примеры
- Конвертировать HTML в plain текст

🔮 Шаг 04:

Миссия 4

👤 Антон хочет интегрировать свою модель в Ollama после обучения. Что первым делом он должен проверить?

Подсказка: Убедись, что формат модели поддержит Ollama.

Ответ:

- Совместимость формата модели (safetensors / pytorch) с Ollama
- Наличие CSS-поддержки
- Размер входного слоя HTML
- Удалить все теги перед генерацией

🔮 Шаг 05:

Миссия 5

Сопоставьте этапы обучения модели HTML с их назначением:

Этапы:

Ответ:

- Сбор датасета с HTML и текстом
- Токенизация с учётом тегов
- Fine-tuning на HTML-примерах
- Валидация на unseen HTML

Назначения:

1. Проверка обобщения на новые HTML-страницы
2. Превратить raw HTML в токены, включая теги

3. Применить обучение модели к HTML-примерам
4. Получить данные и примеры для обучения

Подсказка: Сначала нужны данные, потом токенизация, затем обучение, затем проверка.

- **Сбор датасета с HTML и текстом** — Получить данные и примеры для обучения
- **Токенизация с учётом тегов** — Превратить raw HTML в токены, включая теги
- **Fine-tuning на HTML-примерах** — Применить обучение модели к HTML-примерам
- **Валидация на unseen HTML** — Проверка обобщения на новые HTML-страницы

🔮 Шаг 06:

Миссия 6

🔺 Сопоставьте критерий качества HTML-понимания модели с метрикой:

Критерии:

Ответ:

- Правильная структура DOM
- Корректные атрибуты элементов
- Сохранение текста без искажения
- Обработка вложенности тегов

Метрики:

1. BLEU / ROUGE на HTML-генерацию
2. Exact match атрибутов
3. Perplexity на текстовом содержимом
4. Tree edit distance между деревьями DOM

Подсказка: Структура — дерево, атрибуты — точные значения.

- **Правильная структура DOM** — Tree edit distance между деревьями DOM
- **Корректные атрибуты элементов** — Exact match атрибутов
- **Сохранение текста без искажения** — Perplexity на текстовом содержимом
- **Обработка вложенности тегов** — BLEU / ROUGE на HTML-генерацию

🔮 Шаг 07:

Миссия 7

✂ Упорядочите шаги от простого к сложному при интеграции HTML-модели в Ollama:

1. Подготовка модели в поддерживаемом формате
2. Тестирование небольших HTML-примеров в Ollama
3. Обработка ошибок токенизации и паддинга
4. Нагрузочное тестирование с параллельными запросами
5. Мониторинг качества генерации на веб-контенте

Подсказка: Начните с базовой проверки, потом растите нагрузку и мониторинг.

Ответ:

- Подготовка модели в поддерживаемом формате
- Тестирование небольших HTML-примеров в Ollama
- Обработка ошибок токенизации и паддинга
- Нагрузочное тестирование с параллельными запросами
- Мониторинг качества генерации на веб-контенте

🔔 Шаг 08:

Миссия 8

✎ Упорядочите уровни сложности HTML-примера от простого к более сложному:

1. ` Hello world `
2. ` Text `
3. ` Item1 Item2 `
4. ` A B `
5. ` ... ... `

Подсказка: Начинайте с простых тегов, затем вложение, списки, таблицы, комплексная смесь.

Ответ:

- ` Hello world `
- ` Text `
- ` Item1 Item2 `
- ` A B `
- ` ... ... `

🔔 Шаг 09:

Миссия 9

🔗 После интеграции HTML-модель иногда генерирует JSON вместо HTML. Что наиболее вероятно?

Подсказка: Вероятно, потеря контекста или неправильный prompt.

Ответ:

- **Prompt не указывает явно, что нужен HTML-код**
- Модель сломалась
- **JSON** — дефолт формат Ollama
- HTML удалён из словаря модели

🔔 Шаг 10:

Миссия 10

📁 Сопоставьте тип ошибки HTML с корректирующим действием:

Ошибки:

Ответ:

- Пропущен закрывающий тег
- Неправильное вложение тегов
- Неизвестный тег
- Неправильный атрибут

Корректирующие действия:

1. Автоматическое закрытие тега в постобработке
2. Коррекция вложенности через перепарсинг
3. Удаление или замена на known тег
4. Валидировать атрибуты и отклонять неизвестные

Подсказка: Каждому синтаксическому типу — своя стратегия.

- **Пропущен закрывающий тег** — Автоматическое закрытие тега в постобработке

- **Неправильное вложение тегов** — Коррекция вложенности через перепарсинг
- **Неизвестный тег** — Удаление или замена на known тег
- **Неправильный атрибут** — Валидировать атрибуты и отклонять неизвестные

🔧 Шаг 11:

Миссия 11

🔗 Сопоставьте сценарий использования HTML-понимания с подходом обучения:

Сценарии:

Ответ:

- Генерация форм с `` и ``
- **Свойства CSS inline** — `style` атрибут
- HTML отчёты с таблицами и заголовками
- Обработка SEO-разметки (`` , ``)

Подходы:

1. Обучиться распознавать формы и input-output шаблоны
2. Включить CSS inline-примеры в обучение
3. Обучить структуре таблиц и заголовкам
4. Особое внимание к мета-тегам в примерах обучения

Подсказка: Подход должен соответствовать сценарию задачи.

- **Генерация форм с `` и ``** — Обучиться распознавать формы и input-output шаблоны
- **Свойства CSS inline** — `style` атрибут — Включить CSS inline-примеры в обучение
- **HTML отчёты с таблицами и заголовками** — Обучить структуре таблиц и заголовкам
- **Обработка SEO-разметки (`` , ``)** — Особое внимание к мета-тегам в примерах обучения

🔧 Шаг 12:

Миссия 12

🔗 Упорядочите виды тестов HTML-модели от лёгких к сложным:

1. Минимальные теги без вложений
2. Теги с атрибутами
3. Вложенные теги
4. Комплексные страницы (с таблицами, списками и формами)
5. Страницы с ошибками HTML (небаланс, вложенность и т.д.)

Подсказка: Тестируй сначала простые случаи, затем сложные и ошибочные.

Ответ:

- Минимальные теги без вложений
- Теги с атрибутами
- Вложенные теги
- Комплексные страницы (с таблицами, списками и формами)
- Страницы с ошибками HTML (небаланс, вложенность и т.д.)

🔧 Шаг 13:

Миссия 13

📋 Упорядочите этапы оценки производительности HTML-модели встроенной в Ollama:

1. Сравнение метрик (BLEU, tree-edit и т.д.)
2. Проверка latency и throughput
3. Тестирование в реальных сценариях веб-контента
4. Мониторинг ошибок HTML на живом трафике
5. Регулярная дообучка при новых шаблонах

Подсказка: Начинайте с метрических оценок, затем производительность, потом real-world, мониторинг и поддержка.

Ответ:

- Сравнение метрик (BLEU, tree-edit и т.д.)
- Проверка latency и throughput
- Тестирование в реальных сценариях веб-контента
- Мониторинг ошибок HTML на живом трафике
- Регулярная дообучка при новых шаблонах

📁 Модуль 5 - Миссия 5. Управление памятью и контекстом

📖 Урок 1 - 🧠 Context window и KV cache

🔮 Шаг 01:

Миссия 1

Инженер Лена загружает длинный текстовый контекст в LLM, но видит, что только первые 2048 токенов обрабатываются. Что из нижеперечисленного правильно описывает «context window»?

Подсказка: Представьте, что окно — это «окно внимания», через которое модель видит вход.

Ответ:

- **Максимальное число токенов, которые модель может одновременно учитывать из контекста**
- Объём оперативной памяти GPU
- Внутренняя архитектура трансформера KV cache
- Количество слоёв в модели

🔮 Шаг 02:

Миссия 2

При общении с ботом Вася хочет помнить предыдущие 3 сообщения в контексте, но бот «забывает» их. Какая ошибка наиболее вероятна?

Подсказка: Проверьте, как формируется запрос к модели.

Ответ:

- **При формировании prompt он не добавил предыдущие сообщения в input**
- У модели недостаточно параметров
- KV cache переполняется
- Контекст слишком короткий

Шаг 03:

Миссия 3

Девушка-разработчица Саша хочет ограничить максимальный контекст для экономии ресурсов, она настраивает ``max_length=512``. Что произойдёт, если пользователь пришлёт сообщение, которое вместе с историей превысит 512 токенов?

Подсказка: Подумайте, как популярные библиотеки обрабатывают обрезку.

Ответ:

- **Самый старый контекст (начало истории) будет усечён**
- Модель выдаст ошибку «too long»
- Будет записан весь текст несмотря на ограничение
- Модель проигнорирует пользовательское сообщение

Шаг 04:

Миссия 4

Инженер Артем экспериментирует с KV cache: он замечает, что при длинной истории ответы становятся быстрее. Почему так?

Подсказка: KV cache хранит ключи и значения из предыдущих слоёв.

Ответ:

- **Потому что уже вычисленные ключи/значения не пересчитываются заново**
- Модель автоматом уменьшает контекст
- GPU кэширует слои модели
- Контекст window уменьшается

Шаг 05:

Миссия 5

Мария решила вручную очистить KV cache перед каждым новым запросом. Что может случиться?

Подсказка: Подумайте, что модель теряет, если очищать cache.

Ответ:

- **Контекст памяти теряется, и модель не сможет помнить предыдущие токены**
- Ускорение генерации
- Увеличится объём используемой памяти
- Модель станет непредсказуемой

Шаг 06:

Миссия 6

Программист Денис заметил, что иногда KV cache переполняется, и это вызывает сбой. Какой из подходов поможет предотвратить переполнение?

Подсказка: Подумайте об ограничении длины истории.

Ответ:

- **Обрезать старые токены в истории до фиксированного размера**
- Дублировать recent tokens
- Игнорировать переполнение silently
- Отключить KV cache полностью

Шаг 07:

Миссия 7

Инженер Катя сталкивается с терминами, связанными с контекстом и кешем. Сопоставьте термин с правильным описанием:

Ответ:

- Context window
 - KV cache
 - Prompt truncation
 - Memory reset
1. Обрезка части истории, если превышает лимит
 2. Максимальное число токенов, который модель может обработать одновременно
 3. Хранение ключей и значений слоёв для ускорения генерации
 4. Полное очищение сохранённой памяти между сегментами

Подсказка: Вспомните, что делает cache и зачем обрезают текст.

- **Context window** — Максимальное число токенов, который модель может обработать одновременно
- **KV cache** — Хранение ключей и значений слоёв для ускорения генерации
- **Prompt truncation** — Обрезка части истории, если превышает лимит
- **Memory reset** — Полное очищение сохранённой памяти между сегментами

Шаг 08:

Миссия 8

На этапе отладки инженер Иван хочет понять, какие последствия неправильного управления контекстом. Сопоставьте проблему с её причиной:

Ответ:

- Модель забывает предыдущее сообщение
 - Генерация становится медленной
 - Контекст содержит нерелевантные данные
 - Ошибки из-за переполнения кеша
1. Очищение KV cache каждый раз
 2. Слишком длинный контекст, без усечения
 3. Неотфильтрованная история, спам
 4. Недостаточное ограничение кеша / переполнение

Подсказка: Подумайте, какая причина ведёт к каждой проблеме.

- **Модель забывает предыдущее сообщение** — Очищение KV cache каждый раз
- **Генерация становится медленной** — Слишком длинный контекст, без усечения
- **Контекст содержит нерелевантные данные** — Неотфильтрованная история, спам
- **Ошибки из-за переполнения кеша** — Недостаточное ограничение кеша / переполнение

Шаг 09:

Миссия 9

Нина читает статью: «преимущества и недостатки больших окон контекста». Сопоставьте аспект с его эффектом:

Ответ:

- Большое окно
 - Среднее окно
 - Малое окно
1. Высокий расход памяти, медленные вычисления
 2. Баланс между ресурсами и качеством
 3. Не хватает истории, может «забывать» связи

Подсказка: Подумайте, как размер окна влияет на ресурсы и качество.

- **Большое окно** — Высокий расход памяти, медленные вычисления
- **Среднее окно** — Баланс между ресурсами и качеством
- **Малое окно** — Не хватает истории, может «забывать» связи

 Шаг 10:

Миссия 10

Инженер Саша проводит диагностику процесса работы с контекстом. Отсортируйте этапы от начала до конца:

1. Очистка KV cache (если требуется)
2. Сборка истории prompt + новый ввод пользователя
3. Обрезка истории по ограничению окна
4. Генерация через модель с использованием cache

Подсказка: Сначала готовят контекст, потом передают в модель, а cache уже используется внутри.

Ответ:

- Очистка KV cache (если требуется)
- Сборка истории prompt + новый ввод пользователя
- Обрезка истории по ограничению окна
- Генерация через модель с использованием cache

 Шаг 11:

Миссия 11

При ошибке переполнения кеша инженер Лев решает внедрить лимит. Отсортируйте действия по приоритету:

1. Наложить ограничение на длину истории
2. Выбросить старые сообщения
3. Увеличить память кеша (если возможно)
4. Логировать переполнения для анализа

Подсказка: Какие меры минимальны и какие — вспомогательные?

Ответ:

- Наложить ограничение на длину истории
- Выбросить старые сообщения
- Увеличить память кеша (если возможно)
- Логировать переполнения для анализа

Шаг 12:

Миссия 12

Вася ведёт диалог с ботом, и диалоговая история со временем растёт. Отсортируйте варианты стратегии управления историей по степени агрессивности усечения (от мягкой до жёсткой):

1. Обрезать только вопросы бота
2. Удалить самые старые сообщения полностью
3. Обрезать отдельные фразы внутри сообщений
4. Удалять сообщения с наименьшей важностью (по весу)

Подсказка: Агрессивность — это сколько вы «теряете» из истории.

Ответ:

- Обрезать только вопросы бота
- Удалить самые старые сообщения полностью
- Обрезать отдельные фразы внутри сообщений
- Удалять сообщения с наименьшей важностью (по весу)

Шаг 13:

Миссия 13

Толя тестирует последовательность, когда модель сначала сбрасывает `cache`, потом строит контекст, потом генерирует. Но хочет сделать более оптимально. Отсортируйте эту последовательность по нужному порядку:

1. Генерация ответа
2. Сборка новой истории + входа пользователя
3. Обрезка истории по лимиту окна
4. Возможная очистка `cache` перед началом (опциональная)

Подсказка: Кеш очищается только если нужно, контекст строится до генерации.

Ответ:

- Возможная очистка `cache` перед началом (опциональная)
- Сборка новой истории + входа пользователя
- Обрезка истории по лимиту окна
- Генерация ответа

Шаг 14:

Миссия 14

Инженер Андрей реализует эвристику выбора, какие сообщения сбросить из истории. Он предлагает: самые старые, низкооценённые, бот-ответы в первую очередь, нейтральные. Отсортируйте эти критерии по порядку, в котором их стоит применять (сначала самое слабое основание):

1. Удаление нейтральных сообщений
2. Удаление бот-ответов
3. Удаление низкооценённых сообщений
4. Удаление самых старых сообщений

Подсказка: Сначала удаляйте те, что менее ценны, потом те, что скорее всего можно скипать.

Ответ:

- Удаление нейтральных сообщений

- Удаление бот-ответов
- Удаление низкооценённых сообщений
- Удаление самых старых сообщений

Шаг 15:

Миссия 15

Команда хочет внедрить стратегию «слойное усечение»: сначала убрать фразы, потом сообщения, затем блоки, затем полное удаление. Отсортируйте эти уровни усечения в правильном порядке:

1. Удалить целые блоки (абзацы)
2. Удалить слова или фразы внутри сообщений
3. Удалить отдельные сообщения
4. Удалить целые темы или сессии

Подсказка: От самого мелкого уровня до самого крупного.

Ответ:

- Удалить слова или фразы внутри сообщений
- Удалить отдельные сообщения
- Удалить целые блоки (абзацы)
- Удалить целые темы или сессии

Шаг 16:

Миссия 16

Финальный квест: Вася хочет протестировать разные конфигурации управления памятью: он подготовил четыре варианта, и нужно упорядочить их «жесткость» усечения (от мягкого до агрессивного):

1. Оставлять всю историю без усечения
2. Урезать только по сообщениям бота/редко важным
3. Обрезать старую часть, но сохранить ключевые точки
4. Сбрасывать память между сессиями

Подсказка: Чем больше теряется из истории — тем агрессивнее — сортируйте по этому.

Ответ:

- Оставлять всю историю без усечения
- Урезать только по сообщениям бота/редко важным
- Обрезать старую часть, но сохранить ключевые точки
- Сбрасывать память между сессиями

Урок 2 - Перезапуск и восстановление состояния модели

Шаг 01:

Миссия 1

Инженер Игорь перезапустил процесс модели из-за сбоя. Что из нижеуказанного наиболее разумно сделать сразу после запуска, чтобы минимизировать потерю контекста?

Подсказка: Подумайте, как частично восстановить состояние модели.

Ответ:

- Загрузить последний чекпоинт состояния и восстановить память

- Просто начать с чистой модели без истории
- Увеличить learning rate
- Отключить все логику каскадного восстановления

🔮 Шаг 02:

Миссия 2

Анна ведёт диалог через LLM и бот неожиданно перезапустился. Какой минимальный набор данных ей передать новой сессии, чтобы восстановить беседу?

Подсказка: Какие данные нужны, чтобы LLM “помнил” прошлое?

Ответ:

- **Последние N сообщений + сохранённые векторы памяти**
- Все логи сервера
- Текущий токен генерации
- Полный дамп состояния GPU

🔮 Шаг 03:

Миссия 3

Разработчик Максим хочет, чтобы после перезапуска модель автоматически загружала состояние из внешнего хранилища. Какой подход подходит лучше всего?

Подсказка: Подумайте о сохранении (checkpoint) и восстановлении (restore).

Ответ:

- **Сохранять промежуточные слои и память на диск и загружать при старте**
- Перезапускать без сохранения
- Всегда запрашивать у пользователя историю заново
- Хранить состояние только в RAM не на диск

🔮 Шаг 04:

Миссия 4

После восстановления состояния из чекпоинта разработчик Ольга заметила, что ответы иногда не учитывают последние сообщения пользователя. Как это может происходить?

Подсказка: Подумайте, что могло быть не включено в чекпоинт.

Ответ:

- **Чекпоинт не включал свежую часть диалога, только старую память**
- Контекст слишком мал
- KV cache повреждён
- Модель переключилась на другой режим

🔮 Шаг 05:

Миссия 5

Инженер Павел планирует перезапуск сервиса каждую ночь, но хочет, чтобы память диалогов не терялась. Какой подход выбрать?

Подсказка: Сохранение памяти требует надёжного хранилища.

Ответ:

- **Автоматически сохранять state (память и cache) в базу и загружать при старте**
- Каждый раз сбрасывать память

- Просить пользователей переслать историю
- Отключить перезапуск на ночь

🔗 Шаг 06:

Миссия 6

Разработчица Лиза хочет минимизировать диск I/O при сохранении состояния. Что она может сделать?

Подсказка: Подумайте о компрессии и инкрементальных изменениях.

Ответ:

- **Сохранять только дельты изменений, а не весь дамп**
- Сохранять каждый токен отдельно
- Не сохранять ничего
- Дублировать файлы часто

🔗 Шаг 07:

Миссия 7

После аварийного сбоя разработчик Кирилл хочет, чтобы восстановление было устойчивым. Что нужно предусмотреть в системе восстановления?

Подсказка: Подумайте о версиях, консистенции и проверках целостности.

Ответ:

- **Хранить несколько последних версий и проверять контрольные суммы**
- Хранить только одну версию
- Не делать проверок целостности
- Всегда путать версии

🔗 Шаг 08:

Миссия 8

Инженер Света читает документацию по восстановлению состояния. Сопоставьте термин и действие:

Ответ:

- Checkpoint
- Restore
- Incremental save
- Cold start

1. Сохранение полного состояния модели в файл
2. Запуск модели с нуля без контекста
3. Восстановление состояния из сохранённого файла
4. Сохранение только изменений с момента последнего чекпоинта

Подсказка: Подумайте, что делают checkpoint и restore.

- **Checkpoint** — Сохранение полного состояния модели в файл
- **Restore** — Восстановление состояния из сохранённого файла
- **Incremental save** — Сохранение только изменений с момента последнего чекпоинта
- **Cold start** — Запуск модели с нуля без контекста

Шаг 09:

Миссия 9

Проектный архитектор пытается объединить стратегии восстановления. Сопоставьте проблему с подходом:

Ответ:

- Избыточное хранение
 - Замедленное восстановление
 - Неполное состояние
 - Повреждённый дамп
1. Хранить только дельты, не весь дамп
 2. Восстановление последней версии сразу, без поиска версий
 3. Неполный чекпоинт, пропущенные обновления
 4. Добавить контрольные суммы или подписи

Подсказка: Подумайте, как каждый подход решает конкретную проблему.

- **Избыточное хранение** — Хранить только дельты, не весь дамп
- **Замедленное восстановление** — Восстановление последней версии сразу, без поиска версий
- **Неполное состояние** — Неполный чекпоинт, пропущенные обновления
- **Повреждённый дамп** — Добавить контрольные суммы или подписи

Шаг 10:

Миссия 10

Разработчик Олег хочет выбрать методы восстановления. Сопоставьте метод с сценарием:

Ответ:

- Полное восстановление
 - Инкрементальное восстановление
 - Гибридное восстановление
 - Ленивая загрузка состояния
1. Каждый элемент загружается только при необходимости
 2. Сначала полный дамп, затем применяются дельты
 3. Восстановление всего состояния за один шаг
 4. Только изменения загружаются и восстанавливаются

Подсказка: Сравните плюсы и минусы разных стратегий.

- **Ленивая загрузка состояния** — Каждый элемент загружается только при необходимости
- **Гибридное восстановление** — Сначала полный дамп, затем применяются дельты
- **Полное восстановление** — Восстановление всего состояния за один шаг
- **Инкрементальное восстановление** — Только изменения загружаются и восстанавливаются

Шаг 11:

Миссия 11

В документации встречаются термины: «warm restart», «cold restart», «snapshot», «journaling». Сопоставьте термин и понятие:

Ответ:

- warm restart
 - cold restart
 - snapshot
 - journaling
1. Перезапуск, при котором память восстанавливается
 2. Перезапуск с нулевым состоянием модели
 3. Снимок состояния в момент времени
 4. Запись логов операций для воспроизведения изменений

Подсказка: Используйте аналогии с базами данных.

- **warm restart** — Перезапуск, при котором память восстанавливается
- **cold restart** — Перезапуск с нулевым состоянием модели
- **snapshot** — Снимок состояния в момент времени
- **journaling** — Запись логов операций для воспроизведения изменений

 Шаг 12:

Миссия 12

Разработчик Никита планирует процесс восстановления после сбоя. Отсортируйте шаги от того, что делать в первую очередь, до финального этапа:

1. Проверка контрольных сумм сохранённых файлов
2. Загрузка последнего валидного чекпоинта
3. Применение инкрементальных дельт к чекпоинту
4. Запуск модели и валидация результата восстановления

Подсказка: Первый шаг — убедиться, что файлы не повреждены.

Ответ:

- Проверка контрольных сумм сохранённых файлов
- Загрузка последнего валидного чекпоинта
- Применение инкрементальных дельт к чекпоинту
- Запуск модели и валидация результата восстановления

 Шаг 13:

Миссия 13

Восстановление модели может происходить в разных режимах: «полный», «инкрементальный», «гибрид», «ленивая загрузка». Отсортируйте эти режимы по скорости запуска (от самого медленного до самого быстрого):

1. Полный
2. Гибридный
3. Инкрементальный
4. Ленивая загрузка

Подсказка: Подумайте, сколько данных надо сразу загрузить.

Ответ:

- Полный
- Гибридный
- Инкрементальный

- Ленивая загрузка

Шаг 14:

Миссия 14

Татьяна настраивает план откатов восстановления. У неё есть четыре сценария: откат до предыдущего чекпоинта, откат до самого старого, восстановление дельт, аварийный сброс в чистое состояние. Отсортируйте их по надёжности отката (от безопасного к рискованному):

1. Откат до предыдущего чекпоинта
2. Восстановление дельт
3. Откат до самого старого чекпоинта
4. Аварийный сброс в чистое состояние

Подсказка: Самый безопасный — минимальное отклонение от текущего состояния.

Ответ:

- Откат до предыдущего чекпоинта
- Восстановление дельт
- Откат до самого старого чекпоинта
- Аварийный сброс в чистое состояние

Шаг 15:

Миссия 15

При перезапуске сервиса команда сталкивается с четырьмя подходами: без восстановления, только дельты, полный дамп, гибрид. Отсортируйте по степени потери истории (от наименьшей потери до наибольшей):

1. Гибридный
2. Полный дамп
3. Только дельты
4. Без восстановления

Подсказка: Подумайте, в каком варианте теряется минимум информации.

Ответ:

- Гибридный
- Полный дамп
- Только дельты
- Без восстановления

Шаг 16:

Миссия 16

Команда проводит анализ восстановления: они имеют шаги «чекап», «дельты», «загрузка», «валидация». Отсортируйте полную цепочку восстановления после сбоя:

1. Чекап (snapshot) создания
2. Применение дельт после восстановления базового состояния
3. Загрузка состояния модели
4. Валидация корректности восстановленного состояния

Подсказка: Восстановление = загрузка → применение дельт → проверка.

Ответ:

- Чекап (snapshot) создания
- Загрузка состояния модели
- Применение дельт после восстановления базового состояния
- Валидация корректности восстановленного состояния

 Шаг 17:

Миссия 17

Финальный вызов: инженер Ирина хочет сравнить 4 режима восстановления: «cold start», «warm start with full dump», «warm start with deltas», «lazy warm start». Отсортируйте эти режимы по времени запуска (от самого медленного к самому быстрому):

1. cold start
2. warm start with full dump
3. warm start with deltas
4. lazy warm start

Подсказка: Чем меньше данных загружается сразу — тем быстрее старт.

Ответ:

- cold start
- warm start with full dump
- warm start with deltas
- lazy warm start

 Урок 3 - Chain of Thought внутренний диалог LLM

 Шаг 01:

Миссия 1

Инженер Алексей заметил, что его LLM иногда путается при ответах на сложные вопросы. Что из нижеперечисленного лучше всего поможет LLM думать последовательно, чтобы улучшить ответы?

Подсказка: Представьте, что модель ведет внутренний диалог, анализируя каждый шаг.

Ответ:

- **Использовать метод Chain of Thought** — поэтапное объяснение рассуждений
- Просто увеличить объем памяти модели
- Игнорировать промежуточные шаги и сразу выводить ответ
- Использовать только заранее обученные шаблоны ответов

 Шаг 02:

Миссия 2

Марина пишет скрипт для LLM, который должен выводить промежуточные рассуждения. Какой подход наиболее верен для реализации внутреннего диалога?

Подсказка: Внутренний диалог должен быть читаем и логически последовательным.

Ответ:

- **Добавлять пояснения каждого шага, чтобы модель могла анализировать ход мысли**
- Выводить только окончательный ответ без пояснений

- Случайно вставлять промежуточные мысли для вариативности
- Использовать только внешние источники без внутренних объяснений

🔗 Шаг 03:

Миссия 3

Разработчик Иван заметил, что LLM теряет контекст при длинных диалогах. Какой способ поможет лучше сохранить цепочку рассуждений?

Подсказка: Помните о роли памяти и контекста в модели.

Ответ:

- **Разбивать задачу на шаги и поэтапно сохранять вывод каждого шага**
- Записывать только последние два сообщения диалога
- Игнорировать предыдущий контекст и отвечать заново
- Использовать случайные ключевые слова без структуры

🔗 Шаг 04:

Миссия 4

Сопоставьте понятия, связанные с внутренним диалогом LLM, с их правильными определениями.

Ответ:

- Chain of Thought
- Контекст
- Промежуточный вывод
- Память модели

1. Последовательное объяснение модели своих рассуждений.
2. Информация из предыдущих сообщений, влияющая на ответ.
3. Промежуточный результат, который помогает двигаться к финальному ответу.
4. Механизм хранения и доступа к прошлым данным внутри модели.

Подсказка: Вспомните, как модель обрабатывает и хранит информацию в ходе диалога.

- **Chain of Thought** — Последовательное объяснение модели своих рассуждений.
- **Контекст** — Информация из предыдущих сообщений, влияющая на ответ.
- **Промежуточный вывод** — Промежуточный результат, который помогает двигаться к финальному ответу.
- **Память модели** — Механизм хранения и доступа к прошлым данным внутри модели.

🔗 Шаг 05:

Миссия 5

Сопоставьте ситуации из работы программиста с проблемами, которые могут возникать из-за плохого управления внутренним диалогом LLM.

Ответ:

- Модель пропускает важные детали при ответе
- LLM генерирует противоречивые рассуждения
- Долгий ответ теряет структуру и смысл
- Ответ не учитывает предыдущие сообщения

1. Недостаточный контекст

2. Отсутствие поэтапного анализа
3. Проблемы с управлением памятью
4. Потеря памяти о диалоге

Подсказка: Подумайте, как каждый из аспектов внутреннего диалога влияет на качество ответа.

- **Модель пропускает важные детали при ответе** — Недостаточный контекст
- **LLM генерирует противоречивые рассуждения** — Отсутствие поэтапного анализа
- **Долгий ответ теряет структуру и смысл** — Проблемы с управлением памятью
- **Ответ не учитывает предыдущие сообщения** — Потеря памяти о диалоге

Шаг 06:

Миссия 6

Сопоставьте термины из LLM с их функциями в управлении цепочкой мыслей.

Ответ:

- Токен
 - Векторное представление
 - Скрытое состояние
 - Механизм внимания (attention)
1. Минимальная единица текста, обрабатываемая моделью.
 2. Числовое представление слова или токена для вычислений.
 3. Внутреннее состояние модели, содержащее информацию о предыдущем контексте.
 4. Механизм выделения важных частей входа для ответа.

Подсказка: Подумайте о том, как модель понимает и обрабатывает информацию на разных этапах.

- **Токен** — Минимальная единица текста, обрабатываемая моделью.
- **Векторное представление** — Числовое представление слова или токена для вычислений.
- **Скрытое состояние** — Внутреннее состояние модели, содержащее информацию о предыдущем контексте.
- **Механизм внимания (attention)** — Механизм выделения важных частей входа для ответа.

Шаг 07:

Миссия 7

Сопоставьте примеры задач с типом Chain of Thought, который подходит для решения.

Ответ:

- Математическая задача с несколькими шагами
 - Юридический анализ контракта
 - Диалог с пользователем по технической поддержке
 - Создание плана статьи
1. Пошаговое вычисление
 2. Обоснованное рассуждение
 3. Пошаговый разбор запроса
 4. Последовательное структурирование

Подсказка: Каждый тип Chain of Thought помогает структурировать мысли для разных задач.

- **Математическая задача с несколькими шагами** — Пошаговое вычисление
- **Юридический анализ контракта** — Обоснованное рассуждение
- **Диалог с пользователем по технической поддержке** — Пошаговый разбор запроса
- **Создание плана статьи** — Последовательное структурирование

🔮 Шаг 08:

Миссия 8

Помогите Анастасии правильно упорядочить этапы создания Chain of Thought для LLM, чтобы получить качественный ответ.

1. Провести разбор задачи на логические части
2. Запросить у модели промежуточные рассуждения по каждой части
3. Объединить промежуточные выводы в итоговый ответ
4. Проверить, что рассуждения последовательны и не противоречивы

Подсказка: Начинайте с разбора, затем пошаговое объяснение и итог.

Ответ:

- Провести разбор задачи на логические части
- Запросить у модели промежуточные рассуждения по каждой части
- Объединить промежуточные выводы в итоговый ответ
- Проверить, что рассуждения последовательны и не противоречивы

🔮 Шаг 09:

Миссия 9

Расположите шаги, которые программист Иван предпринимает, чтобы исправить потерю контекста в LLM.

1. Добавить сохранение промежуточных результатов рассуждений
2. Улучшить механизм управления памятью модели
3. Проверить и оптимизировать объем передаваемого контекста
4. Тестировать качество ответов на сложные вопросы

Подсказка: Оптимизация памяти начинается с диагностики и заканчивается проверкой результата.

Ответ:

- Добавить сохранение промежуточных результатов рассуждений
- Улучшить механизм управления памятью модели
- Проверить и оптимизировать объем передаваемого контекста
- Тестировать качество ответов на сложные вопросы

🔮 Шаг 10:

Миссия 10

Расположите этапы, через которые проходит LLM при генерации ответа с внутренним диалогом.

1. Получение и анализ запроса
2. Формирование промежуточных рассуждений
3. Объединение промежуточных выводов

4. Формирование итогового ответа

Подсказка: Сначала модель понимает вопрос, потом думает и формирует ответ.

Ответ:

- Получение и анализ запроса
- Формирование промежуточных рассуждений
- Объединение промежуточных выводов
- Формирование итогового ответа

🔮 Шаг 11:

Миссия 11

Разработчица Ольга экспериментирует с Chain of Thought и хочет, чтобы LLM не только рассуждала, но и проверяла свои ответы. Какой подход ей стоит выбрать?

Подсказка: Внутренний диалог можно расширить этапом самопроверки.

Ответ:

- **Добавить этап проверки и исправления промежуточных выводов**
- Просто увеличить количество рассуждений без проверки
- Игнорировать промежуточные результаты и сразу отвечать
- Попросить модель давать максимально короткий ответ

🔮 Шаг 12:

Миссия 12

Алексей внедряет внутренний диалог в свою LLM, но она начинает генерировать слишком длинные и неструктурированные ответы. Что ему стоит сделать?

Подсказка: Контроль структуры — залог понятности.

Ответ:

- **Добавить форматирование и разделение рассуждений на этапы**
- Увеличить длину вывода без изменений структуры
- Использовать случайный порядок мыслей для разнообразия
- Игнорировать длинные ответы и выбирать короткие

🔮 Шаг 13:

Миссия 13

Разработчица Марина сталкивается с тем, что LLM иногда «забывает» начало разговора в длинной цепочке. Что поможет избежать этой проблемы?

Подсказка: Память — ключ к сохранению важного контекста.

Ответ:

- **Использовать внешнюю память или summary для ключевых идей**
- Удалять все предыдущие сообщения для освобождения памяти
- Игнорировать длинные цепочки и отвечать локально
- Перезапускать модель при каждом новом запросе

🔮 Шаг 14:

Миссия 14

Сопоставьте виды памяти LLM с их назначением в контексте цепочки рассуждений.

Ответ:

- Кратковременная память
 - Долговременная память
 - Временная память
 - Внешняя память
1. Хранит информацию из текущего диалога
 2. Сохраняет накопленные знания модели из обучения
 3. Временное хранение промежуточных рассуждений
 4. Используется для хранения данных вне модели (например, базы данных)

Подсказка: Разные виды памяти помогают LLM справляться с объемом и сложностью информации.

- **Кратковременная память** — Хранит информацию из текущего диалога
- **Долговременная память** — Сохраняет накопленные знания модели из обучения
- **Временная память** — Временное хранение промежуточных рассуждений
- **Внешняя память** — Используется для хранения данных вне модели (например, базы данных)

Шаг 15:

Миссия 15

Разработчик Иван заметил, что LLM перестал корректно использовать контекст после долгой сессии. Что могло стать причиной?

Подсказка: Иногда объем доступной памяти для контекста ограничен.

Ответ:

- **Превышение лимита контекстной памяти модели**
- Сбой в сети интернет
- Ошибка в логике пользовательского интерфейса
- Недостаток вычислительных ресурсов процессора

Шаг 16:

Миссия 16

Сопоставьте этапы Chain of Thought с их описаниями:

Ответ:

- Разделение задачи
- Пошаговое рассуждение
- Формирование ответа
- Проверка промежуточных результатов

Подсказка: Chain of Thought — это процесс, разбитый на этапы для улучшения качества вывода.

- **Разделение задачи** — Разбиение сложной проблемы на более простые части.
- **Пошаговое рассуждение** — Логическое продумывание каждого этапа решения.
- **Формирование ответа** — Сбор итогового решения после анализа.
- **Проверка промежуточных результатов** — Контроль правильности каждого шага до финального вывода.

🔊 Шаг 17:

Миссия 17

Отсортируйте шаги оптимального управления памятью LLM при работе с длинным контекстом:

1. Очистка неактуальных данных
2. Загрузка текущего контекста
3. Архивирование старых сообщений
4. Обработка новых запросов

Подсказка: Управление памятью — это баланс между хранением нужных данных и удалением лишних.

Ответ:

- Загрузка текущего контекста
- Обработка новых запросов
- Очистка неактуальных данных
- Архивирование старых сообщений

🔊 Шаг 18:

Миссия 18

Разработчица Ольга внедряет механизм внутреннего диалога LLM. Какой подход поможет повысить качество рассуждений модели?

Подсказка: Иногда полезно заставить модель «думать вслух».

Ответ:

- **Использование цепочек рассуждений (Chain of Thought)**
- Уменьшение контекстного окна
- Полное игнорирование промежуточных шагов
- Случайный перебор ответов

📖 Урок 4 - 🛠️ Настройка параметров генерации

🔊 Шаг 01:

Миссия 1

Вы заметили, что текст ответа слишком «размытый». Какой параметр стоит уменьшить?

Подсказка: Это управляет разнообразием вероятных токенов.

Ответ:

- **temperature**
- max_tokens
- stop
- presence_penalty

🔊 Шаг 02:

Миссия 2

Инженер Лена использует Ollama и хочет, чтобы модель игнорировала очень низкоприоритетные токены. Какой параметр поможет этому?

Подсказка: Частотный штраф влияет на предпочтение часто встречающихся токенов.

Ответ:

- **frequency_penalty**
- top_p
- temperature
- max_tokens

🔮 **Шаг 03:**

Миссия 3

При генерации кода ответ стал очень повторяющимся. Какой параметр стоит увеличить, чтобы стимулировать разнообразие?

Подсказка: Присутствие_penalty повышает штраф за повторение уже сгенерированных токенов.

Ответ:

- **presence_penalty**
- temperature
- max_tokens
- frequency_penalty

🔮 **Шаг 04:**

Миссия 4

Сопоставьте параметр генерации с тем, на что он влияет.

Подсказка: Подумайте: какой параметр управляет «насколько случайно», «максимум длины», «разнообразие» и «повторения».

Параметры:

Ответ:

- temperature
- max_tokens
- top_p
- frequency_penalty
- presence_penalty

Эффекты:

1. Ограничивает общую длину ответа.
2. Уменьшение вероятности повторов.
3. Генерация более «творческих» ответов.
4. Ограничение выбора токенов до «вершинной» массы вероятности.
5. Штраф за уже сгенерированные токены.

- **temperature** — Генерация более «творческих» ответов.
- **max_tokens** — Ограничивает общую длину ответа.
- **top_p** — Ограничение выбора токенов до «вершинной» массы вероятности.
- **frequency_penalty** — Уменьшение вероятности повторов.
- **presence_penalty** — Штраф за уже сгенерированные токены.

🔮 **Шаг 05:**

Миссия 5

Сопоставьте сценарий с параметром, который наиболее эффективно решит проблему.

Подсказка: Подумайте, в какой ситуации ответ слишком длинный или слишком «размытый».

Сценарии:

Ответ:

- Ответ слишком длинный
- Ответ генерирует лишние повторения
- Ответ слишком разнообразный, уходит в неожиданные темы
- Ответ содержит мало опций, слишком консервативен
- Вы хотите отрегулировать порог выбора токенов

Парочка параметров:

1. max_tokens
2. presence_penalty
3. temperature
4. frequency_penalty
5. top_p

- **Ответ слишком длинный** — max_tokens
- **Ответ генерирует лишние повторения** — presence_penalty
- **Ответ слишком разнообразный, уходит в неожиданные темы** — temperature
- **Ответ содержит мало опций, слишком консервативен** — frequency_penalty
- **Вы хотите отрегулировать порог выбора токенов** — top_p

🔮 Шаг 06:

Миссия 6

В сценарии: инженер Артем настроил temperature = 0.9 и top_p = 0.8. Что выскажут эти параметры вместе?

Подсказка: Они часто комбинируются, чтобы управлять случайностью и отбрасыванием крайних вероятностей.

Описание:

Ответ:

- Высокая случайность
- Отсечение самых низких вероятностей
- Более консервативный выбор токенов
- Средняя вариативность
- Почти жёсткий отбор наиболее вероятных токенов

Параметры:

1. temperature = 0.9 + top_p = 0.8
 - **temperature = 0.9 + top_p = 0.8** — Средняя вариативность и отсечение самых низких вероятностей.

🔮 Шаг 07:

Миссия 7

Отсортируйте этапы настройки генерации в Ollama, когда вы хотите управлять контекстом и памятью.

Подсказка: Сначала задаётся общая рамка, затем точные параметры, в конце тестирование.

1. Установить параметры temperature, top_p, penalties

2. Определить границу максимального контекста / длину prompt + ответ
3. Проверить поведение на разных входах (edge cases)
4. Итеративно корректировать на основе ошибок

Ответ:

- Определить границу максимального контекста / длину prompt + ответ
- Установить параметры temperature, top_p, penalties
- Проверить поведение на разных входах (edge cases)
- Итеративно корректировать на основе ошибок

 Шаг 08:

Миссия 8

Отсортируйте, как при настройке параметров генерации вы постепенно снижаете риск «галлюцинаций»:

Подсказка: Начинайте с жёстких ограничений и затем добавляйте «свободы» по необходимости.

1. Снизить temperature
2. Увеличить presence_penalty / frequency_penalty
3. Ограничить max_tokens или длину контекста
4. Ограничить топ-выбор через top_p
5. Тестировать и корректировать в цикле

Ответ:

- Снизить temperature
- Увеличить presence_penalty / frequency_penalty
- Ограничить max_tokens или длину контекста
- Ограничить топ-выбор через top_p
- Тестировать и корректировать в цикле

 Шаг 09:

Миссия 9

Упорядочите подходы, которыми инженер может управлять контекстной памятью в Ollama:

Подсказка: Сначала очищение, потом сжатие, потом селективное хранение.

1. Сжатие контекста / summarization
2. Удаление старых частей контекста
3. Селективное хранение ключевых фактов
4. Ограничение длины окна контекста
5. Регулярный контроль на отклонения

Ответ:

- Сжатие контекста / summarization
- Удаление старых частей контекста
- Селективное хранение ключевых фактов
- Ограничение длины окна контекста
- Регулярный контроль на отклонения

 Шаг 10:

Миссия 10

Инженер Катя пытается заставить модель придерживаться только последних 1000 токенов контекста. Что разумно сделать в Ollama?

Подсказка: Возможно, потребуется как техника «скользящего окна», так и обрезка.

Ответ:

- **Обрезать старую часть контекста и передавать только последние 1000 токенов**
- Передавать весь контекст, игнорируя ограничения
- Увеличить temperature до максимума
- Устанавливать max_tokens = 1000

🧙 Шаг 11:

Миссия 11

Вы хотите, чтобы модель никогда не возвращала слово «банан». Какой приём применить?

Подсказка: Некоторые API поддерживают явно запрещённые токены.

Ответ:

- **Добавить «банан» в список запрещённых токенов (bad_tokens)**
- Установить temperature = 0
- Установить max_tokens маленьким
- Установить presence_penalty высоко

🧙 Шаг 12:

Миссия 12

Сопоставьте термин (из управления памятью / контекстом) и короткое описание.

Подсказка: Вспомните common techniques: sliding window, summarization, key extraction.

Термины:

Ответ:

- Sliding window
- Summarization
- Key fact extraction
- Context pruning
- Chunking

Описания:

1. Обрезка старых частей контекста.
 2. Разбиение длинного контекста на части (чанки).
 3. Создание краткого резюме из части контекста.
 4. Извлечение наиболее важных фактов для хранения.
 5. Использование движущегося окна, скользящего по контексту.
- **Sliding window** — Использование движущегося окна, скользящего по контексту.
 - **Summarization** — Создание краткого резюме из части контекста.
 - **Key fact extraction** — Извлечение наиболее важных фактов для хранения.
 - **Context pruning** — Обрезка старых частей контекста.
 - **Chunking** — Разбиение длинного контекста на части (чанки).

🧙 Шаг 13:

Миссия 13

Сопоставьте ситуацию из жизни инженера с подходом к управлению памятью.

Подсказка: Подумайте, как вы бы поступили, если история сессии становится слишком сквозной.

Ситуации:

Ответ:

- **Диалог длится слишком много шагов** — растёт контекст
- **Большая часть контекста** — несущественные сообщения
- Необходимость сохранять ключевые факты, а не всю переписку
- Нужно разбить очень длинное сообщение на части
- Контекст достигает допустимого размера API

Подходы:

1. Chunking

2. Summarization / сжатие

3. Key fact extraction

4. Context pruning

5. Sliding window

- **Диалог длится слишком много шагов** — растёт контекст — Sliding window
- **Большая часть контекста** — несущественные сообщения — Context pruning
- **Необходимость сохранять ключевые факты, а не всю переписку** — Key fact extraction
- **Нужно разбить очень длинное сообщение на части** — Chunking
- **Контекст достигает допустимого размера API** — Summarization / сжатие

Шаг 14:

Миссия 14

Отсортируйте шаги, как инженер может вводить сжатие контекста:

Подсказка: Сначала выбрать что сжимать, потом алгоритм, потом интеграцию и проверку.

1. Выбрать, какие части контекста подлежат сжатию (чаты, лонг тексты)
2. Применить summarization или абстракцию к выбранным частям
3. Интегрировать возвращённое резюме в контекст как «замену» оригиналов
4. Проверить, что ответы модели остались точными и релевантными
5. **Цикл корректировки: если потеря важности** — уменьшить степень сжатия

Ответ:

- Выбрать, какие части контекста подлежат сжатию (чаты, лонг тексты)
- Применить summarization или абстракцию к выбранным частям
- Интегрировать возвращённое резюме в контекст как «замену» оригиналов
- Проверить, что ответы модели остались точными и релевантными
- **Цикл корректировки: если потеря важности** — уменьшить степень сжатия

Шаг 15:

Миссия 15

Упорядочите, как инженер может постепенно усложнять стратегию управления контекстом:

Подсказка: От простого удаления — к автоматическому сжатию и динамическому выбору.

1. Простое обрезание старых сообщений
2. Ручная аннотация ключевых фактов
3. Применение summarization к блокам

4. Автоматизированный выбор значимых фактов
5. Гибрид: комбинирование summarization + sliding window

Ответ:

- Простое обрезание старых сообщений
- Ручная аннотация ключевых фактов
- Применение summarization к блокам
- Автоматизированный выбор значимых фактов
- Гибрид: комбинирование summarization + sliding window

Шаг 16:

Миссия 16

К финалу: инженер Алекс хочет использовать динамическую стратегию: в начале — жёсткие ограничения, позднее — более гибкие. Какая логика лучше?

Подсказка: Начинайте с безопасного режима, потом адаптируйтесь к ситуации.

Ответ:

- **Стартовать с низкой temperature и строгих штрафов, затем расширять диапазон при стабильности**
- Сразу открыть максимальную свободу и потом ограничить, если ошибки
- Всегда оставаться на одном наборе параметров
- Каждый раз случайно менять параметры

Урок 5 - Квест заставь модель помнить больше, чем положено

Шаг 01:

Миссия 1

Вы запускаете Ollama-инстанс и хотите, чтобы модель «помнила» длинный контекст диалога. Какой начальный приём наиболее логичен?

Подсказка: Сначала стоит попробовать передавать максимальный допустимый контекст.

Ответ:

- **Передать полный контекст до лимита API**
- Уменьшить temperature до 0
- Добавить частотный штраф
- Ограничить max_tokens минимальным

Шаг 02:

Миссия 2

Вы замечаете, что модель обрезает начало диалога, забывая факты, которые были в первых сообщениях. Что вы попробуете?

Подсказка: Возможно, стоит явно удалять менее важные части.

Ответ:

- **Обрезать старые сообщения (prune) и сохранять ключевые факты**
- Всегда передавать весь контекст целиком
- Уменьшить top_p
- Поменять модель на меньшую

🔮 Шаг 03:

Миссия 3

Ваша сессия стала длинной: контекст превышает предел API. Какой приём применить?

Подсказка: Подумайте о сжатии вместо полного удаления.

Ответ:

- **Применить summarization к части контекста перед передачей**
- Обрезать всё до минимального размера
- Увеличить temperature
- Установить max_tokens = 1

🔮 Шаг 04:

Миссия 4

Вы хотите, чтобы модель хранила только ключевые факты (имена, даты, цели), а не все мелочи. Что логично сделать?

Подсказка: Это похоже на фильтрацию перед сохранением.

Ответ:

- **Применить key-fact extraction при сохранении контекста**
- Хранить весь текст полностью
- Никак не фильтровать, довериться модели
- Каждый раз перезаписывать весь контекст нулями

🔮 Шаг 05:

Миссия 5

Сопоставьте технику управления памятью с её описанием.

Подсказка: Подумайте, как разные методы ограничивают, фильтруют или сжимают контекст.

Методы:

Ответ:

- Sliding window
- Summarization
- Key fact extraction
- Context pruning
- Chunking

Описание:

1. Хранение только свежих последних частей диалога
2. Создание краткого резюме длинных блоков
3. Выбор ключевых фактов из информации
4. Удаление менее значимых частей контекста
5. Разбиение длинного текста на части (чанки)

- **Sliding window** — Хранение только свежих последних частей диалога
- **Summarization** — Создание краткого резюме длинных блоков
- **Key fact extraction** — Выбор ключевых фактов из информации
- **Context pruning** — Удаление менее значимых частей контекста
- **Chunking** — Разбиение длинного текста на части (чанки)

🔮 Шаг 06:

Миссия 6

Артём замечает: модель забывает вводное описание проекта, когда диалог растёт. Какой метод предпочтительнее?

Подсказка: Подумайте, что лучше — сжатие или удаление.

Ситуации:

Ответ:

- Превышен предел токенов контекста
- **Большинство текста** — второстепенные реплики
- Важные факты разбросаны в начале
- Очень длинный монолог в середине
- Часть текста можно резюмировать

Методы:

1. Sliding window
2. Context pruning
3. Summarization
4. Chunking
5. Key fact extraction

- **Превышен предел токенов контекста** — Summarization
- **Большинство текста** — второстепенные реплики — Context pruning
- **Важные факты разбросаны в начале** — Key fact extraction
- **Очень длинный монолог в середине** — Chunking
- **Часть текста можно резюмировать** — Sliding window

🔮 Шаг 07:

Миссия 7

Настя пробует комбинировать методы: summarization + sliding window. Что получится?

Подсказка: Часть контекста будет сжата, часть — просто «скользнуть».

Комбинации:

Ответ:

- Summarization + sliding window
- Chunking + pruning
- Key extraction + summarization
- Pruning + sliding window
- Summarization + pruning

Результаты:

1. Сжатые ключевые части + свежие части диалога
2. Удаление и разбивка без сжатия
3. Хранение только ключевых фактов с резюме
4. Удаление старых + движение окна
5. Сжатие мелких частей + удаление лишнего

- **Summarization + sliding window** — Сжатые ключевые части + свежие части диалога
- **Chunking + pruning** — Удаление и разбивка без сжатия

- **Key extraction + summarization** — Хранение только ключевых фактов с резюме
- **Pruning + sliding window** — Удаление старых + движение окна
- **Summarization + pruning** — Сжатие мелких частей + удаление лишнего

🧙 Шаг 08:

Миссия 8

Отсортируйте шаги, как инженер может повысить «память» модели:

Подсказка: Начинайте с базовых трюков, затем комбинируйте методы.

1. Передать весь контекст (до предела)
2. Обрезать малозначимый текст (pruning)
3. Сжать длинные блоки (summarization)
4. Выбрать ключевые факты (key extraction)
5. Комбинировать sliding window + summarization

Ответ:

- Передать весь контекст (до предела)
- Обрезать малозначимый текст (pruning)
- Сжать длинные блоки (summarization)
- Выбрать ключевые факты (key extraction)
- Комбинировать sliding window + summarization

🧙 Шаг 09:

Миссия 9

Вы хотите, чтобы модель «напоминала» о вводных инструкциях при долгом диалоге. Что вы сделаете?

Подсказка: Иногда полезно «переподать» важные инструкции.

Ответ:

- **Вставлять ключевую инструкцию периодически в контекст (re-inject)**
- Полностью забывать инструкцию после начала
- Писать инструкцию в переменную и не передавать
- Уменьшать max_tokens до минимума

🧙 Шаг 10:

Миссия 10

Вы хотите, чтобы модель «вспоминала» детали прошлого разговора, но не всё подряд. Какая стратегия?

Подсказка: Сочетайте извлечение и сжатие.

Ответ:

- **Извлекать ключевые факты и сжимать менее важные части**
- Передавать всё без фильтрации
- Каждый раз сбрасывать память
- Обрезать строго по времени, не по значимости

🧙 Шаг 11:

Миссия 11

🎯 Финал: ваша задача — заставить модель помнить больше, чем положено лимитом API. Какую стратегию вы выберете в комбинированном режиме?

Подсказка: Используйте гибриды из методов, а не один.

Ответ:

- **Re-inject инструкций + summarization + sliding window + key fact extraction**
- Только summarization
- Только sliding window
- Только pruning

📁 Модуль 6 - 🔗 Миссия 6. Рождение агента

📖 Урок 1 - 🤖 Что такое агент и как он рассуждает

💡 Шаг 01:

Миссия 1

Девелопер Иван задумал создать агента, который автоматически отвечает на сообщения пользователей. Какой компонент в агенте отвечает за планирование действий?

Подсказка: Это не просто генератор ответа — он выбирает последовательность шагов.

Ответ:

- **Планировщик (Planner)**
- Натуральный языковой модуль
- Модуль памяти

💡 Шаг 02:

Миссия 2

👩 Лена прототипирует агента, который может решать задачи по математике. Что должно быть у агента, чтобы он обдумывал многозадачно?

Подсказка: Размышление — это возможность рассматривать альтернативы.

Ответ:

- **Поиск альтернативных действий и модель их последствий**
- Запись всего диалога в лог
- Использование шаблонов ответа
- Жёсткое последовательное выполнение команд

💡 Шаг 03:

Миссия 3

👨 Разработчик Сергей добавляет агенту «контекст» беседы. Что является задачей этого контекста?

Подсказка: Агент должен помнить, о чём уже говорили.

Ответ:

- **Сохранять историю и состояние диалога**
- Ускорять рендер UI
- Отображать лог ошибок

🔮 Шаг 04:

Миссия 4

🧠 Сопоставьте компонент агента с его обязанностью:

Компоненты:

Ответ:

- Модуль восприятия
- Планировщик
- Исполнитель (Executor)
- Память агента

Функции:

1. Хранит факты и предыдущие шаги
2. Получает и превращает входной сигнал в понятную форму
3. Запускает шаги агента (выполняет команды)
4. **Формирует стратегию** — какие шаги делать и в каком порядке

Подсказка: Восприятие — это входной модуль, исполнитель — выходной.

- **Модуль восприятия** — Получает и превращает входной сигнал в понятную форму
- **Планировщик** — Формирует стратегию — какие шаги делать и в каком порядке
- **Исполнитель (Executor)** — Запускает шаги агента (выполняет команды)
- **Память агента** — Хранит факты и предыдущие шаги

🔮 Шаг 05:

Миссия 5

🧠 Сопоставьте тип рассуждения агента с его признаком:

Типы рассуждения:

Ответ:

- Вперёд (forward reasoning)
- Назад (backward reasoning)
- Гибридное рассуждение
- Планирование с возвратом (backtracking)

Признаки:

1. Строит шаги от начального состояния к цели
2. Стартует от цели и ищет, что нужно сделать, чтобы её достигнуть
3. **Сочетает оба подхода** — и вперёд, и назад
4. Может откатываться назад при неудаче и пробовать альтернативы

Подсказка: Запоминание путей и возврат к предыдущим ветвям относится к backtracking.

- **Вперёд (forward reasoning)** — Строит шаги от начального состояния к цели
- **Назад (backward reasoning)** — Стартует от цели и ищет, что нужно сделать, чтобы её достигнуть
- **Гибридное рассуждение** — Сочетает оба подхода — и вперёд, и назад
- **Планирование с возвратом (backtracking)** — Может откатываться назад при неудаче и пробовать альтернативы

🔮 Шаг 06:

Миссия 6

🔍 При реализации агента Диана задумалась: можно ли вводить случайность в вариант действий. Какой эффект даёт стохастичность в выборе шага?

Подсказка: Иногда лучше не выбирать всегда один и тот же ответ.

Ответ:

- **Добавляет разнообразие поведения и предотвращает заикливание**
- Всегда снижает качество ответов
- Отвечает быстрее, но менее осмысленно

🔮 Шаг 07:

Миссия 7

📖 Разработчик Павел хочет, чтобы агент сам учился в процессе взаимодействия. Какой механизм поможет?

Подсказка: Это связано с оценкой качества действий и корректировкой поведения.

Ответ:

- **Обратная связь и обучение на основе вознаграждения**
- Хардкод правил
- Просто случайный выбор
- Строгий сценарий без изменений

🔮 Шаг 08:

Миссия 8

🔗 Сопоставьте стратегию рассуждения агента с её преимуществом:

Стратегии:

Ответ:

- Только forward reasoning
- Только backward reasoning
- Гибридное рассуждение
- Backtracking

Преимущества:

1. Простота реализации при одностороннем движении
2. Эффективно, когда цель точна и ограничена
3. Устойчивость к неопределенности и гибкость
4. Позволяет исправлять ошибки и пробовать альтернативы

Подсказка: Гибрид — универсальный подход, но проще — односторонний.

- **Только forward reasoning** — Простота реализации при одностороннем движении
- **Только backward reasoning** — Эффективно, когда цель точна и ограничена
- **Гибридное рассуждение** — Устойчивость к неопределенности и гибкость
- **Backtracking** — Позволяет исправлять ошибки и пробовать альтернативы

🔮 Шаг 09:

Миссия 9

✳ Сопоставьте подход к агенту с характерной особенностью:

Подходы:

Ответ:

- Реактивный агент
- Планирующий агент
- Обучающийся агент
- Модульный агент

Особенности:

1. **Не хранит память** — реагирует на текущее состояние
2. Использует внутренние модели мира и планирует шаги
3. Модифицирует себя со временем на основе опыта
4. Разделён на независимые модули (восприятие, планирование, исполнение)

Подсказка: Многие современные агенты — именно модульные + обучающиеся.

- **Реактивный агент** — Не хранит память — реагирует на текущее состояние
- **Планирующий агент** — Использует внутренние модели мира и планирует шаги
- **Обучающийся агент** — Модифицирует себя со временем на основе опыта
- **Модульный агент** — Разделён на независимые модули (восприятие, планирование, исполнение)

🧙 Шаг 10:

Миссия 10

➡ Расположите этапы рассуждения агента в логическом порядке:

1. Интерпретация входных данных
2. Генерация кандидатов действий
3. Оценка кандидатов и выбор лучшего
4. Выполнение выбранного действия
5. Запись результата в память

Подсказка: Сначала агент воспринимает, затем размышляет и действует, потом обновляет память.

Ответ:

- Интерпретация входных данных
- Генерация кандидатов действий
- Оценка кандидатов и выбор лучшего
- Выполнение выбранного действия
- Запись результата в память

🧙 Шаг 11:

Миссия 11

Представьте агента-ассистента, который планирует по шагам. Расположите стадии планирования:

1. Построение модели мирового состояния
2. Формулирование целей агента
3. Генерация возможных планов
4. Выбор наиболее эффективного плана

5. Исполнение шагов плана

Подсказка: Агент сначала формулирует, что хочет, потом придумывает как.

Ответ:

- Построение модели мирового состояния
- Формулирование целей агента
- Генерация возможных планов
- Выбор наиболее эффективного плана
- Исполнение шагов плана

Шаг 12:

Миссия 12

 Мария добавляет агенту «кору» — слой высшего управления между планировщиком и исполнителем. Зачем нужна кора?

Подсказка: Это что-то вроде «судьи», который может вмешаться.

Ответ:

- **Корректирует план при неожиданностях и конфликтных целях**
- Ускоряет исполнение команд напрямую
- Записывает лог без влияния на поведение

Шаг 13:

Миссия 13

 Артур задумал вводить «ограничение безопасности» в агента, чтобы он не нарушал правила. Как называются такие ограничения?

Подсказка: Это часто называют «контейнером», «сейф-зоной» или «демилитаризованной зоной» внутри агента.

Ответ:

- **Безопасная оболочка (sandbox / safety filter)**
- Глобальная память
- Интерфейс визуализации
- Раффикация логики

Шаг 14:

Миссия 14

♥ Сопоставьте виды ограничений агента с их назначением:

Ограничения:

Ответ:

- Лимиты действий
- Фильтр контента
- Проверка валидности команд
- Ограничения по ресурсам

Назначения:

1. Запрещать недопустимые слова и темы
2. Ограничивать число шагов или время исполнения
3. Убедиться, что команда корректна и безопасна

4. Не дать агенту перегрузить систему

Подсказка: Подумайте: фильтр сдерживает «плохие» ответы, ресурс ограничивает вычисления.

- **Лимиты действий** — Ограничивать число шагов или время исполнения
- **Фильтр контента** — Запрещать недопустимые слова и темы
- **Проверка валидности команд** — Убедиться, что команда корректна и безопасна
- **Ограничения по ресурсам** — Не дать агенту перегрузить систему

Шаг 15:

Миссия 15

 Расположите стадии, как агент проверяет безопасность до выполнения команды:

1. Генерация кандидата команды
2. Проверка на валидность и безопасность
3. Модификация или отклонение опасной команды
4. Запуск безопасной команды исполнителем

Подсказка: Безопасность проверяется перед исполнением — иначе риск слишком велик.

Ответ:

- Генерация кандидата команды
- Проверка на валидность и безопасность
- Модификация или отклонение опасной команды
- Запуск безопасной команды исполнителем

Шаг 16:

Миссия 16

 Когда агент завершил цель, какие шаги он должен выполнить по завершению:

1. Отчёт о достигнутом результате
2. Освобождение ресурсов
3. Запись в память истории и контекста
4. Переход в режим ожидания или новой задачи

Подсказка: Завершение — не просто остановка, но и уборка после себя.

Ответ:

- Отчёт о достигнутом результате
- Освобождение ресурсов
- Запись в память истории и контекста
- Переход в режим ожидания или новой задачи

Шаг 17:

Миссия 17

 В финале проекта команда хочет, чтобы агент мог рассуждать сам о себе: мета-размышление. Что это значит?

Подсказка: Агент оценивает собственные шаги и корректирует свой план.

Ответ:

- **Агент анализирует и корректирует собственную стратегию**
- Агент просто запоминает журнал действий

- Механизм внешнего мониторинга со стороны разработчика

📖 Урок 2 - 🧠 Function calling и подключение инструментов

🔮 Шаг 01:

Миссия 1

Когда агенту LLM нужно вызвать внешнюю функцию через function calling, что важно указать в запросе?

Подсказка: Подумайте, как LLM поймёт, что вы хотите вызвать функцию.

Ответ:

- **specify the function schema and name**
- just include function name without schema
- omit function definition entirely

🔮 Шаг 02:

Миссия 2

Команда разработчика даёт LLM описание инструмента «weather» с функцией get_weather. Как агент должен запросить погоду в Париже?

Подсказка: Подумайте: что LLM передаст в аргументы.

Ответ:

- **call get_weather with {"location":"Paris"}**
- call get_weather with {"city":"Paris"}
- call weather.get(Paris)

🔮 Шаг 03:

Миссия 3

Если агент получает ответ от инструмента, что должен делать дальше?

Подсказка: Подумайте о продолжении диалога после функции.

Ответ:

- **интегрировать результат в ответ пользователю**
- игнорировать результат и отвечать заново
- отправить пустое сообщение

🔮 Шаг 04:

Миссия 4

Когда агент может выбрать между несколькими функциями, что важно предусмотреть?

Подсказка: Подумайте о правильной функции для задачи.

Ответ:

- **указывать аргументы, чтобы выбрать нужную функцию**
- вызывать все функции сразу
- не предоставлять аргументы

🔮 Шаг 05:

Миссия 5

При описании инструмента для агента нужно указать типы аргументов и возвращаемое значение. Зачем?

Подсказка: Подумайте — как LLM поймёт, как строить JSON-аргументы.

Ответ:

- чтобы LLM валидно формировало JSON
- для документации только
- чтобы скрыть аргументы от пользователя

🔮 Шаг 06:

Миссия 6

🎧 Сопоставьте термин из области function calling с его определением.

Подсказка: Подумайте, как LLM взаимодействует с инструментами.

Термины:

Ответ:

- Function schema
- Function call
- Tool response
- Argument validation

Определения:

1. Ответ, который возвращает внешний инструмент после вызова
 2. Структура, описывающая название функции и её аргументы
 3. Процесс проверки аргументов перед вызовом функции
 4. Сам факт вызова функции с JSON-аргументами
- **Function schema** — Структура, описывающая название функции и её аргументы
 - **Function call** — Сам факт вызова функции с JSON-аргументами
 - **Tool response** — Ответ, который возвращает внешний инструмент после вызова
 - **Argument validation** — Процесс проверки аргументов перед вызовом функции

🔮 Шаг 07:

Миссия 7

🎧 Сопоставьте роль участников в архитектуре агента-инструментов.

Подсказка: Подумайте, кто что делает: агент, инструмент, пользователь.

Участники:

Ответ:

- Агент LLM
- Инструмент (tool)
- Пользователь
- Схема функции

Роли/действия:

1. Подготавливает интерфейс вызова функции для агента
2. Задаёт пользователю вопрос или принимает запрос
3. Исполняет функцию и возвращает результат агенту
4. Интерпретирует запрос и решает, какую функцию вызвать

- **Агент LLM** — Интерпретирует запрос и решает, какую функцию вызвать
- **Инструмент (tool)** — Исполняет функцию и возвращает результат агенту
- **Пользователь** — Задает пользователю вопрос или принимает запрос
- **Схема функции** — Подготавливает интерфейс вызова функции для агента

🔮 Шаг 08:

Миссия 8

🧠 Сопоставьте ошибку агента с возможной причиной при function calling.

Подсказка: Подумайте об ошибках JSON, аргументах, имени функции.

Ошибка агента:

Ответ:

- Invalid JSON
- Function not found
- Missing required argument
- Unexpected argument

Причина:

1. Аргумент не описан в схеме
2. Неправильный формат JSON (синтаксическая ошибка) Не указали обязательный ключ аргумента
3. Имя функции не совпадает с объявленным
 - **Invalid JSON** — Неправильный формат JSON (синтаксическая ошибка)
 - **Function not found** — Имя функции не совпадает с объявленным
 - **Missing required argument** — Не указали обязательный ключ аргумента
 - **Unexpected argument** — Аргумент не описан в схеме

🔮 Шаг 09:

Миссия 9

🧠 Расположите шаги создания агента с поддержкой инструментов в правильном порядке:

Подсказка: Представьте, что вы конструируете pipeline.

1. Определить схемы функций для инструментов
2. Интегрировать вызов функции в диалог агента
3. Инструмент возвращает ответ агенту
4. Проверка аргументов перед вызовом функции
5. Агент решает, нужно ли вызвать функцию или ответить сам

Ответ:

- Определить схемы функций для инструментов
- Агент решает, нужно ли вызвать функцию или ответить сам
- Проверка аргументов перед вызовом функции
- Интегрировать вызов функции в диалог агента
- Инструмент возвращает ответ агенту

🔮 Шаг 10:

Миссия 10

🧠 Расположите в хронологическом порядке события при взаимодействии агента и инструмента:

Подсказка: Подумайте, как проходит запрос-ответ.

1. Агент готовит JSON-аргументы
2. Инструмент возвращает данные
3. LLM решает, вызвать функцию или ответить напрямую
4. Проверка аргументов на корректность
5. Инструмент получает вызов и исполняет функцию

Ответ:

- Агент готовит JSON-аргументы
- Проверка аргументов на корректность
- Инструмент получает вызов и исполняет функцию
- Инструмент возвращает данных
- LLM решает, вызвать функцию или ответить напрямую

💡 Шаг 11:

Миссия 11

🧠 Упорядочьте стадии отладки агента с инструментами:

Подсказка: При отладке важно логгировать на каждом этапе.

1. Отслеживание ошибок аргументов
2. Тестовый запрос агента без функций
3. Тест вызова функции вручную
4. Интеграционное тестирование с агентом и инструментом
5. Анализ логов обмена

Ответ:

- Тестовый запрос агента без функций
- Тест вызова функции вручную
- Отслеживание ошибок аргументов
- Анализ логов обмена
- Интеграционное тестирование с агентом и инструментом

💡 Шаг 12:

Миссия 12

🧠 Упорядочите этапы, если агент использует несколько инструментов в одном диалоге:

Подсказка: Важно выбрать, когда вызвать каждый инструмент.

1. Выбор, какой инструмент вызвать первым
2. Подготовка аргументов для первого инструмента
3. Получение ответа и анализ результата
4. Решение, нужен ли второй вызов инструмента
5. Подготовка аргументов и вызов второго инструмента
6. Интеграция итоговых данных и ответ пользователю

Ответ:

- Выбор, какой инструмент вызвать первым
- Подготовка аргументов для первого инструмента

- Получение ответа и анализ результата
- Решение, нужен ли второй вызов инструмента
- Подготовка аргументов и вызов второго инструмента
- Интеграция итоговых данных и ответ пользователю

🔮 Шаг 13:

Миссия 13

🧠 Упорядочите шаги для обработки ошибок от инструмента:

Подсказка: Подумайте, что делать, если инструмент вернул ошибку.

1. Поймать ошибку и логировать её
2. Агент анализирует тип ошибки
3. Попытка повторного вызова или fallback
4. Сообщить пользователю понятным языком
5. Остановить дальнейшие вызовы, если критическая ошибка

Ответ:

- Поймать ошибку и логировать её
- Агент анализирует тип ошибки
- Попытка повторного вызова или fallback
- Сообщить пользователю понятным языком
- Остановить дальнейшие вызовы, если критическая ошибка

🔮 Шаг 14:

Миссия 14

Предположим, инструмент возвращает ответ в поле "result". Как агент должен его вставить в диалог?

Подсказка: Какую формулировку даст пользовательский ответ?

Ответ:

- **Встроить result как часть ответа пользователю**
- Показать полный JSON инструментального ответа
- Игнорировать поле "result"

🔮 Шаг 15:

Миссия 15

Если агент вызывает функцию с неверными типами аргументов, что должно произойти?

Подсказка: Подумайте о безопасности и валидности.

Ответ:

- **Логировать ошибку и отказаться от вызова**
- Попытаться привести типы автоматически
- Игнорировать типы и вызвать любой способ

🔮 Шаг 16:

Миссия 16

🧠 Сопоставьте подход к fallback с ситуацией:

Подсказка: Подумайте, как агент может действовать, если инструмент недоступен.

Сценарии:

Ответ:

- Инструмент временно недоступен
- Инструмент возвращает ошибку формата
- Инструмент дал неожиданные данные
- Инструмент дал частичный ответ

Подходы fallback:

1. Повтор попытки через таймаут или задержку
 2. Вернуться к генерации внутреннего ответа без инструмента
 3. Очистить некорректные части и использовать то, что удалось
 4. Провести валидацию и запросить уточнение у пользователя
- **Инструмент временно недоступен** — Повтор попытки через таймаут или задержку
 - **Инструмент возвращает ошибку формата** — Провести валидацию и запросить уточнение у пользователя
 - **Инструмент дал неожиданные данные** — Вернуться к генерации внутреннего ответа без инструмента
 - **Инструмент дал частичный ответ** — Очистить некорректные части и использовать то, что удалось

Шаг 17:

Миссия 17

 Расположите полную последовательность от запроса пользователя до финального ответа агента:

Подсказка: Включите стадии принятия решения, вызова функции, обработки и ответа.

1. Пользователь задаёт запрос агенту
2. LLM решает, нужен ли вызов функции
3. **Если нужен** — подготовка аргументов и проверка
4. Вызов функции и получение ответа
5. Обработка результата функции агентом
6. Формирование окончательного ответа клиенту

Ответ:

- Пользователь задаёт запрос агенту
- LLM решает, нужен ли вызов функции
- **Если нужен** — подготовка аргументов и проверка
- Вызов функции и получение ответа
- Обработка результата функции агентом
- Формирование окончательного ответа клиенту

Урок 3 - Ollama + LangChain = осознанный агент

Шаг 01:

Миссия 1

Разработчик Анна решила соединить Ollama с LangChain — что первым шагом она должна настроить?

Подсказка: Подумайте, с чего начинается интеграция агента.

Ответ:

- **инициализировать клиент Ollama внутри LangChain**
- писать агентов вручную без LangChain
- называть функции без схем

🔮 **Шаг 02:**

Миссия 2

В диалоге агента нужно предоставить инструменты через LangChain. Как Анна передаёт инструменты в агента?

Подсказка: Подумайте, как LangChain описывает инструменты.

Ответ:

- **через параметр tools при создании агента**
- внутри prompt как текстовое описание
- не передавать инструменты, пусть агент сам ищет

🔮 **Шаг 03:**

Миссия 3

При конфигурации Ollama в LangChain Анна задаёт модель и аргументы. Что важно указать?

Подсказка: Подумайте: модель Ollama должна знать, что она агент с инструментами.

Ответ:

- **enable_function_calling = True**
- disable_streaming = True
- use_only_completion_mode

🔮 **Шаг 04:**

Миссия 4

Агент получил сообщение от пользователя и решает: вызвать инструмент или ответить сам. Как он принимает решение?

Подсказка: Подумайте о ключевых словах и намерении.

Ответ:

- **аналитика намерения: если запрос связан с инструментом — вызвать**
- всегда вызывать инструмент
- никогда не вызывать инструмент

🔮 **Шаг 05:**

Миссия 5

Если инструмент недоступен (сбой сети), что агент должен сделать?

Подсказка: Подумайте о fallback-стратегии.

Ответ:

- **перейти к внутреннему ответу без инструмента**
- бесконечно пытаться вызвать инструмент
- игнорировать ошибку

🔮 Шаг 06:

Миссия 6

🧠 Сопоставьте компоненты агента (Ollama + LangChain) с их обязанностями:

Подсказка: Подумайте, кто что делает при вызове инструмента.

Компоненты:

Ответ:

- LangChain Agent
- LLMChain / PromptTemplate
- Ollama клиент
- Tool в LangChain

Обязанности:

1. отвечает за выбор, вызывать функцию или нет
 2. определяет шаблон запроса и контекст
 3. обеспечивает общение с Ollama-сервером
 4. инкапсулирует логику инструмента и его API
- **LangChain Agent** — отвечает за выбор, вызывать функцию или нет
 - **LLMChain / PromptTemplate** — определяет шаблон запроса и контекст
 - **Ollama клиент** — обеспечивает общение с Ollama-сервером
 - **Tool в LangChain** — инкапсулирует логику инструмента и его API

🔮 Шаг 07:

Миссия 7

🧠 Сопоставьте этапы взаимодействия агента с инструментом в контексте Ollama + LangChain:

Подсказка: Подумайте о передаче данных между слоями.

Этапы:

Ответ:

- Agent вызывает функцию
- Ollama получает запрос
- Tool исполняет действие
- Ответ возвращается обратно

Связь:

1. langchain → ollama
 2. ollama → tool
 3. tool → ollama
 4. ollama → langchain
- **Agent вызывает функция** — langchain → ollama
 - **Ollama получает запрос** — ollama → tool
 - **Tool исполняет действие** — tool → ollama
 - **Ответ возвращается обратно** — ollama → langchain

🔮 Шаг 08:

Миссия 8

🧠 Сопоставьте ситуации с поведением агента:

Подсказка: Подумайте, как агент реагирует на ошибки.

Ситуации:

Ответ:

- Инструмент возвращает JSON-ошибку
- Инструмент вернул пустой ответ
- Инструмент отвечает с задержкой тайм-аута
- Инструмент возвращает валидный результат

Поведение:

1. логировать и fallback на ответ LLM
 2. запросить уточнение у пользователя
 3. повторить запрос с задержкой
 4. внедрить результат в ответ пользователю
- **Инструмент возвращает JSON-ошибку** — логировать и fallback на ответ LLM
 - **Инструмент вернул пустой ответ** — запросить уточнение у пользователя
 - **Инструмент отвечает с задержкой тайм-аута** — повторить запрос с задержкой
 - **Инструмент возвращает валидный результат** — внедрить результат в ответ пользователю

Шаг 09:

Миссия 9

🧠 Сопоставьте термины, используемые при агентах Ollama + LangChain:

Подсказка: Подумайте о «chain», «agent», «tool», «callback».

Термины:

Ответ:

- Chain
- Agent
- Tool
- Callback

Описание:

1. цепочка логики обработки запроса
 2. контроллер, принимающий решения
 3. объект внешней функции / API
 4. место для логирования событий и ошибок
- **Chain** — цепочка логики обработки запроса
 - **Agent** — контроллер, принимающий решения
 - **Tool** — объект внешней функции / API
 - **Callback** — место для логирования событий и ошибок

Шаг 10:

Миссия 10

🧠 Расположите шаги, которые Анна выполняет при настройке агента Ollama + LangChain:

Подсказка: Подумайте, как растёт абстракция.

1. описать инструменты и схемы функций

2. настроить клиент Ollama
3. создать экземпляр агента LangChain с tools и llm
4. настроить цепочки / промпты
5. запустить тестовый диалог

Ответ:

- описать инструменты и схемы функций
- настроить клиент Ollama
- создать экземпляр агента LangChain с tools и llm
- настроить цепочки / промпты
- запустить тестовый диалог

 Шаг 11:

Миссия 11

💡 Расположите стадии обработки пользовательского запроса агентом на Ollama + LangChain:

Подсказка: Включите выбор, вызов и ответ.

1. получить запрос пользователя
2. agent решает вызвать инструмент или нет
3. **если да** — подготовка аргументов
4. вызов Ollama / инструмент
5. получить ответ инструмента и интегрировать
6. ответить пользователю

Ответ:

- получить запрос пользователя
- agent решает вызвать инструмент или нет
- **если да** — подготовка аргументов
- вызов Ollama / инструмент
- получить ответ инструмента и интегрировать
- ответить пользователю

 Шаг 12:

Миссия 12

💡 Расположите шаги отладки агента Ollama + LangChain:

Подсказка: Вспомните: unit → integration → логирование.

1. тестировать отдельные инструменты
2. логировать ввод / вывод агента
3. интеграционное тестирование агента с инструментами
4. отлавливать ошибки аргументов
5. тестировать агент без инструментов

Ответ:

- тестировать отдельные инструменты
- логировать ввод / вывод агента
- интеграционное тестирование агента с инструментами
- отлавливать ошибки аргументов
- тестировать агент без инструментов

Шаг 13:

Миссия 13

💡 Расположите возможные fallback-стратегии агента (при сбоях инструмента):

Подсказка: От простого к более сложному.

1. ответить пользователю сообщением «инструмент недоступен»
2. использовать внутренний генератор LLM без инструментов
3. запросить уточнение у пользователя
4. повторить вызов с задержкой
5. сформировать частичный ответ из того, что получилось

Ответ:

- ответить пользователю сообщением «инструмент недоступен»
- использовать внутренний генератор LLM без инструментов
- запросить уточнение у пользователя
- повторить вызов с задержкой
- сформировать частичный ответ из того, что получилось

Шаг 14:

Миссия 14

💡 Упорядочите стадии, если агент вызывает несколько инструментов последовательно:

Подсказка: Важно логика, ответ и фильтрация.

1. решить вызвать первый инструмент
2. обработать результат первого инструмента
3. решить, нужен второй инструмент
4. вызвать второй инструмент при необходимости
5. интегрировать результаты и ответить пользователю

Ответ:

- решить вызвать первый инструмент
- обработать результат первого инструмента
- решить, нужен второй инструмент
- вызвать второй инструмент при необходимости
- интегрировать результаты и ответить пользователю

Шаг 15:

Миссия 15

💡 Упорядочите этапы построения осознанного агента на Ollama + LangChain с callback'ами:

Подсказка: Включите моменты логирования и прерывания.

1. настройка callback для логов
2. определение инструментов и схем функций
3. создание агента с включённым callback
4. тестовый запрос с логированием
5. анализ логов и исправления

Ответ:

- настройка callback для логов

- определение инструментов и схем функций
- создание агента с включённым callback
- тестовый запрос с логированием
- анализ логов и исправления

🔮 Шаг 16:

Миссия 16

🧠 Расположите полную цепочку от пользовательского запроса до логированного ответа:

Подсказка: Включите callback'и, выбор инструментов, вызов и ответ.

1. запрос пользователя приходит в агент
2. agent оценивает, нужен инструмент
3. callback логирования входа
4. **если нужен** — подготовка аргументов и вызов инструмента
5. получить ответ и обработать
6. callback логирования результата
7. ответ агента пользователю

Ответ:

- запрос пользователя приходит в агент
- agent оценивает, нужен инструмент
- callback логирования входа
- **если нужен** — подготовка аргументов и вызов инструмента
- получить ответ и обработать
- callback логирования результата
- ответ агента пользователю

📖 Урок 4 - 🖥️ Подключение API и выполнение команд

🔮 Шаг 01:

Миссия 1

Вы запускаете Ollama агент и получаете сообщение: «Cannot connect to API». Что первоочередно проверить?

Подсказка: Проблема может быть на стороне сети или конфигурации адреса.

Ответ:

- **Проверить, правильно ли указан URL и порт API**
- Изменить код агента на другой язык программирования
- Перезапустить весь сервер без анализа логов
- Игнорировать ошибку и продолжить работу

🔮 Шаг 02:

Миссия 2

🖥️ Вы хотите вызвать команду через HTTP API: /v1/command . Какой HTTP-метод наиболее логичен?

Подсказка: Команда изменяет состояние — подумайте о семантике методов.

Ответ:

- **POST**

- GET
- DELETE
- HEAD

🔮 Шаг 03:

Миссия 3

🔗 Вы заметили, что ваш API-ключ передаётся в URL как параметр. Что безопаснее?

Подсказка: Возьмите лучший вариант защиты конфиденциальных данных.

Ответ:

- **Передавать ключ в заголовке Authorization**
- Оставить в URL
- Логгировать ключ в открытом виде
- Шифровать весь HTTP трафик вручную

🔮 Шаг 04:

Миссия 4

🕒 Вы вызываете API, и результат медленный. Что можно попробовать первым?

Подсказка: Иногда узким местом является задержка сети или ожидание ответа.

Ответ:

- **Установить таймаут меньшим значением и повторить запрос**
- Увеличить глубину вложенности агента
- Запустить API на другом хосте без анализа логов
- Отключить шифрование

🔮 Шаг 05:

Миссия 5

🌐 Вы хотите подключить внешнюю службу (например, веб-хук) к Ollama агенту. Какую часть API вы, скорее всего, используете?

Подсказка: Это действие — отправка данных на сервер.

Ответ:

- **POST /v1/callback**
- GET /v1/callback
- PUT /v1/command
- DELETE /v1/callback

🔮 Шаг 06:

Миссия 6

Сопоставьте HTTP-статус кода с ситуацией в API Ollama:

Коды:

Ответ:

- 200
- 400
- 401
- 500

Ситуации:

1. Неправильный формат JSON тела запроса
2. Успешное выполнение команды
3. Серверная ошибка в агенте
4. Неверный или отсутствующий API-ключ

Подсказка: Представьте, как API отвечает на ошибку клиента или своего сбоя.

- **200** — Успешное выполнение команды
- **400** — Неправильный формат JSON тела запроса
- **401** — Неверный или отсутствующий API-ключ
- **500** — Серверная ошибка в агенте

Шаг 07:

Миссия 7

 Сопоставьте параметры API с их назначением:

Параметры:

Ответ:

- timeout
- retries
- endpoint
- headers

Назначения:

1. Максимальное число попыток при неудаче
2. URL-адрес вызова API
3. HTTP заголовки, например, Authorization
4. Ожидание ответа в миллисекундах

Подсказка: Подумайте, что делают заголовки, сколько раз пробуем, куда шлём, сколько ждём.

- **timeout** — Ожидание ответа в миллисекундах
- **retries** — Максимальное число попыток при неудаче
- **endpoint** — URL-адрес вызова API
- **headers** — HTTP заголовки, например, Authorization

Шаг 08:

Миссия 8

 Сопоставьте команды агента с их назначением:

Команды:

Ответ:

- start_agent
- stop_agent
- reload_model
- send_message

Назначения:

1. Остановить работу агента

2. Перезагрузить используемую модель
3. Запустить агента из состояния ожидания
4. Отправить сообщение агенту и получить ответ

Подсказка: Команды называются прямо по действию.

- **start_agent** — Запустить агента из состояния ожидания
- **stop_agent** — Остановить работу агента
- **reload_model** — Перезагрузить используемую модель
- **send_message** — Отправить сообщение агенту и получить ответ

Шаг 09:

Миссия 9

 Сопоставьте схемы URL с действием агента:

Схемы:

Ответ:

- /v1/agent/status
- /v1/agent/command
- /v1/agent/config
- /v1/agent/logs

Действия:

1. Получение текущего состояния агента
2. Отправка команды агенту
3. Получение или корректировка конфигурации агента
4. Получение логов агента

Подсказка: Подумайте, что означает часть пути после /agent/.

- **/v1/agent/status** — Получение текущего состояния агента
- **/v1/agent/command** — Отправка команды агенту
- **/v1/agent/config** — Получение или корректировка конфигурации агента
- **/v1/agent/logs** — Получение логов агента

Шаг 10:

Миссия 10

 Сопоставьте параметры команды агента с их эффектом:

Параметры:

Ответ:

- timeout_ms
- payload
- async
- priority

Эффекты:

1. Данные, которые команда передаёт агенту
2. Время ожидания ответа
3. Запрос выполнить асинхронно, не дожидаясь ответа
4. Приоритет выполнения команды при очередности

Подсказка: Названия параметров прямо намекают на смысл.

- **timeout_ms** — Время ожидания ответа
- **payload** — Данные, которые команда передаёт агенту
- **async** — Запрос выполнить асинхронно, не дожидаясь ответа
- **priority** — Приоритет выполнения команды при очередности

🔧 Шаг 11:

Миссия 11

Расположите шаги отправки команды агенту через API в правильном порядке:

1. Обработать ответ агента
2. Сформировать тело запроса с нужными параметрами
3. Отправить HTTP запрос к API
4. Проверить корректность ответа и статус

Подсказка: Представьте жизненный цикл запроса.

Ответ:

- Сформировать тело запроса с нужными параметрами
- Отправить HTTP запрос к API
- Проверить корректность ответа и статус
- Обработать ответ агента

🔧 Шаг 12:

Миссия 12

🔧 Расположите шаги отладки ошибки «timeout» в вашем клиенте:

1. Увеличить значение timeout
2. Проверить сетевое соединение и пинги
3. Проанализировать логи агента и трассировки
4. Выполнить повторный запрос

Подсказка: От простого к сложному.

Ответ:

- Проверить сетевое соединение и пинги
- Увеличить значение timeout
- Проанализировать логи агента и трассировки
- Выполнить повторный запрос

🔧 Шаг 13:

Миссия 13

🔧 Расположите типичные реакции агента при запроса send_message (в порядке возникновения):

1. Возвратить поле status
2. Выполнить внутреннюю логику и передать на модель
3. Ответить с данными или ошибкой
4. Принять и распознать входящее сообщение

Подсказка: Подумайте, как сообщение проходит через агент внутрь и обратно.

Ответ:

- Принять и распознать входящее сообщение
- Выполнить внутреннюю логику и передать на модель
- Возвратить поле status
- Ответить с данными или ошибкой

 Шаг 14:

Миссия 14

 Расположите фазы жизненного цикла агента при запуске API-микросервиса:

1. Регистрация на discovery / service registry
2. Загрузка модели и инициализация зависимостей
3. Открытие HTTP-конца API
4. Запуск фоновых задач агента

Подсказка: Что должно быть готово перед тем, как агент начнёт отвечать?

Ответ:

- Загрузка модели и инициализация зависимостей
- Запуск фоновых задач агента
- Открытие HTTP-конца API
- Регистрация на discovery / service registry

 Шаг 15:

Миссия 15

Расположите этапы надёжного взаимодействия клиента с агентом через API:

1. Обработка ошибок и повторов
2. Подготовка запроса с API-ключом и заголовками
3. Анализ ответа и проверка полей status/errors
4. Отправка запроса
5. Валидация результата (например, schema validate)

Подсказка: Подумайте, как проходит защищённый и устойчивый цикл запроса.

Ответ:

- Подготовка запроса с API-ключом и заголовками
- Отправка запроса
- Обработка ошибок и повторов
- Анализ ответа и проверка полей status/errors
- Валидация результата (например, schema validate)

 Урок 5 - Квест агент, который исправляет HTML и тестирует результат

 Шаг 01:

Миссия 1

Вы — инженер-агент, и ваш первый шаг — понять, что сломано в HTML отчёте. Что вы проверите первым?

Подсказка: Начните с основы документа — структура, теги <html>, <head>, <body>.

Ответ:

- **Наличие корневых тегов `<html>` и `<body>`**
- Цвет фона страницы
- Размер шрифта заголовка
- Количество внешних CSS-файлов

🔮 Шаг 02:

Миссия 2

🌐 Агент нашёл тег `<div id="wrapper">`, но внутри него есть незакрытый `<p>`. Какой подход предпочтительнее?

Подсказка: Автоматически закрывать теги — риск; лучше исправить структуру.

Ответ:

- **Добавить закрывающий `</p>` там, где нужно**
- Удалить весь блок `<p>`
- Оставить как есть, браузер сам закроет

🔮 Шаг 03:

Миссия 3

📁 Вы заметили, что тег `<img>` не содержит атрибут `alt`. Что лучше сделать?

Подсказка: Атрибут `alt` — важен для доступности и SEO.

Ответ:

- **Добавить информативный `alt`**
- Оставить без `alt`
- Удалить тег `<img>` полностью

🔮 Шаг 04:

Миссия 4

🔗 Агент хочет убедиться, что после правок страница всё ещё валидна. Какой инструмент вы бы выбрали?

Подсказка: Ищите валидатор HTML, который проверит структуру и вложенность тегов.

Ответ:

- **W3C HTML Validator**
- CSS Lint
- JavaScript консоль браузера

🔮 Шаг 05:

Миссия 5

📱 После правок агент наблюдает, что один раздел страницы отображается неправильно в мобильной версии. Что проверите сначала?

Подсказка: Начните с мета-тегов и адаптивных стилей.

Ответ:

- **Мета-тег `viewport` и медиа-запросы CSS**
- Шрифты заголовков
- Картинки большого размера

- Порядок скриптов

🔮 Шаг 06:

Миссия 6

Агент корректирует HTML-фрагменты и сопоставляет термины со значениями.

Подсказка: Вспомните, что такое «валидность», «семантика», «доступность». Не путайте их.

Термины:

Ответ:

- Валидность
- Семантика
- Доступность
- Кроссбраузерность

Определения:

1. Поддержка разных браузеров и устройств
 2. Соответствие документа стандартам HTML
 3. Логичное использование тегов по смыслу
 4. Удобство для пользователей с ограничениями
- **Валидность** — Соответствие документа стандартам HTML
 - **Семантика** — Логичное использование тегов по смыслу
 - **Доступность** — Удобство для пользователей с ограничениями
 - **Кроссбраузерность** — Поддержка разных браузеров и устройств

🔮 Шаг 07:

Миссия 7

✂ Агент хочет распределить HTML-ошибки по категориям. Сопоставьте ошибку с категорией.

Подсказка: Некоторые ошибки — структурные, другие — логические или визуальные.

Ошибки:

Ответ:

- Незакрытый тег
- Неправильное вложение элементов (`<p>` внутри `<span>`)
- Отсутствие атрибута `alt` в `<img>`
- Использование устаревшего тега `<font>`

Категории:

1. Структурная ошибка
 2. Семантическая ошибка
 3. Доступность
 4. Устаревшие теги / устаревшие практики
- **Незакрытый тег** — Структурная ошибка
 - **Неправильное вложение элементов (`<p>` внутри `<span>`)** — Семантическая ошибка
 - **Отсутствие атрибута `alt` в `<img>`** — Доступность
 - **Использование устаревшего тега `<font>`** — Устаревшие теги / устаревшие практики

Шаг 08:

Миссия 8

📁 Агент систематизирует типы тестов HTML-кодов. Сопоставьте тип теста и его цель.

Подсказка: Тесты могут проверять структуру, визуалку и поведение.

Типы тестов:

Ответ:

- Юнит-тест фрагмента HTML
- Скриншотный тест (визуальный)
- Интеграционный тест с браузером
- Линтер HTML

Цели:

1. Проверка визуального соответствия макету
 2. Проверка структуры, закрытия тегов и порядка вложенности
 3. Проверка взаимодействия HTML с CSS и JS
 4. Автоматическая проверка на ошибки синтаксиса и правилах кодирования
- **Юнит-тест фрагмента HTML** — Проверка структуры, закрытия тегов и порядка вложенности
 - **Скриншотный тест (визуальный)** — Проверка визуального соответствия макету
 - **Интеграционный тест с браузером** — Проверка взаимодействия HTML с CSS и JS
 - **Линтер HTML** — Автоматическая проверка на ошибки синтаксиса и правилах кодирования

Шаг 09:

Миссия 9

Агент описал последовательность этапов исправления HTML и тестирования. Упорядочьте шаги:

1. Написание юнит-теста для фрагмента HTML
2. Линтинг и первичная проверка синтаксиса
3. Интеграционное тестирование в браузере
4. Просмотр визуального сравнения (скриншотный тест)
5. Проверка и исправление структуры HTML

Подсказка: Сначала проверяем синтаксис, потом исправляем, затем тестируем частями, после — визуально, и наконец весь результат.

Ответ:

- Линтинг и первичная проверка синтаксиса
- Проверка и исправление структуры HTML
- Написание юнит-теста для фрагмента HTML
- Интеграционное тестирование в браузере
- Просмотр визуального сравнения (скриншотный тест)

Шаг 10:

Миссия 10

⌘ Агент устанавливает порядок приоритета исправлений HTML-ошибок. Упорядочьте от самого важного к менее критичному:

1. Синтаксические ошибки (незакрытые теги, незавершённые атрибуты)
2. Ошибки доступности (отсутствие alt, aria-атрибутов) Неправильная семантика (использование <div> вместо семантических тегов)
3. Визуальные расхождения макета
4. Избыточные комментарии и пустые теги

Подсказка: Сначала исправляем то, что ломает страницу, потом удобство, затем эстетику.

Ответ:

- Синтаксические ошибки (незакрытые теги, незавершённые атрибуты)
- Ошибки доступности (отсутствие alt, aria-атрибутов)
- Неправильная семантика (использование <div> вместо семантических тегов)
- Визуальные расхождения макета
- Избыточные комментарии и пустые теги

💡 Шаг 11:

Миссия 11

Агент оценивает шаги валидации и тестирования страницы. Расставьте последовательность:

1. Запуск W3C валидатора
2. Автоматический линтинг HTML
3. Юнит-тест отдельных компонентов
4. Интеграционное тестирование страницы в браузере
5. Сравнение скриншотов с эталоном

Подсказка: Сначала статические проверки, потом фрагменты, потом целый контекст и визуал.

Ответ:

- Автоматический линтинг HTML
- Запуск W3C валидатора
- Юнит-тест отдельных компонентов
- Интеграционное тестирование страницы в браузере
- Сравнение скриншотов с эталоном

💡 Шаг 12:

Миссия 12

🔧 Агент планирует процесс правок и тестирования полной страницы. Упорядочьте этапы:

1. Создание резервной копии исходного HTML
2. Линтинг и исправление синтаксических ошибок
3. Мелкие правки структуры и семантики
4. Юнит-тесты фрагментов после правок
5. Интеграционное тестирование в эмуляторе браузера
6. Скриншотное сравнение визуала

Подсказка: Сначала сохраните, затем проверяйте синтаксис, потом структуру, потом тесты, и в конце визуал.

Ответ:

- Создание резервной копии исходного HTML

- Линтинг и исправление синтаксических ошибок
- Мелкие правки структуры и семантики
- Юнит-тесты фрагментов после правок
- Интеграционное тестирование в эмуляторе браузера
- Скриншотное сравнение визуала

Шаг 13:

Миссия 13

🔍 Агент упорядочивает приоритетные проверки после финального правка. Порядок от наиболее важного к менее важному:

1. Валидность HTML
2. Проверка доступности (alt, aria)
3. Кроссбраузерное тестирование
4. Визуальное сравнение со скриншотом
5. Удаление комментариев и избыточного кода

Подсказка: Сначала технические проверки, затем удобство, далее визуальный контроль, и потом чистота кода.

Ответ:

- Валидность HTML
- Проверка доступности (alt, aria)
- Кроссбраузерное тестирование
- Визуальное сравнение со скриншотом
- Удаление комментариев и избыточного кода

Шаг 14:

Миссия 14

Агент тестирует несколько версий страницы. Упорядочите версии по приоритету проверки:

1. Версия до правок (baseline)
2. Версия с исправлениями структуры
3. Версия с исправлениями доступности
4. Версия с визуальными правками
5. Финальная версия после всех тестов

Подсказка: Сравнивайте изменения шаг за шагом, постепенно наращивая исправления.

Ответ:

- Версия до правок (baseline)
- Версия с исправлениями структуры
- Версия с исправлениями доступности
- Версия с визуальными правками
- Финальная версия после всех тестов

Шаг 15:

Миссия 15

✧ Агент готовит финальный чек-лист для релиза HTML. Упорядочите пункты чек-листа по логике:

1. Проверка валидности HTML
2. Оценка доступности (alt, aria, заголовки)
3. Кроссбраузерное тестирование
4. Сравнение скриншотов
5. Удаление избыточного кода

Подсказка: Чек-лист — как экзамен: сначала технические пункты, потом визуал, в конце чистота.

Ответ:

- Проверка валидности HTML
- Оценка доступности (alt, aria, заголовки)
- Кроссбраузерное тестирование
- Сравнение скриншотов
- Удаление избыточного кода

🔮 Шаг 16:

Миссия 16

🕒 Агент планирует поддержку правок. Упорядочите рекомендации по поддержке исправленного HTML:

1. Настроить линтер и правила для HTML
2. Внедрить юнит-тесты для будущих фрагментов
3. Автоматические визуальные тесты при деплое
4. Регулярная проверка доступности
5. Периодический прогон W3C валидатора

Подсказка: Постоянство важнее разовых правок — думайте о будущем.

Ответ:

- Настроить линтер и правила для HTML
- Внедрить юнит-тесты для будущих фрагментов
- Автоматические визуальные тесты при деплое
- Регулярная проверка доступности
- Периодический прогон W3C валидатора

🔮 Шаг 17:

Миссия 17

🧩 Финальный квест: агент собирает весь рабочий процесс исправления и тестирования HTML. Упорядочите все основные этапы:

1. Сохранение резервной копии оригинала
2. Линтинг и первичная проверка синтаксиса
3. Исправление структуры и тегов
4. Проверка доступности (alt, aria)
5. Юнит-тестирование фрагментов
6. Интеграционное тестирование в браузере
7. Скриншотное сравнение визуала
8. Удаление избыточного кода и финальные правки

Подсказка: Это итоговый маршрут агента — от копии к финальной чистой версии с тестами и визуальной проверкой.

Ответ:

- Сохранение резервной копии оригинала
- Линтинг и первичная проверка синтаксиса
- Исправление структуры и тегов
- Проверка доступности (alt, aria)
- Юнит-тестирование фрагментов
- Интеграционное тестирование в браузере
- Скриншотное сравнение визуала
- Удаление избыточного кода и финальные правки

Модуль 7 - ⚡ Миссия 7. Оптимизация ядра

Урок 1 - 📊 Принципы квантования Q4, Q5, Q8

Шаг 01:

Миссия 1

★ Инженер Маша пытается выбрать битовую глубину квантования для новой модели. Какое преимущество даёт квантование до Q4?

Подсказка: Меньше бит — меньше память и объём модели.

Ответ:

- **Сильно уменьшенный размер модели и меньшие требования к памяти**
- Максимальная точность вывода на всех задачах
- Отсутствие деградации в скорости генерации

Шаг 02:

Миссия 2

🕒 Павел экспериментирует: он уже сделал Q8-квантование, но думает о Q5. Что может быть риском при переходе на Q5?

Подсказка: Меньшее количество бит может ухудшить представление чисел.

Ответ:

- **Увеличение ошибки аппроксимации весов**
- Увеличение размера модели
- Полное отсутствие ускорения вывода

Шаг 03:

Миссия 3

🔗 Когда стоит применять Q8-квантование в инженерной практике?

Подсказка: Q8 часто выбирают как компромисс между размером и качеством.

Ответ:

- **Когда нужен баланс между качеством и ресурсами**
- Когда важна максимальная точность даже при малых ресурсах
- Когда важна простота реализации выше всего

🔮 Шаг 04:

Миссия 4

🔮 Коллега предлагает гибридный подход: часть слоёв квантовать до Q4, часть — до Q8. Какое преимущество будет?

Подсказка: Можно квантовать «меньше чувствительные» слои агрессивнее.

Ответ:

- Сбалансировать уменьшение размера и сохранение качества
- Упростить весь конвейер обучения
- Исключить необходимость проверки точности

🔮 Шаг 05:

Миссия 5

🔮 При переходе с Q8 на Q4 команда заметила «шумы» в генерации. Что из этого вероятно причина?

Подсказка: Ошибки округления сильнее при малом числе бит.

Ответ:

- Ошибки квантования весов приводят к искажениям
- Переполнение буфера ввода
- Неправильное форматирование вывода

🔮 Шаг 06:

Миссия 6

Если вы хотите протестировать три варианта битности — Q4, Q5 и Q8 — какую стратегию лучше выбрать?

Подсказка: Сначала протестируйте самые агрессивные режимы, потом «откатитесь».

Ответ:

- Сначала Q4 и Q5, затем Q8 в резерве
- Сначала Q8, потом Q5 и Q4
- Только Q8 сразу

🔮 Шаг 07:

Миссия 7

Команда хочет понять, как разные битности соответствуют диапазону представления. Сопоставьте битность и её типичный диапазон (в относительных единицах).

Подсказка: Смотрите: Q4 — очень ограниченный, Q8 — почти как обычное целое 8-битное.

Битности:

Ответ:

- Q4
- Q5
- Q8

Диапазоны:

1. ~16 уровней (−8 ... +7)
2. ~32 уровня (−16 ... +15)
3. ~256 уровней (−128 ... +127)

- **Q4** — ~16 уровней (−8 ... +7)
- **Q5** — ~32 уровня (−16 ... +15)
- **Q8** — ~256 уровней (−128 ... +127)

🔮 Шаг 08:

Миссия 8

🏢 Инженер Лев хочет сопоставить влияние битности на характеристики. Сопоставьте битность и эффект.

Подсказка: При снижении битности — размер уменьшается, но возможна потеря качества.

Битности:

Ответ:

- Q4
- Q5
- Q8

Эффекты:

1. Минимальный размер, высокие потери качества
 2. Средний компромисс между качеством и размером
 3. Максимальное качество среди квантованных режимов
- **Q4** — Минимальный размер, высокие потери качества
 - **Q5** — Средний компромисс между качеством и размером
 - **Q8** — Максимальное качество среди квантованных режимов

🔮 Шаг 09:

Миссия 9

✂ Разработчики хотят понять, к каким частям модели применять агрессивное квантование. Сопоставьте компонент и рекомендацию.

Подсказка: Некоторые слои менее чувствительны к ошибкам — их можно квантовать жёстче.

Компоненты:

Ответ:

- Слои внимания (attention)
- Выходные проекции (output heads)
- Промежуточные слои (intermediate dense)

Рекомендации:

1. Оставить чуть более высокую битность (Q8) для сохранения качества
 2. Можно агрессивно квантовать (Q4 или Q5)
 3. Использовать среднюю битность (Q5) как компромисс
- **Слои внимания (attention)** — Оставить чуть более высокую битность (Q8) для сохранения качества
 - **Выходные проекции (output heads)** — Можно агрессивно квантовать (Q4 или Q5)
 - **Промежуточные слои (intermediate dense)** — Использовать среднюю битность (Q5) как компромисс

🔮 Шаг 10:

Миссия 10

🧑‍🔬 Команда анализирует ошибки при разной битности. Сопоставьте битность и ожидаемую ошибку RMSE (в абстрактных единицах).

Подсказка: Чем меньше битов — тем выше ошибка квантования.

Битности:

Ответ:

- Q4
- Q5
- Q8

Ошибки:

1. ≈ 0.20
 2. ≈ 0.10
 3. ≈ 0.02
- **Q4** — ≈ 0.20
 - **Q5** — ≈ 0.10
 - **Q8** — ≈ 0.02

🔮 Шаг 11:

Миссия 11

👤 Инженер Кира хочет сопоставить задачу и подходящую битность. Сопоставьте задачу и рекомендуемую битность.

Подсказка: Для чувствительных задач — более высокая битность.

Задачи:

Ответ:

- Ранжирование и точность (precision tasks)
- Генерация текста общего характера
- Сжатие эмбеддингов

Рекомендации битности:

1. Q8
 2. Q5 или Q8
 3. Q4 или Q5
- **Ранжирование и точность (precision tasks)** — Q8
 - **Генерация текста общего характера** — Q5 или Q8
 - **Сжатие эмбеддингов** — Q4 или Q5

🔮 Шаг 12:

Миссия 12

Порядок шагов для внедрения квантования модели. Расположите шаги в правильной последовательности:

1. Оценка точности на исходной модели
2. Квантуем веса с выбранной битностью (Q4/Q5/Q8)
3. Финетюнинг или корректировка шкал (если нужно)

4. Сравнение качества между битностями
5. Выбор режима, который подходит для продакшена

Подсказка: Сначала измерения, затем эксперименты, потом выбор.

Ответ:

- Оценка точности на исходной модели
- Квантуем веса с выбранной битностью (Q4/Q5/Q8)
- Финетюнинг или корректировка шкал (если нужно)
- Сравнение качества между битностями
- Выбор режима, который подходит для продакшена

Шаг 13:

Миссия 13

 Упорядочите этапы при смешанном квантовании (гибрид Q4 и Q8):

1. Выделить слои, чувствительные к точности
2. Квантуем менее чувствительные слои до Q4
Квантуем чувствительные слои до Q8
3. Оценить итоговую модель в целевых задачах
4. При необходимости скорректировать битности

Подсказка: Сначала анализ слоёв, потом квантование и проверка.

Ответ:

- Выделить слои, чувствительные к точности
- Квантуем менее чувствительные слои до Q4
- Квантуем чувствительные слои до Q8
- Оценить итоговую модель в целевых задачах
- При необходимости скорректировать битности

Шаг 14:

Миссия 14

 Порядок исследования ошибок квантования при разных битностях:

1. Квантуем модель до Q4
2. Считаем метрики качества на тестовом наборе
3. Квантуем модель до Q5
4. Считаем метрики качества для Q5
5. Квантуем модель до Q8 и сравниваем результаты

Подсказка: Сравнивайте по шагам, а не сразу всё разом.

Ответ:

- Квантуем модель до Q4
- Считаем метрики качества на тестовом наборе
- Квантуем модель до Q5
- Считаем метрики качества для Q5
- Квантуем модель до Q8 и сравниваем результаты

Шаг 15:

Миссия 15

🕒 Порядок мер защиты от деградации после агрессивного квантования:

1. ЛИНЕЙНАЯ корректировка шкал (calibration)
2. Финетюнинг некоторых слоёв
3. Обратный переход на более высокий бит при сильном падении качества
4. Сравнение модели с базовой версией
5. Мониторинг качественных метрик

Подсказка: После квантования часто нужно калибровать и проверять.

Ответ:

- ЛИНЕЙНАЯ корректировка шкал (calibration)
- Финетюнинг некоторых слоёв
- Обратный переход на более высокий бит при сильном падении качества
- Сравнение модели с базовой версией
- Мониторинг качественных метрик

🔧 Шаг 16:

Миссия 16

🕒 Как логично ранжировать подходы по увеличению сложности (от простого к сложному):

1. Q8-квантование базовой модели
2. Q5-квантование всей модели
3. Адаптивное (гибридное) квантование Q4+Q8
4. Квантуем + финетюнинг и калибровка
5. Итеративный выбор битности по слоям

Подсказка: Сложность растёт по мере гибкости и тонкой настройки.

Ответ:

- Q8-квантование базовой модели
- Q5-квантование всей модели
- Адаптивное (гибридное) квантование Q4+Q8
- Квантуем + финетюнинг и калибровка
- Итеративный выбор битности по слоям

🔧 Шаг 17:

Миссия 17

🔧 Полный порядок экспериментов команды ЛЛМ-инженеров при исследовании квантования:

1. Оценка исходной модели (baseline)
2. Квантование в Q8 и оценка качества
3. Квантование в Q5 и оценка качества
4. Квантование в Q4 и оценка качества
5. Сравнение режимов и выбор лучшего
6. Финетюнинг / калибровка выбранного варианта

Подсказка: Начинайте с менее агрессивных режимов, затем двигайтесь к более жёстким, анализируя на каждом шаге.

Ответ:

- Оценка исходной модель (baseline)
- Квантование в Q8 и оценка качества

- Квантование в Q5 и оценка качества
- Квантование в Q4 и оценка качества
- Сравнение режимов и выбор лучшего
- Финетюнинг / калибровка выбранного варианта

Урок 2 - Сравнение производительности CPU GPU

Шаг 01:

Миссия 1

 Анна запускает модель на CPU и GPU — что она заметит при работе на GPU?

Подсказка: GPU оптимизирован для параллелизма и матричных операций.

Ответ:

- **Ускорение вычислений при больших батчах**
- Замедление операций на векторных данных
- Снижение энергопотребления без потери скорости

Шаг 02:

Миссия 2

 Андрей сравнивает время инференса на одном и том же слое: CPU даёт 300 мс, GPU — 50 мс. Что это иллюстрирует?

Подсказка: Основная сила GPU — параллельная обработка.

Ответ:

- **Преимущество GPU при параллельных операциях**
- CPU эффективнее при больших матрицах
- Одинаковая архитектура, просто разная реализация

Шаг 03:

Миссия 3

 Валя думает: если модель маленькая и слой — всего 32×32, будет ли GPU всегда выигрывать?

Подсказка: Учтите накладные расходы на передачу данных.

Ответ:

- **Не обязательно** — CPU может быть быстрее из-за накладных расходов
- GPU всегда быстрее при любых матрицах
- Скорость одинакова независимо от размера

Шаг 04:

Миссия 4

 Когда модель обрабатывает батч размером 1, на каком устройстве скорее всего будет минимальное время отклика?

Подсказка: GPU часто требует времени на запуск ядра и загрузку данных.

Ответ:

- **CPU может быть быстрее для небольшого батча**
- GPU всегда быстрее, даже для батча = 1
- Оба дадут одинаковое время

🔮 Шаг 05:

Миссия 5

🔗 Иван замеряет throughput: сколько токенов в секунду выдаёт модель. На GPU — 200 ток/с, на CPU — 40 ток/с. Какой вывод?

Подсказка: Throughput измеряет пропускную способность.

Ответ:

- **GPU обеспечивает значительно большую пропускную способность**
- CPU даёт лучший latency
- Производительность одинакова при достаточном батче

🔮 Шаг 06:

Миссия 6

🌐 Марк хочет сравнить energia/токен. Он видит, что GPU быстрее, но потребляет больше энергии. Что важнее для серверов?

Подсказка: Баланс скорости и энергозатрат важен в продакшене.

Ответ:

- **Энергозатраты могут нивелировать выигрыш скорости**
- Скорость всегда важнее энергозатрат
- Энергозатраты несут незначительную нагрузку для серверов

🔮 Шаг 07:

Миссия 7

🔗 Сопоставьте метрику и её смысл в сравнении CPU vs GPU.

Подсказка: Подумайте, что измеряют latency, throughput и energy per token.

Метрики:

Ответ:

- Latency (задержка)
- Throughput (пропускная способность)
- Энергия на токен

Описание:

1. Время ответа на один запрос / один батч
2. Сколько токенов можно сгенерировать за секунду
3. Энергопотребление делённое на количество токенов

- **Latency (задержка)** — Время ответа на один запрос / один батч
- **Throughput (пропускная способность)** — Сколько токенов можно сгенерировать за секунду
- **Энергия на токен** — Энергопотребление делённое на количество токенов

🔮 Шаг 08:

Миссия 8

Сергей хочет оценить накладные расходы. Сопоставьте тип накладных расходов с источником:

Подсказка: GPU требует времени на копирование данных и запуск ядер.

Тип накладных расходов:

Ответ:

- Копирование данных CPU ↔ GPU
- Запуск ядра (kernel launch)
- Синхронизация между потоками

Источник:

1. Передача памяти между устройствами
2. Оверхед стартового вызова на GPU
3. Ожидание завершения операций / барьеры

- **Копирование данных CPU ↔ GPU** — Передача памяти между устройствами
- **Запуск ядра (kernel launch)** — Оверхед стартового вызова на GPU
- **Синхронизация между потоками** — Ожидание завершения операций / барьеры

🔮 Шаг 09:

Миссия 9

🔍 Команда хочет сопоставить сценарий с тем, на каком устройстве он предпочтительнее.

Подсказка: Учтите размер батча, сложность операций и нагрузку.

Сценарии:

Ответ:

- Маленький батч, низкая сложность
- Большой батч, сложные матрицы
- Гибридная нагрузка (часть на CPU, часть на GPU)

Предпочтение:

1. CPU эффективнее
 2. GPU предпочтителен
 3. Смешанное решение (CPU + GPU вместе)
- **Маленький батч, низкая сложность** — CPU эффективнее
 - **Большой батч, сложные матрицы** — GPU предпочтителен
 - **Гибридная нагрузка (часть на CPU, часть на GPU)** — Смешанное решение (CPU + GPU вместе)

🔮 Шаг 10:

Миссия 10

📊 Елена хочет понять рост ускорения. Сопоставьте отношение ускорения с характеристикой:

Подсказка: Если speedup = 8x, значит GPU в 8 раз быстрее по throughput или latency.

Отношения ускорения:

Ответ:

- 2x
- 5x
- 10x

Типичные случаи:

1. Умеренное ускорение для средних моделей
2. Хороший выигрыш на крупных матрицах

3. Большие выигрыши на тяжёлых слоях и больших батчах

- **2x** — Умеренное ускорение для средних моделей
- **5x** — Хороший выигрыш на крупных матрицах
- **10x** — Большие выигрыши на тяжёлых слоях и больших батчах

Шаг 11:

Миссия 11

📋 Упорядочьте шаги для оценки производительности CPU и GPU:

1. Запустить модель на CPU, замерить метрики
2. Запустить модель на GPU, замерить метрики
3. Убедиться, что размер батчей одинаков
4. Сравнить latency, throughput и энергию
5. Интерпретировать результаты для выбора

Подсказка: Убедитесь, что условия тестов согласованы перед сравнением.

Ответ:

- Запустить модель на CPU, замерить метрики
- Запустить модель на GPU, замерить метрики
- Убедиться, что размер батчей одинаков
- Сравнить latency, throughput и энергию
- Интерпретировать результаты для выбора

Шаг 12:

Миссия 12

⚙️ Порядок оптимизационных шагов при переходе на GPU:

1. Минимизировать копирования между CPU и GPU
2. Объединить ядра (kernel fusion)
3. Использовать батчи большего размера
4. Использовать тензорные ядра (если есть)
5. Профилировать узкие места и оптимизировать

Подсказка: Сначала устраняем накладные расходы, затем усиливаем вычислительные ядра.

Ответ:

- Минимизировать копирования между CPU и GPU
- Объединить ядра (kernel fusion)
- Использовать батчи большего размера
- Использовать тензорные ядра (если есть)
- Профилировать узкие места и оптимизировать

Шаг 13:

Миссия 13

📋 Упорядочите факторы, которые усиливают выигрыш GPU над CPU:

1. Большой размер батча
2. Большие матрицы операций
3. Использование специализированных библиотек (cuBLAS, cuDNN)
4. Множество параллельных операций

5. Низкий накладной режим копирования и запуска ядер

Подсказка: Чем больше параллелизма и меньше накладных расходов — тем больше выигрыш GPU.

Ответ:

- Большой размер батча
- Большие матрицы операций
- Использование специализированных библиотек (cuBLAS, cuDNN)
- Множество параллельных операций
- Низкий накладной режим копирования и запуска ядер

Шаг 14:

Миссия 14

🔗 Упорядочите стадии гибридного выполнения (CPU + GPU):

1. Часть операций (например, малые слои) оставить на CPU
2. Передать тяжёлые блоки на GPU
3. Синхронизировать результаты между устройствами
4. Собрать итоговый выход модели
5. Анализ производительности каждого этапа

Подсказка: Сначала решение, что на CPU, что на GPU, затем передача, синхронизация, сбор и анализ.

Ответ:

- Часть операций (например, малые слои) оставить на CPU
- Передать тяжёлые блоки на GPU
- Синхронизировать результаты между устройствами
- Собрать итоговый выход модели
- Анализ производительности каждого этапа

Шаг 15:

Миссия 15

📊 Упорядочите метрики по степени важности при сравнении устройств (с учётом продакшена):

1. Latency для пользовательских задач
2. Throughput для нагрузочных сценариев
3. Энергия на токен (эффективность)
4. Накладные расходы на синхронизацию и копирование
5. Простота интеграции и отладки

Подсказка: В продакшне важно не только скорость, но и эффективность и удобство поддержки.

Ответ:

- Latency для пользовательских задач
- Throughput для нагрузочных сценариев
- Энергия на токен (эффективность)
- Накладные расходы на синхронизацию и копирование
- Простота интеграции и отладки

Шаг 16:

Миссия 16

🗒 Упорядочите этапы принятия решения: на CPU, GPU или гибрид:

1. Замерить производительность на CPU и GPU
2. Оценить энергоэффективность и накладные расходы
3. Совместимость с инфраструктурой (оборудование, драйверы)
4. Определить, где выгоден GPU и где CPU
5. Выбрать режим: однородный или гибридный

Подсказка: Решение принимается на основе измерений, затрат и инфраструктуры.

Ответ:

- Замерить производительность на CPU и GPU
- Оценить энергоэффективность и накладные расходы
- Совместимость с инфраструктурой (оборудование, драйверы)
- Определить, где выгоден GPU и где CPU
- Выбрать режим: однородный или гибридный

Шаг 17:

Миссия 17

🗒 Упорядочите шаги итогового процесса сравнения CPU и GPU и принятия решения в проекте:

1. Собрать метрики latency, throughput, энергия
2. Проанализировать накладные расходы (копирования, синхронизация)
3. Сопоставить результаты с требований проекта
4. Принять архитектурное решение (CPU, GPU или гибрид)
5. Осуществить интеграцию и финальное тестирование

Подсказка: Процесс завершается интеграцией выбранного решения и проверкой.

Ответ:

- Собрать метрики latency, throughput, энергия
- Проанализировать накладные расходы (копирования, синхронизация)
- Сопоставить результаты с требований проекта
- Принять архитектурное решение (CPU, GPU или гибрид)
- Осуществить интеграцию и финальное тестирование

Урок 3 - 🧠 Ускорение attention и уменьшение VRAM

Шаг 01:

Миссия 1

🔍 Инженер ЛЛМ Анна заметила, что при генерации с длинным контекстом модель начинает тормозить: рост времени attention-калькуляции. Что из вариантов может помочь сразу снизить нагрузку на VRAM?

Подсказка: думайте о способах уменьшить размер матриц до перемножения.

Ответ:

- **Снизить длину последовательности контекста**
- Увеличить batch size
- Добавить больше слоёв модели

🔮 Шаг 02:

Миссия 2

🕒 Пётр экспериментирует с attention-механизмом: он хочет заменить плотный (dense) attention на что-то более щадящее по памяти. Какой подход наиболее вероятен?

Подсказка: вспомните методы с прерывистым или локальным вниманием.

Ответ:

- **Использовать локальный (windowed) attention**
- Увеличить проекционные слои внимания
- Полностью отказаться от attention
- Перевести на рекуррентные слои

🔮 Шаг 03:

Миссия 3

🕒 Инженер Лена решает внедрить sparsity в attention. Какой из способов наиболее прямолинеен?

Подсказка: sparsity означает, что не все пары токенов взаимодействуют.

Ответ:

- **Разбить контекст на блоки и делать attention только внутри блоков**
- Всегда делать attention по всем парам
- Увеличить hidden size
- Уменьшить количество голов внимания

🔮 Шаг 04:

Миссия 4

Андрей хочет уменьшить квантование весов attention-модулей, чтобы сэкономить VRAM за счёт lower precision. Какой тип квантования предпочтителен для GPU при минимальной потере качества?

Подсказка: подумайте о целочисленных форматах с поддержкой современных ускорителей.

Ответ:

- **INT8-квантование**
- INT4-квантование
- UINT16
- FLOAT64

🔮 Шаг 05:

Миссия 5

🕒 Чтобы снизить пиковое потребление памяти при attention, Мария хочет применять частичное вычисление (шардирование или chunking). Что лучше всего делать?

Подсказка: разбивайте огромную матрицу на части, чтобы не держать всё сразу.

Ответ:

- **Разбивать последовательность на чанки и вычислять attention по частям**
- Сразу считать всю матрицу полноразмерно
- Увеличить число attention-голов
- Уменьшить количество слоёв

🔮 Шаг 06:

Миссия 6

✦ Сопоставьте термин и краткое его описание, применительное к ускорению attention и уменьшению VRAM:

Термины:

Ответ:

- FlashAttention
- Kernel Fuser
- Memory-efficient attention
- Low-rank factorization

Описания:

1. Метод, минимизирующий копии тензоров и промежуточные аллокации
2. Аппроксимация матриц внимания через произведение низкоранговых матриц
3. Специализированный алгоритм attention, оптимизированный под GPU
4. Подход, уменьшающий пиковую память за счёт перераспределения вычислений

Подсказка: вспомните, какие технологии минимизируют временные буферы и заливающиеся выделения.

- **FlashAttention** — Специализированный алгоритм attention, оптимизированный под GPU
- **Kernel Fuser** — Метод, минимизирующий копии тензоров и промежуточные аллокации
- **Memory-efficient attention** — Подход, уменьшающий пиковую память за счёт перераспределения вычислений
- **Low-rank factorization** — Аппроксимация матриц внимания через произведение низкоранговых матриц

🔮 Шаг 07:

Миссия 7

✦ Сопоставьте подход и его эффект на VRAM или скорость при attention:

Подходы:

Ответ:

- Репараметризация слоёв (reparameterization)
- Sparse attention mask
- Mixed precision (FP16/AMP)
- Chunked attention (разбиение на части)

Эффекты:

1. Уменьшение использования памяти на промежуточные буферы матриц
2. Снижение числа вычисляемых пар токенов
3. Ускорение матричных операций за счёт половинной точности
4. Снижение пикового потребления памяти при больших последовательностях

Подсказка: подумайте, какой подход влияет на вычисляемые пары, а какой — на точность.

- **Reparameterization** — Уменьшение использования памяти на промежуточные буферы матриц
- **Sparse attention mask** — Снижение числа вычисляемых пар токенов

- **Mixed precision (FP16/AMP)** — Ускорение матричных операций за счёт половинной точности
- **Chunked attention** — Снижение пикового потребления памяти при больших последовательностях

🔮 Шаг 08:

Миссия 8

🔮 Сопоставьте типы sparsity и их характерные особенности:

Типы sparsity:

Ответ:

- Block sparsity
- Pattern sparsity
- Random sparsity
- Structured sparsity

Особенности:

1. Заранее фиксированные позиции нулей в весах, удобные для ускорения
2. Нерегулярные нули, часто худшие для ускорения на GPU
3. Синтетические схемы блоков нулей/ненулей
4. Выделенные шаблоны (pattern) нулей/ненулей, поддерживаемые аппаратно

Подсказка: подумайте, какие sparsity-дизайны поддерживаются на уровне аппаратуры.

- **Block sparsity** — Синтетические схемы блоков нулей/ненулей
- **Pattern sparsity** — Выделенные шаблоны (pattern) нулей/ненулей, поддерживаемые аппаратно
- **Random sparsity** — Нерегулярные нули, часто худшие для ускорения на GPU
- **Structured sparsity** — Заранее фиксированные позиции нулей в весах, удобные для ускорения

🔮 Шаг 09:

Миссия 9

🔮 Сопоставьте технологию ускорения attention и ограничение, которое она накладывает:

Технологии:

Ответ:

- Kernel fusion
- Mixed precision
- Chunked attention
- Low-rank approximation

Ограничения:

1. Потенциальная потеря точности
2. Не позволяет обрабатывать весь контекст целиком одновременно
3. Сложность реализации на GPU
4. Подходит не для всех типов матриц внимания

Подсказка: каждая технология несёт компромисс между скоростью, памятью и качеством.

- **Kernel fusion** — Сложность реализации на GPU

- **Mixed precision** — Потенциальная потеря точности
- **Chunked attention** — Не позволяет обрабатывать весь контекст целиком одновременно
- **Low-rank approximation** — Подходит не для всех типов матриц внимания

🔗 Шаг 10:

Миссия 10

Упорядочите шаги, какие предпримет инженер, чтобы интегрировать FlashAttention в ядро Ollama:

1. Тестирование корректности вывода с новым модулем
2. Обновление документации API для пользователей
3. Замена стандартного attention-кода на FlashAttention внутри ядра
4. Профилирование производительности до и после

Подсказка: начните с измерения, затем заменяйте, затем проверяйте, и в конце обновляйте документацию.

Ответ:

- Тестирование корректности вывода с новым модулем
- Обновление документации API для пользователей
- Замена стандартного attention-кода на FlashAttention внутри ядра
- Профилирование производительности до и после

🔗 Шаг 11:

Миссия 11

🔗 Упорядочите этапы внедрения chunked attention (разбиение на части) в inference pipeline:

1. Сбор ошибок и логов на граничных чанках
2. Разработка логики разбиения последовательности на чанки
3. Интеграция chunk-логики в существующий flow inference
4. Запуск тестов на единичных и граничных последовательностях

Подсказка: сначала проект, потом интеграция, потом проверка, потом отладка в граничных случаях.

Ответ:

- Сбор ошибок и логов на граничных чанках
- Разработка логики разбиения последовательности на чанки
- Интеграция chunk-логики в существующий flow inference
- Запуск тестов на единичных и граничных последовательностях

🔗 Шаг 12:

Миссия 12

🕒 Упорядочите стратегию, как постепенно снижать precision attention-модулей без сильного ухудшения качества:

1. Сравнение метрик качества после изменений
2. Плавное снижение precision на отдельных слоях
3. Откат изменений, если качество слишком упало
4. Наблюдение влияния на память и скорость

Подсказка: постепенность и проверки — ключ к безопасному переходу.

Ответ:

- Сравнение метрик качества после изменений
- Плавное снижение precision на отдельных слоях
- Откат изменений, если качество слишком упало
- Наблюдение влияния на память и скорость

 Шаг 13:

Миссия 13

 Упорядочите задачи, связанные с внедрением sparse attention mask:

1. Генерация маски sparsity (pattern) для attention
2. Модификация ядра attention для учёта маски
3. Профилирование производительности с маской и без неё
4. Тестирование качества генерации при новом attention

Подсказка: сначала генерируешь маску, интегрируешь, тестируешь и профилируешь.

Ответ:

- Генерация маски sparsity (pattern) для attention
- Модификация ядра attention для учёта маски
- Профилирование производительности с маской и без неё
- Тестирование качества генерации при новом attention

 Шаг 14:

Миссия 14

 Упорядочите шаги применения low-rank приближения к матрицам Q, K, V:

1. Обучение/файнтюнинг с низкоранговым представлением
2. Декомпозиция матриц на факторы (например SVD-style)
3. Оценка отклонения от оригинальных матриц внимания

Подсказка: сначала декомпозиция, потом интеграция и затем проверка качества.

Ответ:

- Обучение/файнтюнинг с низкоранговым представлением
- Декомпозиция матриц на факторы (например SVD-style)
- Оценка отклонения от оригинальных матриц внимания

 Шаг 15:

Миссия 15

 Упорядочите стадии анализа производительности attention после оптимизации:

1. Сбор профилей времени и памяти
2. Сравнение до/после оптимизации
3. Выявление узких мест (hotspots)
4. Выбор следующего метода оптимизации

Подсказка: аналитика — итеративный цикл сбора, сравнения, дискавери и планирования.

Ответ:

- Сбор профилей времени и памяти

- Сравнение до/после оптимизации
- Выявление узких мест (hotspots)
- Выбор следующего метода оптимизации

🔧 Шаг 16:

Миссия 16

🔗 Упорядочите стратегию комбинирования разных оптимизаций attention для максимального эффекта:

1. Применить kernel fusion + mixed precision
2. Добавить sparse attention mask к уже улучшенному ядру
3. Внедрить chunking и профилировать по частям
4. Оценить качество и откатить неудачные комбинации

Подсказка: комбинируйте постепенно, тестируя все промежуточные варианты.

Ответ:

- Применить kernel fusion + mixed precision
- Добавить sparse attention mask к уже улучшенному ядру
- Внедрить chunking и профилировать по частям
- Оценить качество и откатить неудачные комбинации

🔧 Шаг 17:

Миссия 17

🔗 Финальное задание: упорядочите этапы, которые инженер выполнит перед выпуском ядра с оптимизированным attention:

1. Мониторинг использования VRAM на разных задачах производственного масштаба
2. Регрессионное тестирование на репрезентативных датасетах
3. Сравнение скорости и качества с оригинальным ядром
4. Обновление версии ядра и задокументирование изменений

Подсказка: сначала тестирование качества и совместимости, затем нагрузочное тестирование, затем релиз и документация.

Ответ:

- Мониторинг использования VRAM на разных задачах производственного масштаба
- Регрессионное тестирование на репрезентативных датасетах
- Сравнение скорости и качества с оригинальным ядром
- Обновление версии ядра и задокументирование изменений

📖 Урок 4 - 📁 Анализ ggml_quantize и логов inference

🔧 Шаг 01:

Миссия 1

📖 Инженер Оля запускает команду `ggml\_quantize` на модели и видит: «quant: mode = q4_0». Что это значит?

Подсказка: попробуйте вспомнить, какие режимы квантования поддерживаются в ggml.

Ответ:

- Квантование до 4 бит без знака
- **Квантование до 4 бит со знаком**

- Квантование до 8 бит плавающей точности
- Квантование с плавающей точностью 16 бит

🔮 Шаг 02:

Миссия 2

🔍 После quantize Олег запускает inference и в логах видит «bad quant alignment». Что наиболее вероятно сломано?

Подсказка: подумайте, что могло пойти не так при упаковке весов.

Ответ:

- **Нарушена выравниваемость (alignment) в памяти**
- Неправильный формат логов
- Ошибка в tokenizer
- Ошибка в языке модели

🔮 Шаг 03:

Миссия 3

👂 Маша сравнивает время inference на оригинальной модели и на квантованной. Если квантованная работает быстрее — почему это возможно?

Подсказка: подумайте про уменьшение объёмов операций и нагрузку на память.

Ответ:

- **Потому что меньше объём операций с весами и меньшая нагрузка на шину памяти**
- Потому что модель генерирует меньше токенов
- Потому что логика attention отключена
- Потому что менее точная модель

🔮 Шаг 04:

Миссия 4

⚙️ В логах inference Антон видит строку «apply_logits_buffer: size=...». Что это значит с точки зрения внутренней работы?

Подсказка: вспомните, как накапливаются логиты перед выбором токена.

Ответ:

- **Буфер логитов, в который аккумулируются предсказания перед нормализацией**
- Буфер attention весов
- Буфер input токенов
- Буфер градиентов

🔮 Шаг 05:

Миссия 5

🔪 После quantize в режиме q5_1 Лена замечает рост ошибки во многих случаях. Что наиболее разумно проверить в первую очередь?

Подсказка: проверьте, что не нарушена структура весов после квантования.

Ответ:

- **Проверить восстановление весов деквантованием**
- Проверить версию Python
- Проверить токенизацию

- Проверить лимит длины контекста

🔮 Шаг 06:

Миссия 6

📖 Сопоставьте ключевые поля лога inference с их значением и ролью:

Поля лога:

Ответ:

- n_past
- n_ctx
- n_batch
- n_threads

Роли / значения:

1. Количество токенов, уже обработанных в сессии
2. Общий размер контекстного окна модели (максимум)
3. Размер батча при генерации (сколько токенов генерируем одновременно)
4. Число потоков (ядра CPU / GPU) для вычислений

Подсказка: свяжите аббревиатуры с «past», «context», «batch», «threads».

- **n_past** — Количество токенов, уже обработанных в сессии
- **n_ctx** — Общий размер контекстного окна модели (максимум)
- **n_batch** — Размер батча при генерации (сколько токенов генерируем одновременно)
- **n_threads** — Число потоков (ядра CPU / GPU) для вычислений

🔮 Шаг 07:

Миссия 7

🔍 Сопоставьте виды квантования ggml_quantize с особенностями, которые они приносят:

Виды квантования:

Ответ:

- q4_0
- q4_1
- q5_0
- q5_1

Особенности:

1. Простейшее 4-битное квантование, минимальный overhead
2. 4-битное квантование с компенсацией шкалирования (scale-aware)
3. 5-битное квантование с базовой опцией
4. 5-битное квантование с дополнительной корректировкой (bias или оптимизация)

Подсказка: вспомните обозначения _0 и _1 — простое / улучшенное.

- **q4_0** — Простейшее 4-битное квантование, минимальный overhead
- **q4_1** — 4-битное квантование с компенсацией шкалирования (scale-aware)
- **q5_0** — 5-битное квантование с базовой опцией
- **q5_1** — 5-битное квантование с дополнительной корректировкой (bias или оптимизация)

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте поля лога quantize с тем, что они обозначают:

Поля:

Ответ:

- `n_rows`
- `n_cols`
- `q_byte_size`
- `scale_min / scale_max`

Значения:

1. Число строк (`dim0`) матрицы весов
2. Число столбцов (`dim1`) матрицы весов
3. Объём памяти, выделенной под квантованные веса (в байтах)
4. Минимальный и максимальный масштабы (`scale`) для квантования

Подсказка: названия достаточно прозрачно говорят о размере и шкалах.

- **`n_rows`** — Число строк (`dim0`) матрицы весов
- **`n_cols`** — Число столбцов (`dim1`) матрицы весов
- **`q_byte_size`** — Объём памяти, выделенной под квантованные веса (в байтах)
- **`scale_min / scale_max`** — Минимальный и максимальный масштабы (`scale`) для квантования

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте сообщения об ошибках quantize и возможные причины:

Сообщения:

Ответ:

- "quantization failed: overflow"
- "unsupported quant mode"
- "scale too small"
- "alignment error"

Причины:

1. Выбран режим квантования, не поддерживаемый текущей архитектурой
2. Шкала (`scale`) получилась нулевой или близкой к нулю, что мешает нормализации
3. Материалы веса слишком велики для выбранного диапазона квантования (переполнение)
4. Нарушение выравнивания (`alignment`) памяти при упаковке

Подсказка: подумаем, какая из ошибок связана с режимом, какая — с памятью.

- **quantization failed: overflow** — Материалы веса слишком велики для выбранного диапазона квантования (переполнение)
- **unsupported quant mode** — Выбран режим квантования, не поддерживаемый текущей архитектурой
- **scale too small** — Шкала (`scale`) получилась нулевой или близкой к нулю, что мешает нормализации
- **alignment error** — Нарушение выравнивания (`alignment`) памяти при упаковке

🔧 Шаг 10:

Миссия 10

🔧 Упорядочите шаги процесса ``ggml_quantize`` при квантовании матрицы весов:

1. Подсчёт минимального и максимального значений (scale)
2. Рассчёт квантованных значений (integer representation)
3. Упаковка целых чисел в компактный формат (с битами)
4. Сохранение квантованных весов и метаданных (scale, zero point и т.д.)

Подсказка: сначала шкалы, затем квантование, затем упаковка и сохранение.

Ответ:

- Подсчёт минимального и максимального значений (scale)
- Рассчёт квантованных значений (integer representation)
- Упаковка целых чисел в компактный формат (с битами)
- Сохранение квантованных весов и метаданных (scale, zero point и т.д.)

🔧 Шаг 11:

Миссия 11

🔧 Упорядочите этапы анализа логов inference после quantize:

1. Поиск сообщений об ошибках (overflow, alignment и др.)
2. Сравнение времени загрузки (load) до и после quantize
3. Анализ пикового потребления памяти (VRAM или RAM)
4. Оценка качества выходного текста (разница с оригиналом)

Подсказка: сначала проверяем ошибки, затем метрики, затем качество.

Ответ:

- Поиск сообщений об ошибках (overflow, alignment и др.)
- Сравнение времени загрузки (load) до и после quantize
- Анализ пикового потребления памяти (VRAM или RAM)
- Оценка качества выходного текста (разница с оригиналом)

🔧 Шаг 12:

Миссия 12

🔧 Упорядочите шаги, которые предпримет инженер перед массовым применением quantize к нескольким моделям:

1. Создание скрипта автоматического квантования с отчётами логов
2. Бэкап оригинальных моделей и весов
3. Параллельное тестирование моделей на representative задачах
4. Сравнение качества, скорости и памяти между оригиналом и квантованной версией

Подсказка: сначала безопасность (бэкап), потом автоматизация, потом тесты, затем сравнение.

Ответ:

- Создание скрипта автоматического квантования с отчётами логов
- Бэкап оригинальных моделей и весов
- Параллельное тестирование моделей на representative задачах
- Сравнение качества, скорости и памяти между оригиналом и квантованной версией

Шаг 13:

Миссия 13

 Упорядочите шаги, когда лог inference показывает «overflow» после quantize:

1. Отменить квантование и вернуться к оригиналу (roll back)
2. Проанализировать, для каких слоёв произошёл переполнения
3. Попробовать другой режим квантования (например q5_1 вместо q4)
4. Запустить тестовое inference на небольшом фрагменте с отслеживанием границ переполнения

Подсказка: сначала локализовать проблему по слоям, затем тестировать альтернативы, затем откат, затем проверка.

Ответ:

- Отменить квантование и вернуться к оригиналу (roll back)
- Проанализировать, для каких слоёв произошёл переполнения
- Попробовать другой режим квантования (например q5_1 вместо q4)
- Запустить тестовое inference на небольшом фрагменте с отслеживанием границ переполнения

Шаг 14:

Миссия 14

 Упорядочите стратегию улучшения результатов после первоначального quantize, если качество слишком ухудшилось:

1. Попробовать другой режим квантования (q4_1, q5_1)
2. Добавить корректирующие коэффициенты (bias, scale adjustments)
3. Изучить, какие слои наиболее чувствительны к квантованию
4. Смешанное хранение (оставлять некоторые слои в FP32)

Подсказка: комбинируйте режимы и гибридные схемы, чтобы минимизировать деградацию.

Ответ:

- Попробовать другой режим квантования (q4_1, q5_1)
- Добавить корректирующие коэффициенты (bias, scale adjustments)
- Изучить, какие слои наиболее чувствительны к квантованию
- Смешанное хранение (оставлять некоторые слои в FP32)

Шаг 15:

Миссия 15

 Упорядочите этапы, если вы хотите логировать подробные данные quantize и inference в production:

1. Добавить подробные метки (tags) в логах quantize (режим, слои, масштабы)
2. Собирать логи inference (время, память, ошибки)
3. Инструментализировать сбор метрик в систему мониторинга
4. Анализ и визуализация логов, поиск аномалий

Подсказка: сначала логируйте, потом собирайте, потом интегрируйте и анализируйте.

Ответ:

- Добавить подробные метки (tags) в логах quantize (режим, слои, масштабы)

- Собирать логи inference (время, память, ошибки)
- Инструментализировать сбор метрик в систему мониторинга
- Анализ и визуализация логов, поиск аномалий

🔮 Шаг 16:

Миссия 16

Упорядочите полный рабочий цикл: quantize → inference → лог-анализ → корректировка:

1. Запустить quantize с первыми настройками
2. Произвести inference и записать логи
3. Проанализировать логи и найти слабые места
4. Скорректировать режим квантования или стратегию слоёв
5. Повторить цикл с новыми параметрами

Подсказка: это итеративный процесс «измерил — изменил — проверил — повторил».

Ответ:

- Запустить quantize с первыми настройками
- Произвести inference и записать логи
- Проанализировать логи и найти слабые места
- Скорректировать режим квантования или стратегию слоёв
- Повторить цикл с новыми параметрами

🔮 Шаг 17:

Миссия 17

🏆 Финальный квест: упорядочите все шаги, которые Андрей выполнит перед выпуском версии Ollama с интегрированным `ggml\_quantize` и логированием:

1. Регрессионное тестирование качества и сравнение с базовой моделью
2. Нагрузочное тестирование на разных сценариях и оценка пиков памяти
3. Интеграция логирования quantize и inference
4. Подготовка релиза, документации и миграционных инструкций
5. Мониторинг в продакшн и откат при аномалиях

Подсказка: порядок: тесты, нагрузка, логирование, релиз, мониторинг.

Ответ:

- Регрессионное тестирование качества и сравнение с базовой моделью
- Нагрузочное тестирование на разных сценариях и оценка пиков памяти
- Интеграция логирования quantize и inference
- Подготовка релиза, документации и миграционных инструкций
- Мониторинг в продакшн и откат при аномалиях

📖 Урок 5 - 🎯 Квест добейся ускорения генерации в 2 раза

🔮 Шаг 01:

Миссия 1

⚡ Инженер Лена запускает базовый inference и фиксирует время T_0 . Она хочет ускорить генерацию в 2x — с чего начать?

Подсказка: подумайте про снижение вычислительной нагрузки на операции matrix multiplication.

Ответ:

- **Внедрить mixed precision (FP16 / AMP)**
- Увеличить максимальную длину контекста
- Добавить больше attention-голов
- Уменьшить размер батча

🔮 **Шаг 02:**

Миссия 3

✨ Аня решает использовать kernel fusion и оптимизацию ядер для линейных слоёв. Почему это может ускорить генерацию?

Подсказка: подумайте, как объединение операций уменьшает накладные расходы.

Ответ:

- **Потому что уменьшаются лишние копии тензоров и синхронизации**
- Потому что увеличивается точность
- Потому что модель генерирует меньше токенов
- Потому что память освобождается быстрее

🔮 **Шаг 03:**

Миссия 4

После применения fusion и FP16 Олег замечает, что пиковая VRAM всё ещё высока. Какой метод может помочь с этим?

Подсказка: подумайте про разбитие вычислений на части.

Ответ:

- **Chunked attention / разбивка на части**
- Увеличить batch size
- Уменьшить число потоков
- Удалить attention слои

🔮 **Шаг 04:**

Миссия 5

👹 После всех оптимизаций Ира всё ещё не достигает 2х ускорения. Какой следующий приём она может использовать?

Подсказка: вспомните методы аппроксимации матриц.

Ответ:

- **Low-rank approximation attention**
- Увеличить глубину сети
- Переход на int8
- Увеличить hidden size

🔮 **Шаг 05:**

Миссия 6

📊 Сопоставьте методы ускорения генерации с их основным преимуществом:

Методы:

Ответ:

- Mixed precision
- Kernel fusion
- Chunked attention
- Low-rank approximation

Преимущества:

1. Уменьшение накладных расходов на промежуточные операции
2. Снижение объёма операций матричного умножения
3. Снижение пикового потребления памяти
4. Ускорение операций за счёт более лёгкой точности

Подсказка: комбинируйте, как метод влияет на операции и память.

- **Mixed precision** — Ускорение операций за счёт более лёгкой точности
- **Kernel fusion** — Уменьшение накладных расходов на промежуточные операции
- **Chunked attention** — Снижение пикового потребления памяти
- **Low-rank approximation** — Снижение объёма операций матричного умножения

 Шаг 06:

Миссия 7

 Сопоставьте оптимизации и потенциальные риски:

Оптимизации:

Ответ:

- Mixed precision
- Kernel fusion
- Chunked attention
- Low-rank approximation

Риски:

1. Потеря численной стабильности
2. Сложная отладка ядер
3. Появление границ между чанками
4. Снижение качества из-за аппроксимации

Подсказка: риск often напрямую отражает компромисс метода.

- **Mixed precision** — Потеря численной стабильности
- **Kernel fusion** — Сложная отладка ядер
- **Chunked attention** — Появление границ между чанками
- **Low-rank approximation** — Снижение качества из-за аппроксимации

 Шаг 07:

Миссия 8

Сопоставьте внутренняя метрика (лог) и её роль для диагностики ускорения:

Метрики:

Ответ:

- compute_time

- mem_peak
- kernel_launches
- tensor_copy_overhead

Роли:

1. Время основных операций вычисления
2. Пиковое потребление памяти
3. Количество запусков ядер GPU / блоков
4. Накладные расходы на копирование тензоров

Подсказка: названия часто прямо указывают на роль метрики.

- **compute_time** — Время основных операций вычисления
- **mem_peak** — Пиковое потребление памяти
- **kernel_launches** — Количество запусков ядер GPU / блоков
- **tensor_copy_overhead** — Накладные расходы на копирование тензоров

🔔 Шаг 08:

Миссия 9

🔗 Сопоставьте стратегии комбинирования оптимизаций и когда их применять:

Стратегии:

Ответ:

- FP16 + kernel fusion
- Kernel fusion + chunking
- FP16 + low-rank
- Полный микс (все методы сразу)

Когда применять:

1. Когда хотим сбалансировать скорость и память
2. Когда fusion уже на месте и память все ещё узкий горлышко
3. Когда память есть, но хотим снизить операции
4. Когда изначальный прирост слишком мал и нужен максимальный комбинированный эффект

Подсказка: стратегия должна сочетать методы с взаимодополняющими преимуществами.

- **FP16 + kernel fusion** — Когда хотим сбалансировать скорость и память
- **Kernel fusion + chunking** — Когда fusion уже на месте и память все ещё узкий горлышко
- **FP16 + low-rank** — Когда память есть, но хотим снизить операции
- **Полный микс (все методы сразу)** — Когда изначальный прирост слишком мал и нужен максимальный комбинированный эффект

🔔 Шаг 09:

Миссия 10

🔗 Упорядочите шаги, которые инженер проделает, чтобы проверить, что ускорение действительно в 2х:

1. Сравнить latency / throughput до и после оптимизации
2. Проверить качество выходного текста (разница с оригиналом)
3. Собрать профили времени и памяти по слоям
4. Выполнить нагрузочное тестирование на разных сценариях

Подсказка: начните с измерений, затем качество, затем детальный анализ, затем стресс-тесты.

Ответ:

- Сравнить latency / throughput до и после оптимизации
- Проверить качество выходного текста (разница с оригиналом)
- Собрать профили времени и памяти по слоям
- Выполнить нагрузочное тестирование на разных сценариях

Шаг 10:

Миссия 11

 Упорядочите этапы внедрения chunked + fusion в production-pipeline:

1. Интеграция fusion и чанкования в inference код
2. Тестирование на граничных последовательностях
3. Профилирование и сравнение с прежним режимом
4. Мониторинг поведения при пиковых нагрузках

Подсказка: сначала интеграция, потом тесты, потом профили, потом мониторинг.

Ответ:

- Интеграция fusion и чанкования в inference код
- Тестирование на граничных последовательностях
- Профилирование и сравнение с прежним режимом
- Мониторинг поведения при пиковых нагрузках

Шаг 11:

Миссия 12

 Упорядочите корректировки, если качество падения после оптимизации слишком велико:

1. Откат некоторых слоёв к FP32
2. Попробовать другую стратегию low-rank / другую ранг-оценку
3. Изменить chunk size / границы чанков
4. Скорректировать порядок fusion / разбивки

Подсказка: начинайте с менее агрессивных изменений, затем переходите к сильным.

Ответ:

- Откат некоторых слоёв к FP32
- Попробовать другую стратегию low-rank / другую ранг-оценку
- Изменить chunk size / границы чанков
- Скорректировать порядок fusion / разбивки

Шаг 12:

Миссия 13

 Упорядочите шаги для автоматизированного отбора конфигурации оптимизаций:

1. Генерация комбинаций параметров (precision, chunk size, fusion опции)
2. Запуск тестового inference и логирование метрик для каждой конфигурации
3. Сортировка конфигураций по скорости и качеству
4. Выбор лучшей конфигурации и её валидация на большом наборе данных

Подсказка: используйте метод grid-search / sweep-подход.

Ответ:

- Генерация комбинаций параметров (precision, chunk size, fusion опции)
- Запуск тестового inference и логирование метрик для каждой конфигурации
- Сортировка конфигураций по скорости и качеству
- Выбор лучшей конфигурации и её валидация на большом наборе данных

 Шаг 13:

Миссия 14

🔗 Упорядочите цикл итераций оптимизации до достижения цели 2х:

1. Применить текущую конфигурацию оптимизаций
2. Измерить скорость, память, качество
3. Сравнить с целевым ускорением и порогом деградации качества
4. Выбрать новую конфигурацию или зафиксировать её
5. Повторить при необходимости

Подсказка: итерации — ключ к поиску оптимального сочетания.

Ответ:

- Применить текущую конфигурацию оптимизаций
- Измерить скорость, память, качество
- Сравнить с целевым ускорением и порогом деградации качества
- Выбрать новую конфигурацию или зафиксировать её
- Повторить при необходимости

 Шаг 14:

Миссия 15

🔗 Упорядочите шаги для запуска A/B теста между оптимизированной и базовой версией:

1. Запуск inference по тем же входам на обеих версиях
2. Сбор метрик скорости, памяти, качества
3. Статистический анализ различий (например t-test)
4. Выбор победителя и принятие решения о переходе

Подсказка: важно сравнить на одинаковых входах и провести статистику.

Ответ:

- Запуск inference по тем же входам на обеих версиях
- Сбор метрик скорости, памяти, качества
- Статистический анализ различий (например t-test)
- Выбор победителя и принятие решения о переходе

 Шаг 15:

Миссия 16

📦 Упорядочите шаги, перед тем как предъявить новую оптимизацию команде разработчиков:

1. Подготовить документацию: как работает, ограничения, как откатиться
2. Представить метрики и графики ускорения / качества
3. Организовать демонстрацию новой версии в действии
4. Обсудить возможные риски и план мониторинга

Подсказка: сначала подготовка материалов, затем демонстрация и обсуждение.

Ответ:

- Подготовить документацию: как работает, ограничения, как откатиться
- Представить метрики и графики ускорения / качества
- Организовать демонстрацию новой версии в действии
- Обсудить возможные риски и план мониторинга

 **Шаг 16:**

Миссия 17

▣ Финальная последовательность: что должен сделать инженер перед выпуском версии с 2x ускорением:

1. Провести полный регрессионный тест (качество и семантика)
2. Запустить нагрузочные тесты и измерить пиковую нагрузку
3. Провести A/B тесты на репрезентативных данных
4. Подготовить релиз, документацию и инструкцию по откату
5. Внедрить мониторинг и алерты после релиза

Подсказка: последовательность: тесты → нагрузка → A/B → релиз → мониторинг.

Ответ:

- Провести полный регрессионный тест (качество и семантика)
- Запустить нагрузочные тесты и измерить пиковую нагрузку
- Провести A/B тесты на репрезентативных данных
- Подготовить релиз, документацию и инструкцию по откату
- Внедрить мониторинг и алерты после релиза

Модуль 8 - Миссия 8. Векторная память и поиск знаний (RAG)

Урок 1 - Что такое RAG и зачем он нужен

 **Шаг 01:**

Миссия 1

Инженер Оля хочет объяснить коллегам, что такое RAG (Retrieval-Augmented Generation). Какое определение ближе всего к истине?

Подсказка: RAG сочетает в себе поиск и генерацию.

Ответ:

- **Метод, где генеративная модель дополняется внешним поиском знаний**
- Техника, где модель полностью генерирует ответ без источников
- Простая база данных фактов, которую модель запоминает навсегда

 **Шаг 02:**

Миссия 2

Разработчик Игорь обсуждает варианты архитектуры: зачем нам добавлять поиск внешней базы к LLM в RAG?

Подсказка: LLM не хранит всё знание — вспомните ограничение контекста.

Ответ:

- Чтобы модель могла игнорировать контекст
- **Чтобы она могла достать актуальные факты из внешней базы**
- Чтобы уменьшить размер модели
- Чтобы избежать генерации текста

🔮 Шаг 03:

Миссия 3

Во время митинга София говорит, что RAG помогает обновлять знания без дообучения модели. Это означает, что:

Подсказка: Подумайте, что обычно делает дообучение.

Ответ:

- **Можно подключать новые документы в базу поиска, не трогая веса модели**
- Модель автоматически сама обучается на новых текстах
- Модель забывает старые данные

🔮 Шаг 04:

Миссия 4

Никита называет RAG «гибридным подходом». Что именно он имеет в виду?

Подсказка: Думайте «retrieve + generate».

Ответ:

- Смешение двух языков
- **Сочетание этапа поиска и этапа генерации**
- Одновременное использование GAN и LLM
- Дублирование знаний внутри модели

🔮 Шаг 05:

Миссия 5

Сопоставьте роль компонента RAG с его задачей (коллективная задача инженеров).

Компоненты RAG:

Ответ:

- Retriever (модуль поиска)
- Документная база / индекс
- Generator (модель генерации)
- Фильтр / ранжирование

Задачи:

1. Выбор самых релевантных документов
2. Генерация финального ответа на основе входа и найденных текстов
3. Хранение и структурирование документов для эффективного поиска
4. Оценка и сортировка найденных документов

Подсказка: Подумайте, кто что делает: кто ищет, кто сортирует, кто генерирует.

- **Retriever (модуль поиска)** — Выбор самых релевантных документов
- **Документная база / индекс** — Хранение и структурирование документов для эффективного поиска

- **Generator (модель генерации)** — Генерация финального ответа на основе входа и найденных текстов
- **Фильтр / ранжирование** — Оценка и сортировка найденных документов

🔗 Шаг 06:

Миссия 6

Инженер Вика рисует блок-схему RAG: сопоставьте этап с его назначением.

Этапы RAG:

Ответ:

- Поисковый запрос (query)
- Расширение / преформирование запроса
- Загрузка документов
- Объединение с входом и передача в генератор

Назначения:

1. Улучшить формулировку запроса для точного поиска
2. Извлечь релевантные документы по запросу
3. Сформировать окончательный вход модели для генерации ответа
4. Просто взять запрос пользователя

Подсказка: Подумайте, в каком порядке запрос обрабатывается — сначала он может быть модифицирован, потом тянем документы и потом комбинируем.

- **Поисковый запрос (query)** — Просто взять запрос пользователя
- **Расширение / преформирование запроса** — Улучшить формулировку запроса для точного поиска
- **Загрузка документов** — Извлечь релевантные документы по запросу
- **Объединение с входом и передача в генератор** — Сформировать окончательный вход модели для генерации ответа

🔗 Шаг 07:

Миссия 7

Команда обсуждает, как разграничить что отдавать в генератор. Сопоставьте термин с логикой:

Термины:

Ответ:

- Контекстное окно (context window)
- Документный контекст
- Prompt-инжиниринг
- Chunking (разбиение документов)

Логика:

1. Разбиение больших документов на части меньшего размера
2. Текст, передаваемый модели вместе с вопросом
3. Выбор, какие документы учитывать вне окна модели
4. Построение такого запроса, чтобы модель правильно поняла задачу

Подсказка: Контекстное окно — это лимит, prompt — способ подачи, chunking — работа с большими текстами.

- **Контекстное окно (context window)** — Текст, передаваемый модели вместе с вопросом
- **Документный контекст** — Выбор, какие документы учитывать вне окна модели
- **Prompt-инжиниринг** — Построение такого запроса, чтобы модель правильно поняла задачу
- **Chunking (разбиение документов)** — Разбиение больших документов на части меньшего размера

🧙 Шаг 08:

Миссия 8

В диалоге Тимур спрашивает, какие типы индексов можно использовать для RAG. Сопоставьте тип индекса с примером:

Типы индексов:

Ответ:

- Inverted index
- Vector index (векторный)
- Hybrid index
- Keyword index

Примеры:

1. Поиск документов по ключевым словам (термам)
2. Использование embedding-векторов для семантического поиска
3. Комбинация термального и векторного поиска
4. Классический обратный индекс слова к документам

Подсказка: Векторный — это embedding, inverted — обратный, hybrid — смесь.

- **Inverted index** — Классический обратный индекс слова к документам
- **Vector index (векторный)** — Использование embedding-векторов для семантического поиска
- **Hybrid index** — Комбинация термального и векторного поиска
- **Keyword index** — Поиск документов по ключевым словам (термам)

🧙 Шаг 09:

Миссия 9

Во время обсуждения Лена говорит: «мы можем нормализовать текст перед поиском». Сопоставьте метод нормализации с его эффектом.

Методы:

Ответ:

- Лемматизация
- Стемминг
- Lowercase (приведение к нижнему регистру)
- Удаление стоп-слов

Эффекты:

1. Сократить слово до корня без учёта грамматики
2. Отбросить модальные «и», «или», «на» и т.п.
3. Привести все буквы к нижнему регистру
4. Привести слово к его базовой форме с учётом морфологии

Подсказка: Лемматизация — грамматически корректный базис, стемминг — простой корень.

- **Лемматизация** — Привести слово к его базовой форме с учётом морфологии
- **Стемминг** — Сократить слово до корня без учёта грамматики
- **Lowercase (приведение к нижнему регистру)** — Привести все буквы к нижнему регистру
- **Удаление стоп-слов** — Отбросить модальные «и», «или», «на» и т.п.

🔮 Шаг 10:

Миссия 10

🔍 Устройте этапы типичного pipeline RAG в правильном порядке:

1. Обработка ответа пользователю
2. Генерация ответа с учётом найденных документов
3. Формулировка запроса / его расширение
4. Поисковый запрос и извлечение документов
5. Ранжирование релевантности документов

Подсказка: Подумайте, что должно быть первым и последним: сначала запрос, потом поиск, потом генерация, и в конце — ответ пользователю.

Ответ:

- Формулировка запроса / его расширение
- Поисковый запрос и извлечение документов
- Ранжирование релевантности документов
- Генерация ответа с учётом найденных документов
- Обработка ответа пользователю

🔮 Шаг 11:

Миссия 11

🔍 Расположите шаги при обновлении базы документов в RAG:

1. Перестроение индекса (reindexing)
2. Добавление новых документов в хранилище
3. Удаление неактуальных или устаревших документов
4. Уведомление или сигнал модели о новом индексе
5. Проверка консистентности базы

Подсказка: Сначала надо убрать мусор, потом добавить, затем перестроить, проверить и уведомить систему.

Ответ:

- Удаление неактуальных или устаревших документов
- Добавление новых документов в хранилище
- Перестроение индекса (reindexing)
- Проверка консистентности базы
- Уведомление или сигнал модели о новом индексе

🔮 Шаг 12:

Миссия 12

✳ Порядок отладки системы RAG при неожиданном ответе:

1. Проверить, что документы в базе адекватны
2. Проверить запрос, который передаётся к retriever
3. Проверить ранжирование / фильтрацию результатов
4. Протрассировать, как генератор комбинирует текст
5. Изменить prompt-инжиниринг и протестировать

Подсказка: Часто причина — именно в запросе или ранжировании, но генератор тоже может быть виноват.

Ответ:

- Проверить запрос, который передаётся к retriever
- Проверить ранжирование / фильтрацию результатов
- Проверить, что документы в базе адекватны
- Протрассировать, как генератор комбинирует текст
- Изменить prompt-инжиниринг и протестировать

🔮 **Шаг 13:**

Миссия 13

📖 Команда обновляет RAG для новой предметной области. Расположите действия в порядке:

1. Сбор и очистка новых документов
2. Создание embeddings и индексирование
3. Обновление настроек retriever / ранжировщика
4. Тестовые запросы и оценка качества
5. Обучение prompt / адаптация генератора

Подсказка: Сначала база, потом индекс, потом модели, потом тестирование и доработка.

Ответ:

- Сбор и очистка новых документов
- Создание embeddings и индексирование
- Обновление настроек retriever / ранжировщика
- Тестовые запросы и оценка качества
- Обучение prompt / адаптация генератора

🔮 **Шаг 14:**

Миссия 14

✳ Придётся масштабировать RAG-систему: расставьте этапы масштабирования:

1. Шардирование индекса / распределение по узлам
2. Кэширование популярных запросов и ответов
3. Балансировка нагрузки запросов
4. Оптимизация производительности поиска (approximate поиск)
5. Мониторинг и логирование метрик качества ответов

Подсказка: Сначала структурируем на узлы, затем добавляем оптимизации и наблюдение.

Ответ:

- Шардирование индекса / распределение по узлам
- Кэширование популярных запросов и ответов
- Балансировка нагрузки запросов

- Оптимизация производительности поиска (approximate поиск)
- Мониторинг и логирование метрик качества ответов

🔮 Шаг 15:

Миссия 15

🔗 Сценарий: команда внедряет цепочку RAG + фильтрации + постпроцессинга. Расположите этапы:

1. Поиск релевантных документов
2. Оценка и фильтрация документов (spam, нерелевантные)
3. Генерация ответа
4. Постпроцессинг ответа (форматирование, проверка фактов)
5. Возврат ответа пользователю

Подсказка: Фильтрация перед генерацией, а после генерации — доводка ответа.

Ответ:

- Поиск релевантных документов
- Оценка и фильтрация документов (spam, нерелевантные)
- Генерация ответа
- Постпроцессинг ответа (форматирование, проверка фактов)
- Возврат ответа пользователю

🔮 Шаг 16:

Миссия 16

✨ Расставьте этапы расширенного варианта: RAG + контроль фактов + интерактивное уточнение:

1. Пользователь задаёт вопрос
2. Формирование / уточнение запроса (возможно уточняющий вопрос)
3. Поиск и извлечение документов
4. Ранжирование и фильтрация
5. Генерация ответа с учётом документов
6. Проверка фактов (fact-checking) и коррекция
7. Выдача окончательного ответа пользователю

Подсказка: Уточнение может быть до поиска, проверка фактов — после генерации, перед отдачей.

Ответ:

- Пользователь задаёт вопрос
- Формирование / уточнение запроса (возможно уточняющий вопрос)
- Поиск и извлечение документов
- Ранжирование и фильтрация
- Генерация ответа с учётом документов
- Проверка фактов (fact-checking) и коррекция
- Выдача окончательного ответа пользователю

Урок 2 - Интеграция Ollama с векторными БД

Шаг 01:

Миссия 1

Инженер Артём обсуждает с коллегой: зачем интегрировать Ollama с внешней векторной БД?

Подсказка: Подумайте, что модель сама по себе не хранит огромные массивы векторов локально.

Ответ:

- Чтобы хранить **embeddings** и делать быстрый семантический поиск
- Чтобы заменить генератор модели
- Чтобы убрать необходимость prompt-инжиниринга

Шаг 02:

Миссия 2

Когда Алиса настраивает интеграцию, она говорит: «Нам нужен клиент, который общается с БД». Что он должен поддерживать?

Подсказка: Подумайте, как клиент отправляет запрос на поиск по embedding.

Ответ:

- Метод поиска ближайших векторов (**k-nearest**) через **API**
- Метод запуска генерации без документов
- Метод обучения модели внутри клиента
- Метод графового поиска

Шаг 03:

Миссия 3

Борис жалуется, что при большом количестве документов поиск медленный. Какой вариант поможет ускорить интеграцию Ollama → векторная БД?

Подсказка: Используйте приближённые методы.

Ответ:

- Использовать **approximate nearest neighbors (ANN)** индекс
- Отключить генерацию текста
- Уменьшить размер корпуса до минимума
- Увеличить размер prompt

Шаг 04:

Миссия 4

Во время интеграции Катя говорит: «Нужно нормализовать embeddings и хранить их единым форматом». Почему это важно?

Подсказка: Подумайте о метриках расстояния.

Ответ:

- Чтобы все векторы сравнивались по одной шкале (например, **L2** или **cosine**)
- Чтобы уменьшить размер embedding
- Чтобы генератор быстрее работал
- Чтобы избежать использования prompt

🔮 Шаг 05:

Миссия 5

В переписке с заказчиком Максим пишет, что интеграция должна быть модульной. Что он подразумевает?

Подсказка: Подумайте, что можно менять без влияния на остальную систему.

Ответ:

- **Можно подменять клиент векторной БД без переделки генератора**
- Все части системы должны быть в одном модуле
- Нельзя разносить части на разные сервисы
- Нужно объединить модель и базу в один сервис

🔮 Шаг 06:

Миссия 6

Сопоставьте компонент интеграции Ollama ↔ векторная БД с его функцией:

Компоненты:

Ответ:

- Клиент векторной БД
- Конвертор embedding
- Индекс (ANN / деревья)
- Преобразование запроса (query embedding)

Функции:

1. Преобразование пользовательского запроса в вектор
2. Общение с базой для поиска векторов
3. Структурирование векторов для быстрого поиска
4. Преобразование модели embedding-формата в формат БД

Подсказка: Кто отвечает за коммуникацию, кто — за структуру данных?

- **Клиент векторной БД** — Общение с базой для поиска векторов
- **Конвертор embedding** — Преобразование модели embedding-формата в формат БД
- **Индекс (ANN / деревья)** — Структурирование векторов для быстрого поиска
- **Преобразование запроса (query embedding)** — Преобразование пользовательского запроса в вектор

🔮 Шаг 07:

Миссия 7

Инженер Олег говорит: «Мы можем хранить разные пространства embedding». Сопоставьте тип embedding с его характерным признаком:

Тип embedding:

Ответ:

- Словесный embedding (word-level)
- Предложенческий embedding (sentence-level)
- Блочный embedding (document chunk)
- Контекстно-зависимый embedding (dynamic)

Признаки:

1. Вектор отражает часть документа
2. Вектор зависит от всего контекста запроса
3. Один embedding на весь абзац или предложение
4. Embedding на уровне слова или токена

Подсказка: Словесный — самый мелкий уровень, контекстный — динамический.

- **Словесный embedding (word-level)** — Embedding на уровне слова или токена
- **Предложенческий embedding (sentence-level)** — Один embedding на весь абзац или предложение
- **Блочный embedding (document chunk)** — Вектор отражает часть документа
- **Контекстно-зависимый embedding (dynamic)** — Вектор зависит от всего контекста запроса

🔮 Шаг 08:

Миссия 8

При интеграции Лена говорит: «Нужно согласовать метрики расстояния». Сопоставьте метрику с её применением:

Метрики:

Ответ:

- Cosine similarity
- Dot product
- Euclidean (L2)
- Manhattan (L1)

Применения:

1. Используется часто при нормализованных векторах (angle similarity)
2. Прямое скалярное произведение векторов
3. Классическое евклидово расстояние
4. Сумма по абсолютным разностям координат

Подсказка: Cosine — угол, dot — произведение, L2 — расстояние, L1 — абсолютные суммы.

- **Cosine similarity** — Используется часто при нормализованных векторах (angle similarity)
- **Dot product** — Прямое скалярное произведение векторов
- **Euclidean (L2)** — Классическое евклидово расстояние
- **Manhattan (L1)** — Сумма по абсолютным разностям координат

🔮 Шаг 09:

Миссия 9

Девушка-инженер Марина говорит: «Нам важно балансировать точность и скорость». Сопоставьте параметры при настройке БД с их влиянием:

Параметры:

Ответ:

- Количество соседей k
- Размер дерева / рандомных проекций
- Количество проб (префиксные search tries)
- Рейтинг фильтрации (threshold)

Влияние:

1. **Чем больше k** — тем точнее, но медленнее
2. Более большие структуры ускоряют поиск, но растят память
3. Больше число проб увеличивает вероятность найти лучший сосед
4. Фильтрация снижает шум, но может отбросить релевантные

Подсказка: Больше параметров — большая нагрузка, фильтрация — компромисс.

- **Количество соседей k** — Чем больше k — тем точнее, но медленнее
- **Размер дерева / рандомных проекций** — Более большие структуры ускоряют поиск, но растят память
- **Количество проб (префиксные search tries)** — Больше число проб увеличивает вероятность найти лучший сосед
- **Рейтинг фильтрации (threshold)** — Фильтрация снижает шум, но может отбросить релевантные

Шаг 10:

Миссия 10

🔗 Команда хочет встроить Ollama → векторная БД: расположите этапы интеграции:

1. Выбор или развертывание БД (Milvus, Pinecone и др.)
2. Определение формата embedding и метрики
3. Настройка клиента и API для поиска
4. Индексирование корпуса документов (создание embeddings и загрузка)
5. Тестирование поиска и проверка качества

Подсказка: Сперва выбрать систему, затем согласовать формат, потом загрузить и проверить.

Ответ:

- Выбор или развертывание БД (Milvus, Pinecone и др.)
- Определение формата embedding и метрики
- Настройка клиента и API для поиска
- Индексирование корпуса документов (создание embeddings и загрузка)
- Тестирование поиска и проверка качества

Шаг 11:

Миссия 11

⚙️ Если команда расширяет интеграцию на новые коллекции: упорядочите шаги:

1. Преобразование документов в нужный формат
2. Генерация embedding через модель Ollama
3. Нормализация / стандартизация embedding
4. Загрузка embedding в векторную БД
5. Обновление индексов / reindexing / rebuild

Подсказка: Сначала подготовка, затем генерация, нормализация, загрузка, обновление индексов.

Ответ:

- Преобразование документов в нужный формат
- Генерация embedding через модель Ollama
- Нормализация / стандартизация embedding

- Загрузка embedding в векторную БД
- Обновление индексов / reindexing / rebuild

🔮 Шаг 12:

Миссия 12

✂ При мониторинге интеграции: расположите действия по приоритизации проблем:

1. Замедление поиска (время ответа растёт)
2. Низкая релевантность результатов
3. Проблемы с памятью / перегрузка индекса
4. Расхождение метрик embedding между Ollama и БД
5. Ошибка клиента API (сеть, сериализация)

Подсказка: Сначала критичные узкие места, потом несоответствия, потом мелкие ошибки.

Ответ:

- Замедление поиска (время ответа растёт)
- Проблемы с памятью / перегрузка индекса
- Низкая релевантность результатов
- Расхождение метрик embedding между Ollama и БД
- Ошибка клиента API (сеть, сериализация)

🔮 Шаг 13:

Миссия 13

🔧 Расположите этапы устранения рассинхронизации embedding:

1. Обнаружить несоответствующие embedding (по статистике)
2. Регенерировать embedding для проблемных документов
3. Нормализовать новые embedding согласно выбранной метрике
4. Обновить их в базе через клиент API
5. Перестроить индекс, если требуется

Подсказка: Сначала диагностика, затем генерация, нормализация, загрузка и обновление индекса.

Ответ:

- Обнаружить несоответствующие embedding (по статистике)
- Регенерировать embedding для проблемных документов
- Нормализовать новые embedding согласно выбранной метрике
- Обновить их в базе через клиент API
- Перестроить индекс, если требуется

🔮 Шаг 14:

Миссия 14

🌐 При мультисервисной архитектуре интеграции: расставьте шаги взаимодействия:

1. Сервис Ollama вызывает клиент векторной БД через RPC / HTTP
2. Клиент формирует запрос embedding → API БД
3. БД возвращает список идентификаторов и расстояний
4. Сервис получает документы по идентификаторам из хранилища документов
5. Сервис передаёт модели Ollama вход + тексты и получает ответ

6. Сервис возвращает ответ конечному пользователю

Подсказка: Сначала запрос → ID → тексты → генерация → ответ.

Ответ:

- Сервис Ollama вызывает клиент векторной БД через RPC / HTTP
- Клиент формирует запрос embedding → API БД
- БД возвращает список идентификаторов и расстояний
- Сервис получает документы по идентификаторам из хранилища документов
- Сервис передаёт модели Ollama вход + тексты и получает ответ
- Сервис возвращает ответ конечному пользователю

Шаг 15:

Миссия 15

 При масштабировании интеграции: расставьте оптимизации по приоритету:

1. Кэширование embedding и результатов поиска
2. Параллелизация запросов к БД
3. Шардирование / распределение индекса по нодам
4. Использование компактных (quantized) векторов
5. Мониторинг качества поиска и коррекция параметров

Подсказка: Начинайте с простых ускорений, затем распределение, затем контроль.

Ответ:

- Кэширование embedding и результатов поиска
- Параллелизация запросов к БД
- Шардирование / распределение индекса по нодам
- Использование компактных (quantized) векторов
- Мониторинг качества поиска и коррекция параметров

Шаг 16:

Миссия 16

Когда внедряют fallback на текстовый поиск, расположите цепочку:

1. Попытка семантического поиска по embedding
2. **Если слишком низкая релевантность** — fallback к текстовому поиску (keyword)
3. Объединение результатов двух подходов
4. Фильтрация и ранжирование итогового списка
5. Передача результатов в генератор Ollama

Подсказка: Семантика первым, fallback — резервный, затем комбинируем и фильтруем.

Ответ:

- Попытка семантического поиска по embedding
- **Если слишком низкая релевантность** — fallback к текстовому поиску (keyword)
- Объединение результатов двух подходов
- Фильтрация и ранжирование итогового списка
- Передача результатов в генератор Ollama

Шаг 17:

Миссия 17

★ Финальная задача: расположите полный сценарий запроса через интеграцию Ollama + векторная БД:

1. Пользователь задаёт вопрос
2. Сервис Ollama / API формирует embedding запроса
3. Клиент векторной БД запрашивает ближайшие векторы
4. БД возвращает идентификаторы документов + расстояния
5. Сервис вытягивает тексты документов по ID
6. Проверка / фильтрация релевантности документов
7. Передача запроса + тексты в модель Ollama
8. Генерация ответа с учётом документов
9. Постобработка / проверка фактов, корректировка
10. Возврат окончательного ответа пользователю

Подсказка: Подумайте, какие шаги — до генерации, какие — после; фильтрация перед моделью, коррекция после.

Ответ:

- Пользователь задаёт вопрос
- Сервис Ollama / API формирует embedding запроса
- Клиент векторной БД запрашивает ближайшие векторы
- БД возвращает идентификаторы документов + расстояния
- Сервис вытягивает тексты документов по ID
- Проверка / фильтрация релевантности документов
- Передача запроса + тексты в модель Ollama
- Генерация ответа с учётом документов
- Постобработка / проверка фактов, корректировка
- Возврат окончательного ответа пользователю

Урок 3 - Embeddings и поиск по смыслу

Шаг 01:

Миссия 1

Разработчик Анна внедряет embedding-модель для поиска документов. Какой подход даст наиболее качественное семантическое вхождение?

Подсказка: Учитывайте, что важна не просто близость слов, а смысл.

Ответ:

- **Использовать модель, обученную на большом корпусе текстов (например, OpenAI embeddings)**
- Просто усреднить векторы слов
- Использовать TF-IDF
- Использовать мешок слов

Шаг 02:

Миссия 2

 Игорь хочет сравнить два embedding-вектора. Какой метрикой лучше измерять их семантическую близость?

Подсказка: Косинусная мера часто используется для embeddings.

Ответ:

- **Косинусное сходство**
- Евклидово расстояние всегда лучше
- Хэммингово расстояние
- Манхэттенское расстояние

🔗 **Шаг 03:**

Миссия 3

👤 Катя строит систему RAG: на каком этапе она должна вычислять embedding запроса?

Подсказка: Запрос нужно представить в том же пространстве, что и документы.

Ответ:

- **Перед поиском семантически близких документов**
- После отбора документов
- После генерации ответа
- Ни разу не нужен embedding для запроса

🔗 **Шаг 04:**

Миссия 4

В команде рекомендуют нормализацию embedding-векторов. Зачем это нужно?

Подсказка: Нормализация влияет на метрику сравнения.

Ответ:

- **Чтобы все векторы имели единичную длину и сравнение было справедливым**
- Чтобы уменьшить размер вектора
- Чтобы поменять направление вектора
- Чтобы увеличить различия между близкими векторами

🔗 **Шаг 05:**

Миссия 5

🌐 При внедрении в веб-приложение выбирают, где хранить embeddings: в базе SQL или vector database. Что важнее при выборе vector DB?

Подсказка: Обратите внимание на специализированные индексы.

Ответ:

- **Наличие эффективного ANN-индекса (HNSW, IVF и т.д.)**
- Большой объём SQL-таблицы
- Поддержка сложных SQL-запросов
- Наличие триггеров на insert

🔗 **Шаг 06:**

Миссия 6

🔗 При хранении embeddings в векторной базе: что делать при вставке большого батча документов?

Подсказка: Подумайте об эффективности обновления индекса.

Ответ:

- **Пакетно строить индекс, а не вставлять по одному**

- Вставлять по одному синхронно
- Отключать все индексы
- Вставлять только часть и потом дополнять вручную

🔮 Шаг 07:

Миссия 7

🔊 Сопоставьте термины embeddings / ANN / RAG / FAISS с их определениями:

Подсказка: Подумайте, какая роль у каждого в модуле поиска знаний.

Термины:

Ответ:

- Embeddings
- ANN (Approximate Nearest Neighbors)
- RAG (Retrieval-Augmented Generation)
- FAISS

Определения:

1. Подход, при котором генерация опирается на извлечённые документы
 2. Библиотека от Meta для эффективного поиска векторных соседей
 3. Нечёткий поиск ближайших соседей для скорого ответа
 4. Представление текста в виде числового вектора
- **Embeddings** — Представление текста в виде числового вектора
 - **ANN (Approximate Nearest Neighbors)** — Нечёткий поиск ближайших соседей для скорого ответа
 - **RAG (Retrieval-Augmented Generation)** — Подход, при котором генерация опирается на извлечённые документы
 - **FAISS** — Библиотека от Meta для эффективного поиска векторных соседей

🔮 Шаг 08:

Миссия 8

📖 Алексей имеет corpus из разных доменов. Сопоставьте стратегию с её описанием:

Подсказка: Подумайте, как обеспечить баланс между универсальностью и точностью.

Стратегии:

Ответ:

- Глобальная embedding-модель
- Специальная модель на домен
- Гибридный подход
- Локальный fine-tuning на запросы

Описания:

1. Использовать одну модель на все домены без дообучения
 2. Тренировать модель только на текстах конкретного домена
 3. Комбинировать глобальную модель с доменными адаптациями
 4. Периодически дообучать модель под запросы пользователей
- **Глобальная embedding-модель** — Использовать одну модель на все домены без дообучения

- **Специальная модель на домен** — Тренировать модель только на текстах конкретного домена
- **Гибридный подход** — Комбинировать глобальную модель с доменными адаптациями
- **Локальный fine-tuning на запросы** — Периодически дообучать модель под запросы пользователей

🔮 Шаг 09:

Миссия 9

✂ Сопоставьте операцию с её эффектом при поиске по embedding:

Подсказка: Каждая операция улучшает разные аспекты поиска.

Операции:

Ответ:

- Нормализация векторов
- Кластеризация векторов
- Прореживание документа (document pruning)
- Ранжирование результатов

Эффекты:

1. Уменьшение шума и удаление нерелевантных части текста
2. Упрощение поиска, группировка похожих векторов
3. Обеспечение корректного сравнения расстояний
4. Упорядочение кандидатов по степени релевантности

- **Нормализация векторов** — Обеспечение корректного сравнения расстояний
- **Кластеризация векторов** — Упрощение поиска, группировка похожих векторов
- **Прореживание документа (document pruning)** — Уменьшение шума и удаление нерелевантных части текста
- **Ранжирование результатов** — Упорядочение кандидатов по степени релевантности

🔮 Шаг 10:

Миссия 10

🔍 При гибридном поиске текст + embedding: сопоставьте компоненту с её задачей:

Подсказка: Гибрид сочетает точные фильтры и семантику.

Компоненты:

Ответ:

- Булев фильтр (metadata)
- BM25 / классический текстовый поиск
- Embedding-поиск
- Конкатенация рангов

Задачи:

1. Отсеивать документы по строгим критериям (датам, категориям)
2. Собирать кандидатов по ключевым словам
3. Оценивать семантическую близость запроса и текста
4. Объединять результаты разных методов в итоговый ранг

- **Булев фильтр (metadata)** — Отсеивать документы по строгим критериям (датам, категориям)

- **BM25 / классический текстовый поиск** — Собирать кандидатов по ключевым словам
- **Embedding-поиск** — Оценивать семантическую близость запроса и текста
- **Конкатенация рангов** — Объединять результаты разных методов в итоговый ранг

🧭 Шаг 11:

Миссия 11

Разработчик тестирует систему embedding-поиска. Сопоставьте кейс с ожидаемым поведением:

Подсказка: Рассмотрите случаи близких запросов и синонимичных выражений.

Кейсы:

Ответ:

- Запрос «машинное обучение» и «ML»
- Словосочетание в обратном порядке
- Новый термин, не в корпусе
- Запрос с опечаткой

Ожидаемое поведение:

1. Найти те же документы (устойчивая семантика)
 2. Найти похожие документы, возможно с меньшей оценкой
 3. Предсказать близость по контексту (экстраполяция)
 4. Обработать опечатку как близкий запрос
- **Запрос «машинное обучение» и «ML»** — Найти те же документы (устойчивая семантика)
 - **Словосочетание в обратном порядке** — Найти похожие документы, возможно с меньшей оценкой
 - **Новый термин, не в корпусе** — Предсказать близость по контексту (экстраполяция)
 - **Запрос с опечаткой** — Обработать опечатку как близкий запрос

🧭 Шаг 12:

Миссия 12

📋 Упорядочьте этапы pipeline RAG с embedding-поиском:

1. Генерация ответа LLM на основе извлечённых документов
2. Вычисление embedding запроса
3. Семантический поиск среди документов по embedding
4. Предобработка корпуса и построение индекса
5. Объединение результатов (фильтры + ранжирование)

🧭 **Подсказка:**

Подумайте, что должно быть готово до поступления запроса, и что идёт после.

Ответ:

- Предобработка корпуса и построение индекса
- Вычисление embedding запроса
- Семантический поиск среди документов по embedding
- Объединение результатов (фильтры + ранжирование)
- Генерация ответа LLM на основе извлечённых документов

🔮 Шаг 13:

Миссия 13

Упорядочите шаги при доработке embedding-индекса после добавления новых документов:

1. Инкрементальная вставка новых embeddings
2. Ребалансировка или реконструкция индекса
3. Нормализация новых векторов
4. Рейтирование всего пространства (если нужно)
5. Перезапуск сервиса поиска (если требуется)

🔮 Подсказка:

Новые векторы нужно подготовить перед вставкой, и после большой вставки может потребоваться реконструкция.

Ответ:

- Нормализация новых векторов
- Инкрементальная вставка новых embeddings
- Ребалансировка или реконструкция индекса
- Рейтирование всего пространства (если нужно)
- Перезапуск сервиса поиска (если требуется)

🔮 Шаг 14:

Миссия 14

🔮 Упорядочите приоритеты, если качество поиска по embedding начинает ухудшаться:

1. Проверка распределения embedding (есть ли «скопления»)
2. Анализ выборки данных на нерелевантные документы
3. Перестроение или обновление индекса
4. Проверка качества модели embedding и возможный fine-tuning
5. Улучшение ранжирования (сигналы внешние)

🔮 Подсказка:

Сначала диагностируем, затем пробуем менее инвазивные корректировки, и только потом — глобальные изменения.

Ответ:

- Проверка распределения embedding (есть ли «скопления»)
- Анализ выборки данных на нерелевантные документы
- Перестроение или обновление индекса
- Проверка качества модели embedding и возможный fine-tuning
- Улучшение ранжирования (сигналы внешние)

🔮 Шаг 15:

Миссия 15

⚙️ Упорядочите шаги, которые выполнит алгоритм при поиске top-k документов:

1. Вычислить embedding запроса
2. Определить candidate set через ANN-индекс
3. Нормализовать embedding запрос и документов
4. Вычислить точные расстояния для кандидатов
5. Отсортировать кандидатов и вернуть top-k

💡 **Подсказка:** Сначала переводим запрос в `embedding`, затем фильтруем кандидатов, затем уточняем и ранжируем.

Ответ:

- Вычислить `embedding` запроса
- Нормализовать `embedding` запрос и документов
- Определить `candidate set` через ANN-индекс
- Вычислить точные расстояния для кандидатов
- Отсортировать кандидатов и вернуть `top-k`

💡 **Шаг 16:**

Миссия 16

Упорядочите типы тестов, которые стоит провести при отладке системы `embedding`-поиска:

1. Проверка на простые базовые запросы (очевидные результаты)
2. Тестирование синонимов и перефразов
3. Тестирование с шумом и опечатками
4. Стресс-тесты с большим количеством запросов и документов
5. Регрессионное тестирование после обновления модели

💡 **Подсказка:** Начинаем с базовых случаев, затем сложных, и в конце — нагрузочное и регрессии.

Ответ:

- Проверка на простые базовые запросы (очевидные результаты)
- Тестирование синонимов и перефразов
- Тестирование с шумом и опечатками
- Стресс-тесты с большим количеством запросов и документов
- Регрессионное тестирование после обновления модели

💡 **Шаг 17:**

Миссия 17

🔗 Упорядочите этапы внедрения системы RAG с векторной памятью в `production`:

1. Сбор и предобработка корпуса документов
2. Обучение / выбор `embedding`-модели
3. Построение и тестирование ANN-индекса на `dev` данных
4. Интеграция с `backend` и API (поисковый модуль)
5. Мониторинг качества запросов и обратная связь пользователей
6. Итеративное улучшение модели `embedding` и ранжирования

💡 **Подсказка:** Сначала готовим данные и модель, затем интеграция, затем мониторим и улучшаем.

Ответ:

- Сбор и предобработка корпуса документов
- Обучение / выбор `embedding`-модели
- Построение и тестирование ANN-индекса на `dev` данных
- Интеграция с `backend` и API (поисковый модуль)
- Мониторинг качества запросов и обратная связь пользователей
- Итеративное улучшение модели `embedding` и ранжирования

Урок 4 - Встраивание контекста в промпт

Шаг 01:

Миссия 1

Елена готовит промпт к LLM с контекстом: что следует включить первым?

Подсказка: Контекст нужен до вопроса, чтобы модель «понимала», на что опираться.

Ответ:

- **Сначала вставить вспомогательный контекст (факты, данные)**
- Сначала задать вопрос модели
- Сначала перечислить требования к стилю ответа
- **Сначала вставить инструкцию «Ты — эксперт»**

Шаг 02:

Миссия 2

 Максим решает, как ограничить объём контекста: что разумно делать?

Подсказка: Взвесьте trade-off между количеством контекста и стоимостью обработки.

Ответ:

- **Ограничивать контекст до N последних предложений или символов**
- Вставлять весь текст без ограничений
- Игнорировать контекст вовсе
- Делать случайный сэмплинг предложений

Шаг 03:

Миссия 3

 Даша хочет динамически встроить пользовательские данные (например, профиль) в промпт. Как лучше?

Подсказка: Данные должны быть релевантны запросу и аккуратно оформлены.

Ответ:

- **Включать только релевантные поля пользователя в структурированном виде**
- Вставлять весь JSON-объект профиля как текст
- Не включать профиль вовсе
- Дублировать профиль как три разных фрагмента

Шаг 04:

Миссия 4

 При обновлении знаний системы, что нужно делать с embed-контекстом?

Подсказка: Контекст — это ваша память, её нужно актуализировать.

Ответ:

- **Регенерировать embeddings и обновить контекстные вставки**
- Оставить старые embeddings навсегда
- Не обновлять контекст, только задавать вопрос
- Удалять весь контекст

🔮 Шаг 05:

Миссия 5

🔮 В промпт встраивается список релевантных документов. Как их выбрать?

Подсказка: Лучше не вставлять слишком много — это может запутать модель.

Ответ:

- **Отбирать top-k самых релевантных документов**
- Вставлять весь корпус документов
- Выбирать случайные документы
- Вставлять документы непрерывным текстом без разделителей

🔮 Шаг 06:

Миссия 6

🔮 При добавлении контекста Костя думает: стоит ли прерывать инструкцию разметкой? Что сделать?

Подсказка: Ясное разделение частей промпта помогает модели ориентироваться.

Ответ:

- **Разделять контекст, инструкцию и запрос с помощью четких меток (например, ###)**
- Смешивать все тексты подряд
- Вставлять маркеры в начале строки, но не разделять секции
- Не использовать маркировки

🔮 Шаг 07:

Миссия 7

🔮 Сопоставьте часть промпта с её функцией в контексте:

Подсказка: Подумайте, какую роль играет каждый сегмент.

Части промпта:

Ответ:

- Контекстные знания
- Системная инструкция
- Пользовательский запрос
- Ограничения формата ответа

Функции:

1. Ограничить длину, стиль, формат ответа
2. Задать фоновые факты, на которые опираться
3. Прямой вопрос к модели, что именно спросить
4. Установить роль модели и метаправила

- **Контекстные знания** — Задать фоновые факты, на которые опираться
- **Системная инструкция** — Установить роль модели и метаправила
- **Пользовательский запрос** — Прямой вопрос к модели, что именно спросить
- **Ограничения формата ответа** — Ограничить длину, стиль, формат ответа

🔮 Шаг 08:

Миссия 8

Сопоставьте подход к контексту с возможным риском:

Подсказка: Контекст может мешать, если он плохого качества или слишком длинный.

Подход:

Ответ:

- Вставить весь текст контекста
- Оставить только заголовки и ключевые абзацы
- Фильтровать контекст с пониженной релевантностью
- Обновлять контекст только по пороговому изменению

Риски:

1. Перегрузка промпта, модель запутается
 2. Упустить важные детали, если убрать слишком много
 3. Оставить шум, который ухудшит ответы
 4. Контекст быстро устареет, если обновлять редко
- **Вставить весь текст контекста** — Перегрузка промпта, модель запутается
 - **Оставить только заголовки и ключевые абзацы** — Упустить важные детали, если убрать слишком много
 - **Фильтровать контекст с пониженной релевантностью** — Оставить шум, который ухудшит ответы
 - **Обновлять контекст только по пороговому изменению** — Контекст быстро устареет, если обновлять редко

Шаг 09:

Миссия 9

 Сопоставьте тип контекстной вставки с её характерной чертой:

Подсказка: Контекст может быть кратким, расширенным или структурированным.

Тип:

Ответ:

- Краткий абзац
- Развёрнутый текст
- Список пунктов
- Таблица с ключ-значениями

Черты:

1. Читаемость и компактность
 2. Богатство деталей, но риск избыточности
 3. Структурированное выделение важных пунктов
 4. Чётко разделённые поля, легко парсить
- **Краткий абзац** — Читаемость и компактность
 - **Развёрнутый текст** — Богатство деталей, но риск избыточности
 - **Список пунктов** — Структурированное выделение важных пунктов
 - **Таблица с ключ-значениями** — Чётко разделённые поля, легко парсить

Шаг 10:

Миссия 10

 Сопоставьте технику встраивания контекста с её эффектом:

Подсказка: Некоторые техники улучшают точность, другие — устойчивость к шуму.

Техника:

Ответ:

- Маркировка секций (###)
- Использование «...» тегов
- Прокси-упоминания (ссылки на id документов)
- Резюмирование контекста

Эффекты:

1. Модель явно понимает границы контекстных блоков
 2. Структурное разграничение помогает выделять контекст
 3. Уменьшение объема вставки, но риск потери связи
 4. Упрощает сложный контекст, оставляя главное
- **Маркировка секций (###)** — Модель явно понимает границы контекстных блоков
 - **Использование «...» тегов** — Структурное разграничение помогает выделять контекст
 - **Прокси-упоминания (ссылки на id документов)** — Уменьшение объема вставки, но риск потери связи
 - **Резюмирование контекста** — Упрощает сложный контекст, оставляя главное

Шаг 11:

Миссия 11

Сопоставьте формат встраиваемого контекста с уместной ситуацией:

Подсказка: Контекст должен соответствовать типу данных.

Формат:

Ответ:

- Неформатированный текст
- JSON
- Markdown
- Короткий абзац + ссылка на документ

Ситуации:

1. **Контекст** — лог непрерывных событий
 2. Структурированные пользовательские данные (пол, возраст и т.д.)
 3. Техническая спецификация, где важны заголовки и списки
 4. Большой текст, который не помещается целиком
- **Неформатированный текст** — Контекст — лог непрерывных событий
 - **JSON** — Структурированные пользовательские данные (пол, возраст и т.д.)
 - **Markdown** — Техническая спецификация, где важны заголовки и списки
 - **Короткий абзац + ссылка на документ** — Большой текст, который не помещается целиком

Шаг 12:

Миссия 12

 Упорядочьте этапы формирования финального промпта с контекстом:

1. Добавление инструкции роли модели
2. Встроить отобранные контекстные фрагменты

3. Вставить пользовательский запрос
4. Добавить ограничения формата ответа
5. Маркировать границы (разделители)

Подсказка:

Сначала указываем роль, затем контекст, потом вопрос, потом ограничения, и всё разделяем ясно.

Ответ:

- Добавление инструкции роли модели
- Маркировать границы (разделители)
- Встроить отобранные контекстные фрагменты
- Вставить пользовательский запрос
- Добавить ограничения формата ответа

Шаг 13:

Миссия 13

 Упорядочите шаги, если вставленный контекст увеличивает латентность ответа:

1. Сократить число контекстных фрагментов
2. Резюмировать контекстные тексты
3. Проверить, всё ли контекст релевантен
4. Оптимизировать модель под длинные промпы (если возможно)
5. Удалить наименее полезные фрагменты

Подсказка:

Сначала диагностируйте лишние части, затем пробуйте сокращения и оптимизации.

Ответ:

- Сократить число контекстных фрагментов
- Резюмировать контекстные тексты
- Проверить, всё ли контекст релевантен
- Оптимизировать модель под длинные промпы (если возможно)
- Удалить наименее полезные фрагменты

Шаг 14:

Миссия 14

Упорядочите приоритеты при улучшении контекстного промпта:

1. Улучшение качества контекстных фрагментов (релевантность)
2. Переработка структуры разделения контекста
3. Оптимизация длины промпта
4. Тестирование разных шаблонов вставки
5. Автоматизация отбора и вставки контекста

 **Подсказка: Сначала контент, потом форма, потом автоматизация — так логично эволюционировать.**

Ответ:

- Улучшение качества контекстных фрагментов (релевантность)
- Переработка структуры разделения контекста
- Оптимизация длины промпта

- Тестирование разных шаблонов вставки
- Автоматизация отбора и вставки контекста

🔮 Шаг 15:

Миссия 15

🔍 При росте объёма контекста, как разумно перераспределить структуру:

1. Объединить фрагменты в тематические блоки
2. Внедрить внутренние заголовки разделов
3. Вставить ссылки на внешние документы (при необходимости)
4. Сократить менее важные детали внутри блоков
5. Повторно оценить релевантность после изменений

🔮 **Подсказка: Сначала структурируйте, затем сокращайте и проверяйте актуальность.**

Ответ:

- Объединить фрагменты в тематические блоки
- Внедрить внутренние заголовки разделов
- Вставить ссылки на внешние документы (при необходимости)
- Сократить менее важные детали внутри блоков
- Повторно оценить релевантность после изменений

🔮 Шаг 16:

Миссия 16

🔍 Упорядочите виды тестов промптов с контекстом:

1. Базовые кейсы с простым контекстом
2. Синонимы и перефразы при том же контексте
3. Противоречивый контекст (две версии информации)
4. Контекст + шум / нерелевантные фрагменты
5. **Крайние случаи** — очень длинный или пустой контекст

🔮 **Подсказка: Идём от простого к сложному: сначала нормальные случаи, затем хаотичные, затем крайние.**

Ответ:

- Базовые кейсы с простым контекстом
- Синонимы и перефразы при том же контексте
- Противоречивый контекст (две версии информации)
- Контекст + шум / нерелевантные фрагменты
- **Крайние случаи** — очень длинный или пустой контекст

🔮 Шаг 17:

Миссия 17

🔍 Упорядочите этапы внедрения контекстных промптов в систему:

1. Сбор корпуса контекстных фрагментов
2. Разработка шаблонов вставки и маркировок
3. Интеграция с генератором (LLM)
4. Тестирование качества ответов с контекстом
5. Мониторинг производительности и латентности

6. Итеративная корректировка шаблонов и отбора контекста

💡 **Подсказка:** Сначала готовим контекст и шаблоны, затем интегрируем, тестируем, мониторим и улучшаем.

Ответ:

- Сбор корпуса контекстных фрагментов
- Разработка шаблонов вставки и маркировок
- Интеграция с генератором (LLM)
- Тестирование качества ответов с контекстом
- Мониторинг производительности и латентности
- Итеративная корректировка шаблонов и отбора контекста

📖 Урок 5 - 🎯 Квест создай RAG-помощника по документации

💡 Шаг 01:

Миссия 1

Инженер Оля хочет начать создание RAG-ассистента по документации. С чего лучше начать?

Подсказка: Без корпуса документов ассистент «не знает, о чём говорить».

Ответ:

- **Собрать и структурировать корпус документации**
- Сразу писать промпты к LLM
- Запустить интерфейс для пользователей
- Прописать юнит-тесты для генерации ответов

💡 Шаг 02:

Миссия 2

📖 Марк готовит корпус: какие документы включить в RAG-систему?

Подсказка: Важны актуальные технические гайды, API-референсы, примеры кода.

Ответ:

- **API референсы, гайды, примеры кода и changelog**
- Все документы, включая маркетинг
- Только примеры кода
- Только high-level обзоры без детализации

💡 Шаг 03:

Миссия 3

🧠 Лена думает о сегментации: как разбить документацию для embedding?

Подсказка: Слишком большие фрагменты плохо усваиваются, слишком мелкие — теряется контекст.

Ответ:

- **Разбивать на логические секции (пункты, подпункты) разумного размера**
- Оставлять весь документ как один фрагмент
- Разбивать на строки по 1 предложению
- Разбивать на случайные куски фиксированной длины

🔮 Шаг 04:

Миссия 4

🔍 Артём внедряет embedding: какую модель лучше использовать на старте?

Подсказка: Используйте проверенные общие модели перед кастомизацией.

Ответ:

- **Предобученную open source или коммерческую embedding-модель**
- Модель, обученную на случайных данных
- Модель без предобучения
- Просто использовать Bag-of-Words

🔮 Шаг 05:

Миссия 5

🗺 На каком этапе стоит построить ANN-индекс по embedding?

Подсказка: Индекс нужен после того, как embeddings готовы.

Ответ:

- **После генерации embedding для всех фрагментов**
- Сразу перед сегментацией
- После интеграции интерфейса
- Никогда не строить индекс

🔮 Шаг 06:

Миссия 6

🔍 Катя тестирует поиск: что должно быть корректно на базовом тесте?

Подсказка: Запрос должен возвращать максимально релевантный фрагмент.

Ответ:

- **Запрос «функция X» возвращает описание этой функции из документации**
- Запрос «функция X» возвращает случайный фрагмент
- Запрос с пустым текстом возвращает что угодно
- Запрос «foo» возвращает весь корпус

🔮 Шаг 07:

Миссия 7

Сопоставьте стадии с их задачей при создании RAG-ассистента:

Подсказка: Подумайте, какая стадия что делает в pipeline.

Стадии:

Ответ:

- Сбор и сегментация документации
- Генерация embedding фрагментов
- Построение ANN-индекса
- Поисковый запрос + ранжирование кандидатов

Задачи:

1. Создать компактные логические фрагменты из документов
2. Преобразовать текст в числовую векторную форму

3. Обеспечить быстрый поиск ближайших фрагментов

4. Выбрать лучшие фрагменты по релевантности

- **Сбор и сегментация документации** — Создать компактные логические фрагменты из документов
- **Генерация embedding фрагментов** — Преобразовать текст в числовую векторную форму
- **Построение ANN-индекса** — Обеспечить быстрый поиск ближайших фрагментов
- **Поисковый запрос + ранжирование кандидатов** — Выбрать лучшие фрагменты по релевантности

Шаг 08:

Миссия 8

 Сопоставьте тип документации с оптимальной стратегией вставки в контекст:

Подсказка: Более формальные или объемные тексты требуют аккуратной встраиваемости.

Тип документации:

Ответ:

- Краткий FAQ
- Детальный API референс
- Технический обзор архитектуры
- Изменения в последнем релизе (changelog)

Стратегии вставки:

1. Вставлять весь текст FAQ как один фрагмент
 2. Разбивать API по методам / модулям
 3. Резюмировать ключевые разделы обзора и вставлять абзацами
 4. Вставлять только последние изменения как отдельный блок
- **Краткий FAQ** — Вставлять весь текст FAQ как один фрагмент
 - **Детальный API референс** — Разбивать API по методам / модулям
 - **Технический обзор архитектуры** — Резюмировать ключевые разделы обзора и вставлять абзацами
 - **Изменения в последнем релизе (changelog)** — Вставлять только последние изменения как отдельный блок

Шаг 09:

Миссия 9

 Сопоставьте проблему, с которой может столкнуться RAG-ассистент по документации, и способ её смягчения:

Подсказка: Проблемы часто связаны с шумом, противоречием и устареванием.

Проблема:

Ответ:

- Устаревшая информация в контексте
- Противоречивые версии в документации
- Слишком длинный контекст, превышающий лимит токенов
- Нерелевантные фрагменты мешают качеству ответа

Меры смягчения:

1. Регулярно обновлять корпус и embeddings
 2. Выбирать наиболее свежую или подавлять старые версии
 3. Сокращать контекст: резюме, фильтры
 4. Фильтровать и ранжировать по релевантности
- **Устаревшая информация в контексте** — Регулярно обновлять корпус и embeddings
 - **Противоречивые версии в документации** — Выбирать наиболее свежую или подавлять старые версии
 - **Слишком длинный контекст, превышающий лимит токенов** — Сокращать контекст: резюме, фильтры
 - **Нерелевантные фрагменты мешают качеству ответа** — Фильтровать и ранжировать по релевантности

Шаг 10:

Миссия 10

 Сопоставьте формат выдачи из RAG-ассистента с его целью:

Подсказка: Формат может варьироваться от цитат до сводки и расширенного ответа.

Формат ответа:

Ответ:

- Цитата из документации с указанием источника
- Сводка / краткий ответ
- Подробное объяснение с примерами кода
- Ссылки на релевантные разделы

Цель:

1. Показать точное утверждение из официальной документации
 2. Дать компактный ответ, если нужно быстрое понимание
 3. Углубить понимание с примерами и деталями
 4. Помочь пользователю быстро перейти к нужному фрагменту
- **Цитата из документации с указанием источника** — Показать точное утверждение из официальной документации
 - **Сводка / краткий ответ** — Дать компактный ответ, если нужно быстрое понимание
 - **Подробное объяснение с примерами кода** — Углубить понимание с примерами и деталями
 - **Ссылки на релевантные разделы** — Помочь пользователю быстро перейти к нужному фрагменту

Шаг 11:

Миссия 11

 Упорядочьте этапы отладки RAG-ассистента до запуска в production:

1. Тестирование запросов и проверка релевантности
2. Оптимизация структуры контекстов и шаблонов промптов
3. Мониторинг времени ответа и узких мест
4. Интеграция в клиентское приложение
5. Обратная связь от пользователей и улучшения
6. Создание метрик качества (например, точность, полнота)

💡 **Подсказка: Сначала тестируем и доводим качество, затем интегрируем, мониторим и улучшаем.**

Ответ:

- Тестирование запросов и проверка релевантности
- Оптимизация структуры контекстов и шаблонов промптов
- Мониторинг времени ответа и узких мест
- Интеграция в клиентское приложение
- Обратная связь от пользователей и улучшения
- Создание метрик качества (например, точность, полнота)

🔮 Шаг 12:

Миссия 12

📋 Упорядочите шаги оценки качества ответов RAG-ассистента:

1. Сбор выборки типовых запросов пользователей
2. Проверка, какой фрагмент документации выдан
3. Сравнение ответа с эталоном (если есть)
4. Оценка метрик (точность, полнота, свежесть)
5. Корректировка промптов или контекста на основе анализа

💡 **Подсказка: Сначала собирайте реальные запросы, затем оценивайте результат и вносите улучшения.**

Ответ:

- Сбор выборки типовых запросов пользователей
- Проверка, какой фрагмент документации выдан
- Сравнение ответа с эталоном (если есть)
- Оценка метрик (точность, полнота, свежесть)
- Корректировка промптов или контекста на основе анализа

🔮 Шаг 13:

Миссия 13

⌚ Упорядочите действия, если латентность ответов чрезмерна:

1. Уменьшить число контекстных фрагментов
2. Сократить длину фрагментов (резюмирование)
3. Оптимизировать ANN-индекс (перестроить, ребалансировать)
4. Кэшировать результаты частых запросов
5. Профилировать узкие места системы (CPU, I/O и т.д.)

💡 **Подсказка: Сначала упростите контекст, затем оптимизируйте инфраструктуру и добавьте кэширование.**

Ответ:

- Уменьшить число контекстных фрагментов
- Сократить длину фрагментов (резюмирование)
- Оптимизировать ANN-индекс (перестроить, ребалансировать)
- Кэшировать результаты частых запросов
- Профилировать узкие места системы (CPU, I/O и т.д.)

🔮 Шаг 14:

Миссия 14

🔄 Упорядочите циклы итераций для совершенствования RAG-ассистента:

1. Получить обратную связь пользователей
2. Проанализировать слабые места ответов
3. Изменить шаблоны промптов или контекст
4. Обновить embeddings / корпус, если нужно
5. Запустить повторное тестирование и мониторинг

🔮 **Подсказка: Цикл включает feedback → анализ → правки → обновление → проверку.**

Ответ:

- Получить обратную связь пользователей
- Проанализировать слабые места ответов
- Изменить шаблоны промптов или контекст
- Обновить embeddings / корпус, если нужно
- Запустить повторное тестирование и мониторинг

🔮 Шаг 15:

Миссия 15

📦 При расширении документации: как встроить новую часть без сбоев?

1. Сгенерировать embedding для новых фрагментов
2. Проверить релевантность новых фрагментов к существующему корпусу
3. Вставить фрагменты в ANN-индекс (инкрементально или перестроить)
4. Потестировать примеры запросов, касающиеся новой темы
5. Проверить, что старые ответы не ухудшились (регрессия)

🔮 **Подсказка: Новые части нужно обработать, интегрировать и протестировать — как расширение приложения.**

Ответ:

- Сгенерировать embedding для новых фрагментов
- Проверить релевантность новых фрагментов к существующему корпусу
- Вставить фрагменты в ANN-индекс (инкрементально или перестроить)
- Потестировать примеры запросов, касающиеся новой темы
- Проверить, что старые ответы не ухудшились (регрессия)

🔮 Шаг 16:

Миссия 16

🛡️ Упорядочите шаги защиты ассистента от «галлюцинаций» (неверных ответов):

1. Ограничивать генерацию только контекстными фрагментами (retrieval-only режим)
2. Добавлять проверки фактов/контекстов в ответ
3. Кросс-валидация ответа с другим источником документации
4. Отбрасывать кандидаты с низкой релевантностью перед генерацией
5. Отслеживать и логировать ответы с низким доверием для анализа

🔮 **Подсказка: Недостоверные генерации лучше предотвратить заранее: уточнение, проверка и фильтрация важны.**

Ответ:

- Ограничивать генерацию только контекстными фрагментами (retrieval-only режим)
- Добавлять проверки фактов/контекстов в ответ
- Кросс-валидация ответа с другим источником документации
- Отбрасывать кандидаты с низкой релевантностью перед генерацией
- Отслеживать и логировать ответы с низким доверием для анализа

🔗 Шаг 17:

Миссия 17

🔗 Упорядочите конечные шаги для запуска RAG-ассистента по документации на продуктив:

1. Запуск на тестовой среде, проверка корректности
2. Нагрузочное тестирование (производительность, латентность)
3. Мониторинг качества ответов в режиме реального времени
4. Сбор статистики: метрики релевантности, частые запросы, ошибки
5. Итеративное улучшение (контекст, промпты, корпус)
6. Развёртывание на продакшн и сопровождение

🔗 **Подсказка: Пошаговый путь: тестирование → нагрузка → мониторинг → улучшения → продакшн.**

Ответ:

- Запуск на тестовой среде, проверка корректности
- Нагрузочное тестирование (производительность, латентность)
- Мониторинг качества ответов в режиме реального времени
- Сбор статистики: метрики релевантности, частые запросы, ошибки
- Итеративное улучшение (контекст, промпты, корпус)
- Развёртывание на продакшн и сопровождение

📁 Модуль 9 - Миссия 9. Взлом ядра Ollama

📖 Урок 1 - ⚙️ Взаимодействие с llama.cpp через CGo

🔗 Шаг 01:

Миссия 1

Иван решает начать эксперимент с llama.cpp через CGo. С чего логичнее всего начать?

Подсказка: Сначала стоит установить биндинг и убедиться, что Go может вызывать функции C.

Ответ:

- **Собрать простой обёртчик CGo к функции llama_init**
- Сразу запускать модель через высокоуровневый API
- Начать с обёртки UI-компонента
- Написать тесты для генерации текста

🔗 Шаг 02:

Миссия 2

🔗 Пётр хочет вызывать функцию llama_init с CGo. Какой способ правильный?

Подсказка: Используйте синтаксис C.имя при вызове.

Ответ:

- **C.llama_init(args) в Go**
- llama_init(args) напрямую
- CGo.llama_init(args)
- Go.llama_init(args)

🔮 Шаг 03:

Миссия 3

Когда вызывать C.llama_free или подобную функцию освобождения?

Подсказка: Освобождение ресурсов — часть контрактов C-API.

Ответ:

- **После завершения работы с моделью и перед выходом из программы**
- Никогда, GC Go всё освободит
- Перед каждым вызовом llama_init
- Во время генерации текста

🔮 Шаг 04:

Миссия 4

📦 Анна хочет передать Go-строку в функцию C. Что нужно сделать?

Подсказка: Go строка должна быть преобразована в C.char* с фиксацией памяти.

Ответ:

- **Использовать C.CString и defer C.free**
- Просто передать Go строку
- Использовать unsafe.Pointer без выделения
- Конвертировать в []byte и передать

🔮 Шаг 05:

Миссия 5

🔍 Сергей вызывает функцию, и он видит панику «cgo: c pointer passed to Go». Что делать?

Подсказка: Нельзя хранить указатели C в Go без специального трюка.

Ответ:

- **Не сохранять указатель C в Go, копировать данные**
- Отключить проверку cgo
- Хранить указатель как uintptr
- Передавать указатель Go в C

🔮 Шаг 06:

Миссия 6

🔗 Сопоставьте сущности C / Go / CGo с их ролью при использовании llama.cpp:

Подсказка: Подумайте, кто управляет памятью и кто вызывает функции.

Сущности:

Ответ:

- Go код
- C / llama.cpp функции

- Обёртка CGo
- Указатели C

Роли:

1. Вызывает функции C из Go
 2. Реализует низкоуровневые функции модели, аллокации, генерации
 3. Промежуточный слой между Go и C
 4. Управляются C, нельзя хранить в Go напрямую без копирования
- **Go код** — Вызывает функции C из Go
 - **C / llama.cpp функции** — Реализует низкоуровневые функции модели, аллокации, генерации
 - **Обёртка CGo** — Промежуточный слой между Go и C
 - **Указатели C** — Управляются C, нельзя хранить в Go напрямую без копирования

Шаг 07:

Миссия 7

🔍 Сопоставьте подход к управлению памятью с рисками:

Подсказка: Взаимодействие C и Go — зона утечек и паник.

Подход:

Ответ:

- Освобождать сразу после использования
- Копировать данные из C в Go и затем освобождать
- Использовать статическую память (глобальные буферы)
- Оставлять память до завершения всей программы

Риски:

1. Утечка памяти, если не освободить
 2. Случайное освобождение до использования
 3. Конкуренция и гонки при глобальных буферах
 4. Большой пик потребления и нестабильность
- **Освобождать сразу после использования** — Утечка памяти, если не освободить
 - **Копировать данные из C в Go и затем освобождать** — Случайное освобождение до использования
 - **Использовать статическую память (глобальные буферы)** — Конкуренция и гонки при глобальных буферах
 - **Оставлять память до завершения всей программы** — Большой пик потребления и нестабильность

Шаг 08:

Миссия 8

🔍 Сопоставьте тип ошибок с возможной причиной при использовании CGo + llama.cpp:

Подсказка: Ошибки часто связаны с границами памяти или несовместимыми типами.

Ошибка:

Ответ:

- segfault / сегфаулт
- panic: runtime error

- cgo: pointer outside C
- некорректное число аргументов

Причина:

1. Неправильное управление указателями, выход за границы
 2. Ошибки в Go (nil deref) при обработке результата C
 3. Передача Go указателей в C или обратное без копирования
 4. Несоответствие сигнатур функций между C и CGo
- **segfault / сегфаулт** — Неправильное управление указателями, выход за границы
 - **panic: runtime error** — Ошибки в Go (nil deref) при обработке результата C
 - **cgo: pointer outside C** — Передача Go указателей в C или обратное без копирования
 - **некорректное число аргументов** — Несоответствие сигнатур функций между C и CGo

🔗 Шаг 09:

Миссия 9

🔗 Упорядочьте шаги запуска простой генерации через CGo + llama.cpp:

1. Инициализация модели через CGo
2. Передача контекста / промпта как C строка
3. Вызов генератора (C.llama_generate)
4. Копирование результата из C в Go буфер
5. Освобождение C памяти, если нужно
6. Возврат строки Go пользователю

🔗 **Подсказка: Порядок: инициализация → подача данных → генерация → извлечение → очистка → возврат.**

Ответ:

- Инициализация модель через CGo
- Передача контекста / промпта как C строка
- Вызов генератора (C.llama_generate)
- Копирование результата из C в Go буфер
- Освобождение C памяти, если нужно
- Возврат строки Go пользователю

🔗 Шаг 10:

Миссия 10

Упорядочьте шаги оборачивания нескольких параметров (например, температура, max_tokens):

1. Определить структуру параметров в Go
2. Сконвертировать значения в C типы (int, float)
3. Сформировать C структуру / аргументы
4. Передать указатель на структуру C в функцию llama_generate
5. После использования освободить структуру, если требуется

🔗 **Подсказка: Сначала на уровне Go, затем перевод в C, передача, и очистка.**

Ответ:

- Определить структуру параметров в Go
- Сконвертировать значения в C типы (int, float)
- Сформировать C структуру / аргументы

- Передать указатель на структуру C в функцию llama_generate
- После использования освободить структуру, если требуется

🔗 Шаг 11:

Миссия 11

🔍 Упорядочьте шаги диагностики проблемы «не выводится текст»:

1. Проверить, вернулся ли указатель / буфер из C
2. Проверить, не произошла ли ошибка segfault / panic
3. Убедиться, что промпт передан корректно как C строка
4. Проверить, что генератор llama_generate был вызван
5. Проверить освобождение / очистку памяти, не вытёр ли результат

🔗 Подсказка: Диагностика — шаг за шагом: от конца (результат) к началу (ввод).

Ответ:

- Проверить, вернулся ли указатель / буфер из C
- Проверить, не произошла ли ошибка segfault / panic
- Убедиться, что промпт передан корректно как C строка
- Проверить, что генератор llama_generate был вызван
- Проверить освобождение / очистку памяти, не вытёр ли результат

🔗 Шаг 12:

Миссия 12

⚠ Упорядочьте шаги безопасной обработки C строк в многонитковом режиме:

1. Каждая горутина делает C.String отдельно
2. Копирует строку в Go память и освобождает C память
3. Никогда не делится одним C указателем между горутинами без защиты
4. Использует sync.Mutex или каналы при доступе к общим структурам C
5. Освобождает память через C.free после использования

🔗 Подсказка: Нити должны быть изолированы и не пересекаться по указателям C.

Ответ:

- Каждая горутина делает C.String отдельно
- Копирует строку в Go память и освобождает C память
- Никогда не делится одним C указателем между горутинами без защиты
- Использует sync.Mutex или каналы при доступе к общим структурам C
- Освобождает память через C.free после использования

🔗 Шаг 13:

Миссия 13

🔍 Упорядочьте шаги расширения поддержки новых функций llama.cpp (например, логиты, регенерация):

1. Добавить декларации функций в llama.cpp header и реализацию
2. Обновить CGO директивы и включить новые заголовки
3. Добавить Go-обёртки для новых функций
4. Написать тесты, вызывающие новые функции через Go
5. Проверить управление памятью и edge-кейсы новых функций

🔗 **Подсказка: Изменения сначала на уровне C, затем связать их через CGo, потом покрыть тестами.**

Ответ:

- Добавить декларации функций в llama.cpp header и реализацию
- Обновить CGo директивы и включить новые заголовки
- Добавить Go-обёртки для новых функций
- Написать тесты, вызывающие новые функции через Go
- Проверить управление памятью и edge-кейсы новых функций

🔗 **Шаг 14:**

Миссия 14

🔗 Упорядочьте цикл итераций при оптимизации производительности CGo + llama.cpp:

1. Профилировать узкие места (C vs Go части)
2. Оптимизировать код C (уменьшить копирование, inline функции)
3. Минимизировать переходы C ↔ Go (batch вызовы)
4. Кэшировать результаты на Go стороне при возможности
5. Проверить влияние на результат и корректность после оптимизации

🔗 **Подсказка: Оптимизация — циклический процесс: измерил → улучшил → проверил → повторил.**

Ответ:

- Профилировать узкие места (C vs Go части)
- Оптимизировать код C (уменьшить копирование, inline функции)
- Минимизировать переходы C ↔ Go (batch вызовы)
- Кэшировать результаты на Go стороне при возможности
- Проверить влияние на результат и корректность после оптимизации

🔗 **Шаг 15:**

Миссия 15

🔗 Упорядочьте шаги добавления тестов безопасности (fuzz, boundary test):

1. Генерация случайных строк-вводов для llama_generate
2. Проверка, что не происходит паника / segfault
3. Проверка результатов модели на корректность (не пусто, не мусор)
4. Использование краш-репортов для исправлений
5. Регрессионное тестирование после изменений

🔗 **Подсказка: Тесты безопасности сначала пытаются систему, затем фиксируют и проверяют стабильность.**

Ответ:

- Генерация случайных строк-вводов для llama_generate
- Проверка, что не происходит паника / segfault
- Проверка результатов модели на корректность (не пусто, не мусор)
- Использование краш-репортов для исправлений
- Регрессионное тестирование после изменений

🔧 Шаг 16:

Миссия 16

🔧 Упорядочьте финальные шаги для публикации Go-библиотеки с поддержкой llama.cpp:

1. Убедиться, что API стабильна и документирована
2. Скомпилировать кросс-сборки для разных платформ
3. Обновить Readme с примерами использования через Go
4. Написать интеграционные примеры (Go + llama.cpp)
5. Опубликовать релиз и сопровождать issues
6. Мониторинг обратной связи и исправление багов

🔧 **Подсказка: Сначала стабильность и документация, затем сборки, затем публикация и поддержка.**

Ответ:

- Убедиться, что API стабильна и документирована
- Скомпилировать кросс-сборки для разных платформ
- Обновить Readme с примерами использования через Go
- Написать интеграционные примеры (Go + llama.cpp)
- Опубликовать релиз и сопровождать issues
- Мониторинг обратной связи и исправление багов

📖 Урок 2 - 🧠 Профилирование потоков и памяти

🔧 Шаг 01:

Миссия 1

Вы инженер Ollama и впервые запускаете анализатор потоков. Какой инструмент вы бы применили первым?

Подсказка: Начните с чего-то простого, чтобы получить представление о потоках.

Ответ:

- **Запустить профилировщик потоков (thread profiler)**
- Запустить анализатор сети
- Запустить статическую проверку кода

🔧 Шаг 02:

Миссия 2

Вы смотрите на график загрузки CPU и видите пиковые всплески на одном из потоков. Что может быть причиной?

Подсказка: Поток может быть заблокирован на I/O или висеть в бесконечном цикле.

Ответ:

- Поток простаивает
- **Бесконечный цикл в коде**
- Ожидание на синхронизацию
- Поток завершён

🔧 Шаг 03:

Миссия 3

Вы хотите измерить, сколько памяти выделяет каждый поток. Какой подход наиболее прямой?

Подсказка: В динамической среде используйте профилировщик памяти.

Ответ:

- **Запустить профилировщик памяти (heap / allocation profiler)**
- Запустить анализ статистики
- Отключить сборщик мусора

Шаг 04:

Миссия 4

Сопоставьте термины с их значениями:

Термины:

Ответ:

- Heap snapshot
- Allocation stack trace
- GC pause
- Thread dump

Определения:

1. Снимок всех живых объектов в куче
2. Запись цепочки вызовов при выделении памяти
3. Прерывание работы на сборку мусора
4. Список стэков всех потоков в момент времени

Подсказка: Heap snapshot связан с объектами, а thread dump с потоками.

- **Heap snapshot** — Снимок всех живых объектов в куче
- **Allocation stack trace** — Запись цепочки вызовов при выделении памяти
- **GC pause** — Прерывание работы на сборку мусора
- **Thread dump** — Список стэков всех потоков в момент времени

Шаг 05:

Миссия 5

Сопоставьте признаки проблемы с возможным источником:

Признаки:

Ответ:

- Утечка памяти
- Постоянные пиковые CPU
- Блокировка потоков
- Большие аллокации за один раз

Источники:

1. Неосвобождённые ресурсы (listeners, буферы)
2. Неправильный алгоритм с бесконечным циклом
3. Неправильная синхронизация, deadlock
4. Загрузка большого объекта кэша

Подсказка: Утечки чаще связаны с ресурсами, блокировки — с синхронизацией.

- **Утечка памяти** — Неосвобождённые ресурсы (listeners, буферы)

- **Постоянные пиковые CPU** — Неправильный алгоритм с бесконечным циклом
- **Блокировка потоков** — Неправильная синхронизация, deadlock
- **Большие аллокации за один раз** — Загрузка большого объекта кэша

🔗 Шаг 06:

Миссия 6

Сопоставьте этап профилирования с его задачей:

Этапы:

Ответ:

- Сбор данных
- Анализ
- Оптимизация
- Валидация

Задачи:

1. Понимание узких мест
2. Изменение кода
3. Запуск профилировщика и логгирование
4. Проверка, что поведение не изменилось

Подсказка: Сначала собираем, потом анализируем, потом меняем и проверяем.

- **Сбор данных** — Запуск профилировщика и логгирование
- **Анализ** — Понимание узких мест
- **Оптимизация** — Изменение кода
- **Валидация** — Проверка, что поведение не изменилось

🔗 Шаг 07:

Миссия 7

Вы запустили профилировщик памяти и нашли множество короткоживущих объектов. Какой механизм может помочь понять, где они создаются?

Подсказка: Используйте трассировку аллокации.

Ответ:

- **Трассировка стеков аллокации (allocation stack trace)**
- Сбор полного heap snapshot
- Профилировщик сети

🔗 Шаг 08:

Миссия 8

Расположите этапы расследования утечки памяти в правильном порядке:

1. Проанализировать алгоритм выделения объектов
2. Собрать heap snapshot
3. Сравнить два snapshot'a (до/после нагрузки)
4. Выявить объекты, рост которых неостанавливается
5. Найти цепочку аллокации этих объектов
6. Исправить утечку и протестировать снова

Подсказка: Сначала снимки, затем визуальный анализ, потом локация источника и правка.

Ответ:

- Собрать heap snapshot
- Сравнить два snapshot'a (до/после нагрузки)
- Выявить объекты, рост которых неостанавливается
- Найти цепочку аллокации этих объектов
- Проанализировать алгоритм выделения объектов
- Исправить утечку и протестировать снова

 Шаг 09:

Миссия 9

Расположите шаги профилировки многопоточного приложения:

1. Собрать данные каждого потока
2. Агрегировать показатели
3. Выявить аномалии между потоками
4. Сравнить поведение под нагрузкой и без
5. Оптимизировать Hot-spots
6. Проверить, что программа работает корректно

Подсказка: Процесс похож на обычный роллинг анализа, но для потоков.

Ответ:

- Собрать данные каждого потока
- Агрегировать показатели
- Выявить аномалии между потоками
- Сравнить поведение под нагрузкой и без
- Оптимизировать Hot-spots
- Проверить, что программа работает корректно

 Шаг 10:

Миссия 10

Последовательность разрешения блокировки потоков:

1. Снять thread dump
2. Проанализировать взаимное ожидание локов
3. Найти класс и метод, где захват lock'a происходит
4. Внести изменения, минимизировать секцию синхронизации
5. Перезапустить и проверить отсутствие deadlock'ов

Подсказка: Сначала диагностика, затем исправление.

Ответ:

- Снять thread dump
- Проанализировать взаимное ожидание локов
- Найти класс и метод, где захват lock'a происходит
- Внести изменения, минимизировать секцию синхронизации
- Перезапустить и проверить отсутствие deadlock'ов

 Шаг 11:

Миссия 11

После профилирования вы обнаружили, что один метод вызывает большое число временных объектов. Что логичнее сделать?

Подсказка: Подумайте об уменьшении аллокаций.

Ответ:

- **Переписать метод, чтобы переиспользовать объекты или пулы**
- Увеличить память JVM
- Отключить GC
- Игнорировать, если не влияет на производительность

🔮 Шаг 12:

Миссия 12

Вы предлагаете внедрить cgroup-ограничения памяти для процесса Ollama. Что важно учесть?

Подсказка: Переполнение контейнера может вызывать ООМ внутри процесса.

Ответ:

- **Оставить резерв под пики и не задавать слишком жёсткие лимиты**
- Задать лимит равным текущей кучи
- Отключить swap внутри cgroup

🔮 Шаг 13:

Миссия 13

Сопоставьте метрики с их значением в контексте профилирования:

Метрики:

Ответ:

- RSS
- Heap used
- Thread count
- GC time

Значения:

1. Физическая память, занятая процессом
2. Количество потоков процесса
3. Объём занятой в куче памяти
4. Время, потраченное на сборку мусора

Подсказка: RSS — это просто память процесса, Heap used — только часть кучи.

- **RSS** — Физическая память, занятая процессом
- **Heap used** — Объём занятой в куче памяти
- **Thread count** — Количество потоков процесса
- **GC time** — Время, потраченное на сборку мусора

🔮 Шаг 14:

Миссия 14

Сопоставьте ошибку с возможным признаком:

Ошибки:

Ответ:

- OutOfMemoryError
- StackOverflowError
- Deadlock
- Too many threads

Признаки:

1. Потеря отклика, все потоки заблокированы
2. Память исчерпана
3. Глубокая рекурсия вызывает расход стека
4. Создание тысяч потоков приводит к OOM или падению

Подсказка: StackOverflow — рекурсия, too many threads — много потоков.

- **OutOfMemoryError** — Память исчерпана
- **StackOverflowError** — Глубокая рекурсия вызывает расход стека
- **Deadlock** — Потеря отклика, все потоки заблокированы
- **Too many threads** — Создание тысяч потоков приводит к OOM или падению

 Шаг 15:

Миссия 15

Расположите виды профилирования по степени затратности (от лёгкого к тяжёлому):

1. Слепые метрики (CPU, memory counters)
2. Sampling-профилирование
3. Инструментальное профилирование (instrumentation)
4. Трассировка аллокаций с полным стеком

Подсказка: Чем глубже подробность, тем больше накладных расходов.

Ответ:

- Слепые метрики (CPU, memory counters)
- Sampling-профилирование
- Инструментальное профилирование (instrumentation)
- Трассировка аллокаций с полным стеком

 Шаг 16:

Миссия 16

Финальный квест: упорядочьте шаги отталкиваясь от полного профиля потока до исправления hot-spot'a:

1. Собрать профилирование CPU и памяти одновременно
2. Выделить горячие точки (hot spots)
3. Проанализировать стек и аллокации горячих точек
4. Оптимизировать алгоритм горячих точек
5. Повторно профилировать и проверить, что улучшение есть
6. Документировать решение и критерии успеха

Подсказка: Последний этап — обязательно задокументировать выводы и подтвердить эффект.

Ответ:

- Собрать профилирование CPU и памяти одновременно

- Выделить горячие точки (hot spots)
- Проанализировать стек и аллокации горячих точек
- Оптимизировать алгоритм горячих точек
- Повторно профилировать и проверить, что улучшение есть
- Документировать решение и критерии успеха

Урок 3 - Добавление поддержки новой модели (Phi, Gemma)

Шаг 01:

Миссия 1

Вы — инженер Ollama, начинаете интеграцию новой модели Phi. С чего начнёте?
Подсказка: Сначала нужно убедиться, что модель совместима по API.

Ответ:

- Проверить формат входных/выходных тензоров модели Phi
- Сразу запускать нагрузочное тестирование
- Добавить GUI-обёртку для Phi

Шаг 02:

Миссия 2

Вы получили библиотеку с моделью Gemma, но не уверены в её зависимостях. Что проверите первым?

Подсказка: Иногда версии библиотек конфликтуют.

Ответ:

- Совместимость версий PyTorch / TensorFlow, требуемых Gemma
- Размер бинарника модели Gemma
- Лицензия модели Gemma

Шаг 03:

Миссия 3

При запуске модели Phi вы видите ошибку «undefined operator». Какое действие наиболее логично?

Подсказка: Возможно, оператор модели отсутствует в среде исполнения.

Ответ:

- Добавить реализацию недостающего оператора или подключить библиотеку с ним
- Игнорировать ошибку, если модель всё равно запускается
- Переключиться на Gemma временно

Шаг 04:

Миссия 4

Сопоставьте этапы интеграции новой модели с их задачами:

Ответ:

- Сериализация модели
- Загрузка в runtime
- Поддержка батчинга

- Обработка ошибок оператора

Подсказка: Каждому этапу соответствует специфическая задача — от подготовки данных до их оптимальной подачи.

- **Сериализация модели** — Преобразование в формат, понятный ядру Ollama (например, GGUF).
- **Загрузка в runtime** — Инициализация модели в оперативной памяти.
- **Поддержка батчинга** — Организация параллельной обработки нескольких запросов.
- **Обработка ошибок оператора** — Отладка и регистрация неизвестных операторов.

🔮 Шаг 05:

Миссия 5

Сопоставьте популярные модели и их ключевые особенности:

Подсказка: Учитывайте архитектуру, задачи и производительность.

Ответ:

- **Gemma** — Оптимизирована для CPU и edge-инференса.
- **Phi-2** — Небольшая модель от Microsoft, ориентированная на обучение reasoning-способностей.
- **Mistral** — Вариант Transformer без позиции (ALiBi), с высокой скоростью.
- **LLaMA 3** — Ориентирована на полноразмерное обучение и inference в облаке.

🔮 Шаг 06:

Миссия 6

Вы работаете с новыми моделями в Ollama. Сопоставьте действия с типичными ошибками:

Подсказка: Ошибки часто говорят о проблеме в слое, конфигурации или устройстве.

Ответ:

- **ValueError: shape mismatch** — Несовместимость размеров входных данных.
- **Cuda error: device-side assert triggered** — Ошибка на GPU — возможно, выход за границы индексов.
- **ImportError: No module named transformers** — Отсутствует библиотека 🤖 Transformers.
- **RuntimeError: expected scalar type Float** — Тип тензора не совпадает с ожидаемым.

🔮 Шаг 07:

Миссия 7

Разработчик Вася подключает Phi, но не уверен, какие параметры обязательны. Сопоставьте параметры и их назначение:

Подсказка: Подумайте, какие из них влияют на качество и поведение генерации.

Ответ:

- **temperature** — Контролирует степень случайности в ответе.
- **top_k** — Ограничивает выбор токенов k самыми вероятными.
- **max_tokens** — Максимальное число токенов в ответе.
- **repetition_penalty** — Наказывает за повторение одних и тех же токенов.

🔮 Шаг 08:

Миссия 8

Что делают эти команды в процессе сборки модели?

Подсказка: Часть отвечает за упаковку, другая — за оптимизацию.

Ответ:

- **gguf-convert** — Преобразует модель в формат GGUF.
- **quantize.py** — Уменьшает размер модели, снижая точность.
- **merge_tokenizer** — Объединяет токенизаторы при fine-tune.
- **ggml-build** — Компилирует модель для запуска в ggml-библиотеке.

🔮 Шаг 09:

Миссия 9

Ваш коллега Мира сравнивает Phi и Gemma. Сопоставьте модели и их характеристики:

Подсказка: Учитывайте размер, назначение и производительность.

Ответ:

- **Phi-2** — Компактная, ориентирована на reasoning и обучение.
- **Gemma 7B** — Подходит для edge-устройств, оптимизирована по памяти.
- **Phi-3** — Продолжение линии Phi с обучением на mix-quality data.
- **Gemma 2B** — Модель для маломощных серверов с низким latency.

🔮 Шаг 10:

Миссия 10

При интеграции модели на C++ вы видите странные названия функций. Сопоставьте им их назначение:

Подсказка: Подумайте, что может означать "decode", "forward", "init" и т.д.

Ответ:

- **model_init()** — Загружает модель и готовит память.
- **model_forward()** — Выполняет прямой проход через модель.
- **token_decode()** — Преобразует выход модели в читаемый текст.
- **load_weights()** — Загружает веса из файла.

🔮 Шаг 11:

Миссия 11

Вы работаете с LLM и обнаружили странности в генерации. Сопоставьте возможную причину с симптомом:

Подсказка: Некоторые баги в генерации — результат ошибок в настройках или данных.

Ответ:

- **Модель повторяет одно и то же слово** — Слишком низкая temperature или высокая repetition_penalty.
- **Модель выдаёт бессвязный текст** — Слишком высокая temperature.
- **Ответ модели резко обрывается** — Слишком маленькое значение max_tokens.
- **Модель игнорирует ввод пользователя** — Проблемы с prompt или неправильный токенизатор.

🔮 Шаг 12:

Миссия 12

Расположите шаги добавления новой модели Phi в Ollama в правильном порядке:

Подсказка: Начните с загрузки модели и закончите тестированием вывода.

Ответ:

- Скачать модель и подготовить веса
- Конвертировать в формат GGUF
- Интегрировать в Ollama с нужным config
- Запустить тест и проверить генерацию

Шаг 13:

Миссия 13

Расположите шаги подготовки кастомной модели под Ollama:

Подсказка: Учитывайте, что сначала идёт обучение, потом оптимизация.

Ответ:

- Обучение модели
- Квантование для уменьшения размера
- Конвертация в GGUF
- Интеграция в Ollama

Шаг 14:

Миссия 14

Алекс добавляет модель Gemma. Помогите ему: расставьте шаги диагностики при ошибке запуска:

Подсказка: Сначала логика, потом логи, потом исправления.

Ответ:

- Проверить логи запуска Ollama
- Сравнить структуру модели с документацией
- Проверить зависимости и версию Python
- Запустить модель с отладкой

Урок 4 - Отладка и внесение правок в исходники

Шаг 01:

Миссия 1

Вы замечаете, что в коде ядра Ollama есть повторяющийся фрагмент чтения конфигурации в разных модулях. Что вы сделаете первым?

Подсказка: Дублирование — признак, что код можно вынести.

Ответ:

- **Выделить общий метод и заменить дублирующийся код вызовом этого метода**
- Оставить дублирование ради простоты
- Копировать и вставить код из одного места в другое

Шаг 02:

Миссия 2

✂ При отладке ядра вы видите некорректное значение переменной в модуле парсинга. Что наиболее логично сделать?

Подсказка: Используйте минимальное вмешательство для диагностики.

Ответ:

- **Добавить временный лог (print/debug) возле вычисления этой переменной**
- Удалить весь код и написать заново
- Игнорировать ошибку, возможно, она не влияет

Шаг 03:

Миссия 3

Вы обнаружили, что новый тест на модуль генерации текста нестабилен и иногда падает из-за времени ожидания. Как вы действуете?

Подсказка: Подумайте о том, каким способом можно стабилизировать тест.

Ответ:

- **Увеличить таймаут или использовать мок объектов внешних зависимостей**
- Удалить тест полностью
- Снизить нагрузку сервера в продакшене
- Закомментировать часть функционала

Шаг 04:

Миссия 4

 Вы готовитесь внести правку в исходники ядра. Что вы сделаете перед началом работы?

Подсказка: Защита от регрессий — залог устойчивости.

Ответ:

- **Запустить все существующие тесты и убедиться, что они проходят**
- **Никаких проверок** — сразу правлю код
- Удалить старые тесты, они мешают

Шаг 05:

Миссия 5

 При выполнении кода ядра вы наталкиваетесь на исключение NullPointerException в глубоком методе. Что — лучшее первое действие?

Подсказка: Выясните, какой именно элемент null.

Ответ:

- **Добавить проверку null и лог значения перед использованием объекта**
- Просто заглушить исключение
- Игнорировать, надеясь, что ошибка исчезнет
- Удалить весь вызов метода

Шаг 06:

Миссия 6

 Сопоставьте термины и определения, применимые при работе с исходниками ядра Ollama:

Термины:

Ответ:

- Юнит-тест
- Интеграционный тест
- Мок

- Регрессионный тест

Определения:

1. Тест, проверяющий отдельный модуль или функцию изолированно
2. Тест, проверяющий взаимодействие нескольких компонентов системы
3. Заглушка или имитация внешнего компонента
4. Тест, который гарантирует, что баги из прошлых версий не возвращаются

Подсказка: Подумайте, где и как используются моки и зачем нужны регрессионные проверки.

- **Юнит-тест** — Тест, проверяющий отдельный модуль или функцию изолированно
- **Интеграционный тест** — Тест, проверяющий взаимодействие нескольких компонентов системы
- **Мок** — Заглушка или имитация внешнего компонента
- **Регрессионный тест** — Тест, который гарантирует, что баги из прошлых версий не возвращаются

Шаг 07:

Миссия 7

 Сопоставьте этапы отладки и их назначение:

Этапы:

Ответ:

- Инструмент трассировки (trace)
- Постепенное упрощение
- Брейкпоинт
- Логирование

Назначения:

1. Останавливаться на конкретной строке и смотреть состояние
2. Уменьшать сложность задачи, исключая части
3. Выводить значения переменных и шаги управления
4. Собирать подробные шаги исполнения

Подсказка: Подумайте, когда удобнее поставить брейкпоинт, а когда лог лучше.

- **Брейкпоинт** — Останавливаться на конкретной строке и смотреть состояние
- **Постепенное упрощение** — Уменьшать сложность задачи, исключая части
- **Логирование** — Выводить значения переменных и шаги управления
- **Инструмент трассировки (trace)** — Собирать подробные шаги исполнения

Шаг 08:

Миссия 8

 Сопоставьте ошибки с возможной причиной в ядре:

Ошибки:

Ответ:

- Утечка памяти
- Блокировка потока
- Непредвиденное исключение
- Неправильный результат генерации

Причины:

1. Неосвободившийся ресурс или накопление ссылок
2. Взаимное ожидание между потоками
3. Необработанный случай в коде
4. Логическая ошибка в алгоритме

Подсказка: Подумайте, какая причина чаще встречается при неправильной логике.

- **Утечка памяти** — Неосвободившийся ресурс или накопление ссылок
- **Блокировка потока** — Взаимное ожидание между потоками
- **Непредвиденное исключение** — Необработанный случай в коде
- **Неправильный результат генерации** — Логическая ошибка в алгоритме

Шаг 09:

Миссия 9

 Вы готовите правку ядра, связанного с модулем кэширования. Сопоставьте, что нужно проверить после правки:

Пункты:

Ответ:

- Целостность кэша
- Конкурентный доступ
- Потеря данных
- Производительность

Что проверить:

1. Что кэш возвращает корректные значения
2. Что при многопоточном доступе не происходит гонок
3. Что данные не теряются при сбросе кэша
4. Что добавленная логика не сильно замедлила систему

Подсказка: Как кэш используется в реальной нагрузке? Проверьте все аспекты.

- **Целостность кэша** — Что кэш возвращает корректные значения
- **Конкурентный доступ** — Что при многопоточном доступе не происходит гонок
- **Потеря данных** — Что данные не теряются при сбросе кэша
- **Производительность** — Что добавленная логика не сильно замедлила систему

Шаг 10:

Миссия 10

 Расположите этапы внесения правки в исходники ядра Ollama в правильной последовательности:

1. Модификация кода
2. Запуск модульных и интеграционных тестов
3. Анализ задачи и план правки
4. Код-ревью и вливание изменений
5. Запуск системы в тестовой среде

Подсказка: Сначала планирование, затем изменение, затем проверка и интеграция.

Ответ:

- Анализ задачи и план правки
- Модификация кода
- Запуск модульных и интеграционных тестов
- Запуск системы в тестовой среде
- Код-ревью и вливание изменений

 Шаг 11:

Миссия 11

 Расположите шаги диагностики ошибки времени отклика:

1. Измерение узкого места (profiling)
2. Локализация проблемной функции
3. Проверка внешних зависимостей (сеть, БД)
4. Изменение алгоритма или кода
5. Повторное измерение производительности

Подсказка: Сначала измеряйте, затем локализируйте и затем изменяйте.

Ответ:

- Измерение узкого места (profiling)
- Локализация проблемной функции
- Проверка внешних зависимостей (сеть, БД)
- Изменение алгоритма или кода
- Повторное измерение производительности

 Шаг 12:

Миссия 12

 Расположите шаги отката неудачной правки:

1. Остановить текущую сборку (если в CI)
2. Вернуть код к прошлой версии в системе контроля
3. Запустить тесты на откате
4. Убедиться, что система вернулась в рабочее состояние
5. Сообщить команде о причинах отката

Подсказка: Откат должен быть безопасным, проверенным и информативным.

Ответ:

- Остановить текущую сборку (если в CI)
- Вернуть код к прошлой версии в системе контроля
- Запустить тесты на откате
- Убедиться, что система вернулась в рабочее состояние
- Сообщить команде о причинах отката

 Шаг 13:

Миссия 13

 Расположите шаги профилирования производительности ядра:

1. Запустить профилировщик / tracer
2. Собрать данные по вызовам и времени

3. Выделить наиболее затратные участки
4. Оптимизировать и модифицировать код
5. Сравнить до/после измерения

Подсказка: Цикл: измерить → найти узкое место → оптимизировать → проверить.

Ответ:

- Запустить профилировщик / tracer
- Собрать данные по вызовам и времени
- Выделить наиболее затратные участки
- Оптимизировать и модифицировать код
- Сравнить до/после измерения

Шаг 14:

Миссия 14

 Расположите шаги работы с pull-request для внесения правки ядра:

1. Создать PR с описанием изменений
2. Дождаться код-ревью коллег
3. Внести правки по комментариям
4. Пробежать автоматические тесты в CI
5. Слить PR в основную ветку

Подсказка: Код лёгкий к принятию — просто следуй стандартам вашей команды.

Ответ:

- Создать PR с описанием изменений
- Дождаться код-ревью коллег
- Внести правки по комментариям
- Пробежать автоматические тесты в CI
- Слить PR в основную ветку

Шаг 15:

Миссия 15

 Расположите шаги полного цикла отладки до релиза исправления ядра Ollama:

1. Локальная отладка и тестирование
2. Код-ревью и правки
3. Интеграционные тесты и проверка в staging
4. Мониторинг после деплоя
5. Релиз изменений в продакшн

Подсказка: Помните про этапы перед выпуском и после.

Ответ:

- Локальная отладка и тестирование
- Код-ревью и правки
- Интеграционные тесты и проверка в staging
- Мониторинг после деплоя
- Релиз изменений в продакшн

Шаг 16:

Миссия 16

⚡ Расположите шаги анализа регрессионного поведения ядра после правки:

1. Сравнить результаты новых тестов с эталонными
2. Найти модуль, где поведение изменилось
3. Использовать git bisect или аналогичный инструмент
4. Проанализировать изменения в промежуточных коммитах
5. Восстановить рабочий код или внести правку

Подсказка: Git bisect — ваш друг в поиске регрессий.

Ответ:

- Сравнить результаты новых тестов с эталонными
- Найти модуль, где поведение изменилось
- Использовать git bisect или аналогичный инструмент
- Проанализировать изменения в промежуточных коммитах
- Восстановить рабочий код или внести правку

Шаг 17:

Миссия 17

🔍 Последняя миссия: расположите итоговый чек-лист перед выпуском ядра Ollama:

1. Все тесты проходят без ошибок
2. Проведён код-ревью и учтены замечания
3. Отсутствуют предупреждения или ошибки компиляции
4. Производительность на уровне целевого порога
5. Документация обновлена (если интерфейс изменился)

Подсказка: Даже мелкие недочёты могут привести к сбоям.

Ответ:

- Все тесты проходят без ошибок
- Проведён код-ревью и учтены замечания
- Отсутствуют предупреждения или ошибки компиляции
- Производительность на уровне целевого порога
- Документация обновлена (если интерфейс изменился)

Урок 5 - 🎯 Квест расширь Ollama новым параметром и протестируй

Шаг 01:

Миссия 1

Вы хотите расширить Ollama, добавив новый параметр “verbosity_level”. Что вы сделаете в первую очередь?

Подсказка: Начинайте с анализа кода, а не сразу вписывайте изменения.

Ответ:

- **Найти место, где параметры разбираются и добавить параметр туда**
- Добавить параметр лишь в конфигурационный файл без поддержки
- Сразу писать тесты под новый параметр
- Изменить документацию и забыть про реализацию

🔧 Шаг 02:

Миссия 2

Вы добавили параметр “verbosity_level” как строковый ключ, но при запуске видите “Unknown parameter”. Что теперь?

Подсказка: Ошибка говорит, что парсинг параметров не видит ваш ключ.

Ответ:

- **Внести “verbosity_level” в список допустимых параметров в парсере**
- Переименовать параметр в что-то другое без переосмысления
- Откатить весь код до предыдущей версии
- **Игнорировать ошибку** — возможно она исчезнет

🔧 Шаг 03:

Миссия 3

Вы обновили парсер, и теперь параметр принимается. Что вы сделаете дальше?

Подсказка: После парсинга логичен следующий шаг — передача внутрь ядра.

Ответ:

- **Прописать передачу verbosity_level в ядро Ollama**
- Сразу обновить документацию
- Добавить визуальный интерфейс для этого параметра
- Запустить весь проект с другим параметром

🔧 Шаг 04:

Миссия 4

Вы передали параметр в ядро, но поведение не меняется. Почему?

Подсказка: Возможно, ядро игнорирует новый аргумент или использует дефолт.

Ответ:

- **Ядро не читает verbosity_level и использует жестко заданный уровень**
- Параметр передан неправильно как целое число
- Документация описывает другой параметр
- Плагин, который используется, блокирует изменение

🔧 Шаг 05:

Миссия 5

Сопоставьте понятия, связанные с тестированием параметров, с их описаниями:

Подсказка: Вспомните, что такое unit, integration и edge-case.

Понятия:

Ответ:

- Юнит-тест
- Интеграционный тест
- Пределный случай (edge case)
- Регрессионный тест

Определения:

1. Тест, проверяющий поведение модуля в изолированном окружении

2. Тест, проверяющий взаимодействие между компонентами
 3. Ситуация, когда параметр на границе допустимых значений
 4. Тест, гарантирующий, что новые изменения не сломали старую функциональность
- **Юнит-тест** — Тест, проверяющий поведение модуля в изолированном окружении
 - **Интеграционный тест** — Тест, проверяющий взаимодействие между компонентами
 - **Пределный случай (edge case)** — Ситуация, когда параметр на границе допустимых значений
 - **Регрессионный тест** — Тест, гарантирующий, что новые изменения не сломали старую функциональность

🔮 Шаг 06:

Миссия 6

Сопоставьте уровни verbosity с их предполагаемым поведением:

Подсказка: Значения обычно идут от мало к много деталей.

Уровни:

Ответ:

- silent
- minimal
- normal
- debug

Поведение:

1. Выводить только ошибки
 2. Выводить ошибки + основную информацию
 3. Обычный режим с логами средней важности
 4. Выводить все внутренние шаги и детали
- **silent** — Выводить только ошибки
 - **minimal** — Выводить ошибки + основную информацию
 - **normal** — Обычный режим с логами средней важности
 - **debug** — Выводить все внутренние шаги и детали

🔮 Шаг 07:

Миссия 7

Свяжите части процесса отладки параметра verbosity с их ролями:

Подсказка: Пройдите путь от ошибки до исправления.

Шаги:

Ответ:

- Сбор логов
- Анализ логов
- Изменение кода
- Перезапуск с параметром

Роли:

1. Позволяет увидеть, как система работает внутри
2. Выявляет узкие места и места без изменений
3. Внедряет исправления на основе анализа

4. Запускает систему с новыми условиями

- **Сбор логов** — Позволяет увидеть, как система работает внутри
- **Анализ логов** — Выявляет узкие места и места без изменений
- **Изменение кода** — Внедряет исправления на основе анализа
- **Перезапуск с параметром** — Запускает систему с новыми условиями

Шаг 08:

Миссия 8

Упорядочьте этапы интеграции параметра `verbosity_level` в систему:

1. Написание интеграционных тестов с разными уровнями
2. Добавление параметра в интерфейс командной строки
3. Реализация поддержки в ядре Ollama
4. Обновление обработчика парсера конфигурации
5. Запуск системы и проверка правильности

Подсказка: От самого внешнего слоя к внутреннему, затем проверить работу.

Ответ:

- Добавление параметра в интерфейс командной строки
- Обновление обработчика парсера конфигурации
- Реализация поддержки в ядре Ollama
- Написание интеграционных тестов с разными уровнями
- Запуск системы и проверка правильности

Шаг 09:

Миссия 9

Упорядочите жизненный цикл запроса, когда используется `verbosity_level`:

1. Парсинг параметров
2. Логирование внутри ядра
3. Генерация ответа Ollama
4. Вывод пользователю и логи

Подсказка: Параметр проходит от CLI к ядру, затем влияет на вывод.

Ответ:

- Парсинг параметров
- Логирование внутри ядра
- Генерация ответа Ollama
- Вывод пользователю и логи

Шаг 10:

Миссия 10

Упорядочьте действия инженера при обнаружении рассинхрона между логами и результатами:

1. Вернуться к коду логирования
2. Сравнить лог с ожидаемым поведением
3. Запустить тесты с граничными параметрами `verbosity`
4. Вставить дополнительные точки логирования

5. Исправить ошибочную ветвь кода

Подсказка: Логика: проверить → проинструментировать → найти → исправить.

Ответ:

- Запустить тесты с граничными параметрами `verbosity`
- Сравнить лог с ожидаемым поведением
- Вернуться к коду логирования
- Вставить дополнительные точки логирования
- Исправить ошибочную ветвь кода

Шаг 11:

Миссия 11

Вы хотите, чтобы при `verbosity_level = debug` выводились стек-трейсы исключений. Где это следует реализовать?

Подсказка: Вывод ошибок часто централизован в обработчике исключений.

Ответ:

- **В глобальном обработчике исключений ядра**
- В интерфейсе командной строки, после генерации ответа
- В парсере параметров
- В документации и в примере использования

Шаг 12:

Миссия 12

Сопоставьте элементы конфигурации с их функциями по параметру `verbosity`:

Подсказка: Конфигурация задаёт допустимые значения и поведение по умолчанию.

Элементы:

Ответ:

- `default`
- `choices`
- `help description`
- `type validator`

Функции:

1. Описание для пользователя
 2. Ограничение набора допустимых уровней `verbosity`
 3. Установка уровня, если пользователь не указал
 4. Проверка формата значения `verbosity`
- **default** — Установка уровня, если пользователь не указал
 - **choices** — Ограничение набора допустимых уровней `verbosity`
 - **help description** — Описание для пользователя
 - **type validator** — Проверка формата значения `verbosity`

Шаг 13:

Миссия 13

Сопоставьте сценарии с их ожидаемым поведением при `verbosity_level = minimal`:

Подсказка: `minimal` — умеренное количество логов.

Сценарии:

Ответ:

- Успешная генерация результата
- Ошибка API сервиса
- Промежуточные шаги обработки
- Отладочные детали алгоритма

Ожидания:

1. Показать основной результат
2. Показать текст ошибки
3. Показать только шаги высокого уровня
4. Не показывать подробности

- **Успешная генерация результата** — Показать основной результат
- **Ошибка API сервиса** — Показать текст ошибки
- **Промежуточные шаги обработки** — Показать только шаги высокого уровня
- **Отладочные детали алгоритма** — Не показывать подробности

Шаг 14:

Миссия 14

Упорядочьте ветви обработки `verbosity_level` внутри ядра:

1. **Если debug** — логировать все детали
2. **Если silent** — логировать только ошибки
3. **Если minimal** — логировать основные шаги
4. **Если normal** — логировать стандартные сообщения

Подсказка: Сортировка по «от наиболее подробного к наименее подробному» — не совсем: нужно выстроить логические проверки.

Ответ:

- **Если debug** — логировать все детали
- **Если normal** — логировать стандартные сообщения
- **Если minimal** — логировать основные шаги
- **Если silent** — логировать только ошибки

Шаг 15:

Миссия 15

Упорядочьте этапы перед развёртыванием нового параметра `verbosity_level`:

1. Обновление документации
2. Мёрж в основную ветку
3. Ревью кода и проверка тестов
4. Деплой в staging и smoke-тест
5. Релиз в продакшн и мониторинг логов

Подсказка: Без ревью и тестов — рискованно. Без мониторинга — слепо.

Ответ:

- Обновление документации
- Ревью кода и проверка тестов

- Мёрж в основную ветку
- Деплой в staging и smoke-тест
- Релиз в продакшн и мониторинг логов

🔧 Шаг 16:

Миссия 16

Упорядочьте шаги от идеи параметра verbosity до завершённого релиза:

1. Обсуждение идеи с командой
2. Проектирование API параметра
3. Реализация и тестирование
4. Документация и примеры использования
5. Релиз и мониторинг

Подсказка: Сначала обсуждаем, потом проектируем, потом реализуем, тестируем, документируем и выпускаем.

Ответ:

- Обсуждение идеи с командой
- Проектирование API параметра
- Реализация и тестирование
- Документация и примеры использования
- Релиз и мониторинг

📁 Модуль 10 - 🧩 Миссия 10. Автоматизация и CI Ollama

📖 Урок 1 - Dockerизация Ollama и моделей

🔧 Шаг 01:

Миссия 1

Вы приступаете к Dockerизации Ollama. С чего начнёте?

Подсказка: Сначала подумайте об окружении, где всё будет запускаться.

Ответ:

- **Составить Dockerfile для базового образа с зависимостями Ollama**
- Сразу запускать контейнеры в продакшн среде
- Игнорировать зависимости, предполагая, что они уже есть
- Сначала писать CI-скрипт перед Dockerfile

🔧 Шаг 02:

Миссия 2

Контейнер запускается, но модель не грузится (ошибка доступа к файлам). Что делать?

Подсказка: Проверьте, как вы монтируете модель или копируете её внутрь.

Ответ:

- **Убедиться, что путь модели правильно смонтирован или скопирован в контейнер**
- Удалить проверку доступа к файлам
- Запускать модель в хостовой системе, а Ollama в контейнере
- Откатиться к старой версии без контейнера

🔮 Шаг 03:

Миссия 3

Вы хотите уменьшить размер Docker-образа. Какое из решений наиболее разумно?

Подсказка: Минимизировать слои и убрать ненужное.

Ответ:

- **Использовать многоступенчатую сборку (multi-stage build)**
- Добавить все зависимости, даже редко используемые
- Оставить старый большой образ
- Упаковать весь код как один слой без разбивки

🔮 Шаг 04:

Миссия 4

Вы хотите автоматизировать сборку и тестирование Docker-образов через CI. Какой первый шаг?

Подсказка: CI должен знать, как собрать образ и проверить его.

Ответ:

- **Написать CI-pipeline с шагами build, test и push**
- Запускать CI вручную после каждого коммита
- Ограничиться лишь синтаксической проверкой Dockerfile
- Собирать образ вручную и выкатывать через CI

🔮 Шаг 05:

Миссия 5

Сопоставьте понятия из DevOps-конвейера с их описаниями:

Подсказка: CI/CD, pipeline и артефакт — ключевые термины.

Понятия:

Ответ:

- CI
- Pipeline
- Артефакт
- CD

Описания:

1. Последовательность шагов сборки, тестирования и деплоя
 2. Непрерывная интеграция: автоматическая сборка и тестирование при коммите
 3. Цель: автоматический деплой после успешной сборки
 4. Результат сборки, который можно развернуть или доставить
- **CI** — Непрерывная интеграция: автоматическая сборка и тестирование при коммите
 - **Pipeline** — Последовательность шагов сборки, тестирования и деплоя
 - **Артефакт** — Результат сборки, который можно развернуть или доставить
 - **CD** — Цель: автоматический деплой после успешной сборки

🔮 Шаг 06:

Миссия 6

Сопоставьте этапы работы CI при Docker-образах с задачами:

Подсказка: Build, Test, Push, Deploy — основные этапы.

Этапы:

Ответ:

- Build
- Test
- Push
- Deploy

Задачи:

1. Собрать контейнер из Dockerfile
 2. Проверить функциональность внутри контейнера
 3. Отправить образ в registry
 4. Развернуть контейнер в целевой среде
- **Build** — Собрать контейнер из Dockerfile
 - **Test** — Проверить функциональность внутри контейнера
 - **Push** — Отправить образ в registry
 - **Deploy** — Развернуть контейнер в целевой среде

Шаг 07:

Миссия 7

Сопоставьте артефакты Docker-сборки с их применением:

Подсказка: Образ, слой и контейнер — разные виды артефактов.

Артефакты:

Ответ:

- Образ (image)
- Слой (layer)
- Контейнер (container)
- Тэг образа (tag)

Применения:

1. Временная запущенная копия образа
 2. Версия образа в registry
 3. Изменение, добавленное в образ без пересборки всего образа
 4. Идентификатор версии образа
- **Образ (image)** — Версия образа в registry
 - **Слой (layer)** — Изменение, добавленное в образ без пересборки всего образа
 - **Контейнер (container)** — Временная запущенная копия образа
 - **Тэг образа (tag)** — Идентификатор версии образа

Шаг 08:

Миссия 8

Упорядочьте шаги CI для Docker-образа Ollama:

1. Собрать образ
2. Прогнать тесты внутри образа

3. Загрузить образ в registry
4. Развернуть в тестовой среде
5. Проверить работоспособность сервиса

Подсказка: CI идёт от сборки к проверке, затем публикации и проверке в среде.

Ответ:

- Собрать образ
- Прогнать тесты внутри образа
- Загрузить образ в registry
- Развернуть в тестовой среде
- Проверить работоспособность сервиса

Шаг 09:

Миссия 9

Упорядочьте шаги Docker-развертывания модели внутри Ollama:

1. Копирование модели в контейнер
2. Настройка переменных среды
3. Запуск сервиса Ollama с моделью
4. Проверка API запроса к модели

Подсказка: Сначала готовим контейнер, потом запускаем, затем проверяем.

Ответ:

- Копирование модели в контейнер
- Настройка переменных среды
- Запуск сервиса Ollama с моделью
- Проверка API запроса к модели

Шаг 10:

Миссия 10

Упорядочьте этапы создания multi-stage Docker образа:

1. Собрать минимальное окружение для runtime
2. Скомпилировать/собрать весь проект в build stage
3. Скопировать только нужные артефакты из build stage в runtime stage
4. Оптимизировать инструкции RUN, уменьшить слои

Подсказка: Multi-stage начинается с build, затем отделяют runtime.

Ответ:

- Скомпилировать/собрать весь проект в build stage
- Скопировать только нужные артефакты из build stage в runtime stage
- Собрать минимальное окружение для runtime
- Оптимизировать инструкции RUN, уменьшить слои

Шаг 11:

Миссия 11

Упорядочьте этапы проверки безопасности Docker-образа Ollama:

1. Сканирование образа на уязвимости

2. Проверка разрешений файлов внутри контейнера
3. Тестирование запуска под непривилегированным пользователем
4. Обновление базового образа до последних патчей

Подсказка: Безопасность начинается с базы, потом проверяем контейнер.

Ответ:

- Обновление базового образа до последних патчей
- Сканирование образа на уязвимости
- Проверка разрешений файлов внутри контейнера
- Тестирование запуска под непривилегированным пользователем

Шаг 12:

Миссия 12

Вы хотите, чтобы CI пушил только если сборка Docker-образа прошла успешно и тесты зелёные. Какую проверку добавить?

Подсказка: Условие push должно зависеть от статуса предыдущих шагов.

Ответ:

- **Условие в pipeline: push только при успешном завершении build и test**
- Всегда пушить образ, даже если тесты упали
- Отбрасывать тестовый шаг вовсе
- Пушить образ вручную несмотря на CI

Шаг 13:

Миссия 13

Сопоставьте параметры Docker-контейнера с их назначениями:

Подсказка: Учтите volume, ENV, CMD, EXPOSE.

Параметры:

Ответ:

- VOLUME
- ENV
- CMD
- EXPOSE

Назначения:

1. Устанавливает переменные окружения внутри контейнера
 2. Задаёт команду по умолчанию при старте контейнера
 3. Объявляет порты, которые контейнер будет слушать
 4. Указывает каталоги на хосте, монтируемые в контейнер
- **VOLUME** — Указывает каталоги на хосте, монтируемые в контейнер
 - **ENV** — Устанавливает переменные окружения внутри контейнера
 - **CMD** — Задаёт команду по умолчанию при старте контейнера
 - **EXPOSE** — Объявляет порты, которые контейнер будет слушать

Шаг 14:

Миссия 14

Сопоставьте режимы запуска контейнера с их особенностями:

Подсказка: detached, interactive, host, bridge — разные режимы запуска.

Режимы:

Ответ:

- detached
- interactive
- host network
- bridge network

Особенности:

1. Контейнер работает в фоне
 2. Интерактивная сессия с входом в контейнер
 3. Контейнер использует сеть хоста напрямую
 4. Изолированная сеть по умолчанию
- **detached** — Контейнер работает в фоне
 - **interactive** — Интерактивная сессия с входом в контейнер
 - **host network** — Контейнер использует сеть хоста напрямую
 - **bridge network** — Изолированная сеть по умолчанию

Шаг 15:

Миссия 15

Упорядочьте шаги отката (rollback) после неудачного релиза контейнера:

1. Остановить текущий контейнер
2. Вернуть предыдущий стабильный образ
3. Запустить контейнер из стабильного образа
4. Проверить, что система работает корректно
5. Сообщить команде об откате

Подсказка: Сначала отмена, затем восстановление и проверка.

Ответ:

- Остановить текущий контейнер
- Вернуть предыдущий стабильный образ
- Запустить контейнер из стабильного образа
- Проверить, что система работает корректно
- Сообщить команде об откате

Шаг 16:

Миссия 16

Упорядочьте полную схему от локального кода до работающего Docker-сервиса Ollama:

1. Разработка кода локально
2. Создание Dockerfile и тестирование локально
3. CI: сборка, тесты, публикация образа
4. Развёртывание контейнера на staging/production
5. Мониторинг и логирование контейнера в продакшн

Подсказка: Это путь от локального кода к работающему Docker-сервису.

Ответ:

- Разработка кода локально
- Создание Dockerfile и тестирование локально
- CI: сборка, тесты, публикация образа
- Развёртывание контейнера на staging/production
- Мониторинг и логирование контейнера в продакшн

Урок 2 - ⚙️ Настройка CI/CD для LLM

Шаг 01:

Миссия 1

Ваш проект на Ollama только стартует: что первым делом надо настроить в CI/CD для LLM?
Подсказка: Подумайте, какую часть автоматизации хотелось бы видеть первой.

Ответ:

- **Автоматический запуск тестов модели**
- Деплой статического сайта
- Сборка фронтенда без LLM

Шаг 02:

Миссия 2

 Сопоставьте шаг CI/CD с его назначением в контексте LLM-проекта с Ollama:

Подсказка: Помните, что LLM-модели требуют проверки качества вывода и мониторинга.

Ответ:

- **Запуск модульных тестов модели** — Проверка работоспособности критичных функций
- **Сравнение выходов модели с эталоном** — Контроль качества генераций
- **Автоматический деплой модели** — Развёртывание на прод среде
- **Сбор метрик (производительность, латентность)** — Мониторинг SLA

Шаг 03:

Миссия 3

 Инженер Анна хочет, чтобы при каждом пуше в ветку main запускался CI: какой триггер выбрать?

Подсказка: Подумайте, какой тип события наиболее логичен.

Ответ:

- **on push to main branch**
- on pull request to feature branch
- on tag creation only

Шаг 04:

Миссия 4

 Расположите этапы CI/CD ланца для модели Ollama в правильном порядке:

1. Запуск тестов генераций
2. Валидация данных / контроль качества
3. Деплой модели

4. Сбор метрик и логов
5. Триггер CI (push / PR)

Подсказка: Начните с события, затем тесты, затем продакшн и мониторинг.

Ответ:

- Триггер CI (push / PR)
- Валидация данных / контроль качества
- Запуск тестов генераций
- Деплой модели
- Сбор метрик и логов

🔮 Шаг 05:

Миссия 5

🔮 В CI модель выдаёт неожиданно плохие ответы после изменения кода: что делать первым?

Подсказка: На каком этапе лучше поймать регрессию?

Ответ:

- **Запустить сравнение с эталонным выводом**
- Немедленно откатить деплой
- Добавить больше логов в CI

🔮 Шаг 06:

Миссия 6

🔧 Сопоставьте типы тестов CI и их задач в LLM-пайплайне:

Подсказка: Подумайте, что именно проверяет каждый тест.

Ответ:

- **Smoke-тест генерации** — Быстрая проверка, что модель отвечает синтаксически
- **Регрессионный тест** — Сравнение вывода с эталонным примером
- **Производительный тест** — Измерение латентности и throughput
- **Интеграционный тест** — Проверка взаимодействия с внешними данными/сервисами

🔮 Шаг 07:

Миссия 7

🕒 Какую стратегию развертывания модели выбрать, чтобы минимизировать простои при обновлениях?

Подсказка: Популярны “blue/green” и “canary”.

Ответ:

- **Canary deployment**
- All-at-once
- Manual переключение без автоматизации

🔮 Шаг 08:

Миссия 8

📅 Упорядочите фазы canary-деплойа для LLM:

1. Деплой новой версии части трафика
2. Мониторинг показателей (latency, качество вывода)

3. Расширение доли трафика
4. Откаты при аномалиях
5. Полный переход на новую версию

Подсказка: Сначала небольшой релиз, потом проверка, и только потом рост.

Ответ:

- Деплой новой версии части трафика
- Мониторинг показателей (latency, качество вывода)
- Расширение доли трафика
- Откаты при аномалиях
- Полный переход на новую версию

Шаг 09:

Миссия 9

 Вы хотите, чтобы CI автоматически билдил контейнер с моделью: какой шаг вам нужен?

Подсказка: Это стандарт в CI/CD для приложений.

Ответ:

- **Создание Docker-образа модели**
- Запуск UI-тестов
- Сбор статистики фронтенда

Шаг 10:

Миссия 10

 Сопоставьте CI-артефакты и их использование в LLM-проектах:

Подсказка: Под артефактами понимаются файлы, которые сохраняют после сборки.

Ответ:

- **Контейнерный образ** — Используется для деплоя новой версии модели
- **Тар файл с весами** — Артефакт, который загружается в модель-сервер
- **Отчёты тестов** — Артефакт для анализа качества после CI
- **Логи и метрики** — Артефакт для мониторинга и отладки

Шаг 11:

Миссия 11

 Почему стоит кешировать зависимости и веса при CI для LLM?

Подсказка: Подумайте о времени сборки и загрузках.

Ответ:

- **Снижает время CI-прогона**
- Увеличивает точность модели
- Изменяет логику генерации

Шаг 12:

Миссия 12

Упорядочите шаги отката модели в CI/CD:

1. Обнаружение аномалии (плохое качество / метрики)

2. Блокировка трафика на новую версию
3. Возврат на предыдущую стабильную версию
4. Мониторинг после отката
5. Уведомление команды

Подсказка: Как только что-то пойдёт не так — вернуть трафик и проверить состояние.

Ответ:

- Обнаружение аномалии (плохое качество / метрики)
- Блокировка трафика на новую версию
- Возврат на предыдущую стабильную версию
- Мониторинг после отката
- Уведомление команды

Шаг 13:

Миссия 13

 При хранении секретов (ключ API, токен доступа) в CI, что лучше всего использовать?

Подсказка: Не стоит хранить секреты открыто в репозитории.

Ответ:

- **CI secrets / переменные окружения**
- Жёстко закодированные строки
- Файл .secret в ветке main

Шаг 14:

Миссия 14

 Сопоставьте метрику с её назначением при CI/CD LLM:

Подсказка: Латентность, качество, расход ресурсов — ключевые показатели.

Ответ:

- **Latency (задержка)** — Время ответа модели
- **Throughput** — Число запросов в секунду
- **Процент ошибок** — Доля генераций с ошибкой
- **Отклонение качества** — Ухудшение качества по сравнению с базой

Шаг 15:

Миссия 15

Какой контрольный тест стоит запускать перед деплоем новой модели?

Подсказка: Такой тест проверяет, что ничего серьёзно не сломалось.

Ответ:

- **Smoke-генерация на несколько примеров**
- Полный референсный тест на все сценарии
- Запуск только нагрузочных тестов

Шаг 16:

Миссия 16

🔗 Упорядочите действия для масштабирования CI/CD пайплайна с несколькими моделями:

1. Параллельный запуск тестов для каждой модели
2. Разделение этапов: сборка / тест / деплой
3. Кэширование общих зависимостей
4. Оркестрация через матрицу CI
5. Мониторинг по-моделям

Подсказка: Сначала структурируйте, затем параллелите, затем наблюдайте.

Ответ:

- Разделение этапов: сборка / тест / деплой
- Кэширование общих зависимостей
- Оркестрация через матрицу CI
- Параллельный запуск тестов для каждой модели
- Мониторинг по-моделям

💡 **Шаг 17:**

Миссия 17

✨ Вы дошли до финала: какую практика обеспечит устойчивость CI/CD для LLM?

Подсказка: Подумайте о надёжности, откатах и предупреждениях.

Ответ:

- **Автоматические откаты + алерты + тесты качества**
- Ручной старт деплоя без мониторинга
- Откаты без уведомлений и тестов

📖 **Урок 3 - Скрипты автоматических тестов и метрик**

💡 **Шаг 01:**

Миссия 1

Инженер Лена пишет первый тест-скрипт для Ollama: что должно быть в этом скрипте?

Подсказка: Нужно убедиться, что модель отвечает корректно.

Ответ:

- **Отправка запроса к модели и проверка вывода**
- Запуск UI-тестов
- Только сбор метрик без проверки вывода

💡 **Шаг 02:**

Миссия 2

📊 После первого успеха Лена хочет собирать метрики: какая метрика — базовая для старта?

Подсказка: Подумайте об ответе модели и времени ответа.

Ответ:

- **Задержка (latency) ответа**
- Стоимость запуска GPU
- Количество строк кода тестов

🔮 Шаг 03:

Миссия 3

✦ Лена хочет, чтобы тесты запускались автоматически по коммиту: что выбрать?

Подсказка: Это стандарт — триггер CI.

Ответ:

- **Триггер по push в репозиторий**
- Триггер вручную через UI
- Триггер только при выпуске версии

🔮 Шаг 04:

Миссия 4

✦ Лена хочет, чтобы скрипт тестов фиксировал не только успех/неудачу, но и время — зачем?

Подсказка: Метрики помогают видеть деградацию.

Ответ:

- **Чтобы замечать рост задержки со временем**
- Чтобы увеличить объём логов
- Чтобы тест считался медленным

🔮 Шаг 05:

Миссия 5

🔍 В сценарии теста Лена случайно получает нестабильный вывод модели: что делать?

Подсказка: Нужно уменьшить флуктуации и выявить стабильность.

Ответ:

- **Усреднить результаты по нескольким запускам**
- Выбрать только первый ответ
- Игнорировать нестабильность

🔮 Шаг 06:

Миссия 6

🔗 Лена добавляет несколько типов тестов и метрик: сопоставьте тест/метрику и её роль.

Подсказка: Подумайте, какой тест измеряет что.

Ответ:

- **Smoke-тест генерации** — Проверка, что модель выдаёт синтаксически валидный ответ
- **Регрессионный тест вывода** — Сравнение новых ответов с эталонными
- **Тест производительности** — Измерение задержки и пропускной способности
- **Метрика качества** — Оценка содержательной релевантности вывода

🔮 Шаг 07:

Миссия 7

📊 Инженер Илья интегрирует метрики: сопоставьте инструмент/метрику и её применение.

Подсказка: Инструменты часто хранят временные ряды.

Ответ:

- **Prometheus** — Сохранение метрик задержки / ошибок во времени

- **Grafana** — Визуализация временных рядов метрик
- **JSON отчёты** — Артефакт CI для детального анализа
- **Скрипты Python** — Генерация пользовательских метрик из вывода

🔮 Шаг 08:

Миссия 8

Илья хочет, чтобы тесты и метрики связывались: сопоставьте выход CI-теста и тип метрики.
Подсказка: Тест может выдавать несколько числовых показателей.

Ответ:

- **Время ответа (в секундах)** — Latency
- **Количество ошибок** — Error rate
- **Отношение успешных ответов к общему числу** — Accuracy (успешность)
- **Среднее изменение качества по сравнению с базой** — Drift / деградация качества

🔮 Шаг 09:

Миссия 9

🔮 Илья решает, что если метрика latency превысит порог — тест должен упасть: как это реализовать?

Подсказка: В CI-скрипте можно добавить проверки.

Ответ:

- **Добавить assert в скрипт: если latency > threshold, test fails**
- Игнорировать превышения
- Только логировать превышения без падения

🔮 Шаг 10:

Миссия 10

⚠ Для теста качества Илья хочет, чтобы если качество сильно ухудшится — CI отменял деплой: что нужно добавить?

Подсказка: Такое поведение — часть guardrails (предохранители).

Ответ:

- **Порог качества + assert, откат при нарушении**
- Всегда деплоить, вне зависимости от качества
- Только предупреждение без отмены

🔮 Шаг 11:

Миссия 11

📁 Илья расширяет скрипты: сопоставьте команду / библиотеку и её роль в тестах/метриках.

Подсказка: Например, requests, numpy, pytest могут пригодиться.

Ответ:

- **requests** — Отправка HTTP-запросов к API модели
- **pytest** — Организация и запуск тестов
- **numpy** — Статистическая обработка результатов
- **jsonschema** — Валидация структуры вывода модели

🔗 Шаг 12:

Миссия 12

🔍 Лена и Илья объединяют усилия: сопоставьте метрику и действие при её нарушении.

Подсказка: Нарушение порога может вызвать разные реакции.

Ответ:

- **Latency выше порога** — Тест падает / CI фейлится
- **Качество ниже допустимого** — Откат модели
- **Увеличение error rate** — Оповещение команды
- **Drift качества** — Запуск углублённого анализа

🔗 Шаг 13:

Миссия 13

🔗 Расположите шаги автоматического теста + сбор метрик:

1. Отправка запроса к модели
2. Проверка содержательной точности (quality test)
3. Измерение задержки ответа
4. Сохранение результатов как артефакт
5. Сравнение с порогами / assert

Подсказка: Сначала запрос, затем метрики, затем проверка и запись.

Ответ:

- Отправка запроса к модели
- Измерение задержки ответа
- Проверка содержательной точности (quality test)
- Сравнение с порогами / assert
- Сохранение результатов как артефакт

🔗 Шаг 14:

Миссия 14

✂ Расположите шаги, если тест зафиксировал проблему (например, деградацию качества):

1. CI отмечает фейл
2. Откат к стабильной версии модели
3. Оповещение разработчиков
4. Анализ логов и метрик
5. Ручная пересборка и исправление

Подсказка: Проблема — фейл, откат, анализ, исправление.

Ответ:

- CI отмечает фейл
- Откат к стабильной версии модели
- Оповещение разработчиков
- Анализ логов и метрик
- Ручная пересборка и исправление

Шаг 15:

Миссия 15

 При масштабировании тестов на разные модели распорядите действия:

1. Выделение шаблона теста
2. Параллельный запуск тестов на всех моделях
3. Сбор и агрегация метрик
4. Сравнение каждой модели с базовой метрикой
5. Вывод итогового отчёта

Подсказка: Сначала шаблон, потом параллель, потом агрегация и анализ.

Ответ:

- Выделение шаблона теста
- Параллельный запуск тестов на всех моделях
- Сбор и агрегация метрик
- Сравнение каждой модели с базовой метрикой
- Вывод итогового отчёта

Шаг 16:

Миссия 16

 В команде хотят обеспечить безопасность тестов и метрик. Упорядочите шаги:

1. Не сохранять секреты явно в коде
2. Шифровать или использовать защищённое хранилище секретов
3. Передавать секреты в скрипты как переменные окружения
4. Ограничить доступ к метрикам и логам
5. Ротация секретов через промежуток времени

Подсказка: Сначала безопасное хранение, потом передача и управление.

Ответ:

- Не сохранять секреты явно в коде
- Шифровать или использовать защищённое хранилище секретов
- Передавать секреты в скрипты как переменные окружения
- Ограничить доступ к метрикам и логам
- Ротация секретов через промежуток времени

Шаг 17:

Миссия 17

 Финальный квест: упорядочите полный цикл автоматического теста + метрик + принятие решения в CI:

1. Запустить запрос к модели
2. Измерить задержку, оценить качество
3. Сохранить метрики и логи как артефакт
4. Сравнить метрики с порогами / assert
5. **При нарушениях** — фейл или откат
6. Оповестить команду + записать детали

Подсказка: Это финальный сценарий — просто следуйте логике CI-конвейера.

Ответ:

- Запустить запрос к модели
- Измерить задержку, оценить качество
- Сохранить метрики и логи как артефакт
- Сравнить метрики с порогами / assert
- **При нарушениях** — фейл или откат
- Оповестить команду + записать детали

 Урок 4 - **Обновление модели по cron и метрикам качества**

 Шаг 01:

Миссия 1

👤 Инженер Олег хочет, чтобы модель обновлялась ежедневно по расписанию: какой инструмент подходит?

Подсказка: Подумайте об инструменте, запускающем команды по времени на *nix-системах.

Ответ:

- **cron / crontab**
- webhook по HTTP
- Systemd timer без расписания

 Шаг 02:

Миссия 2

📋 Олег настроил cron-задачу, но она не запускается: что первое проверить?

Подсказка: Часто причина — синтаксис расписания или права.

Ответ:

- **Синтаксис строки cron (поле расписания)**
- Версия ОС
- Содержимое .bashrc

 Шаг 03:

Миссия 3

🔧 Олег хочет перезапускать обновление только если качество новой модели выше порога: как реализовать это?

Подсказка: Нужна проверка метрик перед заменой модели.

Ответ:

- **В скрипте сравнить метрики и выполнить деплой только при достижении порога**
- Сразу заменять без проверки
- Откладывать обновление на ручной запуск

 Шаг 04:

Миссия 4

📁 Какой подход к именованию версий модели поможет откатываться?

Подсказка: Используйте понятную и последовательную номенклатуру.

Ответ:

- **Использовать семантическую версию (v1.0, v1.1 и т.д.)**
- Всегда overwrite без версии
- Называть по дате без порядка

🔮 Шаг 05:

Миссия 5

📊 Чтобы оценить качество модели при обновлениях, какую метрику стоит включить?

Подсказка: Нужно измерять не только скорость, но и смысловую точность.

Ответ:

- **Метрика качества ответов (accuracy / relevance)**
- Размер логов
- Номер версии скрипта

🔮 Шаг 06:

Миссия 6

Сопоставьте часть скрипта/логику и её роль при обновлении моделей по cron:

Подсказка: Какие шаги нужны, чтобы не сломать рабочую модель?

Ответ:

- **Запуск обновляющего скрипта по расписанию** — cron инициирует процесс
- **Загрузка новой модели** — Получение свежих весов
- **Запуск тестов качества** — Оценка новой версии по метрикам
- **Сравнение качества с предыдущей версией** — Решение, стоит ли деплой
- **Откат / отказ от обновления** — Возврат к прошлой версии

🔮 Шаг 07:

Миссия 7

📊 Олег записывает метрики: сопоставьте название метрики и что она показывает:

Подсказка: Некоторые метрики измеряют вероятность ошибки, другие — качество.

Ответ:

- **Latency** — Время ответа модели
- **Error rate** — Доля запросов с ошибкой
- **Quality score** — Семантическая точность ответов
- **Drift measure** — Изменение качества относительно базовой версии

🔮 Шаг 08:

Миссия 8

🔍 При реализации сравнения версий: сопоставьте условие и действие, которое должен выполнить скрипт.

Подсказка: Если условие не выполнено — не обновлять модель.

Ответ:

- **новая_quality ≥ порог** — выполнить деплой новой версии
- **новая_quality < порог** — отклонить обновление
- **latency новой версии значительно выше** — отказаться от обновления

- **error rate новой версии выше базовой** — отклонить деплой

🔔 Шаг 09:

Миссия 9

🕒 Чтобы избежать ошибок из-за одиночной неудачной версии, что можно сделать при cron-обновлениях?

Подсказка: Полезна стратегия постепенного обновления или резервной модели.

Ответ:

- **Обновлять сначала тестовую ветку / сапагу, а потом полную**
- Сразу заменять у всех пользователей
- Выключать обновления вовсе

🔔 Шаг 10:

Миссия 10

📁 Как лучше вести хранение старых версий модели, чтобы можно было откатиться?

Подсказка: Версии должны быть адресуемыми и доступными.

Ответ:

- **Хранить версии в репозитории артефактов с версионными именами**
- Затирать старые версии без сохранения
- Хранить только самую свежую

🔔 Шаг 11:

Миссия 11

🔄 При cron-обновлениях важно реагировать на сбои: сопоставьте событие и реакцию скрипта.

Подсказка: Некоторые сбои приводят к откату, другие — к повторной попытке.

Ответ:

- **Не удалось загрузить новую модель** — Забросить обновление и оставить старую
- **Тестовый запуск завершился с ошибкой** — Откат / отказ от обновления
- **Обновление прошло, но метрика качества ухудшилась** — Автоматический откат
- **Обновление в сапагу прошло успешно** — Расширить развертывание на всех пользователей

🔔 Шаг 12:

Миссия 12

📅 Олег хочет вести журнал обновлений: сопоставьте поле лога и его назначение.

Подсказка: Лог поможет отследить, какая версия, когда и почему обновлена.

Ответ:

- **timestamp** — Время запуска обновления cron
- **old_version** — Версия модели до обновления
- **new_version** — Версия модели после обновления
- **reason** — Причина обновления или отказа (по метрикам)

🔔 Шаг 13:

Миссия 13

☞ Упорядочите логические шаги cron-обновления модели:

1. cron запускает скрипт
2. Скрипт загружает новую модель
3. Запускаются тесты качества и метрики
4. Сравниваются метрики с порогами
5. **Если новые метрики лучше** — деплой, иначе откат
6. Запись результата в лог

Подсказка: Следуйте последовательности: инициатор → загрузка → тесты → сравнение → решение → логирование.

Ответ:

- cron запускает скрипт
- Скрипт загружает новая модель
- Запускаются тесты качества и метрики
- Сравниваются метрики с порогами
- **Если новые метрики лучше** — деплой, иначе откат
- Запись результата в лог

🔔 Шаг 14:

Миссия 14

🔗 Расположите шаги реакции при ухудшении качества новой версии:

1. Сравнение метрик показывает ухудшение
2. Откат на предыдущую версию
3. Запись в лог с причиной отката
4. Оповещение команды разработки
5. Переход к обычному расписанию обновлений

Подсказка: Сначала обнаружить проблему, потом откат, далее логика уведомления.

Ответ:

- Сравнение метрик показывает ухудшение
- Откат на предыдущую версию
- Запись в лог с причиной отката
- Оповещение команды разработки
- Переход к обычному расписанию обновлений

🔔 Шаг 15:

Миссия 15

📦 При масштабировании на несколько моделей: упорядочите общий алгоритм cron-обновлений:

1. Запуск обновлений по шаблону cron
2. Для каждой модели: загрузка и тестирование новой версии
3. Сравнение метрик каждой модели с порогом
4. Деплой/откат для каждой модели независимо
5. Сбор логов и отчётов по всем моделям
6. Оповещение о состоянии обновления всех моделей

Подсказка: Множественные модели — циклический и независимый подход.

Ответ:

- Запуск обновлений по шаблону cron
- Для каждой модели: загрузка и тестирование новой версии
- Сравнение метрик каждой модели с порогом
- Деплой/откат для каждой модели независимо
- Сбор логов и отчётов по всем моделям
- Оповещение о состоянии обновления всех моделей

 Шаг 16:

Миссия 16

 Обновления по cron должны быть безопасными: упорядочите меры безопасности:

1. Не хранить секреты API явно в скрипте
2. Использовать защищённое хранилище / vault для токенов
3. Передавать секреты как переменные окружения в cron-скрипт
4. Ограничивать права на выполнение обновлений
5. Вести аудит логов обновлений

Подсказка: Сначала безопасное хранение, затем безопасный доступ, затем аудит.

Ответ:

- Не хранить секреты API явно в скрипте
- Использовать защищённое хранилище / vault для токенов
- Передавать секреты как переменные окружения в cron-скрипт
- Ограничивать права на выполнение обновлений
- Вести аудит логов обновлений

 Шаг 17:

Миссия 17

 Финальный квест: упорядочите весь цикл cron-обновления + реакции:

1. cron запускает скрипт обновления
2. Загрузка новой версии модели
3. Запуск тестов и сбор метрик
4. Сравнение с пороговыми значениями
5. **При улучшении** — деплой, иначе откат
6. Запись лога, уведомление команды
7. Возвращение к штатному расписанию

Подсказка: Это итоговый сценарий — подумайте об управлении, защитах и уведомлениях.

Ответ:

- cron запускает скрипт обновления
- Загрузка новой версии модели
- Запуск тестов и сбор метрик
- Сравнение с пороговыми значениями
- **При улучшении** — деплой, иначе откат
- Запись лога, уведомление команды
- Возвращение к штатному расписанию

Урок 5 - Квест создай автопайлайн обновления LoRA

Шаг 01:

Миссия 1

Инженер Катя готовит новый LoRA-модуль и хочет запустить автоматическое обновление. Какой первый шаг в CI-процессе?

Подсказка: Сначала проверь, работает ли базовый пайплайн.

Ответ:

- **Запустить тестовый билд базового образа**
- Немедленно деплоить LoRA в прод
- Игнорировать pipeline и делать вручную

Шаг 02:

Миссия 2

 Катя хочет, чтобы при коммите в ветку `lora-update` запускался автоматический билд. Какой триггер CI лучше всего использовать?

Подсказка: Триггер привязан к событию в системе контроля версий.

Ответ:

- **On push to branch `lora-update`**
- On tag creation
- On schedule daily

Шаг 03:

Миссия 3

При каждом билде Катя хочет запускать набор тестов LoRA. Что обязательно включить в тестовый шаг?

Подсказка: Проверь, что LoRA не ломает базовую модель.

Ответ:

- **Сравнение выходов модели с и без LoRA**
- Только синтаксический линтинг
- Запуск GUI-теста
- Создание бэкап копий вручную

Шаг 04:

Миссия 4

 Прежде чем деплоить новый LoRA, Катя хочет убедиться, что он соответствует требованиям по качеству. Что она может включить как проверку качества?

Подсказка: Дополнительно к функциональным тестам — метрики.

Ответ:

- **Проверку производительности и качества вывода**
- Только проверку стилистики кода
- Запуск всех тестов в ручном режиме
- Игнорирование метрик и доверие вручную

🔮 Шаг 05:

Миссия 5

👤 Катя хочет обезопасить секреты (API-ключи, токены) в CI. Как лучше это сделать?

Подсказка: Секреты не должны быть в открытом репозитории.

Ответ:

- **Хранить секреты в защищённых переменных CI**
- Записать их прямо в код
- Отправлять секреты почтой при деплое

🔮 Шаг 06:

Миссия 6

✂ Катя составляет список стадий пайплайна для обновления LoRA. Сопоставьте стадию с её задачей.

Подсказка: Подумайте, что происходит в каждой фазе CI/CD.

Стадии:

Ответ:

- Checkout
- Build
- Test
- Deploy

Задачи:

1. Получить код из репозитория
 2. Запустить сборку LoRA-пакета
 3. Проверить работоспособность и метрики
 4. Выложить LoRA на сервер / в хранилище
- **Checkout** — Получить код из репозитория
 - **Build** — Запустить сборку LoRA-пакета
 - **Test** — Проверить работоспособность и метрики
 - **Deploy** — Выложить LoRA на сервер / в хранилище

🔮 Шаг 07:

Миссия 7

📊 Катя определяет метрики для оценки LoRA. Сопоставьте метрику и её назначение.

Подсказка: Думайте об ошибках и времени.

Метрики:

Ответ:

- Latency
- Accuracy delta
- Memory usage
- Throughput

Назначения:

1. Изменение точности относительно базовой модели
2. Время отклика per запрос

3. Объем памяти, потребляемый LoRA

4. Число запросов в секунду

- **Latency** — Время отклика per запрос
- **Accuracy delta** — Изменение точности относительно базовой модели
- **Memory usage** — Объем памяти, потребляемый LoRA
- **Throughput** — Число запросов в секунду

Шаг 08:

Миссия 8

Катя готовит версии LoRA-модулей. Сопоставьте обозначение версии и его смысл.

Подсказка: Семантическое версионирование часто используют.

Версии:

Ответ:

- MAJOR
- MINOR
- PATCH
- PRE-RELEASE

Смысл:

1. Совместимые изменения, новый функционал
 2. Крупные, несовместимые изменения
 3. Мелкие исправления ошибок
 4. Тестовая версия перед релизом
- **MAJOR** — Крупные, несовместимые изменения
 - **MINOR** — Совместимые изменения, новый функционал
 - **PATCH** — Мелкие исправления ошибок
 - **PRE-RELEASE** — Тестовая версия перед релизом

Шаг 09:

Миссия 9

Катя настраивает права пользователей. Сопоставьте роль в CI и её задачу.

Подсказка: Подумайте о разделении обязанностей.

Роли:

Ответ:

- Developer
- Reviewer
- Release Manager
- CI Admin

Задачи:

1. Настройка секретов и прав в CI
 2. Обзор и одобрение pull-request'ов
 3. Запуск и контроль релизов
 4. Разработка и коммиты кода
- **Developer** — Разработка и коммиты кода

- **Reviewer** — Обзор и одобрение pull-request'ов
- **Release Manager** — Запуск и контроль релизов
- **CI Admin** — Настройка секретов и прав в CI

Шаг 10:

Миссия 10

 Упорядочьте этапы автопайплайна обновления LoRA:

1. Deploy нового LoRA
2. Checkout исходного кода
3. Run tests и метрики
4. Build LoRA-модуля
5. Validate качества и откаты

Подсказка: Сначала получаем код, потом билд, потом тесты, затем деплой с возможностью отката.

Ответ:

- Checkout исходного кода
- Build LoRA-модуля
- Run tests и метрики
- Validate качества и откаты
- Deploy нового LoRA

Шаг 11:

Миссия 11

Упорядочите шаги для отката LoRA при провале тестов:

1. Откат к предыдущей версии
2. Уведомление команды об ошибке
3. Анализ логов ошибок
4. Запуск rollback скрипта
5. Проверка, что система вернулась к стабильному состоянию

Подсказка: Сначала уведомить, затем откатить, потом проверить.

Ответ:

- Уведомление команды об ошибке
- Анализ логов ошибок
- Запуск rollback скрипта
- Откат к предыдущей версии
- Проверка, что система вернулась к стабильному состоянию

Шаг 12:

Миссия 12

 Упорядочите ключевые метрики, на которые Катя будет смотреть при принятии решения о деплое (от самой критичной до менее критичной):

1. Accuracy delta
2. Latency
3. Memory usage

4. Throughput

Подсказка: Самое важное — точность, потом задержка и ресурсы.

Ответ:

- Accuracy delta
- Latency
- Memory usage
- Throughput

Шаг 13:

Миссия 13

🔗 Упорядочите приоритеты при доработке пайплайна после первого релиза:

1. Улучшить логирование и оповещения
2. Оптимизировать время сборки
3. Добавить ветки фич для LoRA
4. Повысить стабильность тестов
5. Интегрировать мониторинг в прод среду

Подсказка: Сначала надёжность, затем скорость, затем расширения.

Ответ:

- Повысить стабильность тестов
- Улучшить логирование и оповещения
- Оптимизировать время сборки
- Интегрировать мониторинг в прод среду
- Добавить ветки фич для LoRA

Шаг 14:

Миссия 14

Упорядочите шаги обработки pull-request'a LoRA-обновления:

1. Код ревью
2. Запуск CI-тестов
3. Слияние после зелёного билда
4. Автоматический запуск деплоя после мержа
5. Уведомление команды о статусе

Подсказка: Ревью, тесты, слияние, деплой, уведомление.

Ответ:

- Код ревью
- Запуск CI-тестов
- Слияние после зелёного билда
- Автоматический запуск деплоя после мержа
- Уведомление команды о статусе

Шаг 15:

Миссия 15

🌐 Упорядочите этапы интеграции мониторинга после деплоя LoRA:

1. Сбор метрик с прод окружения
2. Настройка алёртов на отклонения
3. Визуализация метрик (дашборды)
4. Историческое хранение данных
5. Внедрение автоматического триггера отката

Подсказка: Сначала сбор, потом визуализация, потом оповещения и реагирование.

Ответ:

- Сбор метрик с прод окружения
- Историческое хранение данных
- Визуализация метрик (дашборды)
- Настройка алёртов на отклонения
- Внедрение автоматического триггера отката

💡 **Шаг 16:**

Миссия 16

⚙️ Упорядочите шаги миграции старых LoRA-модулей в новую структуру:

1. Экспорт моделей из старого формата
2. Конвертация в новый формат стандарта
3. Тестирование конвертированных моделей
4. Запуск автоматического обновления через пайплайн
5. Валидация на прод наборах данных

Подсказка: От извлечения — к конверсии — к тестам — к валидации — к деплою.

Ответ:

- Экспорт моделей из старого формата
- Конвертация в новый формат стандарта
- Тестирование конвертированных моделей
- Валидация на прод наборах данных
- Запуск автоматического обновления через пайплайн

💡 **Шаг 17:**

Миссия 17

🔄 Упорядочите цикл непрерывного улучшения автопайплайна:

1. Сбор обратной связи от команды
2. Рефакторинг и оптимизация шагов
3. Внедрение новых проверок и тестов
4. Обучение членов команды пользоваться пайплайном
5. Мониторинг и анализ метрик пайплайна

Подсказка: Это цикл: обратная связь → улучшения → тесты → обучение → мониторинг.

Ответ:

- Сбор обратной связи от команды
- Рефакторинг и оптимизация шагов
- Внедрение новых проверок и тестов

- Мониторинг и анализ метрик пайплайна
- Обучение членов команды пользоваться пайплайном

Модуль 11 - Миссия 11. Финальный проект свой ИИ-агент

Урок 1 - Постановка цели и выбор модели

Шаг 01:

Миссия 1

 Алёна хочет, чтобы её агент отвечал на вопросы о финансах. Как сформулировать цель агента?

Подсказка: Цель агента — концентрировать его поведение и границы.

Ответ:

- **Отвечать на вопросы о финансах корректно и безопасно**
- Говорить всё, что приходит в голову
- Игнорировать пользовательские запросы
- Писать стихи про деньги

Шаг 02:

Миссия 2

 Борис хочет, чтобы агент помогал учить языки. Какую модель он должен выбрать?

Подсказка: Выбирай модель, которая хорошо справляется с генерацией текста.

Выбери наиболее подходящую:

Ответ:

- **LLM-модель с сильным языковым ядром**
- Модель, специализирующаяся на компьютерном зрении
- Модель, оптимизированная для обработки аудио

Шаг 03:

Миссия 3

Катя хочет, чтобы агент отвечал с учётом данных пользователя (контекста). Какой компонент обязательно добавить?

Подсказка: Нужно помнить предыдущие сообщения и контекст.

Ответ:

- **Модуль памяти / контекстного буфера**
- Модуль компьютерного зрения
- Модуль анализа аудио
- Модуль шумоподавления

Шаг 04:

Миссия 4

 Максим хочет ограничить действия агента: не форматировать файлы пользователя. Как формализовать такое ограничение?

Подсказка: Используй запрет или правило поведения.

Ответ:

- **Добавить правило: «Не изменяй файлы пользователя»**
- Никкак не ограничивать агент
- Разрешить любые действия

Шаг 05:

Миссия 5

✳ Игорь хочет, чтобы агент мог подгружать базу знаний (фаQ). Как это называется?

Подсказка: Это — подключение внешнего источника знаний.

Ответ:

- **Интеграция внешнего источника / база знаний**
- Сжатие модели
- Генерация случайного текста

Шаг 06:

Миссия 6

📄 Сопоставьте термин и его объяснение для постановки цели агента:

Термины:

Ответ:

- Цель агента
- Ограничения (constraints)
- Среда взаимодействия
- Методы действия

Объяснения:

1. Контекст взаимодействия с пользователем и внешними системами
2. Что агент пытается достичь
3. Запреты, рамки, условия, которые агент не должен нарушать
4. Какие действия агент может совершать (набор команд, API и т.д.)

Подсказка: Подумайте, что такое цель, что — ограничения, что — среда и какие действия.

- **Цель агента** — Что агент пытается достичь
- **Ограничения (constraints)** — Запреты, рамки, условия, которые агент не должен нарушать
- **Среда взаимодействия** — Контекст взаимодействия с пользователем и внешними системами
- **Методы действия** — Какие действия агент может совершать (набор команд, API и т.д.)

Шаг 07:

Миссия 7

🔍 Сопоставьте тип модели и её характеристики:

Типы моделей:

Ответ:

- Большая универсальная модель
- Модель-компаньон с ограниченной специализацией

- Модель-компрессия / distilled
- Модель локального развертывания (on-device)

Характеристики:

1. Низкие требования к ресурсам, можно развернуть локально
2. Универсальна и мощна, может решать разнообразные задачи
3. Узкая специализация, меньше риск ошибок в предмете задачи
4. Упрощена, меньший размер, но близкая по качеству

Подсказка: Универсальная — широкий спектр, компрессия — уменьшение модели, локальная — ресурсная.

- **Большая универсальная модель** — Универсальна и мощна, может решать разнообразные задачи
- **Модель-компаньон с ограниченной специализацией** — Узкая специализация, меньше риск ошибок в предмете задачи
- **Модель-компрессия / distilled** — Упрощена, меньший размер, но близкая по качеству
- **Модель локального развертывания (on-device)** — Низкие требования к ресурсам, можно развернуть локально

🔮 Шаг 08:

Миссия 8

🗨 Сопоставьте элемент спецификации агента и пример:

Элементы:

Ответ:

- Контекст
- Состояние памяти
- Действия / API
- Ограничения

Примеры:

1. Агент не может удалять файлы пользователя
2. История вопросов пользователя, которую агент помнит
3. Подключение к базе данных через API-запросы
4. Текущий ввод пользователя и предыдущий ответ

Подсказка: Контекст — обмен сообщений, память — что хранит, действия — API, ограничения — запреты.

- **Контекст** — Текущий ввод пользователя и предыдущий ответ
- **Состояние памяти** — История вопросов пользователя, которую агент помнит
- **Действия / API** — Подключение к базе данных через API-запросы
- **Ограничения** — Агент не может удалять файлы пользователя

🔮 Шаг 09:

Миссия 9

Сопоставьте критерий выбора модели и мотивацию:

Критерии:

Ответ:

- Качество генерации

- Скорость (latency)
- Ресурсная нагрузка
- Безопасность / фильтры

Мотивации:

1. Минимизировать задержку ответа для пользователя
2. Генерировать тексты с правильной логикой и связностью
3. Модель должна работать на ограниченных вычислительных ресурсах
4. Не допускать нежелательный контент или ошибки безопасности

Подсказка: Качество — за логику, скорость — задержка, ресурсы — нагрузка, безопасность — фильтры.

- **Качество генерации** — Генерировать тексты с правильной логикой и связностью
- **Скорость (latency)** — Минимизировать задержку ответа для пользователя
- **Ресурсная нагрузка** — Модель должна работать на ограниченных вычислительных ресурсах
- **Безопасность / фильтры** — Не допускать нежелательный контент или ошибки безопасности

🔮 Шаг 10:

Миссия 10

✂ Упорядочьте шаги спецификации агента перед реализацией:

1. Определение целей агента
2. Описание ограничений и рамок поведения
3. Выбор подходящей модели
4. Проектирование интерфейсов API / действий
5. Определение контекста и памяти агента

Подсказка: Сначала цель, затем рамки, потом модель, далее как он действует и что он запомнит.

Ответ:

- Определение целей агента
- Описание ограничений и рамок поведения
- Выбор подходящей модели
- Проектирование интерфейсов API / действий
- Определение контекста и памяти агента

🔮 Шаг 11:

Миссия 11

📐 Упорядочьте этапы оценки гипотез моделей:

1. Сбор метрик (качество, latency и пр.)
2. Запуск A/B тестов между моделями
3. Сравнение результатов
4. Выбор победителя и дальнейшее тестирование
5. Внедрение победителя в спецификацию агента

Подсказка: Это экспериментальный цикл: сравнение, выбор, внедрение.

Ответ:

- Запуск A/B тестов между моделями
- Сбор метрик (качество, latency и пр.)
- Сравнение результатов
- Выбор победителя и дальнейшее тестирование
- Внедрение победителя в спецификацию агента

 Шаг 12:

Миссия 12

 Упорядочьте шаги при уточнении ограничений агента после первого теста:

1. Анализ неправильных ответов или нежелательного поведения
2. Добавление новых правил/фильтров
3. Модификация спецификации агента
4. Повторное тестирование агентом в симуляции
5. Оценка изменения поведения модели

Подсказка: Первый шаг — обнаружить проблему, затем ввести ограничение, протестировать и оценить.

Ответ:

- Анализ неправильных ответов или нежелательного поведения
- Добавление новых правил/фильтров
- Модификация спецификации агента
- Повторное тестирование агентом в симуляции
- Оценка изменения поведения модели

 Шаг 13:

Миссия 13

 Упорядочьте этапы итеративного развития агента:

1. Получить обратную связь от пользователей
2. Анализ слабых мест и ошибок агента
3. Корректировка спецификации или модели агента
4. Тестирование улучшенной версии агента
5. Внедрение обновлённого агента в продукцию

Подсказка: Это цикл улучшения: отзывы → анализ → правки → тесты → внедрение.

Ответ:

- Получить обратную связь от пользователей
- Анализ слабых мест и ошибок агента
- Корректировка спецификации или модели агента
- Тестирование улучшенной версии агента
- Внедрение обновлённого агента в продукцию

 Шаг 14:

Миссия 14

 Упорядочьте проверку наиболее важных аспектов перед запуском агента:

1. Тесты по безопасности и ограничениям

2. Тесты на точность и качество ответов
3. Латенси-тесты и нагрузочное тестирование
4. Тесты на согласованность и устойчивость памяти
5. Тест возможного эскейпа ограничений

Подсказка: Сначала безопасность, затем качество, затем производительность, потом устойчивость и контроль.

Ответ:

- Тесты по безопасности и ограничениям
- Тесты на точность и качество ответов
- Латенси-тесты и нагрузочное тестирование
- Тесты на согласованность и устойчивость памяти
- Тест возможного эскейпа ограничений

 Шаг 15:

Миссия 15

 Упорядочьте этапы внедрения агента в продакшн среду:

1. Развертывание агента в staging / тестовом окружении
2. Мониторинг поведения агента и метрик
3. Плавный rollout (канареечный запуск)
4. Финальный переключатель на прод среду
5. Наблюдение и сбор логов в прод окружении

Подсказка: Используйте staged rollout: тестовая зона → мониторинг → постепенное расширение → финальный переход.

Ответ:

- Развертывание агента в staging / тестовом окружении
- Мониторинг поведения агента и метрик
- Плавный rollout (канареечный запуск)
- Финальный переключатель на прод среду
- Наблюдение и сбор логов в прод окружении

 Шаг 16:

Миссия 16

 Упорядочьте этапы анализа после запуска агента:

1. Сбор логов и метрик
2. Сравнение предсказаний агента с эталоном / контрольной выборкой
3. Выявление отклонений и аномалий
4. Формирование отчёта и план корректировок
5. Запуск следующего раунда улучшений

Подсказка: От первичных данных к анализу, плану и новым итерациям.

Ответ:

- Сбор логов и метрик
- Сравнение предсказаний агента с эталоном / контрольной выборкой
- Выявление отклонений и аномалий
- Формирование отчёта и план корректировок

- Запуск следующего раунда улучшений

🔮 Шаг 17:

Миссия 17

🔧 Упорядочьте этапы полного жизненного цикла агента:

1. Спецификация целей и ограничений
2. Выбор и донастройка модели агента
3. Разработка контекста, памяти, API и тестов
4. Тестирование агента, поиск ошибок и корректировки
5. Развёртывание, мониторинг, итеративное улучшение

Подсказка: Это полный цикл: от концепции до постоянного улучшения.

Ответ:

- Спецификация целей и ограничений
- Выбор и донастройка модели агента
- Разработка контекста, памяти, API и тестов
- Тестирование агента, поиск ошибок и корректировки
- Развёртывание, мониторинг, итеративное улучшение

📖 Урок 2 - 🧠 Обучение и адаптация под задачу

🔮 Шаг 01:

Миссия 1

🧠 Когда вы начинаете проект по обучению агента на Ollama, с чего разумнее всего начать?

Подсказка: подумайте, что важно до подачи данных на обучение.

Ответ:

- **Определить целевую задачу и требования к агенту**
- Сразу собрать кучу данных и начать обучать
- Сначала писать UI для агента

🔮 Шаг 02:

Миссия 2

Вы готовите датасет для задачи классификации вопросов. Что важно учесть?

Подсказка: вспомните про баланс классов и качество аннотаций.

Ответ:

- **Убедиться, что классы сбалансированы**
- Обязательно использовать минимум 1000 классов
- Не проверять качество аннотаций
- Игнорировать выбросы в данных

🔮 Шаг 03:

Миссия 3

🔧 Вы запустили обучение агента, и заметили сильное переобучение (loss растёт на валидации). Что попытаться первым?

Подсказка: подумайте про регуляризацию и размер данных.

Ответ:

- **Добавить регуляризацию или dropout**
- Увеличить learning rate
- Уменьшить количество данных
- Игнорировать сигнал валидации

🔮 Шаг 04:

Миссия 4

✂ Вы хотите дообучить агента на новой подзадаче. Какой подход наиболее безопасен?

Подсказка: подумайте об использовании малых шагов и сохранении исходной модели.

Ответ:

- **Файн-тюнить с малым learning rate**
- Обучать с нуля заново
- Менять архитектуру модели кардинально
- Игнорировать старые веса

🔮 Шаг 05:

Миссия 5

🔍 Сопоставьте термин с его функцией в контексте обучения агента:

Термины:

Ответ:

- Transfer learning
- Fine-tuning
- Prompt tuning
- Regularization

Функции:

1. Настройка небольшого дополнительного слоя или параметров prompt
2. Предотвращение переобучения через штрафы или дроп-аут
3. Обучение модели на смежной задаче и перенос знаний
4. Дополнительное обучение уже предварительно обученной модели

Подсказка: подумайте, что делают эти методы, когда модель уже имеет знания.

- **Transfer learning** — Обучение модели на смежной задаче и перенос знаний
- **Fine-tuning** — Дополнительное обучение уже предварительно обученной модели
- **Prompt tuning** — Настройка небольшого дополнительного слоя или параметров prompt
- **Regularization** — Предотвращение переобучения через штрафы или дроп-аут

🔮 Шаг 06:

Миссия 6

📊 Сопоставьте метрику с её применением при оценке агента:

Метрики:

Ответ:

- Accuracy
- F1-score

- Precision
- Recall

Применение:

1. Процент правильно классифицированных примеров
2. Баланс между точностью и полнотой
3. Доля предсказанных как положительные, которые действительно положительные
4. Доля истинных положительных, которые были найдены

Подсказка: вспомните формулы: F1 — гармоническое среднее precision и recall.

- **Accuracy** — Процент правильно классифицированных примеров
- **F1-score** — Баланс между точностью и полнотой
- **Precision** — Доля предсказанных как положительные, которые действительно положительные
- **Recall** — Доля истинных положительных, которые были найдены

Шаг 07:

Миссия 7

 Сопоставьте технику адаптации агента с её кратким описанием:

Техники:

Ответ:

- Adapter layers
- LoRA
- Prompt injection
- Continual learning

Описание:

1. Встройка небольших слоёв в основную сеть для лёгкой корректировки
2. Низкоранговые обновления весов при малом числе параметров
3. Вставка инструкций в prompt для управления поведением агента
4. Обучение на новом потоке данных без забывания старых задач

Подсказка: часто используют лёгкие механизмы, чтобы не трогать всю модель.

- **Adapter layers** — Встройка небольших слоёв в основную сеть для лёгкой корректировки
- **LoRA** — Низкоранговые обновления весов при малом числе параметров
- **Prompt injection** — Вставка инструкций в prompt для управления поведением агента
- **Continual learning** — Обучение на новом потоке данных без забывания старых задач

Шаг 08:

Миссия 8

 Сопоставьте этап проекта агента с его задачей:

Этапы:

Ответ:

- Сбор и аннотирование данных
- Предварительное обучение
- Адаптация под задачу
- Оценка и тестирование

Задачи:

1. Обучить модель на большом корпусе общественных данных
2. Проверить, как агент работает на задачах пользователя
3. Получить разнообразные примеры с метками для конкретной задачи
4. Подстроить агент под узкую задачу, дообучить или настроить prompt

Подсказка: мысленно пройдите жизненный цикл проекта агента от нуля до финала.

- **Сбор и аннотирование данных** — Получить разнообразные примеры с метками для конкретной задачи
- **Предварительное обучение** — Обучить модель на большом корпусе общественных данных
- **Адаптация под задачу** — Подстроить агент под узкую задачу, дообучить или настроить prompt
- **Оценка и тестирование** — Проверить, как агент работает на задачах пользователя

🔮 Шаг 09:

Миссия 9

Расположите шаги от идеи до тестирования ИИ-агента в правильной последовательности:

1. Оценка и тестирование агента
2. Сбор и аннотирование данных
3. Предварительное обучение на общих данных
4. Адаптация агента под целевую задачу
5. Определение требований и постановка задачи

Подсказка: представьте, что вы идёте по дорожке проекта — сначала идея, потом данные, затем обучение и так далее.

Ответ:

- Определение требований и постановка задачи
- Сбор и аннотирование данных
- Предварительное обучение на общих данных
- Адаптация агента под целевую задачу
- Оценка и тестирование агента

🔮 Шаг 10:

Миссия 10

🔁 Расположите стадии дообучения агента так, чтобы избежать катастрофического забывания:

1. Обучение на новом наборе данных
2. Интеграция с механизмом регуляризации / replay buffer
3. Заморозка части весов
4. Оценка производительности на старых задачах
5. Запуск обучения с малыми шагами

Подсказка: сначала защитите старое знание, потом аккуратно учите новое, и проверяйте, что старое не потерялось.

Ответ:

- Заморозка части весов
- Интеграция с механизмом регуляризации / replay buffer

- Запуск обучения с малыми шагами
- Обучение на новом наборе данных
- Оценка производительности на старых задачах

🔮 Шаг 11:

Миссия 11

🔮 У вас есть четыре идеи для улучшений агента. Упорядочите их по предпочтительности внедрения (с чего начать):

1. Оптимизация prompt на краю
2. Добавление новых метрик мониторинга
3. Улучшение качества данных аннотаций
4. Файн-тюнинг части модели

Подсказка: сначала улучшайте данные, затем мониторинг, потом лёгкое улучшение, и уже потом глубокую модель.

Ответ:

- Улучшение качества данных аннотаций
- Добавление новых метрик мониторинга
- Оптимизация prompt на краю
- Файн-тюнинг части модели

🔮 Шаг 12:

Миссия 12

🔮 Сопоставьте проблему с её подходом решения при работе над финальным агентом:

Проблемы:

Ответ:

- Переобучение на мелком наборе данных
- Забывание старых задач
- Неустойчивое поведение агента на крайних примерах
- Низкая скорость ответа

Решения:

1. Использовать регуляризацию и аугментацию
2. Использовать replay buffer / регуляризацию к старым весам
3. Ограничить выходы, добавить фильтры или штрафы
4. Оптимизировать модель, применять лёгкие методы адаптации

Подсказка: каждая проблема часто решается одним из знакомых методов обучения или адаптации.

- **Переобучение на мелком наборе данных** — Использовать регуляризацию и аугментацию
- **Забывание старых задач** — Использовать replay buffer / регуляризацию к старым весам
- **Неустойчивое поведение агента на крайних примерах** — Ограничить выходы, добавить фильтры или штрафы
- **Низкая скорость ответа** — Оптимизировать модель, применять лёгкие методы адаптации

🔧 Шаг 13:

Миссия 13

🔗 Финальная задача: У вас есть четыре действия для запуска агента в продакшн. Упорядочите их логически:

1. Мониторинг поведения и логирование
2. Деплой агента на сервер или устройство
3. Тестирование на краевых сценариях
4. Обеспечение обратной связи пользователей и дообучение

Подсказка: сначала надо проверить, потом запустить, затем следить и улучшать.

Ответ:

- Тестирование на краевых сценариях
- Деплой агента на сервер или устройство
- Мониторинг поведения и логирование
- Обеспечение обратной связи пользователей и дообучение

📖 Урок 3 - 🔗 Интеграция памяти, reasoning и API

🔧 Шаг 01:

Миссия 1

👤 Разработчик Алиса хочет, чтобы агент запоминал краткую историю взаимодействия с пользователем (контекст). Какой подход выбрать первым?

Подсказка: вспомните, что память часто реализуют через буфер недавних сообщений.

Ответ:

- **Использовать кольцевую память последних N сообщений**
- Сохранять всё в базу данных с каждым взаимодействием
- Игнорировать память и каждый раз начинать «с чистого листа»

🔧 Шаг 02:

Миссия 2

🔗 Боб интегрирует секторы агенту внешнее API прогноза погоды. Что нужно учесть первым?

Подсказка: подумайте, как агент узнаёт, когда вызывать API vs отвечать из собственной базы знаний.

Ответ:

- **Определить триггер (условие) для вызова API**
- Вызывать API на каждый запрос без условий
- Хардкодить ответы о погоде
- Отключить память агента

🔧 Шаг 03:

Миссия 3

🔗 Когда агент получает запрос, сначала он анализирует вход. Что важно решить на этом этапе?

Подсказка: важно определить намерение и сущности (intent & entities).

Ответ:

- **Распознать намерение пользователя и ключевые сущности**

- Игнорировать контекст и сразу вызвать API
- Всегда отвечать заранее загруженным шаблоном
- Сразу сохранять запрос в память, не обрабатывая

🧙 Шаг 04:

Миссия 4

У Алисы агент забыл ранее сохранённое сообщение при переключении темы разговора. Что может помочь?

Подсказка: подумайте про стратегию ветвлений контекста и фильтрацию памяти.

Ответ:

- **Использовать сегментированную память по темам**
- Стереть всю память при смене темы
- Оставлять память бесконечной и не фильтровать
- Никогда не учитывать прошлые сообщения

🧙 Шаг 05:

Миссия 5

✂ Разработчик Виктор хочет соединить reasoning (логика) с памятью агента. Что логично реализовать первым?

Подсказка: подумайте, как память может подсказывать шаги рассуждения.

Ответ:

- **Подгружать релевантные факты из памяти перед reasoning**
- Игнорировать память во время reasoning
- Всегда просить пользователя повторить всё
- Хранить reasoning шаги только в метаданных

🧙 Шаг 06:

Миссия 6

🧩 Сопоставьте компонент агента с его ролью в интеграции памяти и API:

Компоненты:

Ответ:

- Менеджер памяти
- API-обёртка
- Модуль рассуждения (reasoning)
- Обработка намерения (intent parser)

Роли:

1. Определяет, когда и как обращаться к внешним API
2. Извлекает из хранилища релевантные контекстные данные
3. Интерпретирует, что хочет пользователь
4. Формирует внутренние логические шаги решения задачи

Подсказка: вспомните, как память, API и reasoning взаимодействуют в агенте.

- **Менеджер памяти** — Извлекает из хранилища релевантные контекстные данные
- **API-обёртка** — Определяет, когда и как обращаться к внешним API

- **Модуль рассуждения (reasoning)** — Формирует внутренние логические шаги решения задачи
- **Обработка намерения (intent parser)** — Интерпретирует, что хочет пользователь

💡 Шаг 07:

Миссия 7

🔗 Сопоставьте стратегию хранения памяти с её особенностью:

Стратегии:

Ответ:

- Кольцевая память
- Сегментированная память
- Long-term memory
- Ассоциативная память

Особенности:

1. Хранение только последних N элементов
2. Разделение памяти по темам или сессиям
3. Сохранение ключевых фактов на весь срок жизни агента
4. Поиск по схожести между сущностями и фактов

Подсказка: вспомните, как человеческая память может быть “долгой” и “контекстной”.

- **Кольцевая память** — Хранение только последних N элементов
- **Сегментированная память** — Разделение памяти по темам или сессиям
- **Long-term memory** — Сохранение ключевых фактов на весь срок жизни агента
- **Ассоциативная память** — Поиск по схожести между сущностями и фактов

💡 Шаг 08:

Миссия 8

🔗 Сопоставьте подход reasoning с его характерной особенностью:

Подходы:

Ответ:

- Chain of Thought
- Tree of Thoughts
- ReAct
- SELF-ASK

Особенности:

1. Модель генерирует последовательность мыслей шаг за шагом
2. Исследование ветвей рассуждений в виде дерева
3. Сочетает действия (action) и размышления (reasoning)
4. Спрашивает подзадачи, прежде чем ответить окончательно

Подсказка: вспомните современные методы LLM-размышлений с API и памятью.

- **Chain of Thought** — Модель генерирует последовательность мыслей шаг за шагом
- **Tree of Thoughts** — Исследование ветвей рассуждений в виде дерева
- **ReAct** — Сочетает действия (action) и размышления (reasoning)
- **SELF-ASK** — Спрашивает подзадачи, прежде чем ответить окончательно

Шаг 09:

Миссия 9

Расположите этапы интеграции памяти, reasoning и API в агента:

1. Обработка намерения пользователя
2. Вызов внешнего API, если нужно
3. Извлечение релевантной памяти
4. Формирование рассуждения и генерация ответа

Подсказка: подумайте, что логично сначала: понять, что хочет пользователь, затем что нужно вспомнить, потом API, потом рассуждение.

Ответ:

- Обработка намерения пользователя
- Извлечение релевантной памяти
- Вызов внешнего API, если нужно
- Формирование рассуждения и генерация ответа

Шаг 10:

Миссия 10

 Расположите шаги, когда агент должен реагировать на запрос с внешним API:

1. Обработка и понимание входа
2. Проверка, нужна ли внешняя информация
3. Вызов API и получение ответа
4. Интеграция ответа API в дальнейшее рассуждение
5. Генерация окончательного ответа пользователю

Подсказка: важно не вызывать API бессмысленно и правильно вставить его результат в рассуждения.

Ответ:

- Обработка и понимание входа
- Проверка, нужна ли внешняя информация
- Вызов API и получение ответа
- Интеграция ответа API в дальнейшее рассуждение
- Генерация окончательного ответа пользователю

Шаг 11:

Миссия 11

 Можно назвать внутренний цикл агента “слушать → помнить → рассуждать → действовать”. Упорядочите эти стадии:

1. Действовать (action / ответ)
2. Слушать (ввод пользователя)
3. Память (retrieve / store контекст)
4. Рассуждать (reasoning / планирование)

Подсказка: цикл начинается с получения ввода, затем память и рассуждение, и завершается действием.

Ответ:

- Слушать (ввод пользователя)
- Память (retrieve / store контекст)
- Рассуждать (reasoning / планирование)
- Действовать (action / ответ)

 Шаг 12:

Миссия 12

⚠ Агент часто обращается к API, даже когда это не нужно, что увеличивает задержки. Что лучше сделать?

Подсказка: подумайте про кэширование и правила вызова API.

Ответ:

- **Кэшировать результаты API и проверять память сначала**
- Вызывать API каждую секунду
- Удалить логику памяти
- Увеличить таймаут

 Шаг 13:

Миссия 13

🔑 При интеграции API с агентом нужно защитить ключи. Как лучше сделать?

Подсказка: подумайте про секреты и доступ из среды исполнения.

Ответ:

- **Хранить ключи в среде (environment variables), не в коде**
- Записать ключи прямо в репозиторий
- Отправлять пользователю ключи
- Отключить защиту ключей

 Шаг 14:

Миссия 14

🧠 Вы хотите, чтобы агент адаптировался к историческим данным пользователя. Что стоит реализовать?

Подсказка: подумайте про дообучение и обновление памяти с течением времени.

Ответ:

- **Периодически переобучать модель и обновлять память**
- Никогда не трогать память после первого запуска
- Очистить память каждый день
- Заморозить модель навсегда

 Шаг 15:

Миссия 15

🔍 Сопоставьте вызов API с возможным способом интеграции его результата в рассуждение агента:

Вызовы API:

Ответ:

- Погода
- Курсы валют
- Поиск новостей
- Проверка базы знаний

Интеграция результата:

1. Внедрить как факт в память и учесть при рассуждении
2. Нормализовать значения и использовать как контекстный факт
3. Сделать семантический поиск по результатам и перейти к ним
4. Сохранить в буфер и выдать как часть ответа

Подсказка: подумайте, как разные API-данные можно аккуратно вставить в цепочку reasoning.

- **Погода** — Внедрить как факт в память и учесть при рассуждении
- **Курсы валют** — Нормализовать значения и использовать как контекстный факт
- **Поиск новостей** — Сделать семантический поиск по результатам и перейти к ним
- **Проверка базы знаний** — Сохранить в буфер и выдать как часть ответа

 Шаг 16:

Миссия 16

🏳️ Финальный сортинг: упорядочите ключевые фазы подготовки агента с памятью, reasoning и API к продакшену:

1. Тестирование на промахах и краевых сценариях
2. Деплой агента и настройка окружения
3. Мониторинг, логирование и обратная связь
4. Итеративное улучшение памяти, reasoning и API-интеграций

Подсказка: подумайте, что логично делать: сначала тесты, потом запуск, потом следить и улучшать.

Ответ:

- Тестирование на промахах и краевых сценариях
- Деплой агента и настройка окружения
- Мониторинг, логирование и обратная связь
- Итеративное улучшение памяти, reasoning и API-интеграций

 Урок 4 - 🤖 Создание полноценного агента

 Шаг 01:

Миссия 1

Вы запускаете базового агента с Ollama, и он не отвечает на запросы. Что вы проверите первым?

Подсказка: Часто проблема — сеть или модель не загружена.

Ответ:

- Перезапустить IDE
- **Проверить доступ к серверу и загрузку модели**
- Изменить цвет консоли
- Удалить файлы логов

🔮 Шаг 02:

Миссия 2

🗉 Агент отвечает, но выдаёт слишком общий текст. Какое действие вы примените?

Подсказка: Нужно скорректировать подсказку (prompt) или инструкцию агента.

Ответ:

- Увеличить размер шрифта
- **Уточнить инструкцию в System Prompt**
- Сделать запрос короче
- Отключить агент

🔮 Шаг 03:

Миссия 3

🗉 Сопоставьте роль агента с её назначением.

Роли:

Ответ:

- System-подсказка
- User-подсказка
- Инструмент (tool)
- Контроль потока (controller)

Назначения:

1. Управляет, какие шаги агент делает и в каком порядке
2. Инструкция от пользователя, что он хочет получить
3. Основная директива, правила поведения агента
4. Дополнительные функции (например, поиск, вычисления)

🔮 Подсказка:

System — задаёт рамки поведения. User — конкретный запрос. Инструменты — внешние возможности. Контроллер — координатор шагов.

- **System-подсказка** — Основная директива, правила поведения агента
- **User-подсказка** — Инструкция от пользователя, что он хочет получить
- **Инструмент (tool)** — Дополнительные функции (например, поиск, вычисления)
- **Контроль потока (controller)** — Управляет, какие шаги агент делает и в каком порядке

🔮 Шаг 04:

Миссия 4

🗉 Агент ведёт диалог, и пользователь спрашивает уточнение. Что вы сделаете?

Подсказка: Лучше задать уточняющий вопрос, чем угадывать.

Ответ:

- **Спросить у пользователя уточнение**
- Предположить и ответить сразу
- Игнорировать и продолжить
- Открыть справочник

🔮 Шаг 05:

Миссия 5

🔮 Агент поддерживает инструменты: сопоставьте инструмент и его пример использования.

Инструменты:

Ответ:

- Поиск в вебе
- Вычислительная функция (calculator)
- База знаний (knowledge base)
- API переводчика

Примеры:

1. Перевести текст с русского на английский
2. Сложить 34 + 58
3. Найти свежие новости
4. Ответ с опорой на локальную базу фактов

🔮 **Подсказка:**

Инструменты расширяют возможности агента — сопоставьте их по назначению.

- **Поиск в вебе** — Найти свежие новости
- **Вычислительная функция (calculator)** — Сложить 34 + 58
- **База знаний (knowledge base)** — Ответ с опорой на локальную базу фактов
- **API переводчика** — Перевести текст с русского на английский

🔮 Шаг 06:

Миссия 6

📦 Если агент может вызвать несколько инструментов, как он решает, какой использовать?

Подсказка: Нужно выбрать инструмент, соответствующий задаче.

Ответ:

- Всегда использовать первый инструмент
- **Выбрать инструмент, наиболее подходящий задаче**
- Вызывать все инструменты параллельно
- Не использовать инструменты

🔮 Шаг 07:

Миссия 7

Упорядочите этапы работы агента при входящем запросе:

1. Использовать инструменты (если нужно)
2. Приём запроса от пользователя
3. Формирование ответа и отправка
4. Анализ и планирование этапов

🔮 **Подсказка:**

Сначала приходит запрос, затем агент планирует шаги, возможно вызывает инструменты, а потом формирует ответ.

Ответ:

- Приём запроса от пользователя
- Анализ и планирование этапов
- Использовать инструменты (если нужно)
- Формирование ответа и отправка

 Шаг 08:

Миссия 8

⚙ Вы хотите ограничить длину ответа агента, чтобы не уйти в слишком подробные рассуждения. Что сделаете?

Подсказка: Можно задать максимальное число токенов или указать стиль ответа.

Ответ:

- **Ограничить max_tokens**
- **Указать стиль «кратко, по делу»**
- Удалить все сигналы stop
- Запросить слишком длинный ответ

 Шаг 09:

Миссия 9

📖 Разные виды памяти агента — сопоставьте и поясните.

Типы памяти:

Ответ:

- Краткосрочная память
- Долгосрочная память
- Контекстная память
- Внешняя база памяти

Описание:

1. Информация, хранящаяся внутри текущего диалога
2. Факты, запоминаемые между сессиями
3. Запись разговоров и данных в файловую или базу систему
4. Содержимое текущего состояния и недавних обменов

 **Подсказка: Краткосрочная и контекстная могут пересекаться; долгосрочная — как база.**

- **Краткосрочная память** — Информация, хранящаяся внутри текущего диалога
- **Контекстная память** — Содержимое текущего состояния и недавних обменов
- **Долгосрочная память** — Факты, запоминаемые между сессиями
- **Внешняя база памяти** — Запись разговоров и данных в файловую или базу систему

 Шаг 10:

Миссия 10

🔒 Вы хотите предотвратить, чтобы агент выполнял вредоносный код через внешние API. Ваш подход?

Подсказка: Надёжность — проверять и фильтровать ввод.

Ответ:

- Не использовать API вообще
- **Валидация и фильтрация всех входных данных**
- Дать агенту полный доступ
- Игнорировать безопасность

🔮 Шаг 11:

Миссия 11

🔮 Агент должен понимать, когда остановиться, чтобы не уйти в бесконечный цикл. Что вы зададите?

Подсказка: Максимальное число шагов или итераций.

Ответ:

- **Ограничить число итераций**
- **Установить тайм-аут на цикл**
- Разрешить бесконечно
- Игнорировать цикл

🔮 Шаг 12:

Миссия 12

🕒 Типы остановок агента — сопоставьте с условиями:

Типы:

Ответ:

- По времени
- По шагам итерации
- По внешнему сигналу
- По достижению цели

Условия:

1. Программа получает стоп-сигнал от пользователя
2. Набрано предельное время исполнения
3. Достигнута нужная метрика или ответ
4. Количество шагов превысило лимит

🔮 **Подсказка: Каждый тип соответствует условию остановки.**

- **По времени** — Набрано предельное время исполнения
- **По шагам итерации** — Количество шагов превысило лимит
- **По внешнему сигналу** — Программа получает стоп-сигнал от пользователя
- **По достижению цели** — Достигнута нужная метрика или ответ

🔮 Шаг 13:

Миссия 13

🔍 Упорядочите стадии разработки агента для финального проекта:

1. Тестирование на краевых сценариях
2. Определение способностей агента (какие инструменты, память)
3. Интеграция модулей: подсказки, планирование, инструменты

4. Оценка и доработка поведения агента
5. Прототип и минимальный рабочий агент

💡 **Подсказка: Сначала план, потом прототип, затем интеграция, тесты и улучшения.**

Ответ:

- Определение способностей агента (какие инструменты, память)
- Прототип и минимальный рабочий агент
- Интеграция модулей: подсказки, планирование, инструменты
- Тестирование на краевых сценариях
- Оценка и доработка поведения агента

💡 **Шаг 14:**

Миссия 14

🕒 На финальном этапе агент дал неожиданный неправильный ответ пользователю. Что вы делаете?

Подсказка: Логируйте контекст, шаги, сравнивайте с эталоном.

Ответ:

- **Изучить логи шага, что агент сделал**
- **Сравнить с идеальным ответом и скорректировать правила**
- Игнорировать ошибку
- Удалить весь агент и начать заново

💡 **Шаг 15:**

Миссия 15

🔧 Расположите фазы финального проекта агента, завершив его:

1. Деплой агент-сервиса
2. Сбор отзывов пользователей
3. Мониторинг и логирование в продакшене
4. Обновление и исправления поведения

💡 **Подсказка: После запуска — слушать фидбэк, следить за работой, потом улучшать.**

Ответ:

- Деплой агент-сервиса
- Сбор отзывов пользователей
- Мониторинг и логирование в продакшене
- Обновление и исправления поведения

Урок 5 - 🏁 Финальный квест демонстрация и отчёт о производительности

💡 **Шаг 01:**

Миссия 1

📺 Вы готовите демонстрацию агента команде, и он ещё не умеет логировать результаты. Что вы сделаете первым?

Подсказка: Без логов не понять, где он ошибается.

Ответ:

- **Добавить базовое логирование (вход, выход)**
- Сразу оптимизировать модель
- Убрать все подсказки
- Запустить демонстрацию без изменений

Шаг 02:

Миссия 2

⇒ На репетиции агент даёт слишком длинный ответ. Как скорректировать это перед презентацией?

Подсказка: Можно задать стиль ответа или ограничить длину.

Ответ:

- **Установить limit_tokens**
- **Указать стиль «кратко»**
- Разрешить любой объём
- Отключить фильтрацию

Шаг 03:

Миссия 3

▮ Вы хотите показать статистику производительности (время ответа, ошибки). Что нужно сделать заранее?

Подсказка: Собирать метрики при тестировании.

Ответ:

- **Внедрить сбор метрик во всех сценариях**
- Просто считать по визуальному впечатлению
- Собирать метрики лишь на продакшене
- Не показывать статистику вовсе

Шаг 04:

Миссия 4

🔍 Вы хотите убедиться, что агент стабильно работает на разных входах. Что подготовите?

Подсказка: Краевые кейсы важны.

Ответ:

- **Набор тестовых сценариев разных длин**
- **Негативные примеры (неожиданные запросы)**
- Только типичные запросы
- Игнорировать ошибки

Шаг 05:

Миссия 5

🔗 Сопоставьте метрику и её смысл в отчёте агента:

Метрики:

Ответ:

- Latency

- Throughput
- Ошибка (Error rate)
- Покрытие сценариев (Coverage)

Смысл:

1. Процент случаев, где агент даёт неверный ответ
2. Сколько запросов в секунду агент обрабатывает
3. Медленность отклика (время ответа)
4. Доля тестовых сценариев, где агент был протестирован

🔗 **Подсказка: Latency — это задержка, throughput — пропускная способность, error rate — ошибки.**

- **Latency** — Медленность отклика (время ответа)
- **Throughput** — Сколько запросов в секунду агент обрабатывает
- **Ошибка (Error rate)** — Процент случаев, где агент даёт неверный ответ
- **Покрытие сценариев (Coverage)** — Доля тестовых сценариев, где агент был протестирован

🔗 Шаг 06:

Миссия 6

Сопоставьте формат метрики и способ её представления в отчёте:

Форматы:

Ответ:

- График во времени
- Гистограмма распределения
- Таблица по сценариям
- Процентное значение

Способы:

1. Показывает, как latency менялась с течением времени
2. Сравнение error rate по сценариям в месяцах
3. Разброс времени ответа на разные запросы
4. Процент успешных запросов из общего числа

🔗 **Подсказка: График — временная линия, гистограмма — распределение, таблица — сценарии, процент — доля.**

- **График во времени** — Показывает, как latency менялась с течением времени
- **Гистограмма распределения** — Разброс времени ответа на разные запросы
- **Таблица по сценариям** — Сравнение error rate по сценариям в месяцах
- **Процентное значение** — Процент успешных запросов из общего числа

🔗 Шаг 07:

Миссия 7

📄 При подготовке отчёта вы хотите подчеркнуть надёжность агента. Какой акцент сделаете?

Подсказка: Надёжность — низкая ошибка + стабильность.

Ответ:

- **Показать error rate и разброс latency**

- Показать только лучший кейс
- Показать только средний ответ
- Не показывать метрики

🔗 Шаг 08:

Миссия 8

✳ Вы хотите показать эволюцию агента: как менялись метрики после улучшения. Что включить в отчёт?

Подсказка: Сравнение до и после.

Ответ:

- **Показать метрики до изменений**
- **Показать те же метрики после изменений**
- Показать только текущие метрики
- Убрать исторические данные

🔗 Шаг 09:

Миссия 9

📊 Сопоставьте раздел отчёта и пример содержимого:

Разделы:

Ответ:

- Метрики производительности
- Ошибка и исключения
- Сценарии тестирования
- Рекомендации по улучшению

Примеры:

1. Среднее время ответа, 95-й процентиль
2. Список кейсов, где агент выдавал некорректный ответ или падал
3. Набор входов, на которых тестировали (краевые, типичные)
4. Что сделать дальше: оптимизация, новые инструменты

🔗 **Подсказка: Каждый раздел — свой тип информации.**

- **Метрики производительности** — Среднее время ответа, 95-й процентиль
- **Ошибка и исключения** — Список кейсов, где агент выдавал некорректный ответ или падал
- **Сценарии тестирования** — Набор входов, на которых тестировали (краевые, типичные)
- **Рекомендации по улучшению** — Что сделать дальше: оптимизация, новые инструменты

🔗 Шаг 10:

Миссия 10

Сопоставьте формат отчёта и преимущество для аудитории:

Форматы:

Ответ:

- Интерактивный дашборд
- PDF-отчёт
- Слайды презентации

- Веб-приложение отчёта

Преимущества:

1. Данные можно фильтровать и смотреть динамику
2. Удобно распечатать и сохранить офлайн
3. Краткий визуальный обзор для команды
4. Доступ через браузер, без установки ПО

💡 **Подсказка: Каждый формат под свою аудиторию.**

- **Интерактивный дашборд** — Данные можно фильтровать и смотреть динамику
- **PDF-отчёт** — Удобно распечатать и сохранить офлайн
- **Слайды презентации** — Краткий визуальный обзор для команды
- **Веб-приложение отчёта** — Доступ через браузер, без установки ПО

💡 **Шаг 11:**

Миссия 11

🔑 Вы хотите продемонстрировать агенту в боевых условиях: что обязательное условие для этого шага?

Подсказка: Нужно обеспечить стабильность и контроль над ошибками.

Ответ:

- **Иметь защиту от необработанных ошибок**
- Запустить без логов
- Дать пользователям полный контроль
- Отключить мониторинг

💡 **Шаг 12:**

Миссия 12

🔔 Вы хотите получать уведомления, если агент начинает вести себя плохо. Что настроите?

Подсказка: Триггеры на аномалии, ошибки, пороговые значения.

Ответ:

- **Алерт при error rate выше порога**
- **Алерт при latency превышении порога**
- Алерт при каждом запросе
- Не настраивать уведомления

💡 **Шаг 13:**

Миссия 13

📋 Упорядочите шаги подготовки финального отчёта:

1. Анализ собранных метрик
2. Визуализация (графики, таблицы) Формулирование выводов и рекомендаций
3. Составление итогового документа (презентация / отчёт)

💡 **Подсказка: Сначала анализ, затем визуализация, потом выводы и оформление.**

Ответ:

- Анализ собранных метрик
- Визуализация (графики, таблицы)

- Формулирование выводов и рекомендаций
- Составление итогового документа (презентация / отчёт)

Шаг 14:

Миссия 14

 Упорядочите этапы проведения демонстрации агента перед заказчиками:

1. Запуск агента на заранее подготовленных сценариях
2. Краткое введение и пояснение возможностей
3. Ответы на живые вопросы от пользователей
4. Обзор статистики и показ метрик

 Подсказка: Сначала ввод, затем демонстрация, потом метрики и вопросы.

Ответ:

- Краткое введение и пояснение возможностей
- Запуск агента на заранее подготовленных сценариях
- Обзор статистики и показ метрик
- Ответы на живые вопросы от пользователей

Шаг 15:

Миссия 15

 Упорядочите фазы пост-демонстрационной работы агента:

1. Сбор отзывов пользователей
2. Фиксация проблем и багов
3. Приоритизация улучшений и план задач
4. Внедрение исправлений и обновление агента

 Подсказка: После демонстрации — отзывы, баги, план, правки.

Ответ:

- Сбор отзывов пользователей
- Фиксация проблем и багов
- Приоритизация улучшений и план задач
- Внедрение исправлений и обновление агента

Шаг 16:

Миссия 16

 Упорядочите шаги мониторинга агента в продакшене:

1. Сбор логов и метрик
2. Настройка дашбордов и алертов
3. Анализ аномалий и предупреждений
4. Реакция: исправления, перезапуск или откат

 Подсказка: Мониторинг — сбор, визуализация, анализ, реакция.

Ответ:

- Сбор логов и метрик
- Настройка дашбордов и алертов
- Анализ аномалий и предупреждений

- Реакция: исправления, перезапуск или откат

Шаг 17:

Миссия 17

🗒 Упорядочите полный жизненный цикл агента после запуска:

1. Мониторинг и логирование
2. Сбор обратной связи и отчётов пользователей
3. Анализ метрик и поведение агента
4. Планирование и внедрение улучшений

 Подсказка: После запуска — следим, слушаем, анализируем, улучшаем.

Ответ:

- Мониторинг и логирование
- Сбор обратной связи и отчётов пользователей
- Анализ метрик и поведение агента
- Планирование и внедрение улучшений

Модуль 12 - Миссия 12. Легенда инженера Ollama

Урок 1 - Рефлексия что ты понял о моделях и интеллекте

Шаг 01:

Миссия 1

Ты — инженер Ollama, и перед тобой стоит вопрос: что из перечисленного лучше всего характеризует «интеллект» модели?

Подсказка: Интеллект — не просто вывод, а способность к обобщению.

Ответ:

- **Умение модели обобщать на данные, которые она не видела**
- Модель просто запоминает набор ответов
- Интеллект значит быстро выдавать ответ
- Интеллект = количество слоёв нейросети

Шаг 02:

Миссия 2

Ты наткнулся на спор: «LLM — это просто статистическая машина». Что ты, как инженер Ollama, выберешь?

Подсказка: Статистика важна, но важны ещё механизмы обучения и представления.

Ответ:

- **LLM** — это статистика + нейронная генерация + внутренние представления
- **LLM** — лишь текстовый шаблон без смысла
- LLM полностью осознанная система
- **LLM** — просто словарь ответов

Шаг 03:

Миссия 3

Твоя команда спрашивает: «Когда модель «понимает» текст?». Как ты отвечаешь?

Подсказка: Понимание — не буквальный перевод, а внутренняя обработка смысла.

Ответ:

- **Когда внутренняя репрезентация текста сохраняет семантические связи**
- Когда модель может переводить на все языки
- Когда модель отвечает быстро
- Когда модель показывает «я знаю»

🔮 Шаг 04:

Миссия 4

Тебе предлагают: «Добавим ещё слоёв, и модель «ума» прибавится». Что ты думаешь?

Подсказка: Большее количество слоёв — не панацея; важно качество обучения и архитектура.

Ответ:

- **Увеличение слоёв может помочь, но без правильной оптимизации смысла не даст**
- Больше слоёв решит все задачи сразу
- Слои не влияют на интеллект
- Нужно малое число слоёв, но огромные веса

🔮 Шаг 05:

Миссия 5

Во время деплоя команда спорит: «Надо ли ограничивать генерацию модели жесткими правилами?». Как ты отвечаешь?

Подсказка: Ограничения важны, но переограничение убивает креативность.

Ответ:

- **Нужны мягкие рамки и защитные правила, но не жёсткие шаблоны**
- Лучше вообще не ставить ограничений
- **Жёсткие правила** — единственный путь к контролю
- Ограничения не влияют на качество генерации

🔮 Шаг 06:

Миссия 6

Задача: сопоставь термин и определение.

Термины:

Ответ:

- Параметры
- Представления (representations)
- Обучение с учителем
- Самонаблюдение (self-supervision)

Определения:

1. Обучение, где модель сама извлекает сигналы из данных без разметки.
2. Векторные эмбединги, в которых модель кодирует смысл входа.
3. **Обучение по размеченным парам «вход — выход».**
4. Настройка весов нейросети, её «параметров».

Подсказка: Параметры — это числа внутри модели, представления — как модель «видит» смысл.

- **Параметры** — Настройка весов нейросети, её «параметров».
- **Представления (representations)** — Векторные эмбединги, в которых модель кодирует смысл входа.
- **Обучение с учителем** — Обучение по размеченным парам «вход—выход».
- **Самонаблюдение (self-supervision)** — Обучение, где модель сама извлекает сигналы из данных без разметки.

🧙 Шаг 07:

Миссия 7

Сопоставь стадии разработки LLM и их описания:

Стадии:

Ответ:

- Предтренировка
- Тонкая настройка (fine-tuning)
- Инференс
- Дистилляция

Описания:

1. Когда обучают модель на большом корпусе без конкретной задачи.
2. Когда модель используется для генерации ответа на запрос.
3. Когда обучают модель на специфичных задачах после базовой тренировки.
4. Когда делают уменьшенную версию модели с сохранением знаний.

Подсказка: Подумай о жизненном цикле модели: сначала обучение на корпусе, потом доработка, потом применение или сжатие.

- **Предтренировка** — Когда обучают модель на большом корпусе без конкретной задачи.
- **Тонкая настройка (fine-tuning)** — Когда обучают модель на специфичных задачах после базовой тренировки.
- **Инференс** — Когда модель используется для генерации ответа на запрос.
- **Дистилляция** — Когда делают уменьшенную версию модели с сохранением знаний.

🧙 Шаг 08:

Миссия 8

Сопоставь вызовы LLM и способы их решения:

Вызовы:

Ответ:

- Генерация токсичного текста
- Срыв контекста (forgetting)
- Переобучение на частных данных
- Низкая эффективность

Способы решения:

1. Regularization, штрафы, ограничение длины обучения.
2. Методы модерации, фильтры, детекторы.
3. Кэширование, оптимизация вычислений, сжатие веса.
4. Периодическое повторное напоминание, техники памяти.

Подсказка: Вспомни: как предотвращают токсичность, как удерживают контекст — методы модерации, память и регуляризация.

- **Генерация токсичного текста** — Методы модерации, фильтры, детекторы.
- **Срыв контекста (forgetting)** — Периодическое повторное напоминание, техники памяти.
- **Переобучение на частных данных** — Regularization, штрафы, ограничение длины обучения.
- **Низкая эффективность** — Кэширование, оптимизация вычислений, сжатие веса.

🔮 Шаг 09:

Миссия 9

От ранних к поздним: расположи этапы жизненного цикла модели LLM:

1. Инференс
2. Обучение с учителем (fine-tuning)
3. Предтренировка
4. Дистилляция

Подсказка: Сначала базовое обучение, потом доработка, потом применение или сжатие.

Ответ:

- Предтренировка
- Обучение с учителем (fine-tuning)
- Инференс
- Дистилляция

🔮 Шаг 10:

Миссия 10

Упорядочи шаги при решении проблемы токсичности в модели:

1. Добавление фильтра на выходной текст
2. Сбор и аннотация токсичного примера
3. Тонкая доработка модели на безопасных данных
4. Мониторинг и сбор обратной связи после деплоя

Подсказка: Сначала надо собрать данные, затем доработать, добавить фильтр и только потом мониторить.

Ответ:

- Сбор и аннотация токсичного примера
- Тонкая доработка модели на безопасных данных
- Добавление фильтра на выходной текст
- Мониторинг и сбор обратной связи после деплоя

🔮 Шаг 11:

Миссия 11

Расположи методы повышения эффективности LLM от простых до сложных:

1. Бинарные веса или квантование
2. Кэширование промежуточных слоёв
3. Прореживание (pruning)
4. Компиляция графов / оптимизированные ядра

Подсказка: Сначала лёгкие трюки, затем структурные оптимизации, затем системные улучшения.

Ответ:

- Кэширование промежуточных слоёв
- Бинарные веса или квантование
- Прореживание (pruning)
- Компиляция графов / оптимизированные ядра

Шаг 12:

Миссия 12

Упорядочи действия инженера Ollama при рефлексии над моделью:

1. Проанализировать ошибки и неожиданные генерации
2. Сравнить поведение модели с ожидаемым интеллектом
3. Сформулировать гипотезы улучшения
4. Планировать эксперименты и доработку

Подсказка: Рефлексия начинается с анализа, затем оценка, гипотезы и план.

Ответ:

- Проанализировать ошибки и неожиданные генерации
- Сравнить поведение модели с ожидаемым интеллектом
- Сформулировать гипотезы улучшения
- Планировать эксперименты и доработку

Шаг 13:

Миссия 13

Упорядочи этапы рефлексивного ревью команды LLM-инженеров:

1. Сбор логов, запросов, метрик
2. Совместное обсуждение неожиданных генераций
3. Запись инсайтов и решений
4. Распределение задач по улучшению

Подсказка: Логика: сначала собрать данные, потом обсудить, записать и распределить работу.

Ответ:

- Сбор логов, запросов, метрик
- Совместное обсуждение неожиданных генераций
- Запись инсайтов и решений
- Распределение задач по улучшению

Шаг 14:

Миссия 14

Во время ретроспективы ты спрашиваешь у команды: «Что нас удивило в работе модели?» Какой ответ отражает зрелое понимание?

Подсказка: Удивление часто рождается там, где модель выходит за рамки ожидания.

Ответ:

- **Когда модель делает логичные связи, которых не было в обучении**
- Когда модель даёт всегда однотипный ответ

- Когда она полностью повторяет вход
- Когда она не отвечает вообще

🌀 Шаг 15:

Миссия 15

На завершающем этапе ты думаешь: «Что важно запомнить о модели?» Какой тезис ты выберешь?

Подсказка: Рефлексия подразумевает обобщение опыта.

Ответ:

- **Модель** — это инструмент с ограничениями, и её стоит постоянно изучать
- Модель работает идеально после одной доработки
- **Модель** — это магия без объяснения
- Модель заменит инженера сразу

🌀 Шаг 16:

Миссия 16

Сопоставь высказывание инженера и его смысл:

Высказывания:

Ответ:

- «Она удивила меня нестандартной ассоциацией»
- «Она забыла часть контекста»
- «Она сгенерировала токсичное выражение»
- «Её ответ слишком предсказуем»

Смыслы:

1. Недостаток вариативности, низкая творческая свобода
2. Контекстный сдвиг, потеря длинной зависимости
3. Нарушение безопасности, угроза доверию
4. Нестандартное обобщение, неожиданное мышление

Подсказка: Каждое высказывание — это отражение одного из вызовов моделей.

- **«Она удивила меня нестандартной ассоциацией»** — Нестандартное обобщение, неожиданное мышление
- **«Она забыла часть контекста»** — Контекстный сдвиг, потеря длинной зависимости
- **«Она сгенерировала токсичное выражение»** — Нарушение безопасности, угроза доверию
- **«Её ответ слишком предсказуем»** — Недостаток вариативности, низкая творческая свобода

🌀 Шаг 17:

Миссия 17

Итоговый порядок: упорядочи стадии рефлексии инженера над интеллектом модели:

1. Описание неожиданностей и ошибок
2. Оценка, соответствие ожиданиям
3. Генерация гипотез улучшения
4. Планирование экспериментов и доработок

Подсказка: Начни с фактов, затем оценка, идеи, план.

Ответ:

- Описание неожиданностей и ошибок
- Оценка, соответствие ожиданиям
- Генерация гипотез улучшения
- Планирование экспериментов и доработок

Урок 2 - Сравнение Ollama, LocalAI и OpenAI API

Шаг 01:

Миссия 1

Вы — инженер Ollama и выбираете backend для локального LLM. Что является преимуществом использования Ollama локально по сравнению с OpenAI API?

Подсказка: подумайте о сетевых задержках и приватности данных.

Ответ:

- **Минимальные задержки и отсутствие внешней передачи данных**
- Более широкий ассортимент моделей, чем у OpenAI
- Гарантия 100 % совпадения ответов с OpenAI

Шаг 02:

Миссия 2

 Вы решили оценить LocalAI. Какой из этих сценариев наиболее подходит для LocalAI?

Подсказка: подумайте о ресурсах и автономности.

Ответ:

- **Запуск LLM на своём сервере без внешнего API**
- Использование облачной инфраструктуры OpenAI
- Сервисы без необходимости локального оборудования

Шаг 03:

Миссия 3

 При интеграции с OpenAI API вы замечаете счёт за токены. Какой из пунктов наиболее прямо влияет на стоимость запроса?

Подсказка: подумайте, что именно OpenAI считает для тарификации.

Ответ:

- **Количество входных + выходных токенов**
- Количество HTTP-запросов в секунду
- Количество разных endpoint'ов

Шаг 04:

Миссия 4

 Инженер команды хочет, чтобы модель быстро переключалась между версиями в локальной среде. Что для этого наиболее важно?

Подсказка: подумайте о совместимости форматов моделей.

Ответ:

- **Поддержка форматов моделей (например, GGML, PyTorch)**
- Автоматическое масштабирование серверов
- Интеграция с UI-фреймворками

Шаг 05:

Миссия 5

 При локальной работе с Ollama вас беспокоит безопасность. Что из следующего — ключевой плюс локального решения?

Подсказка: подумайте, куда не уезжают данные.

Ответ:

- **Данные никогда не покидают вашу машину**
- Открытый исходный код API OpenAI
- Автоматическое шифрование всех токенов на стороне OpenAI

Шаг 06:

Миссия 6

 Сопоставьте платформу и её характерное ограничение:

Платформы:

Ответ:

- Ollama (локально)
- LocalAI
- OpenAI API

Ограничения:

1. Зависимость от интернета и внешних API
2. Необходимость сильного локального оборудования
3. Меньшая экосистема плагинов по сравнению с OpenAI

Подсказка: думайте, какая платформа требует интернет, какая — ресурсы, а какая — экосистему.

- **Ollama (локально)** — Меньшая экосистема плагинов по сравнению с OpenAI
- **LocalAI** — Необходимость сильного локального оборудования
- **OpenAI API** — Зависимость от интернета и внешних API

Шаг 07:

Миссия 7

 Сопоставьте критерий сравнения и его влияние:

Критерии:

Ответ:

- Задержка (latency)
- Стоимость
- Приватность

Влияния:

1. Определяет, насколько быстро ответ приходит к пользователю
2. Связано с оплатой токенов или ресурсов
3. Контроль над тем, куда уходят данные

Подсказка: сопоставьте каждое свойство с его прямым эффектом.

- **Задержка (latency)** — Определяет, насколько быстро ответ приходит к пользователю
- **Стоимость** — Связано с оплатой токенов или ресурсов
- **Приватность** — Контроль над тем, куда уходят данные

🔮 Шаг 08:

Миссия 8

🔍 Сопоставьте этап жизненного цикла запроса LLM и потенциальную точку сравнения:

Этапы запроса:

Ответ:

- Формирование prompt
- Обработка моделью (внутреннее вычисление)
- Постобработка результата

Сравнение может затрагивать:

1. Скорость вычисления
2. Гибкость шаблонов prompt
3. Фильтрация и дополнения результата

Подсказка: подумайте, на каком этапе что имеет значение.

- **Формирование prompt** — Гибкость шаблонов prompt
- **Обработка моделью (внутреннее вычисление)** — Скорость вычисления
- **Постобработка результата** — Фильтрация и дополнения результата

🔮 Шаг 09:

Миссия 9

🏠 Расположите по возрастанию задержку (от наименьшей к наибольшей) для следующих вариантов:

Ответ:

- Запрос к LocalAI на том же сервере
- Запрос к Ollama локально
- Запрос к OpenAI API через интернет

Подсказка: внутри машины — быстрее, через сеть — медленнее.

- Запрос к LocalAI на том же сервере
- Запрос к Ollama локально
- Запрос к OpenAI API через интернет

🔮 Шаг 10:

Миссия 10

📊 Расположите по возрастанию общую стоимость (от дешёвого к дорогому) при активном использовании:

Ответ:

- OpenAI API с высоким трафиком
- Ollama на локальной машине (учитывая эксплуатацию)
- LocalAI на собственном сервере

Подсказка: подумайте о постоянных расходах и о плате за токены.

- Ollama на локальной машине (учитывая эксплуатацию)
- LocalAI на собственном сервере
- OpenAI API с высоким трафиком

🔮 Шаг 11:

Миссия 11

🏠 Расставьте по степени безопасности (от наиболее защищённого к менее защищённому):

Ответ:

- Ollama локально, в изолированной сети
- LocalAI на арендованном сервере без шифрования
- OpenAI API через публичную сеть

Подсказка: чем меньше внешних точек доступа, тем безопаснее.

- Ollama локально, в изолированной сети
- LocalAI на арендованном сервере без шифрования
- OpenAI API через публичную сеть

🔮 Шаг 12:

Миссия 12

📖 Вы разрабатываете учебный продукт, где требуется объединение ответов из двух моделей — локальной (Ollama) и облачной (OpenAI). Какой подход логичнее всего?

Подсказка: можно задействовать оба ресурса для баланса качества и скорости.

Ответ:

- **Гибридный режим: сначала локально, затем дообработка через OpenAI**
- Только OpenAI, чтобы не усложнять
- Только Ollama и отказаться от облака полностью

🔮 Шаг 13:

Миссия 13

👂 В тестовой среде вы сравниваете ответы моделей. Какой критерий НЕ стоит учитывать при сравнении Ollama и OpenAI?

Подсказка: критерии должны быть объективными и технически измеримыми.

Ответ:

- **Как часто модель слушает музыку при генерации ответов**
- Семантическое качество ответа
- Время генерации ответа

🔧 Шаг 14:

Миссия 14

📦 Вы хотите подключить plug-in библиотеку к Ollama, но библиотека рассчитана на OpenAI. Что нужно проверить в первую очередь?

Подсказка: подумайте о форматах и протоколах.

Ответ:

- **Совместимость API-контракта и формат запросов**
- Лицензия библиотеки
- Только размер библиотеки

🔧 Шаг 15:

Миссия 15

🔗 Сопоставьте следующие слова с их значением в контексте интеграции LLM-движков:

Слова:

Ответ:

- Adapter
- Backend
- Endpoint

Значения:

1. Интерфейс, по которому отправляется запрос к модели
2. Компонент, реализующий выполнение модели (локально / удалённо)
3. Промежуточник, позволяющий подключать разные реализации моделей

Подсказка: адаптер — мост, backend — исполнитель, endpoint — точка вызова.

- **Adapter** — Промежуточник, позволяющий подключать разные реализации моделей
- **Backend** — Компонент, реализующий выполнение модели (локально / удалённо)
- **Endpoint** — Интерфейс, по которому отправляется запрос к модели

🔧 Шаг 16:

Миссия 16

🔗 У вас есть три варианта развертывания системы с двумя моделями (Ollama локально, OpenAI облако). Расположите по политике переключения (от простой к сложной):

Ответ:

- Всегда использовать локальную модель, если доступна
- **Сначала пытаться локально, при ошибке** — fallback в облако
- Параллельно запускать обе модели и агрегировать результат

Подсказка: агрегирование требует больше логики.

- Всегда использовать локальную модель, если доступна
- **Сначала пытаться локально, при ошибке** — fallback в облако
- Параллельно запускать обе модели и агрегировать результат

🔧 Шаг 17:

Миссия 17

🔍 Завершающее испытание: расставьте этапы оценки качества гибридной системы (Ollama + OpenAI):

Ответ:

- Сбор метрик задержки и качества
- A/B-тестирование на пользователях
- Сравнение ответов моделей отдельно и в ансамбле
- Выбор стратегии переключения или агрегации

Подсказка: сначала анализ локальных результатов, потом тесты, и затем стратегия.

- Сравнение ответов моделей отдельно и в ансамбле
- Сбор метрик задержки и качества
- A/B-тестирование на пользователях
- Выбор стратегии переключения или агрегации

📖 Урок 3 - 🏠 Анализ производительности собственной сборки

💡 Шаг 01:

Миссия 1

При запуске собственной сборки Ollama вы замечаете, что модель отвечает медленнее, чем ожидалось. Что вы проверите в первую очередь?

Подсказка: Ищите узкое место на входе — чтение данных или загрузка модели.

Ответ:

- **Скорость загрузки модели или бинарных весов**
- Количество слоёв в модели
- Качество токенизации
- Стилль вывода текста

💡 Шаг 02:

Миссия 2

🔍 Ваша сборка работает быстрее на GPU, чем на CPU, но вы хотите понять накладные расходы. Что из этого вы измерите?

Подсказка: Ищите время ожидания ввода-вывода или синхронизацию.

Ответ:

- **Ожидание передачи данных на устройство**
- Число параметров модели
- Число токенов в обучении
- Количество слоёв активации

💡 Шаг 03:

Миссия 3

🔍 Во время теста вы измеряете пропускную способность запросов (throughput). Что это означает?

Подсказка: Представьте, сколько запросов система может обслужить за единицу времени.

Ответ:

- **Число запросов в секунду, которые система может обработать**
- Время ответа на один запрос

- Общее время обучения модели
- Число токенов, выдаваемых в минуту

🔮 Шаг 04:

Миссия 4

📖 Вы измеряете латентность (latency). Что лучше: меньше или больше?

Подсказка: Латентность — это задержка между вводом и ответом.

Ответ:

- **Меньше** — лучше
- **Больше** — лучше
- Неважно
- Должна равняться throughput

🔮 Шаг 05:

Миссия 5

Сопоставьте метрики и их описание в контексте Ollama-движка:

Метрика:

Ответ:

- Throughput (пропускная способность)
- Latency (латентность)
- GPU Utilization (использование GPU)
- IO Wait (ожидание ввода-вывода)

Описание:

1. Процент времени, когда GPU активно вычисляет
2. Время задержки ответа на индивидуальный запрос
3. Количество обработанных запросов в секунду
4. Время, когда процесс ждёт ввода-вывода

Подсказка: Throughput — мощность, latency — время, utilisation — загрузка, IO wait — ожидание диска/сети.

- **Throughput (пропускная способность)** — Количество обработанных запросов в секунду
- **Latency (латентность)** — Время задержки ответа на индивидуальный запрос
- **GPU Utilization (использование GPU)** — Процент времени, когда GPU активно вычисляет
- **IO Wait (ожидание ввода-вывода)** — Время, когда процесс ждёт ввода-вывода

🔮 Шаг 06:

Миссия 6

🔍 Сопоставьте сценарии замедления и их причины:

Сценарий:

Ответ:

- Низкий throughput при малом latency
- Высокая latency и низкая загрузка GPU
- GPU полностью загружен, throughput низкий
- Много IO wait

Причина:

1. Узел ввода-вывода ограничивает скорость
2. Модель проста, CPU блокирует загрузку GPU
3. Неграмотное управление пакетами запросов (overhead)
4. Модель выходит за пределы GPU памяти, идёт свопинг

Подсказка: Как связаны загрузка GPU, ожидания и очереди запросов?

- **Низкий throughput при малом latency** — Неграмотное управление пакетами запросов (overhead)
- **Высокая latency и низкая загрузка GPU** — Модель проста, CPU блокирует загрузку GPU
- **GPU полностью загружен, throughput низкий** — Модель выходит за пределы GPU памяти, идёт свопинг
- **Много IO wait** — Узел ввода-вывода ограничивает скорость

Шаг 07:

Миссия 7

🔧 Сопоставьте методы оптимизации и их эффекты:

Метод:

Ответ:

- Квантизация
- Прюнинг (pruning)
- Пакетная обработка (batching)
- Кэширование результатов

Эффект:

1. Снижение точности ради скорости
2. Удаление незначимых весов, уменьшение размеров модели
3. Обработка сразу группы запросов вместе
4. Ускорение повторяющихся вычислений

Подсказка: Подумайте, что делает каждый способ: уменьшает размер, ускоряет повторные вызовы, группирует запросы.

- **Квантизация** — Снижение точности ради скорости
- **Прюнинг (pruning)** — Удаление незначимых весов, уменьшение размеров модели
- **Пакетная обработка (batching)** — Обработка сразу группы запросов вместе
- **Кэширование результатов** — Ускорение повторяющихся вычислений

Шаг 08:

Миссия 8

📋 Упорядочьте шаги проведения нагрузочного теста Ollama-сервера:

1. Произвести нагрузку запросов в течение заданного времени
2. Собирать метрики (latency, throughput, использование)
3. Запустить сервер в измерительном режиме
4. Проанализировать полученные результаты

Подсказка: Сначала запускаете, затем создаёте нагрузку, затем собираете данные и наконец анализируете.

Ответ:

- Запустить сервер в измерительном режиме
- Произвести нагрузку запросов в течение заданного времени
- Собирать метрики (latency, throughput, использование)
- Проанализировать полученные результаты

 Шаг 09:

Миссия 9

✂ При оптимизации — правильный порядок применений методов. Упорядочьте эти методы:

1. Квантизация модели
2. Пакетирование запросов (batching)
3. Кэширование промежуточных состояний
4. Разделение на потоки и асинхронность

Подсказка: Начните с базовой оптимизации (batching), затем идите к более тонким приёмам.

Ответ:

- Пакетирование запросов (batching)
- Квантизация модели
- Кэширование промежуточных состояний
- Разделение на потоки и асинхронность

 Шаг 10:

Миссия 10

🔍 Упорядочите этапы профилирования исполнения модели:

1. Запустить профайлер (например, nvprof или аналог)
2. Собрать трассировки и логи
3. Проанализировать горячие точки (bottlenecks)
4. Применить оптимизации и повторить цикл

Подсказка: Профилирование — итеративный процесс: измеряйте, анализируйте, улучшайте.

Ответ:

- Запустить профайлер (например, nvprof или аналог)
- Собрать трассировки и логи
- Проанализировать горячие точки (bottlenecks)
- Применить оптимизации и повторить цикл

 Шаг 11:

Миссия 11

🔍 Вы хотите измерить нагрузку сети (bandwidth) между сервером и клиентом при работе Ollama. Какой подход выберете?

Подсказка: Замеряйте объём переданных байт и время.

Ответ:

- **Считать байты, переданные по socket, и время в пути**
- Измерить количество токенов в ответе
- Подсчитать число вызовов API

- Измерить CPU utilization

🔮 Шаг 12:

Миссия 12

🔗 Вы перешли на версию сборки с прюнингом. Что может измениться?

Подсказка: Уменьшение весов часто снижает нагрузку, но может повлиять на качество.

Ответ:

- **Меньше память, выше throughput, чуть выше ошибки**
- Увеличится latency без gain
- Качество ответа полностью сохранится
- Модель перестанет работать

🔮 Шаг 13:

Миссия 13

📖 Сопоставьте методы профилирования и инструменты:

Метод:

Ответ:

- Трассировка CUDA / GPU
- CPU-профилирование (время функций)
- Сетевое профилирование
- IO-профилирование диска/сети

Инструмент:

1. nethogs, iftop
2. nvprof, Nsight Systems
3. strace, iostat
4. cProfile, perf

Подсказка: GPU — nvprof, CPU — perf/cProfile, IO — strace/iostat, сеть — iftop.

- **Трассировка CUDA / GPU** — nvprof, Nsight Systems
- **CPU-профилирование (время функций)** — cProfile, perf
- **Сетевое профилирование** — nethogs, iftop
- **IO-профилирование диска/сети** — strace, iostat

🔮 Шаг 14:

Миссия 14

🔗 Сопоставьте оптимизации с потенциальными негативными эффектами:

Оптимизация:

Ответ:

- Квантизация до 8 бит
- Сильный прюнинг
- Большие батчи (batch sizes)
- Многопоточность / асинхронность

Негативный эффект:

1. Ухудшение качества ответов (шум, ошибки)

2. Потеря важных весов, деградация модели
3. Увеличение latency из-за ожидания полного батча
4. Сложность синхронизации, гонки данных

Подсказка: Оптимизация — это компромисс: скорость против точности или сложности.

- **Квантизация до 8 бит** — Ухудшение качества ответов (шум, ошибки)
- **Сильный прюнинг** — Потеря важных весов, деградация модели
- **Большие батчи (batch sizes)** — Увеличение latency из-за ожидания полного батча
- **Многопоточность / асинхронность** — Сложность синхронизации, гонки данных

Шаг 15:

Миссия 15

 Упорядочите фазы итеративной оптимизации модели Ollama:

1. Сбор базовых метрик производительности
2. Применение оптимизации (квантизация, прюнинг и т.д.)
3. Ретестирование и сравнение метрик
4. Выбор лучших конфигураций и документирование

Подсказка: Итеративный цикл: измеряйте → оптимизируйте → измеряйте → решайте.

Ответ:

- Сбор базовых метрик производительности
- Применение оптимизации (квантизация, прюнинг и т.д.)
- Ретестирование и сравнение метрик
- Выбор лучших конфигураций и документирование

Шаг 16:

Миссия 16

 Финальный квест: расположите этапы полного анализа производительности вашей сборки Ollama:

1. Провести нагрузочный тест
2. Профилировать узкие места
3. Внедрить оптимизации (batching, квантизация и др.)
4. Сравнить метрики до и после и принять решение

Подсказка: Это объединение всех навыков — тестирование, профилирование, оптимизация и анализ.

Ответ:

- Провести нагрузочный тест
- Профилировать узкие места
- Внедрить оптимизации (batching, квантизация и др.)
- Сравнить метрики до и после и принять решение

Урок 4 - 🗨 Презентация проекта

Шаг 01:

Миссия 1

🔊 Вы начинаете готовить презентацию проекта Ollama для технической аудитории. С чего вы начнёте?

Подсказка: Важно сразу установить контекст для слушателей.

Ответ:

- **Кратко описать проблему и цель проекта**
- Погрузиться сразу в детали реализации
- Начать с благодарностей и титульного слайда
- Сразу показать метрики производительности

🔊 Шаг 02:

Миссия 2

📖 Вы готовите слайды с архитектурой системы Ollama. Какую диаграмму стоит показать первым делом?

Подсказка: Сначала дать общее представление.

Ответ:

- **Общая блок-диаграмма модулей**
- Диаграмма последовательностей
- Диаграмма классов
- Диаграмма развёртывания виртуальных машин

🔊 Шаг 03:

Миссия 3

📊 На слайде с результатами вы хотите показать рост производительности. Как лучше визуализировать это?

Подсказка: Важно показать тренд и сравнение.

Ответ:

- **Линейный график с базовой и оптимизированной версией**
- Пироговая диаграмма
- Таблица с цифрами без визуализации
- Кольцевая диаграмма

🔊 Шаг 04:

Миссия 4

На слайде с ограничениями проекта стоит указать риски. Что из этого должно быть включено?

Подсказка: Упомяните внешние зависимости, которые вы не контролируете.

Ответ:

- **Зависимость от пропускной способности сети клиента**
- Название переменной в коде
- Имя дизайнера логотипа
- Цветовую схему слайдов

🔊 Шаг 05:

Миссия 5

🧠 Сопоставьте элементы презентации и их назначение:

Элемент:

Ответ:

- Введение / контекст
- Архитектура системы
- Результаты тестов
- Риски и ограничения

Назначение:

1. Показать, как устроена система и её компоненты
2. Указать, какие внешние факторы могут мешать успеху
3. Дать фон: зачем вообще проект нужен
4. Поделиться эмпирическими данными о производительности

Подсказка: Контекст — «зачем», архитектура — «как», результаты — «что», риски — «чего бояться».

- **Введение / контекст** — Дать фон: зачем вообще проект нужен
- **Архитектура системы** — Показать, как устроена система и её компоненты
- **Результаты тестов** — Поделиться эмпирическими данными о производительности
- **Риски и ограничения** — Указать, какие внешние факторы могут мешать успеху

🔮 Шаг 06:

Миссия 6

📊 Сопоставьте тип визуализации и когда его применить:

Тип визуализации:

Ответ:

- Линейный график
- Столбчатая диаграмма
- Таблица с фактическими числами
- Диаграмма Ганта

Ситуация:

1. Для сравнения производительности разных версий
2. Для планирования этапов проекта
3. Когда нужно показать точные цифры и детали
4. Для отображения трендов по времени

Подсказка: Выбирайте визуализацию в зависимости от задачи — сравнение, план, детали или тренд.

- **Линейный график** — Для отображения трендов по времени
- **Столбчатая диаграмма** — Для сравнения производительности разных версий
- **Таблица с фактическими числами** — Когда нужно показать точные цифры и детали
- **Диаграмма Ганта** — Для планирования этапов проекта

🔮 Шаг 07:

Миссия 7

📋 Упорядочите шаги создания презентации проекта:

1. Собрать метрики и результаты экспериментов
2. Составить структуру и заголовки слайдов

3. Оформить визуализацию и графики
4. Проговорить и отрепетировать презентацию

Подсказка: Логика: от данных → структуры → визуального оформления → репетиции.

Ответ:

- Собрать метрики и результаты экспериментов
- Составить структуру и заголовки слайдов
- Оформить визуализацию и графики
- Проговорить и отрепетировать презентацию

Шаг 08:

Миссия 8

⚡ Упорядочите этапы анализа аудитории перед презентацией:

1. Определить уровень технической подготовки слушателей
2. Выяснить их ожидания и интересы
3. Подготовить ключевые месседжи
4. Подобрать формат и глубину деталей

Подсказка: Сначала узнать, кому вы будете презентовать, потом адаптировать содержание.

Ответ:

- Определить уровень технической подготовки слушателей
- Выяснить их ожидания и интересы
- Подготовить ключевые месседжи
- Подобрать формат и глубину деталей

Шаг 09:

Миссия 9

🗨 Во время выступления один слушатель прерывает вас с техническим вопросом, который уводит от темы. Как поступить?

Подсказка: Контролируйте поток, но покажите уважение к слушателю.

Ответ:

- **Вежливо предложить обсудить вопрос в конце (Q&A)**
- Игнорировать и продолжать
- Сразу углубиться в технический ответ
- Попросить слушателя уйти

Шаг 10:

Миссия 10

👏 В финале презентации вы хотите оставить слушателям «ключевую мысль». Какая формулировка лучше?

Подсказка: Сделайте мысль короткой, запоминающейся, связанной с ценностью проекта.

Ответ:

- **«С нашей оптимизацией Ollama работает вдвое быстрее при том же качестве»**
- «Спасибо за внимание»
- **«Этот проект — наше всё»**
- «Смотрите код на GitHub»

Шаг 11:

Миссия 11

 Сопоставьте части Q&A с правильной стратегией ответа:

Ситуация:

Ответ:

- Не знаете точный ответ
- Вопрос слишком сложный и технический
- Вопрос отвлекает от основной темы проекта
- Вопрос полезен и интересен аудитории

Стратегия ответа:

1. Предложить собрать информацию и ответить позже
2. Ответить коротко, а затем предложить углубление после презентации
3. Кратко связать вопрос с центральной темой и вернуться к плану
4. Поздравить, дать полноценный ответ и возможно задать встречный вопрос

Подсказка: Различайте, когда отвечать широко, а когда удерживать фокус.

- **Не знаете точный ответ** — Предложить собрать информацию и ответить позже
- **Вопрос слишком сложный и технический** — Ответить коротко, а затем предложить углубление после презентации
- **Вопрос отвлекает от основной темы проекта** — Кратко связать вопрос с центральной темой и вернуться к плану
- **Вопрос полезен и интересен аудитории** — Поздравить, дать полноценный ответ и возможно задать встречный вопрос

Шаг 12:

Миссия 12

 Сопоставьте элементы презентации и критерии оценки жюри стартап-презентации:

Элемент презентации:

Ответ:

- Проблема / мотивация
- Техническое решение
- Результаты / метрики
- План развития / roadmap

Критерий оценки:

1. Видно, почему это важно (pain)
2. Алгоритмы, детали, уникальность
3. Данные, цифры, доказательства эффективности
4. Что дальше и как масштабировать

Подсказка: Жюри хочет увидеть: *зачем*, *как*, *что получилось*, *куда идти дальше*.

- **Проблема / мотивация** — Видно, почему это важно (pain)
- **Техническое решение** — Алгоритмы, детали, уникальность
- **Результаты / метрики** — Данные, цифры, доказательства эффективности
- **План развития / roadmap** — Что дальше и как масштабировать

Шаг 13:

Миссия 13

▢ Упорядочите шаги подготовки визуализации графиков для презентации:

1. Выбрать данные, которые хотите показать
2. Выбрать тип графика, подходящий к задаче
3. Настроить оси, легенду, подписи
4. Проверить читаемость и эстетичность

Подсказка: Сначала данные → выбор формы → настройка → визуальное качество.

Ответ:

- Выбрать данные, которые хотите показать
- Выбрать тип графика, подходящий к задаче
- Настроить оси, легенду, подписи
- Проверить читаемость и эстетичность

Шаг 14:

Миссия 14

🕒 Упорядочите этапы «репетиции перед презентацией перед живой аудиторией»:

1. Прогон презентации вслух одному человеку (коллеге)
2. Собрать обратную связь (непонятные места, лишние слайды)
3. Внести правки и улучшения
4. Сделать ещё один прогон со временем и таймингом

Подсказка: Репетиции должны быть итеративны: показывать, слушать, править, проверять.

Ответ:

- Прогон презентации вслух одному человеку (коллеге)
- Собрать обратную связь (непонятные места, лишние слайды)
- Внести правки и улучшения
- Сделать ещё один прогон со временем и таймингом

Шаг 15:

Миссия 15

🗍 Финальный порядок этапов презентации проекта Ollama:

1. Открытие и вводная часть
2. Архитектура и технологические детали
3. Демонстрация результатов и метрик
4. Риски, планы, Q&A и заключение

Подсказка: Это логика «история»: что → как → что достигли → что дальше + вопросы.

Ответ:

- Открытие и вводная часть
- Архитектура и технологические детали
- Демонстрация результатов и метрик
- Риски, планы, Q&A и заключение

Урок 5 - 🧙 Получение звания «Архитектор LLM»

Шаг 01:

Миссия 1

Инженер Ольгама Алекс просматривает логи сервера и замечает, что модель неожиданно выдаёт неконсистентные ответы одному и тому же запросу. Что он должен сделать сначала?

Подсказка: Начинайте с самых простых проверок — часто проблема лежит на поверхности.

Ответ:

- **Проверить, используются ли одинаковые входные данные и конфигурации**
- Сразу переписать весь модуль ответа
- Добавить больше слоёв в архитектуру модели

Шаг 02:

Миссия 2

После проверки конфигурации Алекс замечает: при одинаковом prompt в разных потоках модели выходят слегка разные ответы. Что это может значить?

Подсказка: Подумайте про стохастичность и seeding.

Ответ:

- **Модель использует случайные семплы без фиксации seed**
- Память кэша сломана
- Неправильная версия модели загружена

Шаг 03:

Миссия 3

Алекс хочет обеспечить детерминированность ответа модели при фиксированном prompt. Что он может сделать?

Подсказка: Детерминированность часто связана с параметрами генерации.

Ответ:

- **Задать фиксированный seed генератора**
- **Установить temperature = 0**
- Увеличить размер батча до максимума
- Включить использование dropout при генерации

Шаг 04:

Миссия 4

Алекс замечает, что модель слишком часто выдаёт повторяющийся текст. Как лучше всего это исправить?

Подсказка: Обратите внимание на параметры «repetition penalty» или «frequency penalty».

Ответ:

- **Увеличить repetition / frequency penalty**
- Добавить больше слоёв трансформера
- Уменьшить длину контекста

🔮 Шаг 05:

Миссия 5

В диалоге с моделью Алекс хочет контролировать тональность — например, чтобы ответы были формальными, не дружескими. Как включить это требование?

Подсказка: Можно влиять на стиль через промпты или системные инструкции.

Ответ:

- **Добавить системное сообщение о желаемой тональности**
- Использовать другую модель
- Изменить гиперпараметры temperature

🔮 Шаг 06:

Миссия 6

Алекс хочет оптимизировать подсистемы Ollama и сопоставить каждый компонент с его задачей в LLM-инфраструктуре.

Компоненты:

Ответ:

- Менеджер контекста
- Пулинг моделей
- Кэширование ответов
- Контроллер генерации

Функции:

1. Управление сохранённым состоянием и ветвлением диалога
2. Распределение запросов на свободные модели
3. Сохранение часто задаваемых ответов для ускорения
4. Настройка гиперпараметров генерации (temperature, top_k и др.)

Подсказка: Подумайте, какой модуль взаимодействует с диалоговым контекстом, а какой — с распределением нагрузки.

- **Менеджер контекста** — Управление сохранённым состоянием и ветвлением диалога
- **Пулинг моделей** — Распределение запросов на свободные модели
- **Кэширование ответов** — Сохранение часто задаваемых ответов для ускорения
- **Контроллер генерации** — Настройка гиперпараметров генерации (temperature, top_k и др.)

🔮 Шаг 07:

Миссия 7

Во внутреннем модуле Ollama Алекс видит аббревиатуры: LM, API, CLI, SDK. Нужно сопоставить аббревиатуру и её назначение:

Ответ:

- LM
 - API
 - CLI
 - SDK
1. Интерфейс командной строки для взаимодействия
 2. Язык моделей (Language Model)

3. Библиотека для встраивания в приложения
4. Программный интерфейс для взаимодействия с сервером

Подсказка: Подумайте, что обычно происходит в оболочке (CLI) и что — в программном интерфейсе (API).

- **LM** — Язык моделей (Language Model)
- **API** — Программный интерфейс для взаимодействия с сервером
- **CLI** — Интерфейс командной строки для взаимодействия
- **SDK** — Библиотека для встраивания в приложения

Шаг 08:

Миссия 8

Алекс читает логи ошибок и видит коды: 429, 503, 500, 400. Нужно сопоставить код ошибки с возможной причиной:

Ответ:

- 429
 - 503
 - 500
 - 400
1. Неправильный формат запроса (синтаксическая ошибка)
 2. **Слишком много запросов** — превышен лимит
 3. Сервер перегружен или временно недоступен
 4. Внутренняя ошибка сервера

Подсказка: HTTP-коды несут стандартные значения — вспомните из веб-разработки.

- **429** — Слишком много запросов — превышен лимит
- **503** — Сервер перегружен или временно недоступен
- **500** — Внутренняя ошибка сервера
- **400** — Неправильный формат запроса (синтаксическая ошибка)

Шаг 09:

Миссия 9

В логике маршрутизации запросов Алекс видит такие методы: round_robin, least_loaded, random. Нужно сопоставить метод с его свойством:

Ответ:

- round_robin
 - least_loaded
 - random
1. **Выбор самого загруженного** — противоположно желаемому
 2. Очередное распределение по кругу
 3. Случайный выбор среди всех моделей

Подсказка: Подумайте, как балансировщики обычно работают.

- **round_robin** — Очередное распределение по кругу
- **least_loaded** — Выбор самого загруженного — противоположно желаемому
- **random** — Случайный выбор среди всех моделей

Шаг 10:

Миссия 10

После оценок нагрузки Алекс хочет ввести таймаут на ответы: если модель не отвечает за заданное время — вернуть ошибку. Что он должен учесть при выборе времени таймаута?

Подсказка: Баланс между ожиданием и пользовательским опытом — ключ.

Ответ:

- **Учитывать время выполнения самой модели и накладные задержки сети**
- Ставить максимальный таймаут, чтобы никогда не было ошибок
- Не использовать таймаут вовсе

Шаг 11:

Миссия 11

Алекс хочет отладить цепочку запрос → обработка → ответ. Расположите этапы правильным порядком:

1. Маршрутизация запроса к нужной модели
2. Проверка параметров и валидация запроса
3. Генерация ответа моделью
4. Отправка ответа клиенту

Подсказка: Помните: валидация прежде, чем идти в модель, отправка — в конце.

Ответ:

- Проверка параметров и валидация запроса
- Маршрутизация запроса к нужной модели
- Генерация ответа моделью
- Отправка ответа клиенту

Шаг 12:

Миссия 12

Для устойчивости Ollama, Алекс хочет внедрить fallback-модель, если основная не отвечает. Расположите шаги в правильном порядке:

1. Проверить статус основной модели
2. Если основной не отвечает, перенаправить запрос на backup
3. Логировать ошибку и переключение
4. Возврат ответа от backup-модели клиенту

Подсказка: Сначала проверка, затем fallback, потом логирование и возврат.

Ответ:

- Проверить статус основной модели
- Если основной не отвечает, перенаправить запрос на backup
- Логировать ошибку и переключение
- Возврат ответа от backup-модели клиенту

Шаг 13:

Миссия 13

Алекс разрабатывает систему отката (rollback) при ошибках модели. Расположите правильную последовательность действий:

1. Поймать исключение генерации
2. Откатить версию модели к предыдущей стабильной
3. Перезапустить службу модели
4. Вернуть пользователю сообщение об ошибке

Подсказка: Исключения сначала, откат — потом, уведомление клиента — в конце.

Ответ:

- Поймать исключение генерации
- Откатить версию модели к предыдущей стабильной
- Перезапустить службу модели
- Вернуть пользователю сообщение об ошибке

Шаг 14:

Миссия 14

При развертывании обновления Алекс должен следовать порядку операций. Упорядочите их:

1. Переход модели в состояние «прочие» (standby)
2. Загрузка новой версии модели параллельно
3. Тестирование новой версии на Canary-запросах

- Переключение трафика на новую модель

Подсказка: Подумайте: не отключать всё сразу, тестировать сначала на части трафика.

Ответ:

- Переход модели в состояние «прочие» (standby)
- Загрузка новой версии модели параллельно
- Тестирование новой версии на Canary-запросах
- Переключение трафика на новую модель

Шаг 15:

Миссия 15

Алекс планирует резервное распределение загрузки между дата-центрами. Расположите операции в логическом порядке:

1. Оценка текущей нагрузки по регионам
2. Резервирование ресурсов в дополнительных узлах
3. Настройка маршрутов failover
4. Мониторинг переключений и откатов

Подсказка: Сначала оценка, затем подготовка, потом настройка, наконец контроль.

Ответ:

- Оценка текущей нагрузки по регионам
- Резервирование ресурсов в дополнительных узлах
- Настройка маршрутов failover
- Мониторинг переключений и откатов

Шаг 16:

Миссия 16

Выстраивая автоскейлинг модели, Алекс перечисляет операции в правильном порядке:

1. Измерение метрик загрузки (CPU, latency, QPS)
2. Определение порогов для масштабирования вверх/вниз
3. Запуск или завершение инстансов моделей
4. Перенаправление трафика на новые инстансы

Подсказка: Автоскейлинг — это непрерывный цикл мониторинга, принятия решений и действий.

Ответ:

- Измерение метрик загрузки (CPU, latency, QPS)
- Определение порогов для масштабирования вверх/вниз
- Запуск или завершение инстансов моделей
- Перенаправление трафика на новые инстансы

Шаг 17:

Миссия 17

🔔 Финальная миссия: Алекс хочет подготовить систему Ollama к званию «Архитектор LLM». Расположите шаги по порядку:

1. Провести стресс-тестирование всей инфраструктуры
2. Оценить метрики качества ответов и латентности
3. Провести обзор безопасности и устойчивости к атаке
4. Документировать архитектуру и принять «архитектурный статус»

Подсказка: Без тестов и анализа нет уверенности, а документирование — финал пути.

Ответ:

- Провести стресс-тестирование всей инфраструктуры
- Оценить метрики качества ответов и латентности
- Провести обзор безопасности и устойчивости к атаке
- Документировать архитектуру и принять «архитектурный статус»